MindVideo AI

JAX

من البداية إلى الإنتاج





المؤلف

م/أحمد صالح المغـز

2026

الإصدار الأول · 2026

JAX من الصفر إلى الإنتاج

الدليل الشامل في التعلّم العميق والتبديلات القابلة للتركيب: من jax.numpy إلى أنظمة الإنتاج الموزّعة

كتاب عربي شامل يشرح **JAX 0.12** كما هي: تبديلات قابلة للتركيب فوق مترجم XLA، على **Python 3.14**. من أوّل `jnp.array` إلى نموذج موزّع يعمل في الإنتاج، بأسلوب سردي استفهامي، وأمثلة محلولة، وتمارين مُجابة.

jit ∘ grad ∘ vmap ∘ shard_map

32	6	5	+150	+40
فصلاً دراسياً	أجزاء متدرّجة	ملاحق مرجعية	مثالاً برمجياً	مخططاً توضيحياً

م/أحمد صالح المغز

إعداد وتأليف — رحلة تعلّم عملية ومنهجية

الطبعة: الإصدار الأول (1.0)

بيئة العمل: JAX 0.12 · Flax 0.12 · Optax 0.2 · Python 3.14 · XLA

إِهْدَاء

إِلَى زَوْجَتِي الغَالِيَةِ... رَفِيقَةِ الدَّرْبِ،

الَّتِي كَانَتْ لِصَبْرِي سَنَداً، وَلِهَمِّي رَاوِيَةً،

وَلِهَذَا الكِتَابِ وَقْتاً لَا أَسْتَعِيدُهُ.

مَا بَيْنَ دَفَّتَيْ هَذَا الكِتَابِ مِنْ جُهُودٍ، وَمَا فِيهِ مِنْ نَقْصٍ، كِلَاهُمَا لَكِ الحَقُّ فِيهِ أَوَّلاً؛ فَبِاسْمِكِ، وَبِاسْمِ مَا وهَبْتِهِ مِنْ سَكِينَةٍ وَاتِّزَانٍ، أُهْدِيهِ إِلَيْكِ.

مُقَدِّمَةُ المُؤَلِّف

لَمْ يُكْتَبْ هَذَا الكِتَابُ لِيَكُونَ دَلِيلاً تَقْنِيّاً يَحْفَظُ وَاجِهَاتِ JAX غَيْباً، بَلْ لِيَبْنِيَ نَمُوذَجاً ذِهْنِيّاً يَجْعَلُ كُلَّ تَحْدِيثٍ لَاحِقٍ مَفْهُوماً مِنْ أَصْلِهِ. فَـ JAX لَيْسَتْ مَكْتَبَةً أُخْرَى لِلتَّعَلُّمِ العَمِيقِ؛ بَلْ فِكْرَةٌ وَاحِدَةٌ جَرِيئَةٌ: أنَّ التَّحْوِيلَاتِ القَابِلَةَ لِلتَّرْكِيبِ – `jit` وَ `grad` وَ `vmap` وَ `shard_map` – تَكْفِي لِبِنَاءِ مَا لَا تَحُدُّهُ حُدُودٌ.

كَثِيرٌ مِمَّنْ يَقْرَأُ هَذِهِ الصَّفَحَاتِ سَبَقَتْ لَهُ تَجْرِبَةٌ مَعَ إِطَارِ عَمَلٍ تَقْلِيدِيٍّ، فَيَجِدُ فِي JAX عَدَمَ حَالَةٍ وَنَقَاءً وَظِيفِيّاً يَبْدُوَانِ غَرِيبَيْنِ فِي البِدَايَةِ. لِذَلِكَ نَبْدَأُ كُلَّ فَصْلٍ بِسُؤَالٍ أَوْ مُفَارَقَةٍ، لَا بِتَعْرِيفٍ قَامُوسِيٍّ؛ لِأَنَّ السَّبَبَ – لِمَاذَا تَعْمَلُ JAX عَلَى هَذَا النَّحْوِ – هُوَ تَحْدِيداً مَا لَا تَجِدُهُ فِي التَّوْثِيقِ الرَّسْمِيِّ.

الكِتَابُ مُقَسَّمٌ إِلَى سِتَّةِ أَجْزَاءٍ مُتَدَرِّجَةٍ: مِنْ قِصَّةِ وِلَادَةِ JAX وَبِيئَتِهَا، مَرُوراً بِالنَّوَاةِ الأَرْبَعِ، وَبِنَاءِ الشَّبَكَاتِ العَصَبِيَّةِ، وَالبُنَى الحَدِيثَةِ، وَالأَدَاءِ وَالتَّوَسُّعِ، وَحَتَّى الإِنْتَاجِ وَالمَسْؤُولِيَّةِ المِهْنِيَّةِ. كُلُّ فَصْلٍ يَقِفُ عَلَى أَقْدَامِ مَنْ سَبَقَهُ، وَيُشِيرُ إِلَى مَنْ يَلِيهِ؛ فَلَا حَاجَةَ لِقَطْعٍ سِوَى المَسَارِ المُتَّصِلِ.

نَصِيحَتِي لَكَ وَأَنْتَ تَقْرَأُ: لَا تَكْتَفِ بِقِرَاءَةِ الشِّيفْرَةِ، بَلْ نَفِّذْهَا بِيَدِكَ؛ ثُمَّ اكْسِرْهَا عَمْداً لِتَرَى كَيْفَ تَتَصَرَّفُ عِنْدَ الفَشَلِ، فَالبَحْثُ عَنِ الخَطَأِ نِصْفُ التَّعَلُّمِ. وَحَيْثُمَا وَجَدْتَ خِيَاراً، اعْلَمْ أَنَّهُ وُضِعَ لِيَكُونَ لَكَ طَرِيقاً آمِناً لِلْعَوْدَةِ إِلَى الأَصْلِ.

رَغِبْتُ أَنْ يَخْرُجَ الكِتَابُ عَرَبِيّاً فَصِيحاً، أَكَادِيمِيّاً فِي أَصْلِهِ، وَسَرَدِيّاً فِي لُغَتِهِ؛ فَالمَعْرِفَةُ أَبْقَى مَا اسْتَقَرَّتْ فِي قِصَّةٍ. فَإِنْ آمَنْتَ بِهَذَا النَّهْجِ كَمَا آمَنْتُ بِهِ، فَقَدْ بَدَأْتَ الرِّحْلَةَ، وَعَلَيْكَ بَقِيَّتُهَا.

كلّ كتاب تقني يبدأ بسؤال. وسؤال هذا الكتاب ليس «كيف أكتب `jax.jit`؟» – فهذا يُجاب عنه في سطر واحد – بل: **لماذا يفكّر مجتمع كامل بهذه الطريقة؟** ما الذي يجعل باحثاً في DeepMind وباحثاً في OpenAI ومهندساً في شركة أدوية يستخدمون المكتبة نفسها، رغم أن مشكلاتهم لا تتشابه؟ الجواب ليس في الواجهة البرمجية، بل في نموذج ذهني واحد: الدالّة كائن يمكن تحويله.

في 2018 ظهرت JAX داخل Google كتجربة صغيرة على هامش مشروع `autograd`. لم تكن تهدف إلى منافسة TensorFlow – كانت تهدف إلى الإجابة عن سؤال أنيق: ماذا لو تعاملنا مع التفاضل والتجميع والتوجيه والتوزيع كـ**تحويلات (transformations)** تُطبَّق على دالّة بايثون عادية، بدل أن تكون ميزات مبنيّة داخل إطار عمل ضخم؟ بعد ثمانية أعوام، صار هذا السؤال هو الأساس الذي تُبنى عليه مكتبات Flax وEquinox وOptax، وتُدرَّب به نماذج بمليارات المعاملات على آلاف شرائح TPU.

لمن كُتب هذا الكتاب؟

كُتب لمن يعرف بايثون، ويفهم ما يكفي من الجبر الخطّي ليعرف لماذا نضرب مصفوفتين، ولمن جرّب PyTorch أو TensorFlow ويريد أن يفهم **الطبقة التي تحتها**. لا يشترط الكتاب خبرة سابقة بـJAX؛ لكنه لا يبدأ من الصفر المطلق في التعلّم العميق. من لا يعرف ما هي دالّة الخسارة أو الانتشار العكسي سيجد في الجزء الأول ما يكفيه، لكنه سيحتاج إلى صبر.

> ★ **الوعد**
>
> بعد الفصل الثاني عشر ستكون قادراً على قراءة شيفرة JAX لأيّ بحث منشور وفهم **لماذا** كُتبت بهذا الشكل. وبعد الفصل الثلاثين ستكون قادراً على نشر نموذج يعمل، لا على تشغيل دفتر ملاحظات فقط.

لماذا كتاب آخر عن JAX؟

الوثائق الرسمية ممتازة، لكنها مكتوبة لمن يعرف ما يبحث عنه. والدروس المنتشرة على الإنترنت تشرح `jax.jit` في ثلاثين سطراً ثم تنتقل إلى Flax، فتترك القارئ يكتب شيفرة تعمل ولا يفهم سبب عملها – ثم تنكسر في أوّل نموذج حقيقي. هذا الكتاب يحاول أن يفعل العكس: يبدأ من السبب، ثم يصل إلى الواجهة، ثم يشرح كيف تنكسر الواجهة ومتى.

لذلك ستجد في كل فصل أربعة عناصر لا تتغيّر: **سؤال يفتح الفصل**، و**معادلة أو نموذج ذهني** يفسّره، و**شيفرة قابلة للتشغيل**، و**تمرين يُجاب عنه**. وإن غاب أحدها، فذلك خطأ في الكتاب لا في المنهج.

ما الذي تغطّيه هذه النسخة؟

الكتاب مبنيّ على **JAX 0.12** (إصدار أكتوبر 2026) و**Python 3.14**. هذا مهمّ لأن JAX تتغيّر بسرعة، وبعض ما ستجده في دروس قديمة لم يبقَ صالحاً: `pmap` لم يعد الطريق الموصى به، و`jax.experimental.pjit` اندمج في `jax.jit`، و`jax.custom_remat` أُزيل لصالح `jax.custom_vjp.defremat`، و`jax2tf` أُهمل لصالح `jax.export`. حيثما وُجد فرق بين ما تقوله الدروس القديمة وما تعمل به النسخة الحالية، سأقول ذلك صراحةً.

المكوّن	الإصدار	ملاحظة
Python	`3.14`	JAX 0.12 يتطلّب 3.12 أو أحدث، ويدعم 3.14 و3.14t
jax / jaxlib	`0.12.0`	NumPy ≥ 2.2 إلزامي
Flax	`0.12.10`	Linen مستقرّ، وNNX هو الجيل الجديد
Optax	`0.2.8`	المحسّنات كسلسلة تحويلات
Orbax	`0.12.x`	نقاط الحفظ المتوافقة مع التقطيع

جدول 0.1 — بيئة الكتاب المرجعية

كيف تقرأ الكتاب؟

ثلاثة مسارات ممكنة، وكلّها مشروعة:

- **المسار الكامل:** من الفصل الأول إلى الثاني والثلاثين بالترتيب. مناسب لمن يبدأ من الصفر.
- **المسار العملي:** الفصول 2، 3، 6، 7، 8، 13، 14، 15، 16 — ثمّ العودة إلى ما تحتاجه.
- **المسار المرجعي:** الفهرس والملاحق. اقرأ عند الحاجة، واستخدم الملحق (ج) لبنك المعادلات.

نصيحة واحدة: **لا تقرأ فصل شيفرة دون أن تشغّل شيفرته**. JAX مكتبة تُفهم بالأصابع أكثر مما تُفهم بالعين، ورسائل خطئها — خاصة أخطاء التتبّع — لا تُنسى بعد أن تراها مرّة.

✓ نصيحة عملية

أنشئ مستودعاً واحداً لكل تمارين الكتاب، وفعّل `jax.config.update("jax_debug_nans", True)` أثناء التطوير. ستوفّر على نفسك ساعات من البحث عن `NaN` لا تعرف من أين جاء.

شكر وتنويه

لا يوجد كتاب تقني يُكتب وحده. أفكار هذا الكتاب مدينة لمجتمع JAX المفتوح: لفريق JAX في Google، ولمن كتبوا `autograd` وشرحوا فكرته، ولمن أداروا نقاشات المسألة `#jax` ليلاً. وأي خطأ في الصفحات هو خطأ المؤلّف وحده.

وأخيراً: هذا الكتاب أداة، لا غاية. إن انتهيت منه وأنت تعرف كيف تقرأ شيفرة JAX لم تكتبها، فقد أدّى غرضه.

لكل كتاب عقدٌ غير مكتوب مع قارئه: ما الذي سيقدّمه، وكيف، وما الذي لن يقدّمه. هذا الفصل يوضّح العقد بصراحة، حتى لا تنتظر من الكتاب شيئاً لم يَعِد به – أو تفوّت شيئاً وعد به.

عناصر الفصل الثابتة

كل فصل مبنيّ على خمسة أنواع من العناصر. تعلّمها مرّة واحدة، وستقرأ الكتاب كلّه بسرعة.

خمسة عناصر، وخمس وظائف

الافتتاحية السردية

مشهد أو سؤال يربط الموضوع بالواقع قبل أيّ معادلة. اقرأها كاملة؛ فهي تحدّد المشكلة التي يحلّها الفصل.

المعادلة والنموذج الذهني

كل معادلة يسبقها سؤال «لماذا هذه المعادلة؟» ويتبعها شرح بالكلمات. لا معادلة بلا تفسير.

الشيفرة القابلة للتشغيل

كل كتلة شيفرة كاملة بما يكفي لتشغيلها وحدها، مع تعليقات عربية على الأسطر المهمّة.

الصناديق التوضيحية

ستّة أنواع: ملاحظة، تحذير، نصيحة، مفتاح، رؤية، خطأ شائع. وظيفتها تثبيت ما لا يجب نسيانه.

التمرين والحلّ

تمرين في نهاية كل فصل، يليه حلّ نموذجي كامل. الحلّ مطويّ في النسخة الرقمية، وظاهر في المطبوعة.

دلالة الألوان والرموز

◆ ملاحظة

توضيح جانبي مفيد، لكنه ليس ضرورياً لفهم الفقرة التي يليها.

★ مفتاح

فكرة جوهريّة يجب أن تخرج بها من الفصل. إن نسيت كل شيء آخر، فاحفظ هذه.

⚠ تحذير

سلوك قد يُنتج نتائج خاطئة بصمت – لا خطأً يوقفه المترجم. الأخطر في JAX من نوعه.

✕ خطأ شائع

نمط يقع فيه المبتدئون (وأحياناً المحترفون)، مع تفسير سببه وكيفية تجنّبه.

✓ نصيحة

ممارسة عملية توفّر وقتاً أو ذاكرة، ومجرّبة على شيفرة حقيقية.

✦ رؤية

ربط بين مفهوم في JAX وفكرة أعمّ في التعلم العميق أو في هندسة البرمجيات.

تشغيل الشيفرة

كل الشيفرة مكتوبة لـ**Python 3.14** و**JAX 0.12**. لتشغيلها:

أوامر الطرفية

```
$ python -m venv .venv
$ source .venv/bin/activate        # على Windows: .venv\Scripts\activate
$ pip install "jax[cuda13]" flax optax orbax-checkpoint
$ python -c "import jax; print(jax.__version__, jax.devices())"
```

إن لم يكن لديك GPU، احذف `[cuda13]`؛ ستعمل كل الأمثلة على المعالج، لكن أبطأ. الأمثلة التي تحتاج أكثر من جهاز موسومة صراحةً، ويمكن تشغيلها على جهاز واحد مع `XLA_FLAGS=--xla_force_host_platform_device_count=8`.

✓ مصدر الحقيقة

عند التعارض بين هذا الكتاب والوثائق الرسمية، الوثائق هي المرجع. الإصدارات تتغيّر، والكتاب يشرح الفكرة التي تبقى بعد أن تتغيّر الواجهة.

ما لا يقدّمه الكتاب

- ليس مرجعاً كاملاً لواجهة JAX – فذلك دور الوثائق، وهي آلاف الصفحات.
- ليس مقدّمة إلى التعلّم العميق من الصفر المطلق؛ يلزمك فهم أساسي للشبكات.
- لا يغطّي تدريب نماذج لغوية بمليارات المعاملات عملياً؛ يشرح المبادئ والتقطيع، لا هندسة العناقيد.
- لا يقدّم «أفضل الممارسات» كقوانين مقدّسة؛ يقدّمها كقرارات لها أسباب وثمن.

> الاصطلاحات ليست تفاصيل شكلية. حين يكتب كتابٌ «الشدّة» مرّة و«التينسور» مرّة أخرى، يظنّ القارئ أنهما شيئان. هذا الفصل يثبّت المصطلح والرمز والخط قبل أن نبدأ، حتى يبقى انتباهك للمعنى لا للشكل.

سياسة المصطلحات

أستخدم المصطلح العربي عند أوّل ظهور، وأُتبعه بالإنجليزي بين قوسين، ثم أستمرّ بالعربي أو بالإنجليزي حسب شيوعه في الممارسة. فـ«التبديل» مصطلح واضح، لكن `jit` اسم علم لا يُترجم؛ و«التقطيع» أوضح من sharding، لكن `Mesh` تبقى `Mesh`. القاعدة: **لا أُعرّب ما لا يُعرَّب في الممارسة اليومية**.

العربي	الإنجليزي	ملاحظة
تبديل	transformation	دالّة تأخذ دالّة وتُرجع دالّة: `jit`، `grad`، `vmap`
تتبّع	tracing	تشغيل الدالّة على قيم مجرّدة لبناء الرسم البياني
تجميع	compilation	تحويل الرسم إلى شيفرة جهاز عبر XLA
تقطيع	sharding	توزيع المصفوفة على عدّة أجهزة
شبكة أجهزة	mesh	تنظيم الأجهزة في محاور مسمّاة
شجرة	pytree	بنية متداخلة من قواميس وقوائم
نقاء	purity	غياب الآثار الجانبية
مفتاح	PRNG key	مفتاح عشوائية صريح يُقسَّم ولا يُستهلك
حقل استقبالي	receptive field	مساحة المدخل التي تؤثّر في نخرَج واحد

جدول 0.2 — أهمّ المصطلحات كما تُستخدم في هذا الكتاب

الأرقام

أستخدم **الأرقام الغربية** (0-9) في كل الكتاب، بما في ذلك أرقام الصفحات والفصول. السبب عملي: كل الشيفرة والأشكال والمصفوفات مكتوبة بها، والخلط بين نظامين داخل الصفحة نفسها يُربك العين. أرقام الفصول مكتوبة بالحروف في العناوين («الفصل السابع») لأنها أسماء لا قيماً.

الرموز الرياضية

الرمز	المعنى
x, y	متجه أو مصفوفة مدخل/خرج
W, b	مصفوفة أوزان ومتجه انحياز
θ	مجموعة المعاملات كلّها (pytree)
L	دالّة الخسارة
$\nabla_\theta L$	تدرّج الخسارة بالنسبة إلى المعاملات
$\odot$	ضرب عنصراً بعنصر
$@$	ضرب المصفوفات
$\mathbb{E}[\cdot]$	التوقّع الرياضي
$\sigma(\cdot)$	دالّة سيغمويد أو انحراف معياري حسب السياق

جدول 0.3 — الرموز المتّبعة

اصطلاحات الشيفرة

- الشيفرة بلون داكن، واتجاهها من اليسار إلى اليمين دائماً – حتى داخل صفحة عربية.
- التعليقات داخل الشيفرة بالعربية، والأسماء (المتغيّرات والدوال) بالإنجليزية.
- كل كتلة تحمل اسم اللغة في أعلاها: `Python` أو `أوامر الطرفية`.
- المخرجات تُعرض في صندوق فاتح أسفل الكتلة، موسوم بـ«المخرجات».
- السطر الذي يبدأ بـ `$` أمر طرفية، لا شيفرة بايثون.

✦ لماذا الاتجاه مهمّ؟

لو عُرضت الشيفرة من اليمين إلى اليسار، لانقلبت الأقواس وعلامات الإسناد، وصار `x = y + 1` شيئاً آخر. لذلك تُعزل الشيفرة باتجاه LTR مستقلّ عن اتجاه الفقرة – وهذا قرار طباعي مقصود، لا سهو.

بنية الإحالات

الأشكال مرقّمة بالفصل: «الشكل 7.2» هو الشكل الثاني في الفصل السابع. الجداول كذلك. المعادلات مرقّمة فقط حين يُشار إليها لاحقاً. الإحالات إلى الفصول مكتوبة بالحروف («الفصل الحادي عشر») لتسهيل القراءة الخطّية.

فهرس الكتاب

يضمّ الكتاب ستّة أجزاء و32 فصلاً و5 ملاحق مرجعية. الفهرس قابل للنقر في النسخة الرقمية، وأرقام الصفحات مطابقة لنسخة PDF.

الجزء الأول: الأسس — لماذا JAX، وكيف تفكّر بها

القصة، والبيئة، والمصفوفات، والأجهزة: ما تحتاج معرفته قبل أوّل تبديل

الفصل الأول القصة: لماذا وُلدت JAX؟ **18**
من autograd إلى التبديلات القابلة للتركيب — المشكلة التي حَلّتها، والثمن الذي تدفعه

الفصل الثاني بيئة عمل احترافية على Python 3.14 ..... **28**
التثبيت، والأجهزة، والتحقق، والحتمية، وهيكل المشروع

الفصل الثالث jax.numpy: المصفوفات كما لم تعرفها **36**
الأنواع، والترقية، وx64، والفروق الخفيّة عن NumPy

الفصل الرابع الأشكال والبثّ والفهرسة المتقدّمة **44**
einsum، gather/scatter، وعمليات المقاطع، وقواعد البثّ

الفصل الخامس الأجهزة والذاكرة: jax.Array **52**
أين تعيش البيانات، وكيف تنتقل، ومتى تُنسخ، ولماذا يهمّ ذلك

الجزء الثاني: التبديلات الأربع

jit وgrad وvmap وshard_map: قلب JAX ونموذجها الذهني

الفصل السادس jit: التجميع الذي يغيّر كل شيء **62**
التتبّع، والرسم البياني، وإعادة التجميع، وحدود ما يمكن تجميعه

الفصل السابع grad: التفاضل التلقائي بلا سحر **71**
العكسي والأمامي، value_and_grad، والقيود التي يجب أن تعرفها

الفصل الثامن vmap: الدُفعات بلا حلقات 79
التوجيه التلقائي، والمحاور، والتفاعل مع الحالة والتفرّع

الفصل التاسع النقاء والحالة وأشجار pytree 87
لماذا لا حالة في JAX؟ وكيف تبني نماذج بحالة نظيفة

الفصل العاشر random: العشوائية الصريحة 97
المفاتيح، split، fold_in، وفلسفة التكرار الحتمي

الفصل الحادي عشر من pmap إلى shard_map: التوازي 105
تاريخ التوازي في JAX، والطريق الحديث إلى عدّة أجهزة

الفصل الثاني عشر تركيب التبديلات: القواعد الذهبية 113
jit(grad(vmap(f))): كيف تتركّب، وأين تنكسر، وكيف تتجنّب ذلك

الجزء الثالث: بناء الشبكات العصبية

من الطبقة الأولى إلى حلقة تدريب إنتاجية: Flax Optax Orbax

الفصل الثالث عشر شبكة عصبية من الصفر بـ jax.numpy 122
MLP كامل، وتدرّج يدوي، وتدريب حقيقي بلا مكتبات عالية

الفصل الرابع عشر Flax: من Linen إلى NNX 132
فلسفة المكتبة، وأنماط النماذج، والفرق بين الجيلين

الفصل الخامس عشر Optax: المحسّنات كتحويلات 140
التدرّج، والزخم، وآدم، والجدولة، والقصّ، والتركيب

الفصل السادس عشر حلقة تدريب احترافية 148
الخطوة، والقياس، ونقاط الحفظ، وEMA، والاستئناف

الفصل السابع عشر التنظيم والتعميم 159
Dropout، weight decay، والتطبيع، Mixup، وتعزيز البيانات

الفصل الثامن عشر jaxtyping و Equinox: النمط البايثوني 168
نماذج ككائنات عادية، وأنواع للأشكال، وتحويل آمن

الجزء الرابع: البنى العصبية الحديثة

الصور، والتسلسلات، والانتباه، والمحوّلات، والنماذج التوليدية

الفصل التاسع عشر الشبكات التلافيفية في JAX 177

الالتفاف، والتجميع، والحقل الاستقبالي، وبنى ResNet

الفصل العشرون التسلسلات: RNN وLSTM وGRU عبر scan **186**
الحلقة كدالة، والتدرّج عبر الزمن، والسلاسل الطويلة

الفصل الحادي والعشرون الانتباه والمحوّلات **195**
الانتباه متعدّد الرؤوس، والترميز الموضعي، وذاكرة KV

الفصل الثاني والعشرون GPT صغير من الصفر **203**
الترميز، والكتلة، والتدريب، والتوليد، والتقطيع

الفصل الثالث والعشرون النماذج التوليدية **212**
VAE، وGAN، ونماذج الانتشار: كيف يولّد النموذج ما لم يره

الجزء الخامس: الأداء والتوسّع
XLA، والتقطيع، Pallas، والذاكرة، والدقة العددية

الفصل الرابع والعشرون XLA من الداخل **221**
HLO، والدمج، وحدود التجميع، وذاكرة الترجمة

الفصل الخامس والعشرون التقطيع: Mesh وNamedSharding **229**
GSPMD، وأنماط البيانات والمعاملات والتسلسل، والتحميل الموزّع

الفصل السادس والعشرون Pallas: نوى مخصّصة **238**
متى تكتب نواة؟ وكيف؟ ومن TPU إلى GPU

الفصل السابع والعشرون الذاكرة والأداء **245**
remat، وoffload، وdonation، والقياس، والتشخيص

الفصل الثامن والعشرون الدقة العددية والتكميم **252**
bf16، وfp8، والتكميم، والاستقرار العددي

الجزء السادس: من المختبر إلى الإنتاج
التصدير، والنشر، والاختبار، ودراسات الحالة، والمسؤولية المهنية

الفصل التاسع والعشرون التصدير والنشر **260**
jax.export، وONNX، وFastAPI، والحزمة، والمراقبة

الفصل الثلاثون الاختبار والحتمية وتشخيص الأخطاء **268**
من رسالة الخطأ إلى السبب الجذري: دليل عملي

الفصل الحادي والثلاثون دراسات حالة من الحياة العملية 277
الصحة، والتمويل، والصناعة، والتوصية، والطاقة، والروبوتات

الفصل الثاني والثلاثون الأخلاقيات والخصوصية وخارطة الطريق 288
العدالة، والخصوصية التفاضلية، والأمن، واثنا عشر شهراً

الملاحق

مراجع سريعة تُستشار أثناء العمل

الملحق A خريطة المقابلة بين JAX وPyTorch 296
كل مفهوم بمقابله، مع الفروق الجوهرية

الملحق B مسرد المصطلحات 302
عربي ↔ إنجليزي، مع تعريف موجز

الملحق C بنك المعادلات والخلاصات التشغيلية 306
كل معادلة مهمّة في مكان واحد

الملحق D التمارين الإضافية والحلول 313
تمارين من كل فصل مع حلول كاملة

الملحق E المراجع والمصادر 323
الوثائق الرسمية، والأبحاث المؤسِّسة، والكتب

الجزء الأول

الأسس — لماذا JAX، وكيف تفكّر بها

القصة، والبيئة، والمصفوفات، والأجهزة: ما تحتاج معرفته قبل أوّل تبديل

القصة: لماذا وُلدت JAX؟

من autograd إلى التبديلات القابلة للتركيب – المشكلة التي حَلّتها، والثمن الذي تدفعه

تخيّل أنك باحث في مختبر أدوية، أمامك مليون مركّب كيميائي، ولكل مركّب 2048 وصفاً رقمياً. تريد أن تختبر أثر تغيير صغير في كل وصف على احتمال الفاعلية – أي تريد **المشتقّة** بالنسبة إلى مليون متغيّر. ثم تريد أن تفعل ذلك على 64 بطاقة رسوميات في وقت واحد، وأن تكون النتيجة قابلة للتكرار بعد سنة. الآن تخيّل أن كل هذا يُكتب في ملف واحد، بدالّة واحدة، بلا إطار عمل يخفي عنك ما يحدث. هذا هو الوعد الذي أطلقت JAX نفسها لأجله. فهل أوفت به؟ ولماذا احتاجت إلى ثمانية أعوام لتُفهم فهماً صحيحاً؟

سؤالان لا يجيب عنهما إطار العمل التقليدي

حين تكتب نموذجاً في إطار عمل تقليدي، أنت لا تكتب دالّة رياضية؛ أنت تصف بناءً ثم تُشغّله. الإطار يملك الحلقة، ويملك المحسّن، ويملك الجهاز، ويملك الحالة. هذا مريح جداً في 90% من الحالات. لكنه يصبح قيداً في الحالات التي وُجدت JAX لأجلها:

1. **أريد أن أشتقّ أي شيء.** لا الخسارة فقط: مشتقّة الخرج بالنسبة إلى المدخل، ومشتقّة المشتقّة، وتدرّج دالّة تُرجع مصفوفة كاملة (Jacobian)، وتدرّج دالّة فيها حلقة زمنية.
2. **أريد أن أُجمّع أي شيء.** لا الطبقة فقط: خطوة تدريب كاملة، أو دالّة فيزيائية، أو محاكاة، أو حتى خوارزمية تحسين ليست شبكة عصبية أصلاً.

> لو كانت المكتبة تمنحك التدرّج تلقائياً، فلماذا تحتاج إلى كتابة التدرّج كتحويل؟ لأن التحويل يعني أنك تستطيع تركيبه: أن تأخذ الدالّة، وتشتقّها، ثم تُوجّهها على دُفعة، ثم تُجمّعها، ثم توزّعها — كل ذلك بلا أن يعرف أيّ من هذه الخطوات شيئاً عن الآخر.

من autograd إلى JAX: فكرة واحدة، ثلاث مرات

في 2015 نشر ثلاثة باحثين من هارفارد ورقة عن `autograd`: مكتبة صغيرة تُمكّن بايثون من التفاضل تلقائياً. الفكرة المركزية لم تكن «حساب التدرّج»، بل شيء أعمق: **التفاضل عملية على الدوال، لا على الأرقام**. إذا كان لديك $f: \mathbb{R}^n \to \mathbb{R}$، فإن `grad(f)` دالّة جديدة بنفس التوقيع. وهذا يعني أنك تستطيع كتابة:

$$g = \text{grad}(f) \rightarrow g(x) = \nabla f(x) \rightarrow \text{grad}(g)(x) = \nabla^2 f(x)$$

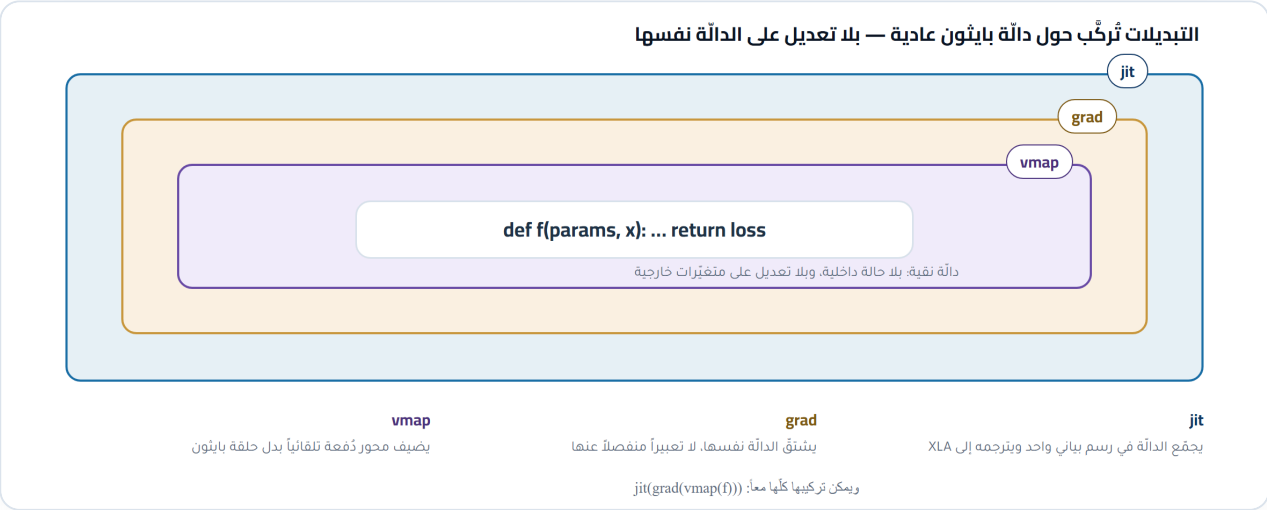
تركيب التفاضل مع نفسه يعطي المشتقّة الثانية بلا كود إضافي

JAX أخذت هذه الفكرة ووسّعتها إلى أربعة تحويلات متماثلة في الشكل. كلّها دوالّ من النوع «دالّة ← دالّة»، وكلّها قابلة للتركيب مع بعضها بلا حدود مصمّمة مسبقاً:

$$\text{jit}(f) : f \rightarrow f_{\text{compiled}}$$
$$\text{grad}(f) : f \rightarrow \nabla f$$
$$\text{vmap}(f) : f \rightarrow f_{\text{batched}}$$
$$\text{shard_map}(f) : f \rightarrow f_{\text{distributed}}$$

التحويلات الأربعة: كل واحد يأخذ دالّة ويُرجع دالّة بنفس التوقيع تقريباً

هذا التماثل هو كل شيء. لأنه يعني أن `jit(grad(vmap(f)))` ليست حيلة، بل نتيجة طبيعية لقاعدة الأنواع: كل تحويل يُرجع دالّة، والدالّة تُقبل كمدخل لأي تحويل آخر. وهذا ما لا تستطيعه الأطر التي تُخفي الحلقة: فيها التدرّج ميزة داخل المحرّك، والتوجيه ميزة أخرى، والتوزيع ثالثة، ولا تُركَّب الثلاثة إلا بما يسمح به المحرّك.



الشكل 1.1 — التبديلات تُركَّب حول دالّة بايثون عادية. الدالّة لا تعرف أنها ستُشتقّ أو تُجمَّع أو تُوجَّه؛ وكل تحويل يرى الدالّة كصندوق أسود.

ولماذا XLA؟ لأن التجميع هو الفرق بين 1× و4×

لو كانت JAX مجرّد واجهة أنيقة فوق NumPy، لما كانت تستحق كتاباً. الفرق الحقيقي أنها تبني **رسمًا بيانيّاً** من دالّتك ثم تسلّمه إلى **XLA** – مترجم كان أصلاً لتسريع TensorFlow على شرائح TPU. وXLA لا ينفّذ عملياتك واحدة واحدة؛ بل يدمجها، ويعيد ترتيبها، ويحذف ما لا يُستخدم، ويخصّص الذاكرة مسبقاً.

المثال الكلاسيكي: كتابة `relu(x @ W + b)` في NumPy تُنتج ثلاث مصفوفات وسيطة في الذاكرة. في XLA تصبح نواة واحدة تقرأ `x` و`W` و`b` مرّة، وتحسب الناتج، وتكتبه مرّة. على مصفوفة كبيرة، هذا الفرق بين أن تكون الذاكرة عنق الزجاجة أو لا تكون.

> ★ **المفتاح**
>
> JAX ليست «NumPy على GPU». JAX هي **لغة بناء برامج** تُترجم إلى XLA. حين تكتب `jnp` فأنت تكتب برنامجاً، لا سلسلة حسابات فورية – وهذا الفرق يفسّر كل سلوك غريب ستقابله لاحقاً.

ولترى الفرق عملياً، قارن النسختين. الأولى تحسب على المعالج بأربع مصفوفات وسيطة، والثانية تُجمّع الحساب كلّه في برنامج واحد:

```python
import numpy as np

x = np.random.randn(4096, 4096).astype("float32")
W = np.random.randn(4096, 4096).astype("float32")
b = np.random.randn(4096).astype("float32")

h = x @ W          # 1 مصفوفة وسيطة
h = h + b          # 2 مصفوفة وسيطة
h = np.maximum(h, 0)   # 3 مصفوفة وسيطة
print(h.shape)
```

```python
import jax
import jax.numpy as jnp

@jax.jit
def block(x, W, b):
    return jax.nn.relu(x @ W + b)

x = jax.random.normal(jax.random.key(0), (4096, 4096))
W = jax.random.normal(jax.random.key(1), (4096, 4096))
b = jnp.zeros(4096)

h = block(x, W, b)      # نواة واحدة، لا مصفوفات وسيطة
```

الفرق ليس في عدد الأسطر، بل في ما يحدث في الذاكرة: النسخة الثانية لا تُنشئ `h` ثلاث مرّات. المترجم يرى التعبير كوحدة واحدة، ويولّد شيفرة تقرأ المدخلات مرّة وتكتب الخرج مرّة. على مصفوفة 4096×4096، هذا يعني توفير مئات الميغابايتات من الحركة – والحركة، لا الحساب، هي ما يستهلك الوقت على GPU.

التبديلات الأربعة: خريطة الكتاب كلّها

يمكنك أن تقرأ الكتاب كلّه كشرح مطوّل لهذا الجدول. احفظه الآن، وستفهم كل فصل مكانه فيه.

التبديل	ماذا يأخذ	ماذا يُرجع	السؤال الذي يجيب عنه
`jax.jit`	دالّة بايثون	دالّة مجمّعة	كيف أُسرّع الحساب وأدمج العمليات؟
`jax.grad`	دالّة عددية	دالّة التدرّج	كيف أشتقّ بلا كتابة المشتقّة يدوياً؟
`jax.vmap`	دالّة لعيّنة	دالّة لدُفعة	كيف أتخلّص من حلقة `for`؟
`jax.shard_map`	دالّة لجهاز	دالّة لعدّة أجهزة	كيف أوزّع الحساب بلا تغيير المنطق؟

جدول 1.1 — التبديلات الأربعة وماذا تغيّر فعلاً

ولاحظ ما لا تجده في الجدول: لا يوجد «محسّن»، ولا «طبقة»، ولا «حلقة تدريب». هذه ليست نقصاً؛ إنها قرار تصميمي. JAX توفّر اللبنات الأولية، وتترك بناء النموذج لمكتبات مثل **Flax** و**Equinox**، والتحسين لـ**Optax**، والحفظ لـ**Orbax**. وهذا هو سبب وجود مكدّس كامل حول مكتبة تبدو صغيرة.

✓ كيف تحفظ الجدول؟

اربط كل تبديل بالفعل الذي يمنعه: `jit` يمنع البطء، و `grad` يمنع الاشتقاق اليدوي، و `vmap` يمنع حلقة `for`، و `shard_map` يمنع نسخ الشيفرة لكل جهاز. حين تنسى أيّها تستخدم، اسأل: «ما الشيء الذي أريد أن أتخلّص منه؟»

لو أردت أن تحسب مشتقّة الخسارة بالنسبة إلى المدخلات لا المعاملات — كما في الهجمات العدائية أو في رسم خرائط الأهمية — أيّ تبديل تستخدم؟ وهل يغيّر ذلك شيئاً في الشيفرة؟ ؟



الشكل 1.2 — مكدّس JAX: كل طبقة تُخفي تعقيد ما تحتها، وتكشف ما يكفي للتحكّم. JAX نفسها تقع في المنتصف: لا تعرف شيئاً عن «الطبقات»، وتعرف كل شيء عن التبديلات.

الثمن: ما تخسره حين تختار JAX

كل تصميم ثمن. ومن يبيعك JAX بلا ثمن يبيعك وهماً. الثمن هنا ثلاثة أشياء، وكلها ستظهر في الفصول القادمة:

⚠ الثمن الأول: لا حالة

لا يمكنك تعديل مصفوفة في مكانها، ولا الاحتفاظ بعدّاد داخل النموذج، ولا طباعة قيمة أثناء التجميع. كل شيء يجب أن يمرّ كمدخل ومخرج. هذا يبدو قيداً في البداية، ثم تكتشف أنه يجعل الشيفرة قابلة للاختبار والتوزيع بشكل لم تجرّبه قبل.

⚠ الثمن الثاني: التتبّع ليس بايثون

داخل `jit`، الدالّة تُشغَّل على قيم «مجرّدة» لا تحمل أرقاماً. أيّ قرار يعتمد على قيمة مدخل (`if x > 0`) سيفشل أو يجمّد على أوّل قيمة رآها. هذا أكثر خطأ يُصادفه المبتدئ، وسنفرد له فصلاً كاملاً.

جرّب هذا بنفسك. الدالّة تبدو بريئة تماماً، وهي مكتوبة ببايثون صحيحة:

Python

```python
import jax
import jax.numpy as jnp

@jax.jit
def bad(x):
    if x[0] > 0:          # قرار يعتمد على قيمة!
        return x * 2
    return x + 1

bad(jnp.array([1.0, 2.0]))
```

‹‹ المخرجات ››

```
TracerBoolConversionError: Attempted boolean conversion of
traced array. The tracer cannot be converted to a Python bool…
```

الرسالة صحيحة ودقيقة، لكنها لا تقول «لا تضع `if` هنا». تقول: «هذه القيمة ليست رقماً، بل رمز». والفرق بين الفهمين هو الفرق بين ساعة من البحث وأسبوع.

⚠ الثمن الثالث: منحنى تعلّم حقيقي

PyTorch تسمح لك بالتفكير بـ«ماذا يحدث الآن؟». JAX تطلب منك التفكير بـ«ما شكل الرسم البياني الذي أبنيه؟». هذا الانتقال يستغرق أسبوعين عادةً، ثم يصبح طبيعياً – وأسرع في التفكير من الأول.

متى تختار JAX؟ ومتى لا؟

السؤال ليس «أيّ مكتبة أفضل؟» بل «أيّ مكتبة تناسب شكل مشكلتي؟». الجدول التالي ليس حكماً نهائياً، بل ملخّص لتجربة عملية.

المعيار	JAX 0.12	PyTorch 2.x
نموذج التفكير	دوالّ نقية + تبديلات	كائنات + تنفيذ فوري
التصحيح	أصعب: أخطاء تتبّع مجرّدة	أسهل: تتبّع مكدّس عادي
الأداء على الأجهزة المتخصّصة	ممتاز (TPU أولاً، XLA)	جيد جداً (CUDA/ROCm)
التوزيع على عدّة أجهزة	مدمج في اللغة (sharding)	DDP/FSDP عبر وحدات
النظام البيئي للنماذج الجاهزة	أصغر، لكنه ينمو بسرعة	الأكبر في المجال
البحث العلمي والاشتقاق المتقدّم	الأقوى	ممكن، لكنه أقل مرونة
النشر في الإنتاج	ممكن (export/ONNX/serving)	أنضج وأكثر أدوات

جدول 1.2 — مقارنة عملية بين JAX وPyTorch (بحسب حالة أكتوبر 2026)

اختر JAX إذا...

- تبحث في أفكار جديدة تحتاج مشتقّات غير قياسية.
- تعمل على TPU أو على عدّة أنواع من العتاد.
- تحتاج توزيعاً على عشرات الأجهزة بمنطق واضح.
- تحبّ أن تفهم ما يحدث تحت الغطاء.

لا تختر JAX إذا...

- تحتاج نموذجاً جاهزاً في يومين.
- فريقك لا يعرف نموذج «الدالّة النقية».
- تعتمد على مكتبات لم تُنقل بعد إلى JAX.
- تحتاج تصحيحاً تفاعلياً سريعاً كل خمس دقائق.

خريطة الطريق

الكتاب ستّة أجزاء. الجزء الأول يبني الأرضية: البيئة، والمصفوفات، والأجهزة. الجزء الثاني هو القلب: التبديلات الأربعة وكيف تتركّب. الجزء الثالث يبني الشبكات فعلياً. الجزء الرابع يغطّي البنى الحديثة: CNN وRNN

والمحوّلات والتوليد. الجزء الخامس للأداء والتوسّع. الجزء السادس ينزل بك إلى الإنتاج والحياة العملية.

ستّة أجزاء، وثلاثون فصلاً بعد هذا

1. الأسس: لماذا JAX، والبيئة، والمصفوفات، والأجهزة
2. التبديلات: jit، grad، vmap، pytree، random، shard_map، التركيب
3. الشبكات: من الصفر، Flax، Optax، حلقة التدريب، التنظيم، Equinox
4. البنى: RNN، CNN، الانتباه، GPT، التوليد
5. الأداء: XLA، التقطيع، Pallas، الذاكرة، الدقّة
6. الإنتاج: التصدير، الاختبار، دراسات الحالة، المسؤولية

تمرين الفصل الأول

1. اكتب دالّة بايثون عادية تحسب $f(x) = x^3 + 2x$. ثم اشتقّها يدوياً. الآن اكتب نسخة تستخدم `jax.grad` وقارن القيمتين عند `x = 1.5` .
2. بأيّ معنى يكون `grad(f)` «دالّة» وليس «عدداً»؟ اذكر مثالاً يوضّح الفرق عملياً.
3. اقرأ الجدول 1.2، واكتب في ثلاثة أسطر أيّ مكتبة تناسب مشروعك الحالي ولماذا. ثم اذكر سبباً واحداً يجعل الاختيار الآخر مقنعاً أيضاً.

الحلّ النموذجي

1. المشتقّة اليدوية هي $f'(x) = 3x^2 + 2$ ، وعند `x = 1.5` تساوي `3(2.25) + 2 = 8.75` .

```python
import jax
import jax.numpy as jnp

def f(x):
    return x ** 3 + 2 * x

df = jax.grad(f)
print(f(1.5), df(1.5))          # 6.375 8.75
print(jax.grad(jax.grad(f))(1.5))  # 9.0  ← 6 المشتقّة الثانية لـx
```

2. لأن `grad(f)` تُرجع شيئاً قابلاً للاستدعاء على أيّ قيمة، لا رقماً محسوباً عند نقطة واحدة. الدليل العملي: السطر الأخير أعلاه – أخذنا تدرّج التدرّج بلا كتابة أيّ كود جديد. لو كان `grad` يُرجع عدداً، لما أمكن استدعاؤه مرّة ثانية.

3. الإجابة مفتوحة، لكن الإجابة الجيّدة تذكر شكل المشكلة لا شعبية المكتبة. مثال: «أختار PyTorch لأن فريقي يبني خدمة تعتمد على `torchvision` ونحتاج تسليماً في أسبوعين؛ والاختيار الآخر مقنع لأن نموذجنا سيبقى ثابتاً سنوات، وJAX ستعطينا عمراً أطول للشيفرة وتوزيعاً أبسط.»

خلاصة الفصل

- JAX ليست إطار عمل، بل **لغة تبديلات** فوق XLA: أربع تحويلات تُركَّب بلا حدود.
- قوّتها من التماثل: كل تحويل يأخذ دالّة ويُرجع دالّة، فتصبح التركيبة طبيعية.
- ثمنها: لا حالة، وتتبّع لا يشبه بايثون، ومنحنى تعلّم حقيقي.
- ليست بديلاً عن PyTorch في كل حالة؛ هي الأفضل حين تكون المشكلة رياضية أو موزّعة.

مصطلحات الفصل

transformation (تبديل)

دالّة تأخذ دالّة وتُرجع دالّة، وتُغيّر طريقة تنفيذها لا معناها.

XLA

مترجم جبري خطّي مُسرَّع، يدمج العمليات ويولّد شيفرة للجهاز.

pure function (دالّة نقية)

دالّة لا تعتمد إلا على مدخلاتها ولا تُغيّر شيئاً خارجها.

tracing (تتبّع)

تشغيل الدالّة على قيم مجرّدة لاستخراج الرسم البياني للحساب.

SPMD

برنامج واحد، بيانات متعدّدة: الشيفرة نفسها تعمل على كل الأجهزة.

بيئة عمل احترافية على Python 3.14

التثبيت، والأجهزة، والتحقق، والحتمية، وهيكل المشروع

في أحد المختبرات، أمضى باحث أسبوعاً كاملاً في مطاردة نتائج «تتغيّر قليلاً» بين تشغيلين متتاليين. جرّب كل شيء: بدّل معدّل التعلّم، غيّر البذرة، أعاد تثبيت المكتبة. في النهاية اكتشف أن المشكلة لم تكن في النموذج: كانت نسخة `jaxlib` مختلفة عن نسخة `jax` بواحد. أسبوع من العمل، بسبب سطر لم يقرأه أحد. هذا الفصل عن ذلك السطر: كيف تبني بيئةً لا تخونك بعد شهر، وتتحقّق منها في ثلاثين ثانية.

لماذا Python 3.14 تحديداً؟

JAX 0.12 يتطلّب Python 3.12 أو أحدث، ويدعم 3.14 و3.14t (النسخة بلا قفل GIL). هذا ليس ترفاً: الإصدارات الأحدث جلبت تحسينات في الأداء وفي نموذج الأنواع، والأهمّ أن **jaxlib** – النواة الأصلية – تُبنى وتُختبر على نطاق ضيّق من إصدارات بايثون. الخروج عن هذا النطاق يعني أنك ستُبنى من المصدر، وهنا تبدأ المشكلات.

> ⚠ **قاعدة صفر**
>
> لا تخلط أبداً بين `jax` و`jaxlib` من إصدارين مختلفين. حدّثهما معاً دائماً في أمر واحد، أو استخدم ملف قيود (`constraints.txt`) يثبّتهما.

> إذا كانت JAX تعمل على Python 3.12 وما بعده، فلماذا يصرّ كثير من الفرق على تثبيت إصدار واحد بعينه في ملف القيود، بدل السماح بالتحديث التلقائي؟

التثبيت: أربع حالات، وأمر واحد

JAX تُوزَّع بواجهة أساسية بلا أي عتاد متخصّص، ثم «إضافات» (extras) لكل نوع عتاد. اختر واحدة فقط:

العتاد	الأمر	ملاحظة
CPU فقط	`pip install -U jax`	كل الأمثلة تعمل، أبطأ

العتاد	الأمر	ملاحظة
NVIDIA (CUDA 13)	`pip install -U "jax[cuda13]"`	المسار الموصى به في 2026
NVIDIA (CUDA 12)	`pip install -U "jax[cuda12]"`	لبيئات قديمة
Google TPU	`pip install -U "jax[tpu]"`	على أجهزة Cloud TPU
AMD (ROCm)	`pip install -U "jax[rocm7-local]"`	بناء محلي؛ راجع الوثائق

جدول 2.1 — أوامر التثبيت بحسب العتاد (JAX 0.12)

مع `uv` – وهو الأسرع اليوم – يصبح الأمر:

أوامر الطرفية

```
$ uv venv --python 3.14
$ source .venv/bin/activate
$ uv pip install -U "jax[cuda13]" flax optax orbax-checkpoint
$ uv pip freeze > requirements.lock
```

✓ لماذا ملف قيود؟

`pip freeze` يثبّت كل شيء بما فيه التبعيات العابرة. بعد شهر، قد يُصدر `jaxlib` تحديثاً يكسر توافقاً دقيقاً؛ ملف القيود يمنع ذلك. اجعله جزءاً من المستودع، لا من حاسوبك فقط.

التحقّق في ثلاثين ثانية

لا تبدأ أيّ مشروع قبل أن تشغّل هذا السكربت وترى خرجه. احفظه باسم `check_env.py`.

```python
import jax
import jax.numpy as jnp
import jaxlib

print("jax     :", jax.__version__)
print("jaxlib  :", jaxlib.__version__)
print("backend :", jax.default_backend())
print("devices :", jax.devices())

# اختبار وظيفي صغير: جمع + تجميع + تدرّج
@jax.jit
def f(x):
    return jnp.sum(jnp.tanh(x) ** 2)

x = jnp.arange(8.0)
print("f(x)    :", f(x))
print("grad    :", jax.grad(f)(x))

# تحقّق من أن التجميع يعمل فعلاً على الجهاز المطلوب
print("lowered :", f.lower(x).compile().devices())
```

الخرج المتوقّع على جهاز فيه بطاقة واحدة:

‹‹ المخرجات ››

```
jax     : 0.12.0
jaxlib  : 0.12.0
backend : gpu
devices : [CudaDevice(id=0)]
f(x)    : 20.741589
grad    : [0.0000000e+00 7.6391354e-01 ...]
lowered : {cuda:0}
```

× خطأ شائع: `backend: cpu` مع بطاقة موجودة

هذا يعني أن JAX لم يجد مكتبات CUDA، لا أن البطاقة غير مدعومة. تحقّق من `nvidia-smi`، ثم من إصدار CUDA المطلوب مقابل المثبّت، ثم من أنك لم تثبّت `jax` العادي فوق نسخة `cuda`.



الشكل 2.1 — خمس خطوات من الصفر إلى أوّل تجميع ناجح. كل خطوة قابلة للتحقّق، ولا تنتقل إلى التالية قبل أن ترى نتيجتها.

الأجهزة: كيف يختار JAX؟

عند الاستيراد، يبحث JAX عن كل الأجهزة المتاحة ويرتّبها بأولوية: TPU ثم GPU ثم CPU. ويمكنك التحكّم بذلك عبر متغيّر البيئة `JAX_PLATFORMS` :

أوامر الطرفية

```
$ JAX_PLATFORMS=cpu python train.py        # للتجريب على المعالج
$ JAX_PLATFORMS=cuda,cpu python train.py   # جرّب CUDA، وارجع إلى CPU عند الفشل
```

وللتحكّم في الذاكرة – وهو أهمّ إعداد على GPU – استخدم:

أوامر الطرفية

```
$ # لا تحجز كل ذاكرة البطاقة مسبقاً (مفيد عند المشاركة)
$ export XLA_PYTHON_CLIENT_PREALLOCATE=false
$ # أو: احجز 80% فقط
$ export XLA_PYTHON_CLIENT_MEM_FRACTION=.8
```

✦ لماذا يحجز XLA كل الذاكرة؟

لأن التخصيص الديناميكي أثناء التجميع يكلّف، ولأن المترجم يخطّط للذاكرة مسبقاً. هذا ممتاز في جهاز مخصّص للتدريب، وسيّئ في دفتر ملاحظات مشترك. لهذا يوجد المتغيّران أعلاه.

لو شاركت بطاقة واحدة بين زميلين، وأطلق كلٌّ منكما عملية تدريب تحجز كل الذاكرة، فما الذي سيحدث؟ وأيّ إعداد يمنع المشكلة قبل أن تحدث؟

الحتمية: أن تحصل على النتيجة نفسها مرّتين

الحتمية في JAX ثلاث طبقات، وكل طبقة تُحلّ بطريقة مختلفة:

1. **حتمية العشوائية.** JAX لا تستخدم مولّداً عالمياً؛ تستخدم مفاتيح صريحة. نفس المفتاح ← نفس الأرقام، دائماً. (الفصل العاشر.)

2. **حتمية الترتيب.** عمليات مثل `segment_sum` قد تُنفَّذ بترتيب غير محدّد على GPU. الحل: `XLA_FLAGS=--xla_gpu_deterministic_ops`، بثمن في الأداء.

3. **حتمية الأجهزة.** تشغيل الشيفرة نفسها على GPU وTPU قد يعطي فروقاً في آخر ثلاث خانات عشرية. لا تعتمد على تطابق تامّ عبر عتاد مختلف.

```python
import jax
# فور حدوثه بدل البحث عنه لاحقاً NaN اكتشاف
jax.config.update("jax_debug_nans", True)
# (معطّل افتراضياً لأسباب أداء) تفعيل الأعداد 64-بت
jax.config.update("jax_enable_x64", True)
```

> ⚠ **تحذير بخصوص x64**
>
> تفعيل `jax_enable_x64` يضاعف استهلاك الذاكرة ويُبطئ GPU بشكل ملموس. لا تفعّله إلا إذا كنت تحتاج دقّة عددية حقيقية (محاكاة، تحليل عددي). معظم نماذج التعلّم العميق لا تحتاجه.

هيكل مشروع لا يخونك

بعد عشرات المشاريع، يتشابه الهيكل الذي يصمد: فصل الإعداد عن الشيفرة، وفصل البيانات عن النموذج، ووجود مكان واحد للحتمية.

نص

```
project/
├── pyproject.toml
├── requirements.lock
├── configs/
│   ├── base.yaml
│   └── exp001.yaml
├── src/
│   ├── __init__.py
│   ├── data.py          # التحميل والتحويل
│   ├── model.py         # تعريف الشبكة
│   ├── train.py         # حلقة التدريب
│   ├── evaluate.py
│   └── utils.py         # الحتمية، السجلات، المسارات
├── tests/
│   ├── test_shapes.py   # اختبار الأشكال قبل القيم
│   └── test_train.py    # خطوة واحدة تنتهي بلا NaN
└── artifacts/           # نقاط الحفظ (خارج git)
```

★ اختبار الأشكال قبل القيم

أكثر من نصف أخطاء JAX أخطاءُ أشكال لا أخطاءُ قيم. اكتب اختباراً يمرّر دفعة وهمية ويتحقّق من `out.shape` – قبل أن تنتظر ساعة من التدريب لتنهار في الخطوة الأخيرة.

تمرين الفصل الثاني

1. أنشئ بيئة جديدة بـ `uv`، وثبّت JAX، وشغّل سكربت التحقّق. ما قيمة `jax.default_backend()` عندك؟ ولماذا؟
2. شغّل السكربت مرّتين، وقارن `grad`. هل النتيجتان متطابقتان تماماً؟ جرّب إضافة `--xla_gpu_deterministic_ops` ولاحظ الفرق.
3. اكتب اختباراً واحداً يتحقّق من أن دالّة `f(x) = x @ W` تُرجع شكلاً صحيحاً لدفعة من 32 عيّنة بـ16 ميزة وخرج بـ10 أصناف.

الحلّ النموذجي

1. على جهاز بلا GPU ستكون القيمة `cpu`؛ ومع بطاقة مدعومة ونسخة `[cuda13]` ستكون `gpu`. المهمّ ألّا تكون `gpu` مع بطاقة موجودة — فذلك يعني مشكلة توافق لا مشكلة عتاد.

2. غالباً ستتطابق القيم على CPU، وقد تختلف في آخر خانة على GPU بسبب ترتيب الجمع في العمليات المتوازية. مع `--xla_gpu_deterministic_ops` يصبح الترتيب ثابتاً، لكن الأداء ينخفض بنسبة تتراوح عادةً بين 10% و30%.

3.

```python
import jax
import jax.numpy as jnp

def test_shape():
    W = jnp.ones((16, 10))
    f = jax.jit(lambda x, W: x @ W)
    out = f(jnp.ones((32, 16)), W)
    assert out.shape == (32, 10), out.shape

test_shape()
print("ok")
```

خلاصة الفصل

- ثبّت `jax` و `jaxlib` معاً، واحفظ ملف قيود في المستودع.
- شغّل سكربت تحقّق يطبع الإصدارات والأجهزة ويجمّع دالّة صغيرة.
- تحكّم في ذاكرة GPU بـ `XLA_PYTHON_CLIENT_*`، وفي الجهاز بـ `JAX_PLATFORMS`.
- الحتمية ثلاث طبقات: العشوائية، والترتيب، والعتاد — ولا تُطلب كلها في كل مشروع.
- اختبر الأشكال قبل القيم: فهي أرخص وأسرع.

مصطلحات الفصل

jaxlib

النواة الأصلية التي تنفّذ العمليات وتتحدّث إلى XLA والأجهزة.

backend (خلفية)

العتاد الذي ينفّذ عليه الحساب: `cpu` أو `gpu` أو `tpu`.

XLA_FLAGS

متغيّر بيئة يمرّر أعلاماً مباشرة إلى مترجم XLA.

deterministic ops

وضع يفرض ترتيباً ثابتاً للعمليات غير الترابطية، بثمن في السرعة.

extras (إضافات)

لاحقات في أمر التثبيت مثل `[cuda13]` تجرّ حزم العتاد المناسبة.

jax.numpy: المصفوفات كما لم تعرفها

الأنواع، والترقية، وx64، والفروق الخفيّة عن NumPy

جرّب هذا: اطبع نوع `jnp.array([1.0, 2.0])` وستجد `float32` لا `float64`. ثم اكتب `x[0] = 5` وستُقابَل برسالة خطأ. ثم قِس حلقة صغيرة وستجد JAX أبطأ من NumPy بأضعاف. ثلاث مفاجآت في ثلاث دقائق، وليست واحدة منها خطأً؛ كلها نتائج مباشرة لنموذج تنفيذ مختلف.

لماذا jnp وليس numpy؟

استبدل `np.` بـ `jnp.` في سكربت قديم، وسيعمل في معظم الحالات. لكن NumPy تُنفّذ `a + b` بحلقة بلغة C على المخزن فوراً، بينما كل دالّة في `jnp` **عنصر أوّلي** (primitive) في لغة وسيطة يفهمها XLA، والناتج `jax.Array` يعيش على الجهاز وقد لا تكون قيمته جاهزة لحظة عودته.

خارج `jax.jit` يترجم JAX كل عنصر وحده ويرسله (op-by-op). وداخله تُلتقط السلسلة كاملة في رسم بياني واحد ويُترجم مرّة: الفرق كألف رسالة قصيرة ورسالة واحدة طويلة.

> ★ **الواجهة عقد، والتنفيذ ليس كذلك**
>
> أن تُرجع `jnp.dot` النتيجة نفسها التي تُرجعها `np.dot` لا يعني أن التكلفة نفسها ولا الأخطاء نفسها.

إذا كانت الواجهة متطابقة إلى هذا الحدّ، فلماذا لم يكتفِ فريق JAX بترقيع NumPy من الداخل بدل بناء مكتبة جديدة؟ ؟

الأنواع والترقية: لماذا يعود float32 دائماً؟

تحسب متوسط خسارة في محاكاة عددية، فتختلف النتيجة عن مرجعك في الخانة الثامنة؛ والسبب سطر واحد: النوع افتراضياً `float32`. تثبّت JAX هذا الافتراضي لأن نصف الذاكرة وضعف عرض النطاق على المسرّعات هو ما تحتاجه الشبكات. والأعداد المكتوبة بلغة بايثون **أنواع ضعيفة** (weak types): العدد `1.0` لا يفرض عرضاً، بل يتبع النوع الآخر.

Python

```python
import jax.numpy as jnp

print(jnp.array(1.0).dtype)                    # float32 لا float64
print((jnp.ones(3, jnp.float32) + 1.0).dtype)  # float32: العدد الضعيف تبع المصفوفة
print((jnp.ones(3, jnp.float16) + 1.0).dtype)  # float16: بقي على عرضه
print(jnp.result_type(1.0, jnp.float32))       # float32
print(jnp.result_type(jnp.bfloat16, jnp.float32))  # float32
```

‹‹ المخرجات ››

```
float32
float32
float16
float32
float32
```

لاحظ السطر الثالث: العدد `1.0` لم يرفع `float16` إلى `float32` كما في NumPy القديم؛ فتحسب مع ثابت دون أن تُفسد نوع مصفوفتك.

> ✕ **خطأ شائع: «float64 هو الأصل، وfloat32 تبسيط»**
>
> في JAX العكس صحيح: `float32` هو الأصل، و`float64` ميزة تُفعَّل صراحةً. من يعود من NumPy يفسّر فروق الدقة على أنها «عدم استقرار» وهي نوعٌ لم يطلبه.

x64: القرار الذي تتّخذه مرّة واحدة

إن كنت تحسب تدرّجات عبر تفاضل عددي، أو تحلّ معادلة تفاضلية، أو تقارن بمرجع `float64`، فأنت تحتاج x64. والمفتاح سطر واحد: `jax.config.update("jax_enable_x64", True)`. والتوقيت مهمّ: الإعداد يغيّر سلوك التتبّع، فضعه قبل إنشاء أيّ مصفوفة وقبل أوّل استدعاء لـ `jax.jit`، أو استخدم `JAX_ENABLE_X64=1`؛ فتغييره في منتصف الجلسة يترك مصفوفات قديمة بنوع مختلف.

> ⚠ **الثمن مزدوج**
>
> تفعيل x64 يضاعف حجم كل موتر وعرض النطاق، ويُبطئ بعض نوى GPU بفارق ملموس لأن العتاد مُحسَّن لـ `float32` و`bfloat16`.

عدم القابلية للتغيير: لماذا لا تستطيع كتابة x[0] = 5؟

السطر `x[0] = 5` يرفع `TypeError`. ولماذا؟ لأن التعديل في المكان يجعل الدالّة تعتمد على الزمن: من عدّل المصفوفة، ومتى، وبأيّ ترتيب. وحين يعتمد الحساب على ترتيب الأحداث يستحيل على المترجم أن يعيد ترتيب العمليات أو أن يوزّعها على أجهزة.

البديل تحديث وظيفي عبر `.at`: تُنتج مصفوفة جديدة ويبقى الأصل كما هو، وتُقرأ السلسلة كجملة: خُذ `x`، عند الموضع صفر، ضع خمسة.

```python
import jax.numpy as jnp

x = jnp.zeros(5)
y = x.at[0].set(5.0).at[2].add(1.0)

print(y)   # [5. 0. 1. 0. 0.]
print(x)   # [0. 0. 0. 0. 0.]  الأصل لم يُمسّ

idx = jnp.array([1, 3])
print(x.at[idx].set(jnp.array([7.0, 9.0])))   # تحديث بعدّة مواضع دفعةً
```

> ✦ **لماذا هذا مكسب لا عبء؟**
>
> لأن المصفوفة صارت قيمة لا مكاناً؛ والمترجم يستطيع حينها أن يعيد استخدامها ويتبرّع بذاكرتها بعد آخر استخدام.

الفروق الخفية عن NumPy

الواجهة واحدة، والقواعد ليست. هذا جدول بالفروق التي تُكلّف وقتاً حقيقياً عند الانتقال.

السلوك	NumPy	JAX
التعديل في المكان	`x[0] = 5`	`x.at[0].set(5)`
معامل `out=`	مدعوم	غير موجود
العشوائية	حالة عالمية عبر `seed`	مفاتيح عبر `jax.random.key`
النوع الافتراضي	`float64` و `int64`	`float32` و `int32`

السلوك	NumPy	JAX
مصفوفات الكائنات	`dtype=object`	غير مدعومة
الأشكال	ديناميكية	ثابتة وقت التتبّع
تساوي عناصر الفرز	حسب الخوارزمية	صرّح بـ `stable=True`
التقطيع	عرض قابل للكتابة	قيمة جديدة فقط
السلاسل والقوائم	مصفوفات نصّية	ممنوعة في الحساب

جدول 3.1 — تسعة فروق بين `numpy` و `jax.numpy`

لنأخذ صفّين ونرى ما يحدث على الطرفية.

```python
import numpy as np
import jax, jax.numpy as jnp

a = np.zeros(3)
np.add(a, 1.0, out=a)          # يعمل: الكتابة في المكان

b = jnp.zeros(3)
try:
    jnp.add(b, 1.0, out=b)     # JAX في out لا معامل
except TypeError as e:
    print("out:", e)

key = jax.random.key(0)        # لا حالة عالمية، بل مفتاح صريح
print(jax.random.normal(key, (2,)))
print(jax.random.normal(key, (2,)))   # القيم نفسها تماماً
```

✓ قاعدة الترجمة الذهنية

حين تنقل شيفرة من NumPy، ابحث عن ثلاثة أنماط: إسناد إلى شريحة، ومعامل `out=`، واستدعاء لدالّة عشوائية.

الأداء: لماذا jnp أبطأ خارج jit؟

قِس حلقة من ألف عملية صغيرة على مصفوفة 64×64، وستجد JAX أبطأ من NumPy وربما بعشرة أضعاف. هي أسرع حين تُترك لتُترجم، لا حين تُستدعى عنصراً عنصراً.

$$T_{total} \approx N \cdot (t_{dispatch} + t_{kernel}) + T_{compile}$$

خارج التجميع تدفع كلفة الإرسال لكل عملية؛ وداخله تدفع كلفة الترجمة مرّة واحدة.

كلفة الإرسال ($t_{dispatch}$) هي ثمن عبور الحدّ بين بايثون والجهاز: بناء الوسائط، والتحقّق من الأنواع، وإطلاق النواة. على موتر كبير تضمحلّ أمام زمن الحساب، وعلى صغير تسيطر عليه – وهذا ما يفعله `jax.jit` : يدفعها مرّة لكل رسم بياني.

Python

```python
import time
import jax, jax.numpy as jnp

def f(a):
    return jnp.tanh(a) * a + 1.0

x = jnp.ones((64, 64))
f(x).block_until_ready()          # تسخين: أوّل نداء يترجم

def bench(fn):
    t0 = time.perf_counter()
    for _ in range(1000):
        fn(x)
    jax.block_until_ready(fn(x))  # لا تقِس قبل أن ينتهي العمل فعلاً
    return round(time.perf_counter() - t0, 3)

print("eager:", bench(f))
print("jit  :", bench(jax.jit(f)))
```

‹‹ المخرجات ››

```
eager: 0.412
jit  : 0.007
```

الفارق في عدد مرّات عبور الحدّ إلى الجهاز، لا في الحساب. وسنعود إلى شروط التجميع في الفصل السادس؛ يكفي الآن أن قياس JAX بلا `block_until_ready` قياس كاذب.

متى تستخدم jnp ومتى تستخدم lax؟

السؤال ليس أيّهما أفضل، بل أيّهما يطابق نيّتك: `jnp` طبقة رفيعة تبثّ الأشكال وترقّي الأنواع وتتصرّف كـNumPy، و`jax.lax` هي العناصر نفسها بلا مجاملة: أشكال مطابقة، وأنواع مطابقة، وتدفّق صريح.

jnp — الوضوح

حساب مصفوفي عادي، بثّ تلقائي، شيفرة تُقرأ كالرياضيات.

lax — التحكّم

حلقة بعدد غير معروف وقت التتبّع (`lax.while_loop`)، أو تفرّع يعتمد على قيمة (`lax.cond`).

القاعدة: اكتب بـ `jnp` حتى تصطدم بجدار. الجدران ثلاثة: شكل يتغيّر، أو تفرّع يعتمد على قيمة، أو حلقة لا تعرف طولها. عندها انزل إلى `lax` .

لو كتبت حلقة `while` بايثون داخل دالّة مجمّعة تنتظر حتى يصغر خطأ ما، فما الذي سيحدث للمترجم؟ ولماذا لا يستطيع الانتظار؟ ؟

تمرين الفصل الثالث

1. ما نوع ناتج `jnp.array([1, 2, 3]) / 2` ؟ ولماذا لا يعطيك `float64` ؟
2. حوّل حلقة تحدّث `x[i]` في مكانه إلى تحديث وظيفي واحد عبر `.at` .
3. قِس زمن الدالّة خارج `jit` وداخله، وفسّر الفرق.

الحلّ النموذجي

1. الناتج `float32` ، لأن المصفوفة الصحيحة تُرقّى إلى النوع العائم الافتراضي؛ وطلب `float64` بلا x64 يُختزل بتحذير.

```python
import jax, jax.numpy as jnp

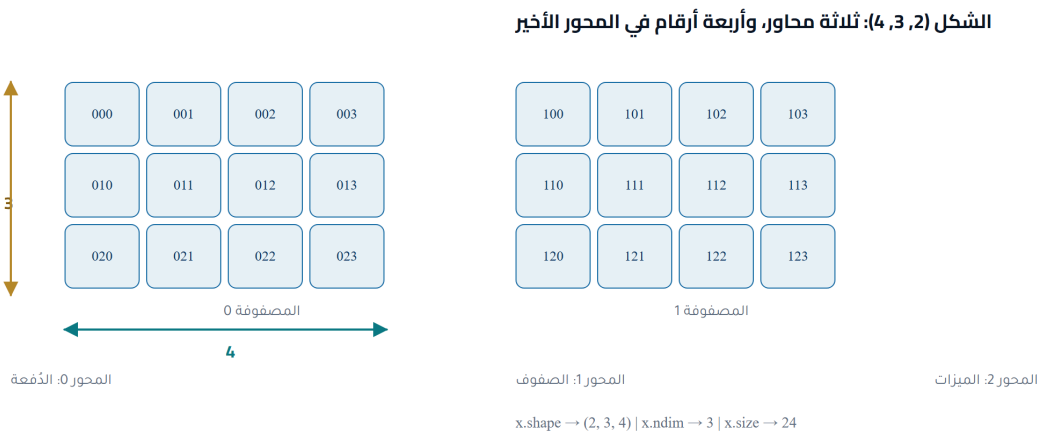
print((jnp.array([1, 2, 3]) / 2).dtype)              # float32
print(jnp.array([1, 2, 3], dtype=jnp.float64).dtype) # float32: x64 معطّل

jax.config.update("jax_enable_x64", True)
print(jnp.array([1, 2, 3], dtype=jnp.float64).dtype) # float64
```

2. سلسلة واحدة، والأصل ثابت:

```python
x = jnp.zeros(4)
x = x.at[0].set(1.0).at[1].set(2.0).at[2].set(3.0)
print(x)        # [1. 2. 3. 0.]
```

3. على CPU يكون الفارق عادةً بين 5× و50× لصالح التجميع، ويتقلّص كلّما كبر الموتر لأن t_{kernel} يطغى على $t_{dispatch}$.



الشكل 3.1 — الشكل أوّل ما يقرأه JAX من المصفوفة، قبل النوع وقبل القيم.

خلاصة الفصل

- `jnp` تشبه NumPy في الواجهة وتختلف في التنفيذ: كل دالّة عنصر أوّلي.
- الافتراضي `float32`، وأعداد بايثون أنواع ضعيفة تتبع المصفوفة.
- x64 قرار يُتّخذ مرّة واحدة قبل إنشاء المصفوفات، وثمنه ذاكرة وسرعة.
- لا تعديل في المكان، وخارج `jit` تدفع كلفة إرسال لكل عملية.

مصطلحات الفصل

primitive (عنصر أوّلي)

أصغر عملية يفهمها XLA، مثل الجمع أو الظلّ الزائدي.

weak type (نوع ضعيف)

نوع لا يفرض عرضاً، فيتبع النوع الآخر.

promotion (ترقية)

قاعدة تحدّد النوع الناتج عند اجتماع نوعين مختلفين.

op-by-op execution

تنفيذ كل عملية على حدة خارج `jit` بكلفة إرسال مستقلة.

functional update (تحديث وظيفي)

إنتاج قيمة جديدة بدل التعديل، كما في `x.at[i].set(v)`.

الأشكال والبثّ والفهرسة المتقدّمة

einsum، gather/scatter، وعمليات المقاطع، وقواعد البثّ

نموذج يعمل، وخسارة تنخفض، ونتائج معقولة. ثم تكتشف أن الدقّة صفر. لا خطأ، لا تحذير، لا انهيار – فقط رقم خاطئ. تتبّع السلسلة، فتجد أن الشيفرة طرحت مصفوفة شكلها `(32, 10)` من أخرى شكلها `(32,)`، فبثّ JAX الأولى إلى `(32, 32)` في صمت. هذا الفصل عن ذلك الصمت.

الشكل هو العقد

كل موتر يحمل وعداً: «سأعطيك دفعةً من 32 عيّنة، كلٌّ منها 10 أرقام». والدالّة التالية تبني على ذلك الوعد. والخطأ لا يقع حين يتغيّر الشكل، بل حين يتغيّر في صمت ويبقى متوافقاً بما يكفي ليمرّ.

لهذا تُقرأ الأشكال بصوت مسموع: `(32, 784)` «دفعة × ميزات»، `(8, 32, 784)` «رؤوس × دفعة × ميزات». وحين يمرّ الشكل بلا معنى في الجملة، فهو غالباً خطأ لم يظهر بعد.

> ★ **الشكل عقد يُوقَّع قبل القيم**
>
> اختبار الشكل أسرع بألف مرّة من اختبار القيمة، ويكشف ما لا تكشفه الخسارة: أن الترتيب نفسه خطأ.

لماذا لا يشتكي JAX حين يطرح `(32, 10)` من `(32,)`؟ وما الذي يجعل هذا السكوت أخطر من رسالة خطأ صريحة؟

قواعد البثّ الثلاث

البثّ ليس سحراً، بل ثلاث قواعد تُطبَّق من اليمين إلى اليسار: حاذِ الأبعاد من الطرف الأيمن، ثم قارن كل زوج: إن تساوى البُعدان مرّ، وإن كان أحدهما واحداً تمدّد، وإلّا فخطأ.

$$(3, 1, 4) \odot (2, 4) \to (3, 2, 4)$$

البُعد الأيمن 4 = 4، ثم 1 يتمدّد إلى 2، ثم 3 يبقى وحده فيُكرَّر.

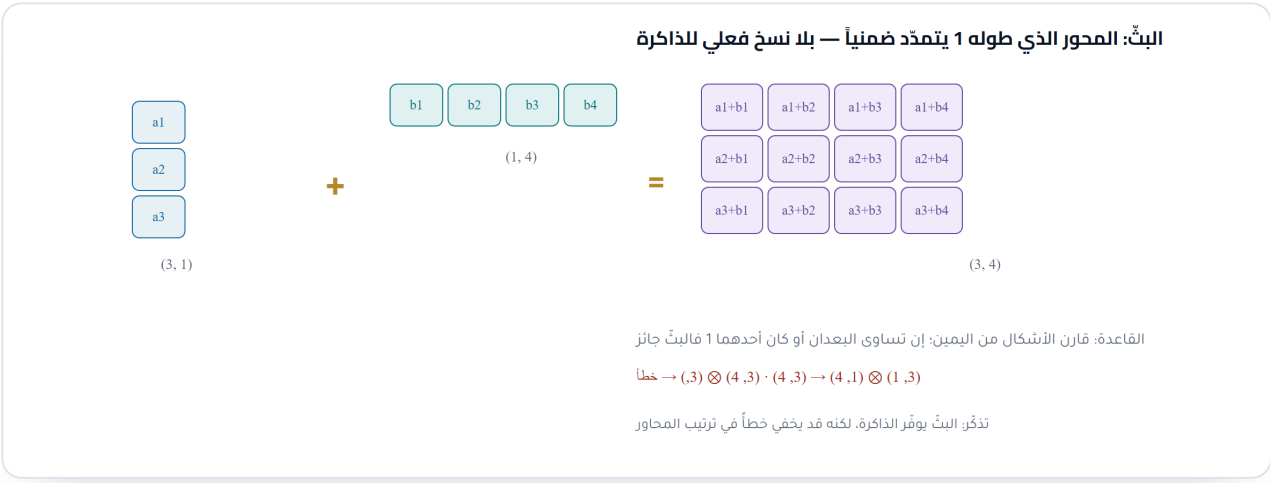
```python
import jax.numpy as jnp

a = jnp.ones((3, 1, 4))
b = jnp.ones((2, 4))
print((a + b).shape)        # (3, 2, 4)

c = jnp.ones((3, 4))
d = jnp.ones((3,))
try:
    c + d                   # 4 لا توافق :3 مقابل
except ValueError as e:
    print("broadcast:", e)
```

لاحظ الحالة الأخيرة: `(3, 4)` و`(3,)` لا يتوافقان، لأن المحاذاة من اليمين تقارن 4 بـ3. لو كان الثاني `(4,)` لمرّ.

> **⚠ البثّ الصامت هو الفخّ**
>
> حين تكون الأبعاد متوافقة بالصدفة يمرّ الحساب وينتج شكلاً أكبر ممّا توقّعت. أضف `assert` على الشكل المتوقّع بعد كل عملية غير بديهية.



الشكل 4.1 — البثّ يبدأ من اليمين: البُعد ذو القيمة واحد يتمدّد، والبُعد الغائب يُعامَل كواحد.

إعادة التشكيل والتبديل

`reshape` يعيد ترتيب القراءة الخطّية للذاكرة، و`transpose` يبدّل محاور، و`moveaxis` ينقل محوراً إلى موضع، و`expand_dims` يضيف محوراً بطول واحد. الأول يغيّر التجميع، والثلاثة الأخرى تغيّر الترتيب.

```python
import jax.numpy as jnp

x = jnp.arange(12).reshape(3, 4)
print(x.T.shape)                    # (4, 3) تبديل
print(jnp.moveaxis(x, 0, 1).shape)  # (4, 3) نقل محور
print(jnp.expand_dims(x, 0).shape)  # (1, 3, 4) إضافة محور

y = jnp.ones((2, 3, 4))
print(y.transpose(0, 2, 1).shape)   # (2, 4, 3)
print(y.reshape(2, 12).shape)       # (2, 12) نفس ترتيب الذاكرة
```

إعادة التشكيل قد تنسخ البيانات فعلاً حين لا يطابق الترتيب الجديد الترتيب الخطّي؛ أمّا التبديل فيغيّر الوصف لا المحتوى.

> **× خطأ شائع: `reshape` حيث تريد `transpose`**
>
> تنتقل من `(B, T, D)` إلى `(B, D, T)` بـ `reshape`، فيمرّ الشكل ويختلط الزمن بالميزات. الأعراض: تدريب يتقارب ببطء بلا سبب ظاهر. القاعدة: إن تغيّر معنى المحاور فاستخدم `transpose`.

einsum: لغة واحدة لكل الضربات

`jnp.einsum` تكتب المعادلة كما تكتبها على الورق: أسماء المحاور حروف، والتكرار يعني الجمع، وما يظهر يسار السهم فقط هو ما يبقى.

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj} \iff \text{"ik,kj→ij"}$$

الحرف k يظهر في المدخلين ولا يظهر في الخرج، فهو محور الجمع.

```python
import jax.numpy as jnp

A, B = jnp.ones((2, 3)), jnp.ones((3, 4))
print(jnp.einsum("ik,kj->ij", A, B).shape)      # (2, 4)

X, Y = jnp.ones((5, 2, 3)), jnp.ones((5, 3, 4))
print(jnp.einsum("bik,bkj->bij", X, Y).shape)   # (5, 2, 4) دفعة كاملة

M = jnp.arange(9).reshape(3, 3)
print(jnp.einsum("ii->", M))                    # الأثر: مجموع القطر
print(jnp.einsum("ii->i", M))                   # القطر نفسه
```

القوة هنا أن المعادلة نفسها تصف الحالة المفردة والمجمّعة: أضف حرفاً في الأول وكرّره في الثاني. وسنستخدم هذا في فصل الانتباه، حيث تُكتب الدرجات كلها بـ `einsum` واحدة.

segmentg scatterg gather

حين تحتاج عناصر بأعيانها لا شرائح متّصلة، استخدم `jnp.take` للجمع و `x.at[idx].set` أو `.add` للتوزيع. وحين تكون لديك مجموعات بأطوال مختلفة فالحلّ `segment_sum` و `segment_max`: تُعطى لكل عنصر رقم مجموعته، فتُجمع المجموعات دفعةً واحدة.

```python
import jax, jax.numpy as jnp

x = jnp.array([10.0, 20.0, 30.0, 40.0])
idx = jnp.array([2, 0, 2])
print(jnp.take(x, idx))          # [30. 10. 30.]
print(x.at[idx].add(1.0))        # [11. 20. 32. 40.] تراكم على المتكرّر

# بيانات غير مستوية: ثلاث مجموعات بأطوال مختلفة
vals = jnp.array([1.0, 2.0, 3.0, 4.0, 5.0])
seg = jnp.array([0, 0, 1, 2, 2])
print(jax.ops.segment_sum(vals, seg, num_segments=3))   # [3. 3. 9.]
```

✓ لماذا المقاطع لا الحلقات؟

لأن الشكل يبقى ثابتاً: خمسة عناصر، ثلاثة مقاطع، دائماً. الحلقات على بيانات غير مستوية تُنتج أشكالاً متغيّرة، وأشكالاً متغيّرة لا تُترجم.

الفهرسة المنطقية والمتقدّمة

`x[mask]` يعمل خارج `jit` ويُعيد شكلاً يعتمد على البيانات. وداخل `jit` يستحيل ذلك: الشكل يجب أن يُعرف قبل التشغيل. البديل `jnp.where(mask, x, 0.0)` : يُبقي الشكل ويصفّر ما لا يطابق.

```python
import jax, jax.numpy as jnp

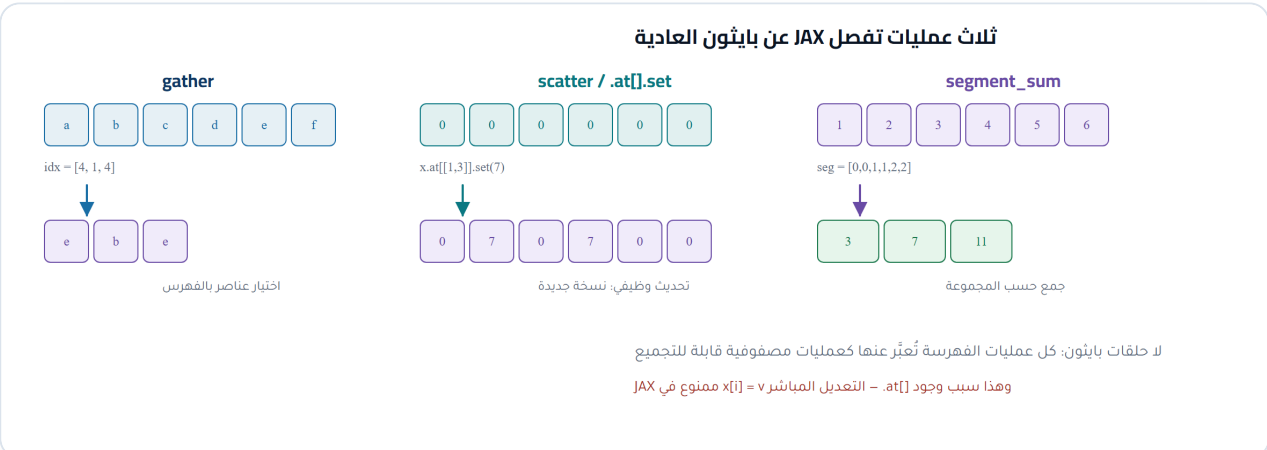
x = jnp.array([-1.0, 2.0, -3.0, 4.0])
mask = x > 0

print(x[mask])                    # [2. 4.] خارج jit: شكل ديناميكي
print(jnp.where(mask, x, 0.0))    # [0. 2. 0. 4.] شكل ثابت
print(jnp.sum(jnp.where(mask, x, 0.0)))   # 6.0 مجموع الموجب فقط

f = jax.jit(lambda v: jnp.sum(jnp.where(v > 0, v, 0.0)))
print(f(x))                       # يعمل: الشكل معروف
```

والقيد نفسه ينطبق على الشرائح: `x[i:j]` بمتغيّرين متتبَّعين يرفع خطأ، لأن الحدّ ليس رقماً معروفاً وقت التجميع. الحلّ `lax.dynamic_slice` مع حجم ثابت.

إذا كان `x[mask]` يعمل خارج التجميع ويفشل داخله، فهل المشكلة في الفهرسة أم في وعد الشكل الذي قطعه المترجم على نفسه؟



الشكل 4.2 — ثلاثة أنماط للفهرسة: شريحة ثابتة، وقناع منطقي، وفهرس متقدّم. الأولان يختلفان في الشكل الناتج، والثالث يحتاج انتباهًا خاصًا.

أخطاء الأشكال الشائعة

أربعة أخطاء تغطّي معظم ما ستراه. اقرأ الرسالة إلى آخرها: JAX يذكر الشكلين معاً في الغالب، وهذا نصف التشخيص.

الرسالة	السبب	العلاج
`Incompatible shapes for broadcasting`	بُعدان غير متساويين ولا أحدهما واحد	أضف محوراً أو بدّل المحاور
`dot_general requires contracting dimensions to have the same shape`	ضرب مصفوفتين غير متوافقتين داخلياً	طابق البُعد الداخلي
`NonConcreteBooleanIndexError`	قناع منطقي داخل `jit`	استبدله بـ `jnp.where`
`ConcretizationTypeError`	حدّ شريحة متتبَّع لا رقم ثابت	استخدم `lax.dynamic_slice`

جدول 4.1 — أربع رسائل خطأ وأسبابها

لاحظ أن الخطأين الأخيرين ليسا خطأي شكل بالمعنى الحسابي، بل خطأ وعد: طلبت من المترجم شيئاً لا يمكن أن يعرفه قبل التشغيل.

متى تعيد التشكيل ومتى تعيد التصميم؟

إعادة التشكيل مجّانية تقريباً، ولهذا تُستعمل كثيراً. لكن كثرة الاستعمال إشارة: إن كنت تعيد التشكيل في كل دالّة فأنت تُصلح عقداً لم تُكتب جيداً. الاختيار الحقيقي هو ترتيب المحاور: `(B, T, D)` أم `(B, D, T)`. اختر ترتيباً واحداً، واكتبه في تعليق أعلى الوحدة، واجعل كل دالّة تُصرّح بما تأخذ وما تُعطي.

> ✦ **التبديل رخيص، والالتباس غالٍ**
>
> ثمن `transpose` زهيد لأن XLA يدمجه في العملية التالية؛ وثمن ترتيب محاور غير مكتوب أسبوع من التصحيح.

تمرين الفصل الرابع

1. لديك `logits` بشكل `(32, 10)` و `labels` بشكل `(32,)`. اكتب الطرح الصحيح الذي يُنتج `(32,)` لا `(32, 32)`.
2. احسب بمصفوفة واحدة المسافات بين كل نقطتين في مصفوفة `(N, D)` باستخدام البثّ.
3. أعد كتابة `x[x > 0]` بصيغة تعمل داخل `jit`.

الحلّ النموذجي

1. الخطأ ناتج عن محاذاة `(32,)` من اليمين، فتُقارن 10 بـ32. الحلّ إضافة محور صريح:

```python
import jax.numpy as jnp

logits = jnp.ones((32, 10))
labels = jnp.arange(32)

print(logits[jnp.arange(32), labels].shape)          # (32,)
print((logits - jnp.expand_dims(labels, 1)).shape)   # (32, 10)
```

2. الفرق بين كل زوج يُبنى بالبثّ بعد إضافة محور:

```python
X = jnp.ones((5, 3))
diff = jnp.expand_dims(X, 1) - jnp.expand_dims(X, 0)   # (5, 5, 3)
dist = jnp.sqrt(jnp.sum(diff ** 2, axis=-1))           # (5, 5)
print(dist.shape)
```

3. `where` يحفظ الشكل، و `sum` يُعيد العدد نفسه:

```python
import jax

f = jax.jit(lambda v: jnp.sum(jnp.where(v > 0, v, 0.0)))
print(f(jnp.array([-1.0, 2.0, -3.0, 4.0])))   # 6.0
```

خلاصة الفصل

- اقرأ الأشكال بصوت مسموع: فالشكل عقد بين الدوالّ لا تفصيلاً داخلياً.
- البثّ من اليمين: تساوٍ أو واحد، وإلّا خطأ – والخطر في الحالة التي تمرّ بالصدفة.
- `reshape` يعيد التجميع، و `transpose` يبدّل المحاور.
- `einsum` تكتب بها الضربات كلها، و `where` يحفظ الشكل.

مصطلحات الفصل

broadcasting (بثّ)

تمدّد الأبعاد ذات القيمة واحد تلقائياً لتتوافق الأشكال.

reshape (إعادة تشكيل)

تغيير توزيع العناصر على المحاور مع الحفاظ على ترتيبها الخطّي.

einsum

صياغة الضربات والجموع بمعادلة حروف تُسمّي المحاور.

segment reduction

جمع أو تعظيم عناصر مجموعات مرقّمة، بشكل ثابت وبلا حلقات.

dynamic shape (شكل ديناميكي)

شكل يعتمد على القيم وقت التشغيل، فلا يُقبل داخل `jit` .

الأجهزة والذاكرة: jax.Array

أين تعيش البيانات، وكيف تنتقل، ومتى تُنسخ، ولماذا يهمّ ذلك

يشتري الفريق بطاقةً بذاكرة 80 غيغابايت، ثم يرى رسالة نفاد ذاكرة عند دفعة صغيرة. يقلّل حجم الدفعة، فتنجح مرةً وتفشل أخرى. يغيّر الإصدار، فلا يتغيّر شيء. المشكلة في الحقيقة ليست في الحجم: هي في مكان البيانات ووقت وصولها. هذا الفصل عن الموتر بوصفه شيئاً له عنوان، لا قيمة فقط: أين يعيش، وكيف ينتقل، ومن يدفع الثمن.

jax.Array: ما الذي تحمله فعلاً

الموتر في JAX ليس مصفوفة أرقام في ذاكرة بايثون. هو وصف لكتلة على جهاز، مع عقد يحدّد كيف تُقرأ: النوع، والشكل، والتقطيع، والأجهزة التي تحملها. والقيمة قد لا تكون وصلت بعد.

لهذا تحمل `jax.Array` خصائص لا تجدها في `numpy.ndarray`: `sharding` يصف كيف توزّع الكتلة، و `devices()` يعدّد الأجهزة التي تحملها، و `aval` يعطي الشكل والنوع اللذين يعرفهما المترجم قبل التشغيل.

```python
import jax, jax.numpy as jnp

x = jnp.ones((4, 8), dtype=jnp.bfloat16)
print(x.shape, x.dtype, x.nbytes)   # (4, 8) bfloat16 64
print(x.device)                     # cuda:0
print(x.sharding)                   # SingleDeviceSharding
print(x.aval)                       # ShapedArray(bfloat16[4,8], weak_type=False)
print(jax.default_backend(), jax.devices())
```

★ **الشكل والنوع عقد، والذاكرة موقع**

`aval` هو ما يعرفه المترجم بلا نظر إلى القيم، و `sharding` ما يعرفه عن توزيع الكتلة. كل ما يخرج عن هذين الوصفين يُجبر على إعادة تجميع أو نسخ.

كيف يختار JAX الجهاز؟

عند الاستيراد يعدّد JAX الأجهزة المتاحة ويرتّبها بأولوية: TPU ثم GPU ثم CPU. `jax.default_backend()` يخبرك بما اختاره، و `jax.devices()` يخبرك بما رآه. وإن أردت التحكّم، فالمتغيّر `JAX_PLATFORMS` يُقرأ قبل التهيئة.

أوامر الطرفية

```
$ JAX_PLATFORMS=cpu python train.py        # للتجريب على المعالج
$ JAX_PLATFORMS=cuda,cpu python train.py   # جرّب CUDA، وارجع إلى CPU عند الفشل
```

وللنقل الصريح دالّتان: `jax.device_put(x, device)` تنقل إلى جهاز محدّد، و `jax.device_get(x)` تُعيد نسخة إلى المضيف. وكلتاهما لا تشتكي حين يكون الموتر على الجهاز المطلوب أصلاً – وهذا ما يجعل التكلفة غير مرئية في الاختبارات.

النقل بين الأجهزة: الثمن الخفي

ثمن النقل ليس في عدد البايتات فقط، بل في الانتظار: الناقل أبطأ من ذاكرة البطاقة بمرتبة كاملة، والنقل يوقف خطّ الأنابيب.

$$t_{\text{copy}} \approx \text{bytes} \,/\, \text{bandwidth}_{\text{bus}}$$

نسخ 64 ميغابايت على ناقل بطيء قد يكلّف أكثر من الحساب نفسه.

القاعدة: أنشئ البيانات على الجهاز الذي سيحسب عليها. `jnp.ones` و `jax.random.normal` تُنتج على الجهاز مباشرة، بينما `np.ones` تُنتج على المضيف ثم تُنسخ.

Python

```python
import numpy as np
import jax, jax.numpy as jnp

host = np.ones((4096, 4096), dtype=np.float32)   # 64 MiB على المضيف
x = jax.device_put(host, jax.devices()[0])       # نسخة واحدة صريحة

y = jnp.ones((4096, 4096))                       # تُنشأ على الجهاز
print(x.devices(), y.devices())
print(jax.device_get(y).shape)                   # عودة إلى المضيف
```

⚠ النسخ الخفيّ في حلقة التدريب

أخطر ما في النقل أنه يحدث ضمنياً: تحويل مصفوفة NumPy إلى `jnp` داخل الخطوة، أو `float(loss)` لطبعه. والأخير يوقف الحلقة أيضاً لأنه يجبر على الانتظار.

التنفيذ غير المتزامن

حين تستدعي دالّة مجمّعة، لا تنتظر حتى ينتهي الحساب؛ تُرسل العمل وتعود. القيمة تُملأ لاحقاً. ولهذا فإن قياس الزمن بلا انتظار يقيس سرعة بايثون في الإرسال، لا سرعة الجهاز في الحساب.

Python

```
import time
import jax, jax.numpy as jnp

x = jnp.ones((4096, 4096))
f = jax.jit(lambda a: a @ a)
f(x).block_until_ready()          # تسخين قبل أي قياس

t0 = time.perf_counter()
f(x)                              # يعود فوراً
print("بلا انتظار:", round(time.perf_counter() - t0, 4))

t0 = time.perf_counter()
f(x).block_until_ready()          # هنا يحدث الحساب فعلاً
print("مع انتظار :", round(time.perf_counter() - t0, 4))
```

‹‹ المخرجات ››

```
بلا انتظار: 0.0002
مع انتظار : 0.1180
```

✕ خطأ شائع: تحسين ما لا تقيسه

ترى رقماً صغيراً فتظنّ العملية سريعة، فتبني عليها قراراً معمارياً. أضف `block_until_ready` إلى كل قياس، وإلى آخر عملية في الحلقة كي لا تتراكم الطلبات في الطابور.

الذاكرة: الحجز والتبرّع

XLA يحجز ذاكرة البطاقة مسبقاً لأن التخصيص أثناء التشغيل يكلّف ويُجزّئ الذاكرة: ممتاز في جهاز مخصّص، وسيّئ في مشترك. والمتغيّران `XLA_PYTHON_CLIENT_PREALLOCATE` و

`XLA_PYTHON_CLIENT_MEM_FRACTION` يضبطان ذلك قبل التهيئة.

المفتاح	الأثر	متى تستخدمه
`XLA_PYTHON_CLIENT_PREALLOCATE=false`	لا يحجز كل الذاكرة مسبقاً	بطاقة مشتركة بين عمليات
`XLA_PYTHON_CLIENT_MEM_FRACTION=.7`	يحدّ الحجز بنسبة	تريد ترك هامش لعمليات أخرى
`XLA_PYTHON_CLIENT_ALLOCATOR=platform`	مخصّص النظام بدل حجز XLA	تشخيص أو ذاكرة موحّدة
`donate_argnums`	يتبرّع بمخزن الوسيط بعد الاستخدام	حلقة تدريب لا تعيد استخدام المدخل

جدول 5.1 — مفاتيح التحكّم في ذاكرة الجهاز

التبرّع هو الأكثر أثراً والأقلّ فهماً. حين تُعلن `donate_argnums=0` تعدك الدالّة ألّا تحتاج المخزن الأصلي بعد الاستدعاء، فيعيد XLA استخدامه للخرج. والشرط واحد: لا تُبقِ مرجعاً إلى القيمة القديمة.

```python
import jax, jax.numpy as jnp

@jax.jit(donate_argnums=0)
def step(w, g):
    return w - 0.01 * g          # الخرج يأخذ مخزن w

w = jnp.ones(1_000_000)
w = step(w, jnp.ones(1_000_000)) # إعادة الإسناد آمنة
print(w.shape)
```

✓ متى يكون التبرّع آمناً؟

حين يكون الوسيط مملوكاً لهذه الحلقة وحدها: وزن مُحدَّث في مكانه، أو مخزن مؤقّت لا يقرأه أحد. أمّا الأوزان التي تحتفظ بنسخة منها لنقطة حفظ، فالتبرّع بها يمحوها بلا إنذار.

أخطاء نفاد الذاكرة: تشخيص لا تخمين

رسالة نفاد الذاكرة لا تقول لك السبب، بل تقول إن المحصّلة تجاوزت السقف. وهناك خمسة أسباب تغطّي معظم الحالات، ولكلٍّ منها مقياس يخصّه.

1. **الدفعة أكبر من اللازم.** القياس: `x.nbytes` لكل مدخل، مضروباً في عدد الموترات الحيّة.

2. **التنشيطات محفوظة للاشتقاق العكسي.** القياس: راقب الاستهلاك أثناء الخطوة نفسها.

3. **نسخ خفيّ كل خطوة.** القياس: راقب `memory_stats()` قبل الخطوة وبعدها؛ إن نما المستخدم بلا توقّف فأنت تُنشئ موترات لا تموت.

4. **إعادة تجميع متكرّرة.** القياس: اطبع عدّاد التجميع؛ كل شكل جديد يضيف برنامجاً وذاكرة.

5. **تجزئة الذاكرة.** القياس: `jax.devices()[0].memory_stats()` يفرّق بين المحجوز والمستخدم.

السبب الثالث هو الأكثر إخفاءً: حلقة تكتب `losses.append(float(loss))` تحتفظ بكل قيمة على المضيف، وتوقف الجهاز كل خطوة. والسبب الرابع يظهر عند تتبّع أشكال متغيّرة: كل شكل يُنتج برنامجاً جديداً لا يُحرَّر تلقائياً.

إذا كان `memory_stats()` يقول إن المحجوز أكبر بكثير من المستخدم، فهل المشكلة في نموذجك أم في طريقة تخصيص الذاكرة؟ وكيف تفرّق بينهما قبل أن تُصغّر الدفعة؟ ؟

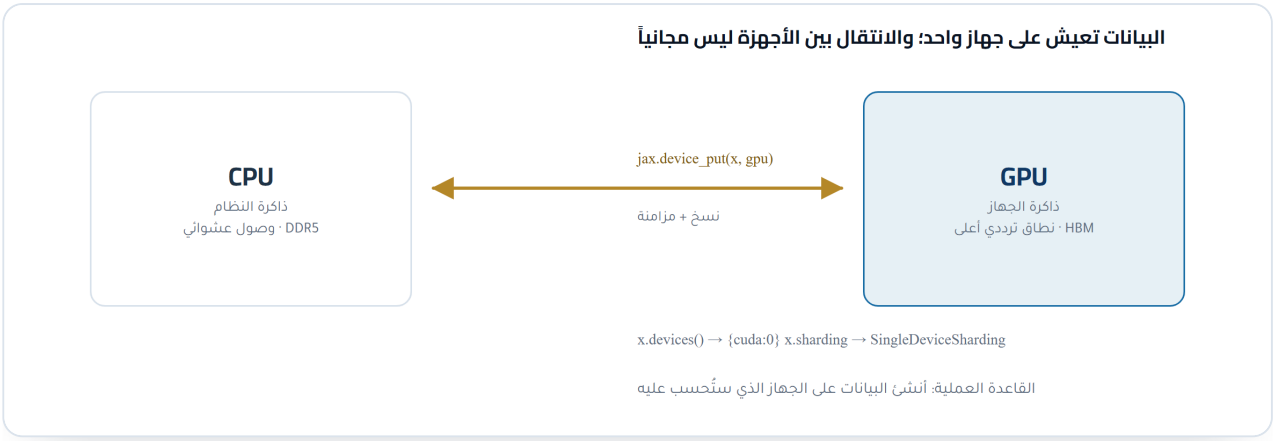
قواعد عملية للأجهزة والذاكرة

تبقى قواعد قليلة تُغنيك عن معظم التشخيص: أنشئ البيانات على الجهاز الهدف، لا تنقل داخل الحلقة، انتظر قبل أن تقيس، وتبرّع بما لا تعود إليه.

✦ الذاكرة تُدار بالتصميم لا بالحيل

تصغير الدفعة تعديل في الأعراض. التعديل الحقيقي تقليل ما تحتفظ به بين الخطوات: حالة النموذج، وحالة المحسّن، والدفعة الحالية.

لو أردت تدريب نموذجين على بطاقة واحدة بذاكرة محدودة، فما الذي تضبطه أولاً: حجم الدفعة، أم التبرّع، أم نسبة الحجز المسبق؟ ولماذا بهذا الترتيب؟ ؟



الشكل 5.1 — مسار البيانات من المضيف إلى الجهاز وما يعود. كل سهم ثمن، وكل ثمن يظهر في القياس إن انتظرت.

تمرين الفصل الخامس

1. أنشئ موتراً بشكل `(1024, 1024)` من نوع `float32`، وتحقّق من `nbytes`. هل يطابق حسابك اليدوي؟
2. قِس زمن ضربة مصفوفة كبيرة مرّتين: مرّة بلا انتظار، ومرّة مع `block_until_ready`. اشرح الفرق.
3. أضف `donate_argnums=0` إلى دالّة تحديث وزن، وقِس الفرق في الذاكرة المستخدمة.

▲ الحلّ النموذجي

1. الحساب: 1024 × 1024 × 4 بايت = 4,194,304 بايت، أي 4 ميغابايت.

```python
import jax, jax.numpy as jnp

x = jnp.ones((1024, 1024), dtype=jnp.float32)
print(x.nbytes)                       # 4194304
print(x.nbytes // (1024 * 1024), "MiB")
```

2. القياس بلا انتظار يقيس الإرسال فقط، لأن الحساب يحدث بعد عودة الدالّة:

```python
import time

f = jax.jit(lambda a: a @ a)
f(x).block_until_ready()              # تسخين

t0 = time.perf_counter(); f(x)
print("بلا انتظار:", round(time.perf_counter() - t0, 5))

t0 = time.perf_counter(); f(x).block_until_ready()
print("مع انتظار :", round(time.perf_counter() - t0, 5))
```

3. التبرّع يقلّل الذاكرة لأن الخرج يستعمل مخزن الوسيط نفسه:

```python
@jax.jit(donate_argnums=0)
def step(w, g):
    return w - 0.01 * g

w = jnp.ones(1_000_000)
g = jnp.ones(1_000_000)
w = step(w, g)                        # لا حجز إضافي بحجم w
print(w.shape)
```

خلاصة الفصل

- `jax.Array` وصف لكتلة على جهاز: نوع وشكل وتقطيع وموقع.
- اختيار الجهاز تلقائي، و `JAX_PLATFORMS` يقيّده قبل التهيئة.
- كل نقل بين المضيف والجهاز ثمن؛ أنشئ البيانات حيث ستحسب.
- لا معنى لأيّ قياس بلا `block_until_ready` .
- الحجز المسبق والتبرّع مفتاحان للذاكرة، ولكلٍّ شرط أمان.

مصطلحات الفصل

aval

القيمة المجرّدة التي يعرفها المترجم: شكل ونوع بلا بيانات.

sharding

وصف توزيع الموتر على الأجهزة.

device_put

دالّة تنقل موتراً إلى جهاز محدّد، وتنسخ إن لزم.

preallocation (حجز مسبق)

حجز XLA لنسبة من ذاكرة الجهاز قبل الحساب لتجنّب التخصيص المتكرّر.

buffer donation (تبرّع بالمخزن)

تعهّد بأن الدالّة لا تحتاج مدخلها بعد الاستدعاء، فيُعاد استخدام مخزنه.

الجزء الثاني

التبديلات الأربع

jit وgrad وvmap وshard_map: قلب JAX ونموذجها الذهني

jit: التجميع الذي يغيّر كل شيء

التتبّع، والرسم البياني، وإعادة التجميع، وحدود ما يمكن تجميعه

كتب أحدهم دالّة من عشرة أسطر، ووضع فوقها `@jax.jit` ، فرأى الزمن يهبط من أربعين ميلي ثانية إلى ثلاثة. ثم غيّر حجم الدفعة من 32 إلى 64، فعاد الزمن إلى أربعين. لم يتغيّر سطر واحد، لكن شيئاً ما أُعيد بناؤه من الصفر. هذا الفصل يفتح الصندوق: أربع مراحل بين سطرك والجهاز – بصمة تُحسب من الأشكال والأنواع.

ماذا يحدث حين تكتب @jax.jit؟

`jit` ليست «مفسّراً أسرع»، بل **حدّ**: ما داخله يُترجم قبل أن يعمل، وما خارجه يبقى بايثون عادياً.

عند الاستدعاء الأوّل تُستدعى دالّتك بمدخلات تحمل شكلاً ونوعاً فقط، فتُسجَّل عملياتها بدل أن تُحسب (**التتبّع**). ثم يُحوَّل السجلّ إلى `jaxpr` ، ويُنزل إلى `HLO` حيث تُدمج العمليات وتُخطَّط الذاكرة، ثم يُنتج **الملفّ التنفيذي** ويُخزَّن بالبصمة.

```python
import time, jax, jax.numpy as jnp

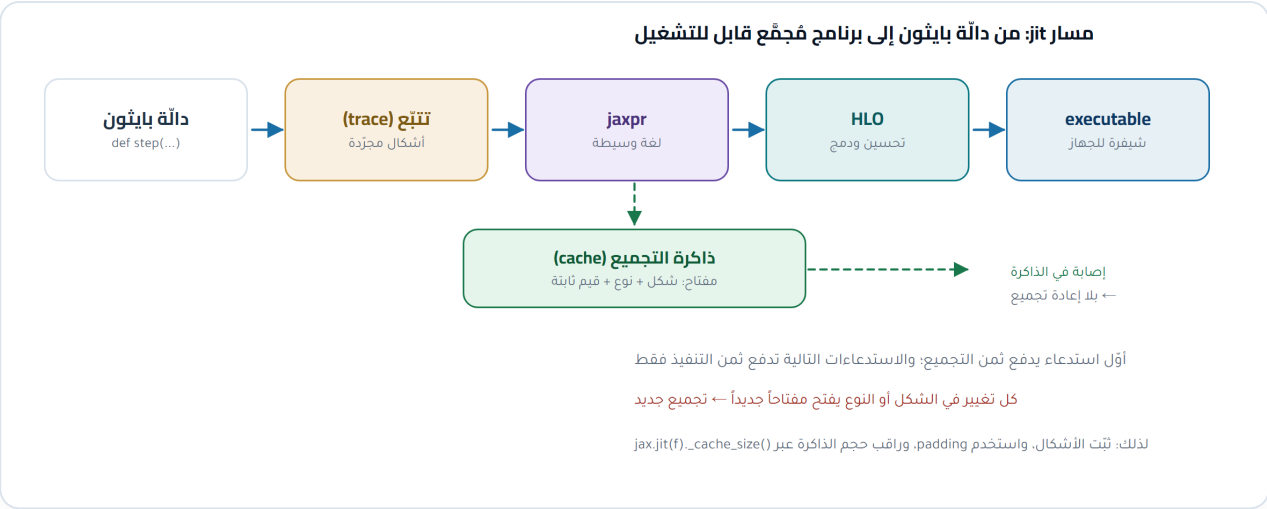
def block(x, w):
    for _ in range(20):
        x = jnp.tanh(x @ w)          # حلقة بايثون تُفكّ عند التتبّع
    return jnp.sum(x)

fast = jax.jit(block)
x = w = jnp.ones((256, 256)) * 0.01
t0 = time.perf_counter(); fast(x, w).block_until_ready()
t1 = time.perf_counter(); fast(x, w).block_until_ready()
print(f"first {(t1-t0)*1e3:.0f} ms, steady {(time.perf_counter()-t1)*1e3:.1f} ms")
```

‹‹ المخرجات ››

```
first 413 ms, steady 1.9 ms
```

في الاستدعاء الثاني نُفِّذ ملفّ واحد مُدمَج، ولم تُنفَّذ الحلقة في بايثون إطلاقاً.



الشكل 6.1 — تتبّع، ثم `jaxpr`، ثم `HLO`، ثم ملفّ تنفيذي مخزَّن.

> ★ **القاعدة التي تفسّر كلّ شيء لاحقاً**
>
> التجميع يحدث مرّة لكل **بصمة**، والبصمة من شكل كلّ وسيط ونوعه وترتيبه ومن قيم الوسائط الثابتة، لا من القيم. كل عطل في هذا الفصل نتيجة مباشرة لهذه الجملة.

التتبّع: كيف تُنفَّذ دالّتك على قيم وهمية

تخيّل مهندساً يرسم خطوات بناء جسر ولن يضع حجراً. المُتتبِّع يمرّر كائنات تحمل `shape` و `dtype` فقط، فتحدث **كل الآثار الجانبية مرّة واحدة**: اطبع داخل دالّة مجمّعة وسيظهر السطر مرّة.

```python
import jax, jax.numpy as jnp

@jax.jit
def f(x):
    print("tracing with", x.shape)  # مرّة واحدة فقط
    return jnp.sum(x * 2.0)

f(jnp.ones(3)); f(jnp.ones(3)); f(jnp.ones(3))

@jax.jit
def g(x):
    jax.debug.print("runtime: {v}", v=jnp.sum(x))   # وقت التنفيذ فعلاً
    return x

g(jnp.arange(4.0)); g(jnp.arange(4.0))
```

```
‹‹ المخرجات ››
tracing with (3,)
runtime: 6.0
runtime: 6.0
```

> ⚠ **الأثر الجانبي قد يكون خطيراً لا مزعجاً**
>
> كتابة ملفّ داخل دالّة مجمّعة تكتبه مرّة عند التتبّع ثم لا تكتبه أبداً، ولا تُصلح ذلك بعدّاد داخلي: قيمته ستُجمَّد كثابت.

> دالّة تطبع سجلّاً لكل عيّنة وتريد أن يبقى السلوك بعد التجميع. هل تحذف الطباعة، أم أن الفرق بين «البناء» و«التشغيل» يستحقّ إعادة التصميم؟ ؟

ما الذي يعيد التجميع؟

المفتاح في الذاكرة المؤقّتة ليس قيمة بل توقيع: لو كان القيمة لكان التجميع بلا معنى، إذ تُنتج كل دفعة مصفوفة جديدة.

التغيير	يعيد التجميع؟	السبب
شكل المصفوفة	نعم	الشكل جزء من البصمة
نوع العنصر	نعم	لكل نوع مسار تعليمات
قيمة وسيط ثابت	نعم	تُثبَّت في الرسم البياني
قيم المصفوفات	لا	لا تدخل البصمة إطلاقاً
متغيّر خارجي	لا – وهذا خطر	يُجمَّد عند التتبّع الأوّل

جدول 6.1 — ما يغيّر البصمة وما لا يغيّرها

السطر الأخير أخطرها لأنه لا يُنتج خطأً: اقرأ ثابتاً من الخارج ثم غيّره، فتبقى النتيجة كما كانت، لأن الرقم القديم صار جزءاً من الرسم.

✕ «غيّرت الإعداد ولم يتغيّر شيء»

يستهلك ساعات لأن الشيفرة تبدو صحيحة: أي رقم يؤثّر في الحساب ولا يأتي من مصفوفة يجب أن يكون وسيطاً صريحاً.

static_argnamesg static_argnums

بعض الوسائط ليست بيانات بل **قرارات بناء**: عدد الطبقات، اسم التنشيط، وضع التدريب. أعلنها ثابتة لتدخل قيمتها في البصمة عن قصد، بدل أن تُجمَّد بصمت.

Python

```
import jax, jax.numpy as jnp

def mlp(x, w, depth, act):
    for i in range(depth):          # depth ثابت ⇒ الحلقة تُفكّ
        x = act(x @ w[i])
    return x

fast = jax.jit(mlp, static_argnums=(2, 3))
ws = [jnp.ones((4, 4)) * 0.5 for _ in range(3)]

fast(jnp.ones((2, 4)), ws, 3, jnp.tanh)
print(fast._cache_size())           # 1
fast(jnp.ones((2, 4)), ws, 2, jnp.tanh)
print(fast._cache_size())           # 2
```

الثمن شرط صارم: الوسيط الثابت يجب أن يكون **قابلاً للتجزئة** لأنه يصير مفتاحاً؛ فالأعداد والنصوص والدوالّ صالحة، والقوائم لا.

✓ ثابتان لا أكثر

كل قيمة ثابتة جديدة تعني ملفّاً تنفيذياً جديداً؛ و `seq_len` بقيم 7 و13 و29 و64 يبني أربعة برامج حيث يكفي واحد.

الأشكال الثابتة: bucketingg padding

دفعة أطوالها 12 و37 و8 و91 لها شكل $(4, L)$ يتغيّر مع كل دفعة، لأن L هو طول أطول جملة؛ فتبني كل خطوة برنامجاً جديداً.

$$\text{cost} \approx N_{\text{shapes}} \times t_{\text{compile}} + N_{\text{steps}} \times t_{\text{run}}$$

حين يقترب عدد الأشكال من عدد الخطوات، يصبح الحدّ الأوّل هو المهيمن.

الحلّ الأوّل: **الحشو إلى طول ثابت** مع قناع في الخسارة: برنامج واحد، وثمنه حسابٌ على رموز وهمية.

الثاني: **الحزم** – أطوال قليلة (128، 256، 512) تُرفع إليها الدفعات.

Python

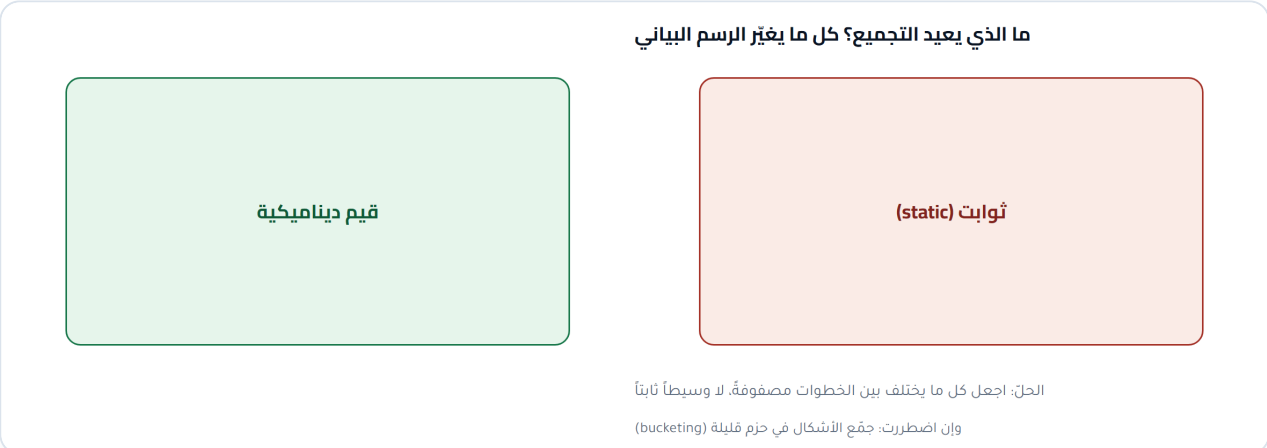
```python
import jax, jax.numpy as jnp

BUCKETS = (16, 32, 64, 128)

def pad_batch(t, buckets=BUCKETS):
    L = next(b for b in buckets if t.shape[1] <= b)
    return jnp.pad(t, ((0, 0), (0, L - t.shape[1])))

@jax.jit
def total(t):
    return jnp.sum(t.astype(jnp.float32))

for raw in (jnp.ones((2, 11)), jnp.ones((2, 30)), jnp.ones((2, 60))):
    print(raw.shape, "->", pad_batch(raw).shape)
```



الشكل 6.2 — يساراً: طول متغيّر ⇒ برنامج جديد كل مرّة. يميناً: أربع حزم ⇒ أربعة برامج فقط.

الأخطاء الشائعة داخل jit

كل خطأ هنا ينبع من مصدر واحد: أنت تكتب بايثون والمُتتبِّع يقرأ حساباً. حين تطلب ما لا يُمكن تمثيله يرفض.

تحويل إلى bool. `if x.sum() > 0:` يسأل عن قيمة لم تُحسب بعد، فتحصل على `TracerBoolConversionError`. العلاج `jax.lax.cond`.

قائمة بدل صفّ. `[1.0, 2.0]` يُنتج `TypeError: not a valid JAX type`؛ العلاج `jnp.array([...])`.

تعديل في المكان. `x[0] = 5.0` يُنتج `TypeError: arrays are immutable`؛ العلاج `x.at[0].set(5.0)`.

حلقة على قيمة. `while x.sum() > 1:` لا ينتهي عند التتبّع؛ العلاج `jax.lax.while_loop`. أمّا `for i in range(10)` فتُفكّ.

استدعاء NumPy. `np.asarray(x)` يُنتج `TracerArrayConversionError`؛ NumPy يريد بيانات والمُتتبِّع يملك شكلاً. استبدله بـ `jax.numpy`.

AOTg donate_argnums

في التدريب تصبح الأوزان القديمة قمامة بمجرّد حساب الجديدة، لكن JAX تحتفظ بالنسختين.
`donate_argnums` تصرّح: «خذه وأعد استخدام ذاكرته»؛ والمكسب ذاكرة.

```python
import jax, jax.numpy as jnp

@jax.jit(donate_argnums=(0,))
def step(w, x):
    return w - 0.01 * (w @ x)

w = x = jnp.ones((512, 512)) * 0.001
w2 = step(w, x)          # w استُهلكت: لا تقرأها بعد الآن

# AOT: ترجمة قبل أن تصل البيانات
compiled = jax.jit(lambda a, b: a - 0.01 * (a @ b)).lower(w2, x).compile()
print(compiled(w2, x).shape)
```

والترجمة المسبقة تفصل التجميع عن التشغيل: `lower` ينتج تمثيلاً منزلاً و`compile` ملفّاً قابلاً للاستدعاء. في خدمة استدلال، ثانية تجميع أمام أوّل طلب أسوأ تجربة ممكنة، والـ `AOT` يلغيها.

كيف تقيس أثر التجميع؟

أكثر قياس خاطئ شيوعاً توقيت الاستدعاء الأوّل ونسبته إلى «سرعة الدالّة». القياس الصحيح يفصل الحالتين، ويستدعي `block_until_ready` قبل إيقاف الساعة.

> ★ **ثلاثة أرقام تحكي القصّة**
>
> زمن البناء، وزمن الحالة المستقرة، وعدد البصمات: الأول جودة الإقلاع، والثاني جودة الحلقة.

دالّتك تعمل في 2 ملي ثانية وتبني في 900، وتُشغَّل مرّة لكل طلب. هل التجميع مكسب أم خسارة هنا؟ ؟

تمرين الفصل السادس

1. اكتب `f(x, k)` تُرجع `jnp.sum(x ** k)`، وجمّعها مع `k` ثابتاً. استدعها بالقيم 1 و2 و3 وعدّ البرامج المخزَّنة.
2. شغّل دالّة فيها `if x.sum() > 0` مع التجميع، وسجّل الخطأ، ثم أصلحها بـ `jax.lax.cond`.
3. قِس زمن البناء والحالة المستقرة على شكلين: `(32, 128)` و `(64, 128)`. هل يتضاعف البناء مع الدفعة؟ اشرح.

الحلّ النموذجي

1. ثلاث قيم ثابتة تعني ثلاث بصمات، أي ثلاثة برامج:

```python
import jax, jax.numpy as jnp

@jax.jit(static_argnums=(1,))
def f(x, k):
    return jnp.sum(x ** k)

x = jnp.arange(1.0, 5.0)
for k in (1, 2, 3):
    print(k, f(x, k))
print("cached:", f._cache_size())   # 3
```

2. الخطأ يذكر `Attempted boolean conversion` ويشير إلى سطر الشرط. الإصلاح يجعل التفريع جزءاً من الرسم:

```python
@jax.jit
def g(x):
    return jax.lax.cond(jnp.sum(x) > 0,
                        lambda v: v * 2.0,
                        lambda v: v * 0.0,
                        x)

print(g(jnp.array([1.0, -0.5])))   # [ 2. -1.]
```

3. لا يتضاعف: التتبّع والـ `jaxpr` يعتمدان على عدد العمليات، والتجميع على الحجم.

خلاصة الفصل

- أربع مراحل: تتبّع، ثم `jaxpr`، ثم `HLO`، ثم ملفّ تنفيذي.
- الأثر الجانبي يحدث عند التتبّع مرّة: الأرقام تُثبَّت في الرسم.
- البصمة من الأشكال والأنواع والثوابت، لا من القيم؛ وكل شكل جديد برنامج جديد.
- قِس البناء والحالة المستقرة منفصلين.

مصطلحات الفصل

trace (تتبّع)

تنفيذ الدالّة على قيم مجرّدة لتسجيل عملياتها.

jaxpr

تمثيل وسيط: متغيّرات وبدائيات بلا بايثون.

HLO

تمثيل XLA الذي يُنزل إليه الـ `jaxpr`.

cache key (بصمة)

مفتاح من الأشكال والأنواع والثوابت.

bucketing (حزم)

رفع الدفعة إلى أقرب طول محدود.

AOT

ترجمة مسبقة عبر `.lower().compile()`.

grad: التفاضل التلقائي بلا سحر

العكسي والأمامي، value_and_grad، والقيود التي يجب أن تعرفها

> سأل طالبٌ: «أين كود الاشتقاق؟». بحث في JAX عن مشتقّ تحليلي لكل دالّة فلم يجد شيئاً: لا جدول مشتقّات، ولا قاعدة سلسلة مكتوبة. والحقيقة أن `grad` لا تعرف كيف تشتقّ x^3 أكثر مما تعرف كيف تشتقّ `tanh`: هي لا تشتقّ شيئاً على الإطلاق، بل تعيد ترتيب الحساب الذي كتبته أنت.

الاشتقاق كتحويل، لا كحساب

خُذ $f(x) = x^3 + 2x$. باليد: $f'(x) = 3x^2 + 2$ ، وعند $x = 2$ تساوي 14 . الطريق اليدوي يحتاج قاعدة لكل دالّة وجهداً يتضاعف مع التعشيش. الطريق الآخر: أي حساب مؤلّف من خطوات، ولكل خطوة مشتقّتها المحلّية، والنتيجة حاصل ضربها.

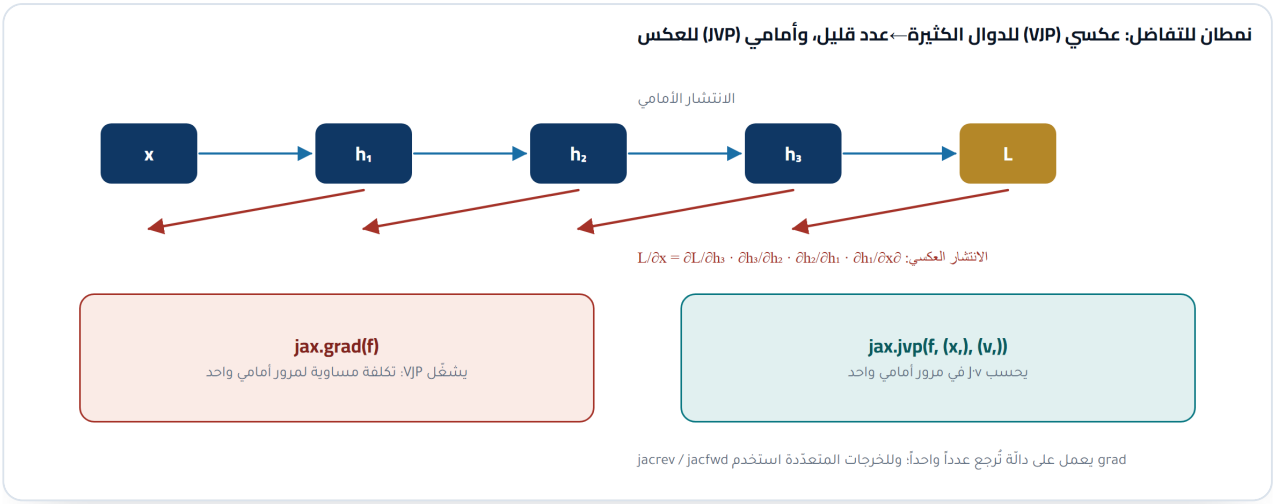
$$\partial f/\partial x = \prod_i \partial y_{i+1}/\partial y_i$$

قاعدة السلسلة ليست صيغة تُحفظ، بل مسار يُقطَع من الخرج إلى الدخل.

```python
import jax, jax.numpy as jnp

f = lambda x: x ** 3 + 2 * x
print(jax.grad(f)(2.0))            # 14.0
print(jax.grad(jax.grad(f))(2.0))  # 12.0
```

لاحظ ما أعادته `grad`: دالّة، لا رقماً. هذا هو التحويل – تأخذ دالّة وتُعيد دالّة، فتُركَّب على نفسها بلا حدّ.



الشكل 7.1 — التدرّج يعود من الخرج إلى الدخل ويجمع المساهمات عند كل عقدة؛ الشفافية هي الرسم نفسه.

> ★ **آلي لا رمزي**
>
> الاشتقاق الرمزي يبني تعبيراً قد يتضخّم أضعافاً. الآلي يطبّق قواعد محلّية على أرقام أثناء المرور، فلا ينتج تعبيراً أكبر من الأصل.

الوضع العكسي: VJP

تخيّل دالّة تأخذ مليون وزن وتُخرج رقماً واحداً – وهي حالة الخسارة في كل شبكة. الأمامي يحتاج مروراً لكل وزن، والعكسي مروراً واحداً. هذه ليست تحسيناً هندسياً بل الفرق بين ممكن ومستحيل.

الواجهة هي `jax.vjp`: تُرجع الخرج ودالّة سحب تأخذ مشتقّة الخرج وتُرجع مشتقّات المدخلات.

```python
import jax, jax.numpy as jnp

def f(x):
    return jnp.array([x[0] * x[1], jnp.sin(x[0])])

x = jnp.array([2.0, 3.0])
y, pullback = jax.vjp(f, x)
print(y)                                  # [6.  0.909]
print(pullback(jnp.array([1.0, 0.0]))[0])  # [3. 2.]
```

التكلفة لا تعتمد على عدد المدخلات بل على عدد المخارج؛ لذلك يكون العكسي رخيصاً حين يُخرج الحساب رقماً واحداً.

دالّة تُخرج 10,000 قيمة من ثلاثة مدخلات. أيّ الوضعين تختار، ولماذا ينقلب الحساب رأساً على عقب مقارنةً بدالّة الخسارة؟ ؟

الوضع الأمامي: JVP

الأمامي يسير مع الحساب: يأخذ اتجاهاً في المدخلات ويمرّره إلى الأمام. تكلفته بعدد المدخلات، وهو الأفضل حين تكون قليلة والمخارج كثيرة – كاشتقاق مسار حركة بالنسبة لثلاثة معاملات.

```python
import jax, jax.numpy as jnp

def g(x):
    return jnp.array([x[0] ** 2, x[1] ** 2, x[0] * x[1]])

y, y_dot = jax.jvp(g, (jnp.array([2.0, 3.0]),), (jnp.array([1.0, 0.0]),))
print(y, y_dot)     # [4. 9. 6.] [4. 0. 3.]
```

`jvp` تُرجع الخرج ومشتقّته في الاتجاه، ولا تحتاج تخزين الحساب الوسيط، فهي أخفّ ذاكرةً من العكسي.

value_and_grad ولماذا هو الأسرع

تحتاج في التدريب إلى الخسارة وتدرّجها معاً. الطريقة الساذجة تدفع مرورين أماميين، لأن `grad` وحدها تعيد حساب الدالّة. و`value_and_grad` تُرجع الاثنين من مرور واحد.

```python
import jax, jax.numpy as jnp

def loss(w, x):
    return jnp.mean(jnp.tanh(x @ w) ** 2)

w, x = jnp.ones((4, 4)) * 0.1, jnp.ones((8, 4))
value, grads = jax.value_and_grad(loss)(w, x)
print(value, grads.shape)

# معلومات إضافية بلا اشتقاق
def loss_aux(w, x):
    h = jnp.tanh(x @ w)
    return jnp.mean(h ** 2), h

(v, h), grads = jax.value_and_grad(loss_aux, has_aux=True)(w, x)
print(v, h.shape)
```

في شبكة كبيرة يكون المرور الأمامي جزءاً كبيراً من الزمن، فحذفه يجعل الخطوة أسرع بنسبة ملموسة بلا أي تغيير في النتيجة العددية.

> ✓ **العادة الصحيحة من البداية**
>
> اكتب `value_and_grad` دائماً، حتى لو لم تحتج القيمة اليوم؛ ستسجّلها غداً.

التدرّج على الأشجار

الأوزان في نموذج حقيقي قاموس من طبقات لا مصفوفة واحدة. و `grad` تفهم ذلك: تمرّر قاموساً فتُرجع قاموساً بالبنية نفسها، بلا تسطيح ولا فهرسة يدوية.

```python
import jax, jax.numpy as jnp

def loss(params, x):
    h = jnp.tanh(x @ params["w1"] + params["b1"])
    return jnp.mean((h @ params["w2"]) ** 2)

params = {"w1": jnp.ones((4, 8)) * 0.1, "b1": jnp.zeros(8), "w2": jnp.ones((8, 1)) * 0.1}
g = jax.grad(loss)(params, jnp.ones((16, 4)))
print(g["b1"].shape)                    # (8,)
print(len(jax.tree.leaves(g)))          # 3

gx, gp = jax.grad(loss, argnums=(1, 0))(params, jnp.ones((16, 4)))
print(gx.shape, gp["w2"].shape)
```

التدرّج يحترم البنية: `jax.tree.map` يعمل عليه كما على الأوزان. و `argnums` يحدّد موضع الاشتقاق؛ وتمرير توپل يُعيد توپلاً بالتنسيق نفسه.

> ⚠ **ترتيب الوسائط ليس تفصيلاً**
>
> `argnums=1` تعني «الوسيط الثاني». خطأ في الرقم يعطيك تدرّجاً شكله صحيح وقيمه بلا معنى، ولن يشتكي أحد.

ما لا يستطيعه grad

الاشتقاق يحتاج بنية جبرية: أعداد حقيقية أو مركّبة تُجمع وتُضرب. ما خرج عن ذلك لا مشتقّة له، لا لأن JAX قصّرت، بل لأن السؤال بلا معنى

المدخلات الصحيحة والمنطقية. `grad` على وسيط `int32` ترفض بـ `TypeError: grad requires real- or complex-valued inputs`. مرّر `float` وحوّل داخل الدالّة إن لزم.

التحكّم المعتمد على القيم. `if x > 0` يُجمَّد عند التتبّع فلا يتفرّع التدرّج. استخدم `jax.lax.cond` أو دالّة ناعمة بدل الحدّ الحادّ.

الأعداد المركّبة. الدوالّ الهولومورفية مرفوضة: `grad of a holomorphic function is not supported`، لأن تعريف التدرّج يفترض خرجاً حقيقياً. استخدم `jax.vjp` مع مشتقّة مرافقة.

NaN في التدرّج. ليس رفضاً بل نتيجة: `sqrt(0)` و`log(0)` و`abs(0)` مشتقّاتها غير معرّفة عند الصفر، فتُلوّث ما بعدها.

hessiang jacfwdg jacrev

التدرّج حالة خاصة: خرج قياسي. أمّا اليعقوبية فتحتاج قراراً – تمرّ على المدخلات (`jacfwd`) أم المخارج (`jacrev`)؟ يحدّده أيّ البعدين أصغر.

التحويل	الخرج	المرورات	متى تستخدمه
`grad`	متّجه بطول n	1 عكسي	خرج قياسي
`jacrev`	مصفوفة m×n	m عكسي	m صغير ng كبير
`jacfwd`	مصفوفة m×n	n أمامي	n صغير mg كبير
`hessian`	مصفوفة n×n	n عكسي	n صغير

جدول 7.1 — تكلفة كل تحويل لدالّة $f: \mathbb{R}^n \to \mathbb{R}^m$

```python
import jax, jax.numpy as jnp

h = lambda v: jnp.array([v[0] ** 2, v[0] * v[1]])
v = jnp.array([2.0, 3.0])

print(jax.jacrev(h)(v))                      # [[3. 2.] [1. 1.]]
print(jax.jacfwd(h)(v))                      # النتيجة نفسها
print(jax.hessian(lambda z: jnp.sum(z ** 3))(v))
```

> ★ **اختر بالبعد الأصغر**
>
> هيسيان لخسارة فيها مليون وزن تُقاس بـ 10^{12} عنصر – مستحيلة. لهذا لا تُحسب صراحةً في التدريب، بل تُستبدل بضرب مصفوفة بمتّجه عبر `jax.vjp` على التدرّج.

أخطاء التدرّج الشائعة

أكثر تدرّج خاطئ تراه ليس خطأً يرفض التشغيل، بل صفراً أو `nan` يمرّ بصمت.

stop_gradient في الموضع الخطأ. وضعها حول الجزء الذي تريد تدريبه يقطع المسار ويُعيد صفراً. استخدمها حيث تمنع التدرّج فعلاً، كأهداف ثابتة في دوالّ خسارة متقدّمة.

ثابت منفصل. بناء ثوابت بـ `numpy` ثم استخدامها داخل الحساب يقطع المسار: لا مشتقّة لشيء لم يمرّ في الرسم. ابنِ الثوابت بـ `jnp` .

float0. الاشتقاق بالنسبة لوسيط صحيح يعطي نوعاً اسمه `float0` : صفر بلا قيمة. تراه عبر `jax.vjp` لا عبر `grad` ، وهو تذكير بأن السؤال غير صالح.

> × **«التدرّج صفر ولا أعرف لماذا»**
>
> قبل أن تتّهم المعادلة، تحقّق من ثلاثة: هل كل ما تدرّبه خرج من `jnp` ؟ هل مرّ عبر `stop_gradient` ؟ هل نوعه عائم؟

خسارتك تنقص في كل خطوة والتدرّج صفر. كيف تنقص الخسارة إن كان التدرّج صفراً؟ وما الذي يخبرك به ذلك عن مصدر العطل؟ ؟

تمرين الفصل السابع

1. احسب مشتقّة $f(x) = x^3 + 2x$ عند 2 باليد وبـ `grad`، ثم تحقّق من المشتقّة الثانية.
2. عرّف دالّة من $\mathbb{R}^2$ إلى $\mathbb{R}^2$، واحسب يعقوبيّتها بالوضعين.
3. ابنِ خسارة على قاموس أوزان، واحسب التدرّج بـ `argnums` ثم بـ `has_aux`.

الحلّ النموذجي

1. $f'(2) = 14$ والثانية $6x = 12$:

```python
import jax, jax.numpy as jnp

f = lambda x: x ** 3 + 2 * x
print(jax.grad(f)(2.0))              # 14.0
print(jax.grad(jax.grad(f))(2.0))    # 12.0
```

2. الوضعان يعطيان المصفوفة نفسها لأن الأبعاد صغيرة:

```python
g = lambda v: jnp.array([v[0] * v[1], v[0] + v[1]])
v = jnp.array([2.0, 3.0])
print(jax.jacrev(g)(v))   # [[3. 2.] [1. 1.]]
print(jax.jacfwd(g)(v))   # [[3. 2.] [1. 1.]]
```

3. التدرّج يحافظ على بنية القاموس، و`has_aux` يمرّر معلومات بلا اشتقاق:

```python
params = {"w": jnp.ones((3, 3)) * 0.1}

def loss(p, x):
    h = x @ p["w"]
    return jnp.sum(h ** 2), h.mean()

(v, aux), grads = jax.value_and_grad(loss, has_aux=True)(params, jnp.ones((2, 3)))
print(v, aux, grads["w"].shape)   # (3, 3)
```

خلاصة الفصل

- `grad` تحويل يُرجع دالّة، لا حساباً لرقم.
- العكسي رخيص حين تقلّ المخارج؛ الأمامي حين تقلّ المدخلات.
- `value_and_grad` يوفّر مروراً أمامياً في كل خطوة.
- التدرّج يحترم أشجار الوسائط، و `argnums` يحدّد موضعه.
- الصفر الصامت و `nan` أخطر من الخطأ الصريح.

مصطلحات الفصل

autodiff

اشتقاق آلي يطبّق قواعد محلّية أثناء التنفيذ.

VJP

سحب المشتقّة من الخرج إلى الدخل.

JVP

دفع مشتقّة اتجاهية إلى الأمام عبر اليعقوبية.

cotangent

القيمة الواردة من الأعلى، تُضرب محلياً وتُمرَّر للأسفل.

float0

نوع صفري بلا قيمة، يظهر عند الاشتقاق بالنسبة لوسيط صحيح.

hessian

مشتقّة ثانية: مصفوفة n×n تكلفتها خطّية مع n.

vmap: الدُفعات بلا حلقات

التوجيه التلقائي، والمحاور، والتفاعل مع الحالة والتفرّع

دالّة تعمل على عيّنة واحدة، وخمسة آلاف عيّنة في الملفّ. الحلّ البديهي حلقة `for`، وهي تعمل وتُخرج أرقاماً صحيحة – ثم تنظر في الزمن فتجد أن التسعين بالمئة منه ذهب إلى بايثون لا إلى الحساب. `vmap` هي الجواب الذي لم تكتبه: تأخذ دالّة العيّنة الواحدة، وتُعيد دالّة الدفعة، بلا حلقة ولا تغيير في المنطق.

الحلقة التي لا نريد كتابتها

ابدأ بالشكل الطبيعي: دالّة تفهم عيّنة واحدة. $\text{score}(w, x) = w \cdot x$، حيث x متّجه بطول أربعة. ثم دفعة من ثمانية: إمّا حلقة، وإمّا `vmap`.

```python
import jax, jax.numpy as jnp

def score(w, x):                # عيّنة واحدة
    return jnp.dot(w, x)

w = jnp.ones(4)
batch = jnp.arange(32.0).reshape(8, 4)

loop = jnp.stack([score(w, x) for x in batch])   # حلقة بايثون
vec = jax.vmap(score, in_axes=(None, 0))(w, batch)
print(loop.shape, jnp.allclose(loop, vec))       # (8,) True
```

النتيجة واحدة، لكن ما يجري مختلف: الحلقة تُطلق ثمانية حسابات صغيرة، كل واحد يدفع ثمن عبور حدود بايثون، و `vmap` تُحوّل الدالّة إلى نسخة تعمل على الدفعة كصفّ واحد.



الشكل 8.1 — `vmap` تضيف بعداً أمام كل بعد موجود، ولا تلمس منطق الدالّة.

> ★ **ليست حلقة أسرع، بل بعداً جديداً**
>
> `vmap` لا تنفّذ دالّتك ثماني مرّات أسرع؛ إنها تعيد كتابتها لتأخذ شكلاً `(8, ...)` . هذا هو سبب اختفاء الحلقة من الشيفرة الناتجة، وسبب أن الأخطاء تظهر في أشكال لا في قيم.

out_axesg in_axes

السؤال الذي تطرحه `vmap` على كل وسيط: أيّ بعد هو الدفعة؟ الجواب في `in_axes` . القيمة `0` تعني «البعد الأوّل دفعة»، و `None` تعني «لا تبعده، مرّره كما هو لكل العيّنات».

```python
import jax, jax.numpy as jnp

def f(w, x):
    return jnp.dot(w, x)

w = jnp.ones(4)
batch = jnp.ones((8, 4))

# x، والأوزان مشتركة، الدفعة في البعد الثاني من
xt = jnp.ones((4, 8))
out = jax.vmap(f, in_axes=(None, 1))(w, xt)
print(out.shape)                                  # (8,)

# الدفعة في البعد الأول، والخرج في بعد جديد
out2 = jax.vmap(f, in_axes=(None, 0), out_axes=1)(w, batch)
print(out2.shape)                                 # (8,)
```

`out_axes` تحدّد أين يقع بعد الدفعة في الخرج؛ الافتراضي صفر.

✕ «shape mismatch» في سطر لا تعرفه

أكثر أخطاء `vmap` أن `in_axes` تصف وسيطاً غير الذي تظنّه. الوسائط تُعدّ بالترتيب، لا بالأسماء. اطبع أشكال الوسائط قبل الاستدعاء، وتذكّر أن `None` لا تعني «تجاهل» بل «مرّر بلا بعد دفعة».

دالّة تأخذ مصفوفة أوزان ومصفوفة مدخلات، وتريد أن تُدرّب أوزاناً مختلفة لكل عيّنة — أي دفعة على الأوزان أيضاً. ماذا تكتب في `in_axes`، وماذا يعني ذلك للنتيجة؟ ؟

vmap فوق pytree

الوسائط في الواقع ليست مصفوفات بل أشجار: قاموس فيه صور ونصوص وأقنعة. و`vmap` تقبل `in_axes` بالبنية نفسها، فيأخذ كل ورقة محورها الخاص.

```python
import jax, jax.numpy as jnp

def step(params, batch):
    h = jnp.tanh(batch["x"] @ params["w"] + params["b"])
    return jnp.sum(h * batch["mask"])

params = {"w": jnp.ones((4, 2)) * 0.1, "b": jnp.zeros(2)}
batch = {"x": jnp.ones((8, 4)), "mask": jnp.ones((8, 2))}

out = jax.vmap(step, in_axes=(None, 0))(params, batch)
print(out.shape)      # (8,)
```

لاحظ `(None, 0)`: الأولى للأوزان كلها مشتركة، والثانية للقاموس كله مُبعَّد من أوّله. هذا يعمل لأن `0` تُطبَّق على كل ورقة في الشجرة، لا على القاموس ككائن.

✓ اكتب الدالّة لعيّنة واحدة دائماً

أسهل طريقة لكتابة دالّة قابلة لـ `vmap` أن تتخيّلها بلا بعد دفعة إطلاقاً. إن احتجت داخل الدالّة إلى `x.shape[0]` لتعرف حجم الدفعة، فأنت تكتبها بشكل خاطئ.

randomg vmap

هنا يظهر فرق جوهري عن NumPy. لا يوجد مولّد عشوائي عالمي في JAX؛ المفتاح قيمة صريحة. فإذا أردت ضجيجاً مختلفاً لكل عيّنة، احتجت مصفوفة مفاتيح – وهذا بالضبط ما تُبعده `vmap` .

```python
import jax, jax.numpy as jnp

def noise(key, shape):
    return jax.random.normal(key, shape)

key = jax.random.key(0)
keys = jax.random.split(key, 8)          # ثمانية مفاتيح مستقلة

draws = jax.vmap(noise, in_axes=(0, None))(keys, (3,))
print(draws.shape)                       # (8, 3)
print(jnp.allclose(draws[0], draws[1]))  # False
```

لو استخدمت المفتاح نفسه لحصلت على ثماني نسخ متطابقة – خطأ يمرّ بصمت لأن الأشكال صحيحة.

> ⚠ **مفتاح واحد، ثماني نسخ**
>
> تمرير المفتاح نفسه إلى `vmap` بلا `split` يعطي ضجيجاً متطابقاً في كل العيّنات. لا رسالة خطأ، ولا شكل خاطئ – فقط تدريب يتوقّف عن التعلّم لأن كل عيّنة ترى التحويل نفسه. افحص الاستقلال باختبار صغير.

vmap gradg: أربع تركيبات

التركيب هو أهمّ ما تفعله `vmap` عملياً. الترتيب يحدّد المعنى، والنتيجتان مختلفتان عددياً لا شكلاً فقط.

التركيب	المعنى	الشكل
`vmap(grad(f), in_axes=(None, 0))`	تدرّج كل عيّنة بالنسبة للأوزان المشتركة	(B, …)
`vmap(grad(f), in_axes=(0, 0))`	تدرّج كل عيّنة بالنسبة لوسائطها	(B, …)
`grad(mean(vmap(f)))`	تدرّج متوسّط الخسارة	مثل الأوزان
`grad(sum(vmap(f)))`	تدرّج مجموع الخسائر	مثل الأوزان

جدول 8.1 — أربع تركيبات وما تعنيه

```python
import jax, jax.numpy as jnp

def loss(w, x):
    return jnp.sum((w * x) ** 2)

w = jnp.ones(4)
batch = jnp.ones((8, 4)) * 2.0

per_example = jax.vmap(jax.grad(loss), in_axes=(None, 0))(w, batch)
total = jax.grad(lambda w: jnp.mean(jax.vmap(loss, in_axes=(None, 0))(w, batch)))(w)

print(per_example.shape)   # (8, 4)
print(total.shape)         # (4,)
```

الأول يعطيك ثمانية اتّجاهات، كل واحد يصلح لتحديث منفصل. والثاني يعطيك اتجاهاً واحداً لمتوسّط الخسارة – وهو ما تحتاجه في تدريب عادي.

> ✦ **لماذا لا يتساويان؟**
>
> متوسّط التدرّجات يساوي تدرّج المتوسّط، لأن الاشتقاق خطّي. لكن تدرّج المجموع ليس مجموع التدرّجات مقسوماً على عدد العيّنات إلا إذا كانت الخسارة خطّية في الدفعة. الخلط بينهما يغيّر مقياس التحديث بلا أن يشتكي أحد.

حدود vmap

`vmap` قوية لكنها ليست سحراً. ثلاثة قيود تشرح معظم ما يخذل الناس.

لا تحكّم معتمد على القيم. `if x > 0` داخل دالّة مُبعَّدة يسأل عن شرط لكل عيّنة على حدة، وهو ما لا يمكن تمثيله كبعد. الحلّ: `jax.lax.cond` إن كان الشرط واحداً للدفعة، أو `jnp.where` إن كان لكل عيّنة، أو `jax.lax.map` إن كان التفرّع بنيوياً حقاً.

لا آثار جانبية. الطباعة والكتابة إلى ملفّ لا تتكرّر لكل عيّنة؛ هي أثر واحد عند التتبّع، كما رأينا في الفصل السادس.

الذاكرة تنمو مع الدفعة. `vmap` تُنشئ نسخة مُبعَّدة من كل قيمة وسيطة. دفعة كبيرة تعني ذاكرة كبيرة، وقد تنفد قبل أن تنفد الأفكار. البديل `jax.lax.map` ينفّذ الحلقة تسلسلياً فيوفّر الذاكرة ويخسر التوازي.

متى لا تستخدم vmap؟

السؤال الأخير هو الأهمّ: متى يكون بعد الدفعة الحقيقي أرخص من `vmap`؟ حين تكون عمليتك أصلاً مصفوفية. ضرب $(B, D) \times (D, K)$ دفعةً واحدة أسرع من `vmap` على متّجه بطول D، لأن الثانية تبني شكلاً (B, D) ثم تضربه.

استخدم `vmap` حين يكون المنطق لعيّنة واحدة بطبيعته: حلقة تدريب مصغّرة، تفرّع على حالة، تدرّج لكل مثال. واستخدم البعد الحقيقي حين تكون العملية جبراً خطّياً واحداً.

> ★ **السؤال الصحيح**
>
> لا تسأل «هل `vmap` أسرع؟» بل «هل منطق عيّنتي قابل للكتابة كمصفوفة؟». إن كان نعم، اكتبه مصفوفة. وإن كان لا، فـ `vmap` هي التي تُنجيك من الحلقة.

دالّة تعالج صورة واحدة وتحتوي على حلقة `while` تتوقّف عند عتبة. هل يمكن تبعيدها بـ `vmap` كما هي؟ وما الذي ستحتاج إلى تغييره أولاً؟ ؟

تمرين الفصل الثامن

1. اكتب `score(w, x)` لعيّنة واحدة، ثم بعّدها بدفعة من 16. قارن الناتج بحلقة بايثون.
2. ابنِ قاموس دفعة فيه `x` و `mask` ، وبعّده مع إبقاء الأوزان مشتركة.
3. احسب تدرّج كل عيّنة وتدرّج المتوسّط، وقارن الشكلين والمعنى.

الحلّ النموذجي

1. النتيجتان متطابقتان، والفرق في الشكل لا في القيمة:

```python
import jax, jax.numpy as jnp

score = lambda w, x: jnp.dot(w, x)
w, batch = jnp.ones(4), jnp.ones((16, 4)) * 3.0

vec = jax.vmap(score, in_axes=(None, 0))(w, batch)
loop = jnp.stack([score(w, x) for x in batch])
print(vec.shape, jnp.allclose(vec, loop))   # (16,) True
```

2. `in_axes=(None, 0)` تُبعد كل ورقة في القاموس من بعدها الأوّل:

```python
def step(p, b):
    return jnp.sum(jnp.tanh(b["x"] @ p["w"]) * b["mask"])

p = {"w": jnp.ones((4, 2)) * 0.1}
b = {"x": jnp.ones((16, 4)), "mask": jnp.ones((16, 2))}
print(jax.vmap(step, in_axes=(None, 0))(p, b).shape)   # (16,)
```

3. لكل عيّنة تدرّجها، وللمتوسّط تدرّج واحد: الأول شكله `(16, 4)` والثاني `(4,)` . الأول يصلح لتحديث منفصل لكل مثال، والثاني لخطوة تدريب واحدة.

خلاصة الفصل

- `vmap` تضيف بعد دفعة بدل كتابة حلقة.
- `in_axes` تحدد محور كل وسيط، و `None` تعني مشترك.
- الأشجار تُبعَّد بورقة لكل ورقة، بالبنية نفسها.
- المفاتيح تُقسَّم قبل التبعيد، وترتيب `vmap` و `grad` يغيّر المعنى.
- الذاكرة تنمو مع الدفعة؛ `lax.map` بديل تسلسلي.

مصطلحات الفصل

vmap

تحويل يضيف بعد دفعة أمام أبعاد الدالّة بلا حلقة ظاهرة.

in_axes

محور الدفعة لكل وسيط؛ `0` للأول و `None` للمشترك.

out_axes

موضع بعد الدفعة في الخرج، افتراضه الصفر.

batching

إعادة كتابة الحساب ليعمل على دفعة كصفّ واحد بدل تكراره.

lax.map

حلقة تسلسلية داخل الرسم البياني، بديل عند ضيق الذاكرة.

per-example grad

تدرّج محسوب لعيّنة واحدة، يُنتج مصفوفة بشكل الدفعة.

النقاء والحالة وأشجار pytree

لماذا لا حالة في JAX؟ وكيف تبني نماذج بحالة نظيفة

جرّب أن تكتب عدّاداً. متغيّر واحد، وزائد واحد، ودالّة تُستدعى في كل خطوة تدريب. في بايثون العادية ينتهي الأمر في سطرين؛ وفي JAX ستحصل على رقم لا معنى له: يتوقّف عند واحد، أو يقفز قفزةً واحدة عند أوّل استدعاء ثم يجمُد. المشكلة ليست في العدّاد، بل في الفكرة التي يمثّلها: أن يكون للدالّة ذاكرةٌ خارج مدخلاتها. هذا الفصل عن تلك الفكرة، وعن البنية التي استبدلتها JAX بها: شجرةٌ من المصفوفات تُمرَّر صراحةً، حتى تعرف أنت قبل المترجم ما الذي تغيّر.

لماذا لا تستطيع JAX أن تحتفظ بعدّاد؟

الدالّة النقية يعتمد خرجها على مدخلاتها وحدها. يبدو التعريف بديهياً حتى تكتب أوّل عدّاد؛ عندئذٍ يصبح الخرج دالّةً في «كم مرّة استُدعيت»، أي في الزمن. والزمن ليس مدخلاً.

ولأن `jax.jit` لا ينفّذ دالّتك بل يقرأها مرّة واحدة – وهي عملية تُسمّى التتبّع – فإن أسطر بايثون الجانبية تنفّذ أثناء القراءة، لا أثناء التشغيل.

```python
import jax
import jax.numpy as jnp

count = 0

@jax.jit
def step(x):
    global count
    count += 1          # ينفّذ عند التتبّع، لا عند التشغيل
    return x + 1.0

for _ in range(3):
    step(jnp.zeros(()))
print("count =", count)
```

‹‹ المخرجات ››

```
count = 1
```

× الخطأ الذي يبدو صحيحاً

الشيفرة أعلاه لا تُطلق استثناءً؛ تُرجع مصفوفة صحيحة ورقماً صغيراً. وهذا أخطر أنماط الخطأ: لا انهيار، بل قيمة معقولة وكاذبة. ولو مرّرت مصفوفة بشكل مختلف لقفز العدّاد إلى اثنين.

وما نكسبه ثلاثة: تخزينٌ مؤقّت، وإعادةُ ترتيب (يحذف XLA حساباً لم يُستخدم لأنه لا أحد يراقب الترتيب)، وتوزيعٌ ممكن: حين تُقطّع المصفوفة على ثمانية أجهزة، فأين تعيش الحالة العامة؟

لو سمحت JAX بمتغيّر عام واحد فقط، فما أوّل شيء سينكسر: التجميع أم التوزيع؟ وأيّهما كان سينكسر بصمت؟ ؟

pytree: البنية التي يراها JAX

حين تمرّر قاموساً إلى دالّة مجمّعة، لا ترى JAX قاموساً. ترى قشرةً (structure) وأوراقاً (leaves): القشرة تنظيم – قاموس أو قائمة أو `namedtuple` – والأوراق هي البيانات.

Python

```
import jax
import jax.numpy as jnp

params = {"w": jnp.ones((2, 3)), "b": jnp.zeros(3)}
grads  = {"w": jnp.full((2, 3), 0.5), "b": jnp.ones(3)}

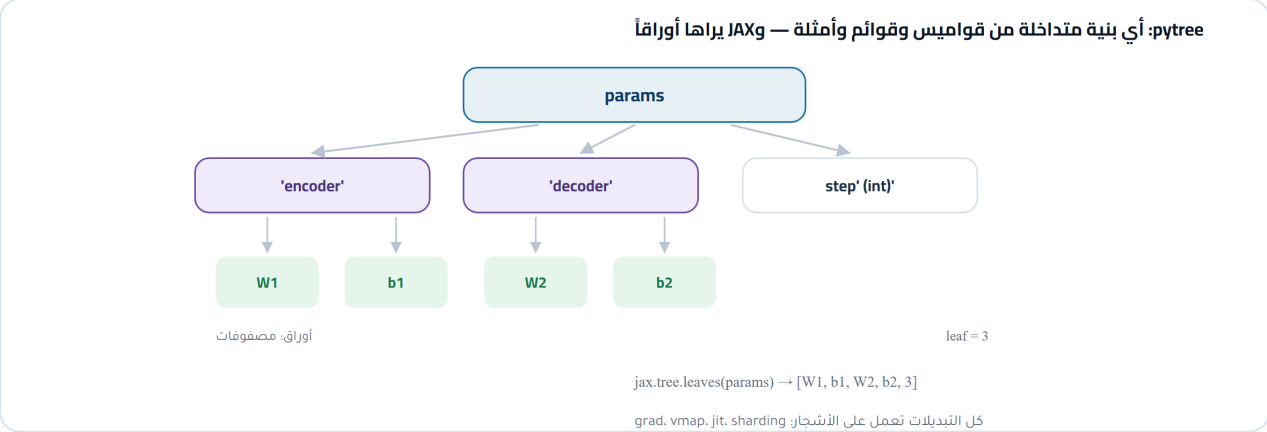
updated = jax.tree.map(lambda p, g: p - 0.1 * g, params, grads)

print(len(jax.tree.leaves(params)))                          # 2
print(jax.tree.structure(updated) == jax.tree.structure(params))  # True
```

‹‹ المخرجات ››

```
2
True
```

> ★ **القشرة جزء من التوقيع**
>
> `jax.tree.structure` يعيد وصف القشرة بلا قيم، وهو قابل للتجزئة ويُستخدم مفتاحاً في ذاكرة الترجمة. تغيير اسم مفتاح من `"w"` إلى `"weight"` يُبطل الترجمة السابقة، ولو بقي عدد المصفوفات وأحجامها كما هو.



الشكل 9.1 — الشجرة التي تراها JAX: العقد الداخلية تنظيم، والأوراق وحدها تصل إلى الحساب.

تسجيل نوعك الخاص

وماذا لو كانت حالتك كائناً من نوعك؟ `Dense` ليس قاموساً، فكيف يعرف JAX ما داخله؟ تخبره: `flatten` يفكّ الكائن إلى أبنائه، و `unflatten` يعيد بناءه.

```python
import jax
import jax.numpy as jnp
from dataclasses import dataclass

@dataclass(frozen=True)
class Dense:
    w: jax.Array
    b: jax.Array

def flatten(d):
    return (d.w, d.b), None          # الأبناء، ثم البيانات الثابتة

def unflatten(aux, children):
    return Dense(*children)

jax.tree_util.register_pytree_node(Dense, flatten, unflatten)

layer = Dense(jnp.ones((3, 4)), jnp.zeros(3))
print(jax.tree.leaves(layer))
```

✓ الثوابت في aux_data، لا بين الأوراق

الوسيط الثاني في `flatten` موضع البيانات الإضافية: ما يلزم لإعادة البناء ولا يُعدّ بيانات. لو أعدت رقم الميزات كابنٍ ثالث، عامله `jit` كورقة وحوّله إلى مُتتبَّع، فتفشل في بناء شكل.

الحالة كمدخل ومخرج

إذن ننقل الحالة إلى التوقيع: تأخذ القديمة وتعيد الجديدة، ولا تحتفظ بشيء بين الاستدعاءين.

```python
import jax
import jax.numpy as jnp

def step(state, x):
    count, total = state
    return (count + 1, total + x), x * 2.0

state = (jnp.array(0), jnp.array(0.0))
step_jit = jax.jit(step)

for i in range(3):
    state, y = step_jit(state, jnp.array(float(i)))

print(state)    # (3, 3.0)
print(y)        # 4.0
```

هذا النمط تعتمده كل مكتبة: Optax يعيد حالة جديدة، وFlax قاموس متغيّرات، و `jax.lax.scan` يحمل الحالة عبر الحلقة. الثمن توقيع أطول ونسخ يدوي.

BatchNorm كمثال حقيقي

> ؟ في BatchNorm، المتوسّط المتحرّك ليس وزناً يتعلّمه الانحدار. فلماذا يُخزَّن مع الأوزان، ولماذا لا يكفي أن يوضع في مكان آخر؟

أثناء التدريب تحسب الطبقة متوسّط الدفعة وتباينها وتطبّع بهما، وتحدّث متوسّطاً متحرّكاً للاستدلال. هذا المتوسّط ليس معاملاً: لا تدرّج له. إنه إحصاء، ولهذا يفصل Flax بين `params` و `batch_stats`:

```python
import jax, jax.numpy as jnp
from flax import linen as nn

class Block(nn.Module):
    @nn.compact
    def __call__(self, x, train: bool):
        x = nn.Dense(8)(x)
        x = nn.BatchNorm(use_running_average=not train)(x)
        return nn.relu(x)

model = Block()
x = jnp.ones((4, 8))
variables = model.init(jax.random.key(0), x, train=True)

y, updates = model.apply(
    {"params": variables["params"], "batch_stats": variables["batch_stats"]},
    x, train=True, mutable=["batch_stats"])

print(y.shape, updates["batch_stats"]["mean"].shape)
```

‹‹ المخرجات ››

```
(4, 8) (8,)
```

> ⚠ **لا تضع `batch_stats` في `params`**
>
> لم نمرّر «النموذج» ككائن حيّ، بل قاموساً أعلنّا أيّ فرع منه قد يتغيّر؛ والإعلان يسمح لـ `jit` بمعرفة شكل الخرج. ولو وُضعت الإحصاءات في `params` لطبّق عليها AdamW تناقص الأوزان فشُوّهت بصمت.

الحالة في NNX

ثم جاء NNX بجواب معاكس: لماذا نفصل؟ أعِد الحالة إلى داخل الكائن. `nnx.Module` يحمل أوزانه وإحصاءاته معاً ويعدّلها في مكانها، لكن داخل نظام يحافظ على قابليتها للتجميع.

```python
import jax
from flax import nnx

class Block(nnx.Module):
    def __init__(self, rngs: nnx.RNGs):
        self.linear = nnx.Linear(8, 8, rngs=rngs)
        self.norm = nnx.BatchNorm(8, rngs=rngs)

    def __call__(self, x):
        return nnx.relu(self.norm(self.linear(x)))

model = Block(nnx.RNGs(0))
graphdef, state = nnx.split(model)      # الشجرة عند الحدود
print(len(jax.tree.leaves(state)))
model = nnx.merge(graphdef, state)      # عودة إلى كائن حيّ
```

الانضباط لم يختفِ بل انتقل: ما زلت تحتاج `nnx.split` عند الحدود، قبل الحفظ وقبل المُحسّن؛ لكن الفصل صار عمليةً تُطلب عند الحاجة لا بنيةً تُفرض دائماً. ولا تخلط Linen وNNX في ملف واحد.

أنماط خاطئة للحالة

ثلاثة أنماط تتكرّر، وكلها تعمل في بايثون العادي وتخون في JAX: قائمة عامّة تُلحق بها السجلات (`log.append(...)` داخل دالّة مجمّعة تعطي عنصراً واحداً بعد مئة استدعاء)، وإغلاق على كائن قابل للتغيير، وذاكرة مؤقّتة على مستوى الوحدة.

وتشترك الثلاثة في عَرَض واحد: تعمل بلا أخطاء وتعطي نتائج تبدو معقولة، ولهذا هي أخطر من الخطأ الذي يوقف التشغيل.

قاعدة: الحالة تُمرَّر، لا تُشار

«تُمرَّر» تعني أنها في التوقيع؛ و«لا تُشار» أنه لا مكانين يملكان النسخة نفسها. القاعدة شرط هندسي: الحالة المُشار إليها تجعل السلوك يعتمد على ترتيب التنفيذ، وهذا الترتيب غير مضمون على GPU.

الاستراتيجية	أين تعيش	مع `jit`؟	الثمن
متغيّر عام	خارج الدالّة	لا	قيمة كاذبة بلا خطأ
إغلاق على كائن	في خلية الدالّة	لا	العيب نفسه، وأخفى
قاموس يُمرَّر ويُعاد	عند المُستدعي	نعم	توقيع أطول ونسخ يدوي

الاستراتيجية	أين تعيش	مع `jit`؟	الثمن
متغيّرات Linen	قاموس `variables`	نعم	فصل صريح بين الأوعية
كائن NNX	داخل الكائن	نعم	`split` عند الحدود

جدول 9.1 — خمس استراتيجيات للحالة، وثمن كل واحدة

تمرين الفصل التاسع

1. عرّف `Pair` كـ `dataclass` وسجّله في pytree، ثم اجمع نسختين منه بـ `jax.tree.map`. ماذا يحدث قبل التسجيل؟
2. حوّل عدّاداً عالمياً إلى حالة مُمرَّرة، وشغّل خمس خطوات بمدخل ثابت يساوي 2.0.
3. اشرح لماذا لا يصحّ وضع إحصاءات BatchNorm في `params`.

الحلّ النموذجي

1. قبل التسجيل يعامل JAX الكائن كورقة واحدة لا يفتحها، فيصل الكائن نفسه إلى الدالّة ويُرفض عند التجميع لأنه ليس مصفوفة.

```python
import jax, jax.numpy as jnp
from dataclasses import dataclass

@dataclass(frozen=True)
class Pair:
    a: jax.Array
    b: jax.Array

jax.tree_util.register_pytree_node(
    Pair,
    lambda p: ((p.a, p.b), None),
    lambda aux, ch: Pair(*ch))

p = Pair(jnp.ones(2), jnp.arange(2.0))
print(jax.tree.map(lambda t: t * 2, p))
```

2. الحالة زوج (عدّاد، مجموع)، والناتج بعد خمس خطوات `(5, 10.0)`: يزيد العدّاد خمساً، ويتراكم المجموع 5×2.0.

3. لأن المُحسّن يرى كل ما في `params` قابلاً للتعلّم: يطبّق عليه تناقص الأوزان ويحسب له عزماً. والإحصاءات نتيجة حساب لا معامل؛ تطبيق التناقص عليها يزيحها نحو الصفر بلا سبب ويشوّه التطبيع في الاستدلال.

خلاصة الفصل

- الدالّة المجمّعة تُقرأ مرّة وتُشغَّل كثيراً؛ الأثر الجانبي يحدث في القراءة.
- pytree = قشرة + أوراق؛ القشرة تدخل في مفتاح الترجمة، والأوراق هي البيانات.
- سجّل نوعك بـ `register_pytree_node` : أبناء في الأول، وثوابت في الثاني.
- الحالة تُمرَّر كمدخل وتُعاد كمخرج؛ هذا ما يجعل الخطوة قابلة للحفظ والتوزيع.
- Flax يفصل `params` عن `batch_stats` ؛ وNNX يعيد الحالة إلى الكائن مع `split` .

مصطلحات الفصل

purity (النقاء)

خاصية دالّة يعتمد خرجها على مدخلاتها وحدها، بلا أثر جانبي.

pytree

بنية متداخلة من حاويات معروفة وأوراق؛ الوحدة التي يفهمها JAX في المعاملات.

leaf (ورقة)

العنصر الذي يصل إلى العمليات الحسابية: مصفوفة أو عدد، لا حاوية.

treedef

وصف القشرة بلا قيم؛ قابل للتجزئة ويُستخدم مفتاحاً في ذاكرة الترجمة.

aux_data

بيانات ثابتة تُرفق بالشجرة لإعادة بناء الكائن، ولا تُعدّ من الأوراق.

random: العشوائية الصريحة

المفاتيح، split، fold_in، وفلسفة التكرار الحتمي

سأل مهندسان في الفريق نفسه السؤال ذاته: «هل نتائجك قابلة للتكرار؟» قال الأول نعم، وأرسل سكربتاً يعمل ويطبع الأرقام نفسها في كل مرّة. وقال الثاني نعم، وأرسل سكربتاً يطبع أرقاماً مختلفة في كل تشغيل. الاثنان يستخدمان JAX، والاثنان وضعا البذرة نفسها. الفرق أن أحدهما كان يسحب عشوائيته من مولّد بايثون العام، والآخر من مفتاح يمرّره بيده. هذا الفصل عن ذلك المفتاح: لماذا ليس رقماً، ولماذا لا يُعاد استعماله، وكيف تبني منه شجرةً تكفي تدريباً كاملاً.

لماذا لا يوجد np.random.seed في JAX؟

في NumPy، حين تنادي `np.random.seed(0)`، فإنك تضع رقماً في مكان خفيّ داخل المكتبة. كل استدعاء لاحق يقرأ ذلك المكان ويعدّله ويقدّم الحالة خطوة. النتيجة أن `np.random.normal()` بدالّة بلا وسائط تعطي أرقاماً مختلفة في كل مرّة: الخرج يعتمد على تاريخ الاستدعاءات، لا على المدخلات.

وهذا ما يرفضه النقاء (الفصل 9). والأسوأ مع ثمانية أجهزة: أيّها يملك ذلك العدّاد؟ إمّا عدّاد مشترك يتنازع عليه الجميع، أو ثمانية عدّادات تعطي ثمانية تيارات. الحلّان سيئان.

```python
import jax, numpy as np, jax.numpy as jnp

@jax.jit
def noise():
    return jnp.asarray(np.random.normal(size=3))

print(noise())
print(noise())      # نفس الأرقام تماماً
```

‹‹ المخرجات ››

```
[ 0.49671414 -0.1382643   0.64768857]
[ 0.49671414 -0.1382643   0.64768857]
```

✕ عشوائية تجمّدت في الرسم

السطر `np.random.normal` نُفّذ مرّة واحدة عند التتبّع، فصار ثابتاً مدفوناً في البرنامج المجمّع. لا خطأ، ولا تحذير: فقط ضجيجٌ لا يتغيّر، ونموذج يتعلّم على الدفعة نفسها في كل خطوة. اعتبر أي استدعاء لمولّد بايثون داخل دالّة مجمّعة خطأً حتى تثبت العكس.

لو وضعت البذرة مرّة واحدة قبل حلقة التدريب، ثم أضفت جهازاً ثانياً، فماذا يحدث للعشوائية على الجهازين؟ وأيّ جزء من البرنامج كان سيقرّر ذلك؟

المفتاح ليس رقماً

`jax.random.key(0)` لا يعيد عدداً، بل مصفوفة صغيرة من نوع خاص: `key<fry>` بشكل `()`. النوع نفسه يمنعك من جمع مفتاحين أو ضربهما في عدد: فالمفتاح ليس قيمة حسابية، بل بطاقة هوية لمولّد.

```python
import jax

k = jax.random.key(0)
print(k.dtype, k.shape)          # key ()
print(jax.random.key_data(k))    # uint32[2]

try:
    k + 1
except TypeError as exc:
    print("لا حساب على المفاتيح:", exc)
```

داخل كل مفتاح مادّة خام من 64 بت؛ والبذرة ليست «الرقم صفر» بل مادة أوّلية يُشتقّ منها تيار كامل. والمولّد – Threefry – عدّاد: يأخذ مفتاحاً وموضعاً، فيعيد أرقاماً بلا حالة محفوظة.

fold_ing split

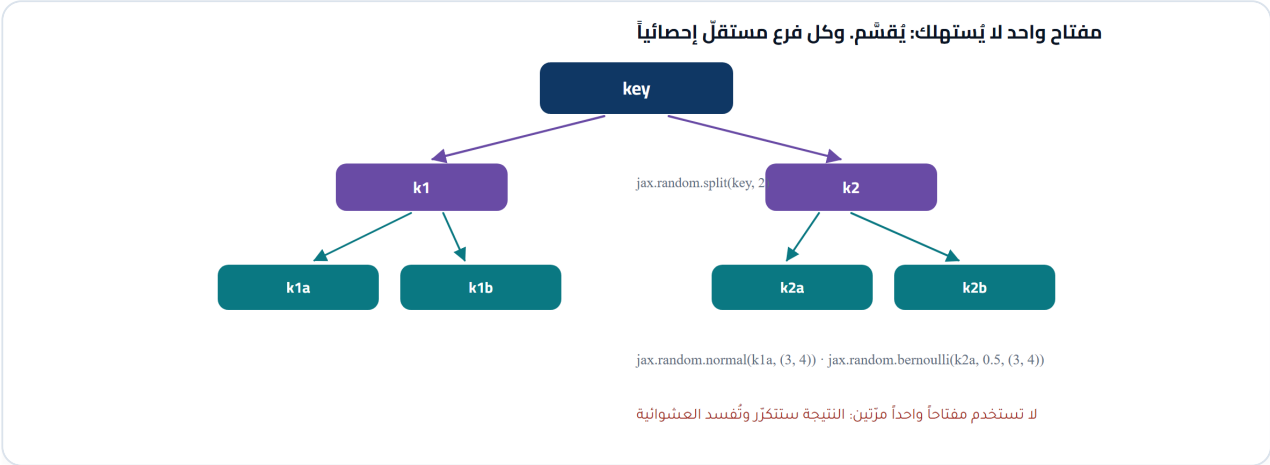
لديك مفتاح واحد، وتحتاج ألف مفتاح مستقلّ. الطريقة الأولى `jax.random.split`: تستهلك مفتاحاً وتنتج مفتاحين – أو `n` مفاتيح – لا علاقة إحصائية بينها. الطريقة الثانية `jax.random.fold_in`: تدمج عدداً صحيحاً في مفتاح فتحصل على مفتاح جديد، دون أن تستهلك الأصل.

```python
import jax

key = jax.random.key(42)
k1, k2 = jax.random.split(key)          # مفتاحان من مفتاح
ka, kb, kc = jax.random.split(k1, 3)    # ثلاثة من الأول

print(jax.random.normal(k1, (2,)))
print(jax.random.normal(k2, (2,)))
print(jax.random.normal(jax.random.fold_in(key, 7), (2,)))
```

تصوّر الشجرة: الجذر مفتاح التدريب، وتحته فروع للتهيئة والتدريب والتقييم، وتحت التدريب فرع لكل خطوة. القاعدة: كل عقدة تُشتقّ من أب أعلى، ولا تُستخدم مرّتين لغرضين.



الشكل 10.1 — شجرة المفاتيح: كل مسار من الجذر إلى الورقة يعطي تياراً مستقلاً، ولا يُسلك المسار نفسه مرّتين.

الوجه	`split`	`fold_in`
المدخل	مفتاح وعدد فروع	مفتاح وعدد صحيح
الأصل	يُستهلك، فلا تُكمل به	يبقى صالحاً للاستعمال
الاستعمال	فروع متوازية لا يعرف بعضها بعضاً	خطوة أو جهاز أو طبقة
الخطر	نسيان استعمال الأصل	تكرار العدد نفسه

جدول 10.1 — `split` مقابل `fold_in`

الأخطاء القاتلة: إعادة استخدام المفتاح

الخطأ الأكثر شيوعاً ليس خطأً في الصياغة، بل في التصوّر: يظنّ كثيرون أن كل استدعاء «يستهلك» المفتاح، كما كانت البذرة تُستهلك في NumPy. في JAX المفتاح قيمة ثابتة: استدعاؤه مرّتين يعطي النتيجة نفسها مرّتين.

في مشروع فيه مئة ملف، كيف تعرف أن مفتاحاً واحداً لم يُستخدم مرّتين لغرضين مختلفين؟ ؟

Python

```
key = jax.random.key(7)

a = jax.random.normal(key, (3,))
b = jax.random.normal(key, (3,))
print(jnp.allclose(a, b))       # True

k1, k2 = jax.random.split(key)
c = jax.random.normal(k1, (3,))
d = jax.random.normal(k2, (3,))
print(jnp.allclose(c, d))       # False
```

‹‹ المخرجات ››

```
True
False
```

⚠ التسريب الصامت

إعادة استخدام مفتاح في تهيئة طبقتين تعني أن الأوزان تبدأ متطابقة، فتكسر التماثل الذي وُجدت العشوائية لأجله. ولن ترى خطأً: الخسارة تنزل، والنموذج يتدرّب، والنتيجة أضعف قليلاً فقط. هذا أسوأ من الانهيار، لأنه لا يترك أثراً تبحث عنه.

التوزيعات الشائعة

كل دالّة توليد في `jax.random` تأخذ مفتاحاً أوّلاً، ثم الشكل. هذا الترتيب ليس ذوقاً: المفتاح هو المدخل الذي يجعل الدالّة نقية، والشكل هو ما يحدّد بنية الخرج.

```python
k1, k2, k3, k4 = jax.random.split(jax.random.key(0), 4)

print(jax.random.uniform(k1, (2, 3), minval=-1.0, maxval=1.0).shape)
print(jax.random.bernoulli(k2, 0.7, (5,)).dtype)
print(jax.random.categorical(k3, jnp.zeros((4, 5))).shape)
print(jax.random.permutation(k4, jnp.arange(6)))
```

‹‹ المخرجات ››

```
(2, 3)
bool
(4,)
[3 0 5 2 1 4]
```

لاحظ `categorical`: تأخذ لوغاريتمات بشكل `(4, 5)` وتعيد أربعة أرقام، واحداً لكل صفّ. و `permutation` تبديلٌ كامل لا اختيار؛ وللاختيار مع التكرار استخدم `jax.random.choice` مع `replace=True`.

العشوائية داخل jit وvmap

بما أن المفتاح مصفوفة، فهو مدخل كأي مدخل: يُمرَّر إلى الدالّة المجمّعة كوسيط، ويُشتقّ منه داخل الرسم. أما في `vmap` فتمرّر مصفوفة مفاتيح، فيصبح لكل عنصر في الدفعة تياره الخاص.

```python
def dropout(key, x, rate):
    keep = jax.random.bernoulli(key, 1.0 - rate, x.shape)
    return jnp.where(keep, x / (1.0 - rate), 0.0)

@jax.jit
def train_step(key, x, step):
    k = jax.random.fold_in(key, step)      # مفتاح جديد لكل خطوة
    return dropout(k, x, 0.2)

keys = jax.random.split(jax.random.key(0), 8)
masks = jax.vmap(lambda k: jax.random.bernoulli(k, 0.5, (4,)))(keys)
print(masks.shape)      # (8, 4)
```

✓ **الخطوة في المفتاح، لا في المتغيّر**

مرّر رقم الخطوة إلى `fold_in` بدل أن تحمل مفتاحاً متغيّراً في حلقة بايثون. بهذا تصبح الخطوة 900 قابلة لإعادة الإنتاج وحدها، حتى لو بدأت من نقطة حفظ في منتصف التدريب، وحتى لو أعدت ترتيب الخطوات على عدّة أجهزة.

التكرار الحتمي عبر الأجهزة

السؤال الذي يقلق الفرق: هل يعطي المفتاح نفسه الأرقام نفسها على CPU وGPU؟ الجواب: نعم، ما دامت الخوارزمية نفسها والشكل نفسه ونوع البيانات نفسه. المولّد حسابٌ صحيح على 32 بت، لا عملية عائمة تعتمد على ترتيب الجمع، ولهذا لا يتأثّر باختلاف العتاد.

لكن الحتمية أضيق مما تظنّ: تتغيّر النتائج إذا غيّرت إصدار `jax`، أو انتقلت إلى مولّد آخر مثل `rbg`، أو فعّلت `jax_enable_x64`. القاعدة: المفتاح يضمن التكرار داخل البيئة نفسها، لا عبر كل البيئات.

★ **ثلاثة أشياء تُخزَّن مع النموذج**

الأوزان، وحالة المُحسّن، ومفتاح التدريب. الثالث يُنسى غالباً، فتستعيد نموذجاً لا يمكن متابعة تدريبه بنتائج قابلة للتكرار. خزّن المفتاح كما تخزّن الأوزان، أو خزّن البذرة ورقم الخطوة واشتقّ المفتاح منهما.

تمرين الفصل العاشر

1. أنتج أربع أقنعة dropout لأربع خطوات من مفتاح واحد، وتحقّق من أنها مختلفة.
2. اشرح لماذا يعطي استدعاء `jax.random.normal` مرّتين بالمفتاح نفسه النتيجة نفسها، وكيف يحلّ `split` المشكلة.
3. ابنِ جدول مفاتيح لخطوتين وجهازين باستخدام `fold_in`، وأثبت أن التشغيل مرّتين يعطي الأرقام نفسها.

الحلّ النموذجي

1. نشتقّ مفتاحاً لكل خطوة بـ `fold_in`، ثم نقارن الأقنعة.

```python
import jax, jax.numpy as jnp

root = jax.random.key(0)
masks = [jax.random.bernoulli(jax.random.fold_in(root, s), 0.5, (6,))
         for s in range(4)]
print(jnp.stack(masks).astype(jnp.int32))
print(len({tuple(m.tolist()) for m in masks}))   # 4 أقنعة مختلفة
```

2. لأن المفتاح قيمة ثابتة لا حالة متغيّرة: الدالّة دالّة في (المفتاح، الشكل) وحدها، فالمدخل نفسه يعطي الخرج نفسه. و `split` يشتقّ مفتاحين مختلفين من الأصل، فيصبح لكل استعمال مدخل مختلف.

3. ندمج رقم الجهاز ورقم الخطوة في مفتاح واحد، فيصبح المسار فريداً:

```python
def step_key(root, device, step):
    return jax.random.fold_in(jax.random.fold_in(root, device), step)

root = jax.random.key(1234)
run1 = [float(jax.random.normal(step_key(root, d, s), ()).sum())
        for d in range(2) for s in range(2)]
run2 = [float(jax.random.normal(step_key(root, d, s), ()).sum())
        for d in range(2) for s in range(2)]
print(run1 == run2)      # True
```

خلاصة الفصل

- لا مولّد عام في JAX: الحالة الخفية تكسر النقاء وتستحيل مع التقطيع.
- المفتاح مصفوفة من نوع `key<fry>`، لا عدداً؛ ولا يُحسب عليه.
- `split` يشتقّ فروعاً ويستهلك الأصل، و `fold_in` يدمج رقماً ويُبقي الأصل.
- إعادة استخدام مفتاح تعطي أرقاماً متطابقة بصمت – وهي أشهر أخطاء الفصل.
- المفتاح يُمرَّر إلى `jit` و `vmap` كأي مدخل، ويُخزَّن مع الأوزان.

مصطلحات الفصل

PRNG key

مصفوفة صغيرة تحمل مادّة المولّد؛ تُمرَّر صراحةً بدل بذرة عامة.

split

اشتقاق مفتاحين أو أكثر من مفتاح أب، باستهلاك الأصل.

fold_in

دمج عدد صحيح في مفتاح لإنتاج مفتاح جديد مع إبقاء الأصل صالحاً.

key<fry>

نوع بيانات المفتاح في JAX 0.12، ويمنع الحساب المباشر على المفاتيح.

threefry

خوارزمية المولّد الافتراضية: عدّاد مع مفتاح، بلا حالة محفوظة.

من pmap إلى shard_map: التوازي

تاريخ التوازي في JAX، والطريق الحديث إلى عدّة أجهزة

في 2018 كان أمامك طريق واحد لتشغيل حساب على عدّة أجهزة: `pmap`. وفي 2021 جاء `pjit` ومعه GSPMD، فتغيّر السؤال من «كيف أوزّع الدفعة؟» إلى «كيف أصف التقطيع؟». وفي 2023 جاء `shard_map` ليعيد لك السيطرة على ما يحدث داخل الجهاز. ثلاث أدوات في خمس سنوات، وثلاثة نماذج ذهنية. هذا الفصل يحكي القصة، ثم يعلّمك أيّها تختار اليوم.

ثلاثة أجيال في خمس سنوات

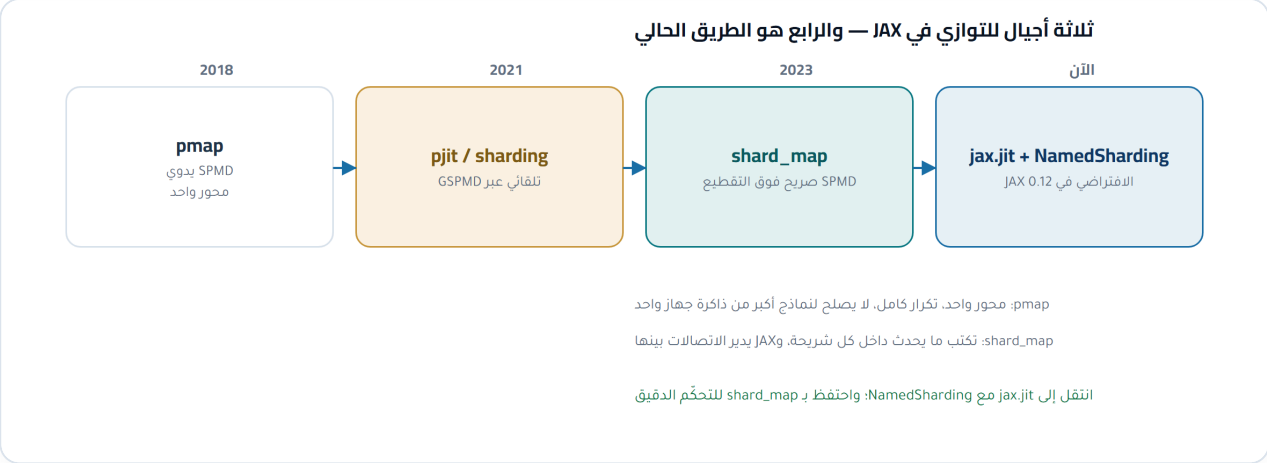
الجيل الأول، `pmap`، كان بسيطاً كفكرة: خذ دالّة وطبّقها على المحور الأول، جهازاً لكل شريحة. والثاني نقل العبء إلى المترجم: تصف التقطيع، ويتولّى GSPMD إدخال الاتصالات. والثالث أعاد ما يحدث داخل الشريحة إلى يدك.

أما اليوم، فالمسار الافتراضي هو `jax.jit` مع `NamedSharding`: تكتب الدالّة كما تكتبها على جهاز واحد، وتصف التقطيع في الحدود. و`jax.experimental.pjit` لم يبقَ واجهةً منفصلة، و`pmap` صار قديماً يُذكر في التاريخ لا في شيفرة جديدة.

SPMD بلغة بسيطة

SPMD تعني «برنامج واحد، بيانات متعدّدة»: الأجهزة الثمانية تشغّل البرنامج نفسه، وكل واحد يرى شريحة مختلفة. لا تكتب ثمانية برامج؛ تكتب واحداً وتصف التقطيع.

والمصفوفة التي تراها **شكلها عالمي**: `(8, 1024)` في نظرك، بينما يحمل كل جهاز `(1, 1024)`. والاتصال (collective) هو ما يعبر الحدود بين الأجهزة.



الشكل 11.1 — الشكل عالمي في نظرك، والشظايا محلية، والاتصال هو الجسر بين المستويين.

> ★ **الشكل عالمي، والبيانات محلية**
>
> كل ما تكتبه خارج `shard_map` يرى الشكل الكامل. وكل ما تكتبه داخله يرى شكل الشريحة. الخلط بين المستويين هو مصدر أكثر أخطاء التوزيع إزعاجاً.

pmap: لماذا لم يعد الخيار؟

`pmap` يطبّق الدالّة على المحور الأول فقط، ويعامل كل وسيط آخر بالتكرار الكامل. مقبول في دفعة بسيطة، وكارثي في نموذج كبير: تكرّر الأوزان ثماني مرّات.

ثم يفرض التفكير بمحور الدفعة دائماً، ويمنع التركيب الحرّ مع تقطيعات أخرى. ولهذا صار قديماً عملياً: اقرأه كتاريخ، ولا تبنِ عليه نظاماً جديداً.

> لماذا يحتاج `pmap` إلى محور أول دائماً، بينما `NamedSharding` لا يحتاج إلى أيّ محور؟ ؟

NamedShardingg mesh

الmesh شبكة مُسمّاة من الأجهزة: تقول إن لديك ثمانية أجهزة مرتّبة على محور اسمه `"x"`، أو شبكة `2 × 4` بمحورين. ثم تصف تقطيع كل مصفوفة بـ `PartitionSpec`.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

devices = jax.devices()                          # ثمانية أجهزة على مضيف واحد
assert len(devices) == 8
mesh = jax.make_mesh((8,), ("x",), devices=devices)
sharding = NamedSharding(mesh, P("x"))

x = jax.device_put(jnp.arange(64.0).reshape(8, 8), sharding)
print(x.sharding)                                # P('x',)
print(x.addressable_shards[0].data.shape)        # (1, 8)

@jax.jit
def col_sum(a):
    return a.sum(axis=0)

print(col_sum(x).shape)                          # (8,) على محور مكرّر
```

و `P("x", None)` تعني: قسّم البُعد الأول على `"x"`، وكرّر الثاني. وللتحكّم الدقيق في الحدود تمرّر `in_shardings` و `out_shardings` إلى `jax.jit`.

الوصف	المعنى	شكل الشريحة
`P("x")`	البُعد الأول على `x`	مقسوم على حجم المحور
`P("x", None)`	الأول مقطّع، والثاني مكرّر	مقسوم في الأول فقط

جدول 11.1 — قراءة `PartitionSpec`

shard_map: تكتب داخل الشريحة

`shard_map` يقلب المنظور: الدالّة التي تكتبها تُنفَّذ على جهاز واحد، وترى المصفوفة بشكلها المحلي. أنت من يكتب الاتصال صراحةً بـ `jax.lax.psum` أو `all_gather`.

```python
from functools import partial
from jax.experimental.shard_map import shard_map

@partial(shard_map, mesh=mesh, in_specs=(P("x"),), out_specs=P("x"))
def local_center(x):
    return x - jax.lax.pmean(x.mean(), "x")

y = local_center(x)
print(y.shape)      # (8, 8) عالمي
```

داخل الدالّة لا يوجد mesh، بل مصفوفة محلية وأسماء محاور للاتصال. ولهذا هو مثالي حين تريد أن تعرف بالضبط أيّ اتصال يحدث، وهو ما يخفيه المقطّع التلقائي.

> ✓ **متى تنزل إلى shard_map؟**
>
> حين تخالف خطّة التقطيع التلقائية ما تريد: توازٍ على المحاور، أو اتصال مخصّص، أو خوارزمية موزّعة تكتبها بنفسك. أما التقطيع العادي للدفعات والأوزان فدعه لـ `NamedSharding`.

الاتصالات: all-reduce، all-gather، reduce-scatter

كل اتصال يجيب عن سؤال مختلف. `psum` يجمع القيم عبر المحور ويعيد المجموع للجميع، وهو ما يحدث في مزامنة التدرّجات. و`all_gather` يعيد لكل جهاز القيم كلها، فتنمو الذاكرة بعدد الأجهزة.

وreduce-scatter يجمع ثم يوزّع: كل جهاز يحصل على جزء من الناتج. وهو ما يستخدمه XLA تحت الغطاء، ولهذا لا تكتبه عادةً بل تتركه للمترجم.

الاتصال	ما يفعله	أين يظهر
`psum`	جمع عبر المحور، والنتيجة للجميع	مزامنة التدرّجات
`pmean`	متوسّط عبر المحور	تطبيع، خسارة موزّعة
`all_gather`	تجميع الشظايا عند الجميع	طبقة تحتاج المدخل كاملاً
reduce-scatter	جمع ثم توزيع الأجزاء	داخل all-reduce الكبير
all-to-all	تبادل غير متجانٍ بين الأجهزة	خبراء، انتباه موزّع

جدول 11.2 — الاتصالات ومتى تظهر

لو قسّمت الأوزان على ثمانية أجهزة ثم احتجت في طبقة ما إلى المصفوفة كاملة، فما ثمن `all_gather`؟ وهل يبقى التوفير في الذاكرة؟ ؟

أيّ نمط أختار؟

الاختيار يبدأ من سؤال: ما الذي لا يتّسع في جهاز واحد؟ إن كانت الدفعة فالتوازي على البيانات، وإن كانت الأوزان فالتوازي على النموذج، وإن كان الاثنان فالتقطيع المُجزَّأ.

النمط	ما يُقطَّع	الاتصال الغالب	متى تختاره
بيانات	الدفعة	`psum` على التدرّجات	النموذج يتّسع في جهاز
موتر	الأوزان على محور داخلي	`all_gather`	طبقة أوسع من ذاكرة الجهاز
FSDP	الأوزان، مع جمع عند الحاجة	all-gather ثم reduce-scatter	نموذج أكبر من الذاكرة
خط أنابيب	الطبقات على مراحل	إرسال بين المراحل	عمق كبير ودفعات صغيرة

جدول 11.3 — أنماط التوزيع وحدودها

لاحظ أن الأنماط تُركَّب: يمكنك تقطيع الدفعة على محور والأوزان على آخر، في شبكة `(4, 2)`. وكلّما زادت المحاور زادت الاتصالات.

أخطاء التوزيع الشائعة

الخطأ الأول هو التكرار الصامت: تنسى التقطيع، فتُوضع المصفوفة كاملة على كل جهاز. لا خطأ، لكن الذاكرة تُستهلك ثماني مرّات. افحص `x.sharding` بعد كل `device_put`.

والثاني ترتيب المحاور: `P("x", "y")` مع mesh بشكل `(2, 4)` يقسم البُعد الأول على محور حجمه اثنان. والثالث القابلية للقسمة: شكل لا يقبل القسمة يُرفض عند الترجمة، والحلّ حشو.

× تقطيع يُلغى بصمت

حين تعيد عملية ما تشكيل المصفوفة (`reshape` أو `transpose`) قد يُعيد المترجم توزيعها على محور مختلف، فتصبح الحدود اتصالات لم تخطّط لها. اطبع `y.sharding` عند كل حدّ تشكّ فيه.

> ⚠ **لا تفترض أن التوزيع يعمل لأنه يعمل**
>
> برنامج موزّع يعطي النتيجة الصحيحة قد يكون أبطأ من نسخة الجهاز الواحد، لأن كل الحدود صارت اتصالات. قِس زمن الخطوة قبل التوزيع وبعده؛ إن لم ينقص، فالتقطيع لا يشتري شيئاً.

تمرين الفصل الحادي عشر

1. أنشئ mesh بشكل `(4, 2)` بمحورين، وقطّع مصفوفة `(8, 16)` على المحور الأول فقط. ما شكل الشريحة؟
2. اكتب دالّة `shard_map` تحسب مجموع كل الأجهزة بـ `psum`، وقارن الناتج بـ `jnp.sum` العادية.
3. اذكر خطأين يجعلان التوزيع «يعمل» بلا فائدة، وكيف تكتشف كل واحد منهما.

الحلّ النموذجي

1. الشريحة `(2, 16)`: البُعد الأول مقسوم على أربعة، والثاني مكرّر بالكامل على المحور `"y"`.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

mesh = jax.make_mesh((4, 2), ("data", "model"))
sh = NamedSharding(mesh, P("data", None))
w = jax.device_put(jnp.ones((8, 16)), sh)
print(w.addressable_shards[0].data.shape)   # (2, 16)
```

2. الدالّة المحلية ترى شريحة، و `psum` يجمعها عبر المحور:

```python
from functools import partial
from jax.experimental.shard_map import shard_map

m1 = jax.make_mesh((8,), ("x",))
f = partial(shard_map, mesh=m1, in_specs=(P("x"),), out_specs=P())
total = f(lambda a: jax.lax.psum(a.sum(), "x"))
print(float(total(jnp.ones((8, 4)))))   # 32.0
```

3. التكرار الصامت: يُكشف بـ `x.sharding`. وترتيب المحاور الخاطئ: يُكشف بمقارنة شكل الشريحة المتوقّع مع `addressable_shards`.

خلاصة الفصل

- ثلاثة أجيال: `pmap` ثم `pjit` ثم `shard_map`؛ والافتراضي اليوم `jax.jit` مع `NamedSharding`.
- SPMD: برنامج واحد على كل جهاز، والشكل عالمي والبيانات محلية.
- `pmap` قديم: محور أول فقط، وتكرار كامل لبقية الوسائط.
- `shard_map` يكتب داخل الشريحة، فتصبح الاتصالات صريحة.
- اختر النمط بسؤال: ما الذي لا يتّسع؟

مصطلحات الفصل

SPMD

برنامج واحد يُنفَّذ على عدّة أجهزة، كلٌّ على شريحة مختلفة من البيانات.

mesh

شبكة مُسمّاة من الأجهزة ذات محاور مثل `"data"` و `"model"`.

PartitionSpec

وصف تقطيع مصفوفة: اسم محور لكل بُعد، أو `None` للتكرار.

collective

عملية تعبر الأجهزة: جمع، أو تجميع، أو توزيع قيم بينها.

shard_map

تحويل يكتب دالّة بمستوى الشريحة، مع اتصالات صريحة وحدود معلنة.

تركيب التبديلات: القواعد الذهبية

jit(grad(vmap(f))): كيف تتركّب، وأين تنكسر، وكيف تتجنّب ذلك

سألك زميل: «هل أضع `vmap` داخل `jit` أم العكس؟». أجبت بسؤال: «ماذا تريد أن تُجمّع؟». لم يفهم. كان يفكّر في التحويلات كأزرار في لوحة تحكّم، وليست كذلك: كل تحويل في JAX دالّة تأخذ دالّة وتُرجع دالّة. وحين تدرك هذا، يتحوّل السؤال من «هل يجوز؟» إلى «ما الذي يحدث في كل ترتيب، وبأيّ ثمن؟». هذا الفصل عن ذلك الثمن.

التركيب ليس ميزة إضافية، بل الخاصية الأساسية

تخيّل مكتبة جعلت `grad` طريقةً على كائن: `model.grad()`. الآن أضف `vmap`: هل `model.vmap().grad()` أم `model.grad().vmap()`؟ كل توليفة تحتاج كائناً ورمزاً جديدين. هكذا تنهار المكتبات عند التركيب.

JAX اختارت العكس: التحويل دالّة من نوع «دالّة ← دالّة». لا شيء منها يعرف شيئاً عن الآخر، ومخرَج كلٍّ منها صالح كمدخَل لأيّ منها، فالتركيب مجّاني بالبناء لا بمجهود الفريق.

لكن «مجّاني» تعني «صحيح»، لا «رخيص». كل تحويل يرى ما فوقه صندوقاً أسود: حين يرى `jit` داخل `vmap` دالّةً تُستدعى مرّة واحدة يجمّعها مرّة واحدة، ولا يعلم أن `vmap` سينسخها على كل عنصر. من هنا يأتي فرق التكلفة.

> ★ **سؤال يُغني عن حفظ الحالات**
>
> أيّ تحويل يرى الدالّة كلها، وأيّها يراها عنصراً واحداً؟ الترتيب يحدّد ما يدخل المترجم، وحجمه يحدّد عدد مرّات التجميع.

خريطة التركيبات

ثمانية تركيبات تغطّي معظم ما ستكتبه، وثمنها:

التركيب	الحكم	ملاحظة التكلفة
`jit(grad(f))`	يعمل، وهو افتراضي التدريب	تجميع واحد لدالّة التدرّج
`grad(jit(f))`	يعمل، لكن بلا فائدة	التجميع داخل التفريق شفّاف
`vmap(jit(f))`	يعمل، والأفضل لأحجام متغيّرة	يجمّع `f` مرّة لكل حجم دفعة
`jit(vmap(f))`	يعمل، والأسرع لدفعة ثابتة	يُعاد التجميع عند كل حجم جديد
`grad(vmap(f))`	يفشل غالباً	`grad` يطلب قياسياً
`vmap(grad(f))`	يعمل، وهو تدرّج كل مثال	ذاكرة = حجم الدفعة × المعاملات
`jit(grad(vmap(f)))`	يعمل، ونمط الخسارة الكلية	الجمع داخل التجميع: الأرخص
`shard_map(vmap(f))`	يعمل، ونمط التقطيع اليدوي	`vmap` داخل الجهاز و`shard_map` عبره

جدول 12.1 — ثمانية تركيبات شائعة، وحكم كلٍّ منها وثمنه



الشكل 12.1 — كل حلقة تحويل تلفّ ما داخلها؛ الخارجي يرى شكلاً واحداً، والداخلي يرى ما فُرض عليه.

في الجدول سطران يعملان وأحدهما «بلا فائدة». هل الخطأ في الدالّة، أم في موضع الحدّ بين التفسير والتنفيذ؟ ؟

الترتيب يغيّر التكلفة لا النتيجة

عدّاد يطبع سطراً عند كل تجميع:

```python
import jax
import jax.numpy as jnp

jax.config.update("jax_log_compiles", True)  # سطر عند كل تجميع

def f(x):
    return jnp.tanh(x) @ jnp.ones((4, 1))

A = jax.jit(jax.vmap(f))   # الدفعة كلها داخل التجميع
B = jax.vmap(jax.jit(f))   # وحدها داخل التجميع f

for n in (8, 16, 32):
    x = jnp.ones((n, 4))
    print("A", n, A(x).shape, "B", n, B(x).shape)
```

‹‹ المخرجات ››

```
Compiling vmap_f ...
A 8 (8, 1) B 8 (8, 1)
Compiling vmap_f ...
A 16 (16, 1) B 16 (16, 1)
Compiling vmap_f ...
A 32 (32, 1) B 32 (32, 1)
Compiling f ...
B 8 (8, 1) B 16 (16, 1) B 32 (32, 1)
```

الأرقام متطابقة، فالنسخ عملية رياضية لا تعتمد على ترتيب التنفيذ. لكن `A` جُمّع ثلاث مرّات: الشكل الذي رآه المترجم تغيّر ثلاث مرّات. أما `B` فجُمّع مرّة لشكل `(4,)`، ثم لفّ `vmap` حول الناتج.

الفرق لا يظهر في اختبار الوحدة بل في الإنتاج: خادم بدفعات متغيّرة سيعيد التجميع عند كل حجم.

> ✓ **ثبّت الشكل بدل أن تثبّت الترتيب**
>
> احشُ الدفعة إلى مضاعفات `8` بقناع، أو استخدم `static_argnums` . حين تستقرّ الأشكال يتكافأ السؤالان.

الحالة داخل التركيب

لماذا يفشل `grad` أحياناً داخل `vmap` ؟ لأنه يحتاج قياسياً، و `vmap` يطلب شيئاً لكل مثال. الحلّ ليس حيلة: `has_aux=True` يجعل `grad` يُرجع (التدرّج، الملحق)، فيوزّع `vmap` الاثنين.

```python
import jax
import jax.numpy as jnp

def loss(params, x, y):
    logits = x @ params["w"] + params["b"]
    l = jnp.mean((logits - y) ** 2)
    return l, logits          # الخسارة + ملحق

params = {"w": jnp.zeros((4, 1)), "b": jnp.zeros(1)}
x, y = jnp.ones((2, 4)), jnp.ones((2, 1))

per_example = jax.vmap(jax.grad(loss, has_aux=True), in_axes=(None, 0, 0))
grads, logits = per_example(params, x, y)
print(jax.tree.map(lambda g: g.shape, grads))
print(logits.shape)
```

‹‹ المخرجات ››

```
{'b': (2, 1), 'w': (2, 4, 1)}
(2, 2, 1)
```

بُعد الدفعة في المقدّمة: تدرّج لكل مثال، والملحق نُسخ أيضاً.

أمّا الحالة القابلة للتعديل فقصة أخرى. `vmap` ينسخ جسم الدالّة على كل عنصر، فإن كتبت نسخةٌ في مصفوفة مشتركة كتبت كلّها فيها بلا ترتيب محدّد. في Linen يظهر هذا حين تستدعي `apply` مع `mutable=["batch_stats"]` داخل `vmap`.

> **× خطأ شائع: قراءة التحديث كأنه جاهز**
>
> تحديثات `batch_stats` تعود بشكل `(B, ...)`، وصيغة الزخم تحتاج مجاميع لا متوسطات. اقرأ صيغة التحديث في المصدر.

> لو لففتَ دالّة تكتب في عدّاد عام بـ `vmap` على 32 عنصراً، فما قيمة العدّاد؟ وهل تُثبت الجواب دون تشغيلها؟ ؟

التبديلات المخصّصة: custom_vjp و custom_jvp

أحياناً تعرف عن دالّتك أكثر ممّا يعرف التفريق التلقائي، والصيغة التحليلية أدقّ عددياً من شجرة العمليات. هنا يعطيك `jax.custom_vjp` المرورين صراحةً:

```python
import jax
import jax.numpy as jnp

@jax.custom_vjp
def softplus(x):
    return jnp.logaddexp(0.0, x)      # أمامي مستقرّ

def softplus_fwd(x):
    y = jnp.logaddexp(0.0, x)
    return y, y                        # نُخزّن y للعكسي

def softplus_bwd(y, g):
    # d/dx softplus(x) = sigmoid(x) = 1 - exp(-softplus(x))
    return (g * (1.0 - jnp.exp(-y)),)

softplus.defvjp(softplus_fwd, softplus_bwd)

print(softplus(30.0), jax.grad(softplus)(30.0))   # 30.0 1.0
print(jax.grad(softplus)(-30.0))                  # ~1.8e-13
```

عند `x = 30` يقترب أي مقام من هيئة `1 + exp(x)` من حدّ الدقّة، أما e^{-30} فصغير.

$$\sigma(x) = 1 / (1 + e^{-x}) = 1 - e^{-\text{softplus}(x)}$$

الصيغتان متكافئتان رياضياً، ومختلفتان عددياً عند الطرفين.

و`jax.custom_jvp` مكمّل: يحدّد المشتقّة الاتجاهية في المرور الأمامي، وهو أوفر حين يكون الحساب الأمامي هو الأغلى. أما `custom_vjp` فأوفر للذاكرة لأنه لا يخزّن الوسائط الوسطية.

> **⚠ لا تُصلح الخطأ العددي بتخمين**
>
> المرور العكسي المخصّص يُستخدم كما هو؛ JAX لا تتحقّق من صحّته.
> `jax.test_util.check_grads` هو الحدّ الأدنى.

التبديلات المخصّصة للتقطيع: custom_partitioning

ماذا لو عرفتَ بنية دالّتك أكثر ممّا يعرف المترجم وأردت فرض تقطيع معيّن؟ `custom_partitioning` يفعل ذلك، وهو موضوع متقدّم بعد `shard_map`.

```python
from jax.experimental.custom_partitioning import custom_partitioning
from jax.sharding import NamedSharding, PartitionSpec as P

@custom_partitioning
def row_softmax(x):
    return jax.nn.softmax(x, axis=-1)

def _partition(mesh, arg_shapes, result_shape):
    s = NamedSharding(mesh, P(None, "tp"))
    return mesh, row_softmax, s, s   # الشبكة، الدالّة، تقطيع المدخل، تقطيع الخرج

row_softmax.def_partition(partition=_partition)
```

الدالّة تقول: «وزّع المحور الأخير على `tp` ولا تقطع الأول». الثمن أنك صرت مسؤولاً عن صحّة التقطيع: الخطأ هنا لا يظهر استثناءً بل أرقاماً خاطئة.

قواعد ذهبية

1. **`jit` في الخارج دائماً.** الاستثناء: تجميع دالّة صغيرة لإعادة استخدامها بأحجام مختلفة، وعندها `vmap(jit(f))` أذكى.
2. **لا تفرّق دالّة مجمّعة.** `grad(jit(f))` يعمل ويضاعف عمل المترجم بلا مقابل.
3. **الدالّة الداخلية تُرجع قياسياً لكل مثال** قبل `vmap(grad(...))`، وإلا فـ `has_aux=True`.
4. **لا حالة قابلة للتعديل داخل `vmap`.** كل كتابة تُنسخ والترتيب غير محدّد.
5. **كل تحويل مخصّص يحتاج اختبار تدرّج**، وإلا كان خطأً صامتاً.

تمرين الفصل الثاني عشر

1. خُذ `f(x) = jnp.sin(x) * x` واكتب `jit(vmap(f))` و `vmap(jit(f))` و `vmap(f)`، وقِس زمن أول استدعاء وثانيه.
2. غيّر حجم الدفعة بين 4 و64 وسجّل عدد التجميعات بـ `jax_log_compiles`.
3. اكتب تدرّجاً مخصّصاً لـ `log1p` بمرور عكسي $g / (1 + x)$، وتحقّق منه عند `x = 1e-8`.

الحلّ النموذجي

1 و2. `vmap(jit(f))` تجمع `f` مرّة واحدة وتبقى ثابتة مهما تغيّر الحجم، لأن ما يدخل التجميع شكله `(4,)`. أما `jit(vmap(f))` فتُعيد التجميع عند كل حجم.

3.

```python
import jax
import jax.numpy as jnp
from jax.test_util import check_grads

@jax.custom_vjp
def log1p(x):
    return jnp.log1p(x)

def fwd(x):
    return jnp.log1p(x), x

def bwd(x, g):
    return (g / (1.0 + x),)

log1p.defvjp(fwd, bwd)
check_grads(log1p, (1e-8,), order=1)
print(log1p(1e-8), jax.grad(log1p)(1e-8))
```

الخرج قريب من `1e-08` و `1.0`. وإن اشتكى `check_grads` فالسبب الفروق المحدودة لا التدرّج.

خلاصة الفصل

- كل تحويل دالّة من دالّة إلى دالّة، فالتركيب صحيح بالبناء.
- الترتيب لا يغيّر الأرقام بل عدد مرّات التجميع.
- `jit` في الخارج هو الافتراضي؛ `vmap(jit(f))` استثناء مدروس.
- `vmap(grad(f))` يحتاج خسارة قياسية لكل مثال.
- الحالة القابلة للتعديل لا تعيش داخل `vmap` .

مصطلحات الفصل

composition (تركيب)

وضع تحويل داخل آخر: مخرَج كل تحويل دالّة صالحة كمدخَل.

trace (تتبّع)

تشغيل الدالّة على قيم رمزية لبناء رسم حسابي بدل أرقام.

has_aux

وسيط في `grad` يجعل الدالّة تُرجع (القيمة، ملحق).

custom_vjp

واجهة لتحديد المرور الأمامي والعكسي يدوياً.

custom_partitioning

واجهة لفرض تقطيع دالّة على شبكة.

الجزء الثالث

بناء الشبكات العصبية

من الطبقة الأولى إلى حلقة تدريب إنتاجية: Orbaxg Optaxg Flax

شبكة عصبية من الصفر بـ jax.numpy

MLP كامل، وتدرّج يدوي، وتدريب حقيقي بلا مكتبات عالية

سألت مهندساً يستخدم إطاراً معروفاً: «ماذا يحدث بين إدخال الصورة ورقم الخسارة؟». رسم مستطيلاً وكتب داخله «النموذج». سألته: «وأين يقع التدرّج؟» قال: «الإطار يحسبه». لم يكن يعرف أن ما يسمّيه إطاراً هو ثلاث أسطر: مرور أمامي، خسارة، تحديث. هذا الفصل يكتب تلك الأسطر الثلاثة بـ `jax.numpy` وحدها، بلا طبقات ولا محسّنات، لتعرف ما الذي ستخفيه عنك المكتبات لاحقاً – وما لن تخفيه أبداً.

لماذا نبدأ من الصفر؟

ثلاثة أسطر لا يخفيها أي إطار فعلياً: كيف تُحوَّل المدخلات إلى مخرجات، وكيف يُقاس الخطأ، وكيف تتحرّك المعاملات. الباقي – التهيئة، والتجميع، وحفظ النقاط – آلات حولها. من يكتبها مرّة بيده يقرأ أي إطار بعدها بوصفه تفصيلاً، لا سحراً.

لو أعطاك زميل نموذجاً يتوقّف عند خسارة `0.693` بالضبط، فما أوّل ثلاثة أسئلة تسألها؟ ولماذا يشير هذا الرقم إلى التهيئة أو التعلّم لا إلى البيانات؟ ؟

البيانات: صناعة مشكلة قابلة للتعلّم

نريد مشكلة لا يحلّها خطّ مستقيم، وصغيرة بما يكفي لتدريبها في ثوانٍ. «القمران»: نصف دائرة أعلى ونصف دائرة أسفل، مع ضجيج يمنع الحفظ. نولّدها بمفاتيح `jax.random` .

```python
import jax
import jax.numpy as jnp

def make_moons(n, key, noise=0.1):
    k1, k2, k3 = jax.random.split(key, 3)
    a = n // 2
    t0 = jax.random.uniform(k1, (a,), maxval=jnp.pi)
    t1 = jax.random.uniform(k2, (n - a,), maxval=jnp.pi)
    x0 = jnp.stack([jnp.cos(t0), jnp.sin(t0)], axis=1)
    x1 = jnp.stack([1.0 - jnp.cos(t1), 0.5 - jnp.sin(t1)], axis=1)
    x = jnp.concatenate([x0, x1]) + noise * jax.random.normal(k3, (n, 2))
    y = jnp.concatenate([jnp.zeros(a), jnp.ones(n - a)]).astype(jnp.int32)
    return x, y

x, y = make_moons(512, jax.random.key(0))
print(x.shape, y.shape, y.mean())
```

‹‹ المخرجات ››

```
(512, 2) (512,) 0.5
```

التهيئة: لماذا لا نبدأ بالأصفار؟

جرّب أن تضع كل الأوزان أصفاراً. كل الخلايا تُخرج الصفر نفسه، فيصل التدرّج إليها متساوياً، فتبقى متساوية إلى الأبد – انهيار تناظر لا خطأ برمجي.

الاختيار الشائع: عيّنة عشوائية بتباين مضبوط. لو كانت مدخلات الطبقة n قيمة بتباين 1، فإن $y = \sum w_i x_i$ يملك تبايناً $n \cdot \mathrm{Var}(w)$؛ ولنُبقيه 1:

$$\mathrm{Var}(w) = 2 / n \Rightarrow \sigma = \sqrt{(2 / n)}$$

تهيئة He: المعامل 2 يعوّض نصف القيم التي يصفّرها `relu`.

```python
def init_params(key, sizes):
    params, key = [], key
    for m, n in zip(sizes[:-1], sizes[1:]):
        key, sub = jax.random.split(key)
        W = jax.random.normal(sub, (m, n)) * jnp.sqrt(2.0 / m)   # He
        params.append({"W": W, "b": jnp.zeros(n)})
    return params

params = init_params(jax.random.key(1), [2, 64, 32, 2])
print([p["W"].shape for p in params])
```

‹‹ المخرجات ››

```
[(2, 64), (64, 32), (32, 2)]
```

> ⚠ **الانحيازات تبدأ أصفراً، والأوزان لا**
>
> تصفير الانحيازات آمن لأن كسر التناظر يقع في الأوزان. أما تصفير الأوزان فقرار معماري بأن كل الخلايا متطابقة.

المرور الأمامي

الشبكة ليست أكثر من حلقة: ضرب مصفوفات، جمع انحياز، دالّة تنشيط:

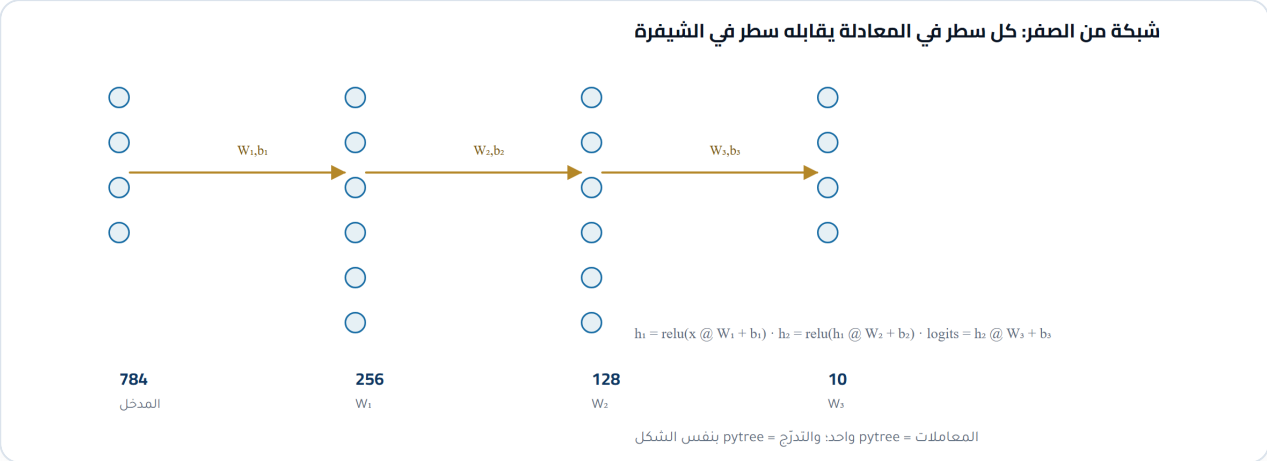
```python
def forward(params, x):
    h = x
    for layer in params[:-1]:
        h = jax.nn.relu(h @ layer["W"] + layer["b"])
    last = params[-1]
    return h @ last["W"] + last["b"]
```

في كل دورة: `(B, m) @ (m, n)` تعطي `(B, n)`، والانحياز `(n,)` يُبثّ على الدفعة. لا بُعد دفعة في المعاملات، فتعمل الدالّة على عيّنة واحدة أو على ألف.

المرحلة	شكل الخرج	المعاملات	العدد
المدخل	`(512, 2)`	–	0

المرحلة	شكل الخرج	المعاملات	العدد
مخفية 1	`(512, 64)`	`W (2,64)` ، `b (64,)`	192
مخفية 2	`(512, 32)`	`W (64,32)` ، `b (32,)`	2,080
الخرج	`(512, 2)`	`W (32,2)` ، `b (2,)`	66
المجموع	–	–	2,338

جدول 13.1 — الأشكال وعدد المعاملات في كل طبقة (دفعة من 512)



الشكل 13.1 — 2 ← 64 ← 32 ← 2. كل سهم ضرب مصفوفات، ولا شيء آخر بين المدخل والخرج.

الخسارة

نريد احتمالاً لكل صنف. `softmax` يحوّل الأرقام إلى احتمالات، لكن حسابه مباشرةً ينفجر عند القيم الكبيرة لأن e^{1000} لا يسع في `float32` . الحلّ طرح الأكبر قبل الأُسّ.

$$L = -(1/B) \sum_b [\, z_{b,y} - \log \sum_j e^{z_{b,j}} \,]$$

الصيغة نفسها مكتوبة على هيئة `logits - logsumexp(logits)` .

```python
import jax.scipy as jsp

def loss_fn(params, x, y):
    logits = forward(params, x)                       # (B, 2)
    logp = logits - jsp.special.logsumexp(logits, axis=-1, keepdims=True)
    picked = logp[jnp.arange(y.shape[0]), y]          # (B,)
    return -jnp.mean(picked)
```

> ★ **الخسارة دالّة، لا كائن**
>
> لأن `loss_fn` تأخذ المعاملات وسيطاً أول، يستطيع `jax.grad` أن يشتقّها بالنسبة إليها. هذا الشرط هو كل ما يربط «نموذجاً» بـ«محسّن» في JAX.

التدرّج

المعاملات هنا قائمة من قواميس، أي pytree. و `jax.grad` لا يهتمّ: يعيد الشجرة نفسها بأوراقها مشتقّات. لا `.backward()` ولا رسم بياني محفوظ.

```python
import optax

optimizer = optax.adamw(learning_rate=1e-2, weight_decay=1e-4)
opt_state = optimizer.init(params)

@jax.jit
def train_step(params, opt_state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state, loss

params, opt_state, l0 = train_step(params, opt_state, x, y)
print(round(float(l0), 4))
```

‹‹ المخرجات ››

```
0.6931
```

`0.6931` هو $\log 2$: خسارة نموذج يُخرج احتمالين متساويين. إن رأيت هذا الرقم بعد مئات الخطوات، فالمعاملات لا تتحرّك فعلاً.

الحلقة الكاملة

الآن كل شيء في مكانه. هذا سكربت واحد يعمل من أوّله إلى آخره:

Python

```python
import jax, jax.numpy as jnp, optax, jax.scipy as jsp

def make_moons(n, key, noise=0.1):
    k1, k2, k3 = jax.random.split(key, 3)
    a = n // 2
    t0 = jax.random.uniform(k1, (a,), minval=0.0, maxval=jnp.pi)
    t1 = jax.random.uniform(k2, (n - a,), minval=0.0, maxval=jnp.pi)
    x0 = jnp.stack([jnp.cos(t0), jnp.sin(t0)], axis=1)
    x1 = jnp.stack([1.0 - jnp.cos(t1), 0.5 - jnp.sin(t1)], axis=1)
    x = jnp.concatenate([x0, x1]) + noise * jax.random.normal(k3, (n, 2))
    y = jnp.concatenate([jnp.zeros(a), jnp.ones(n - a)]).astype(jnp.int32)
    return x, y

def init_params(key, sizes):
    params = []
    for m, n in zip(sizes[:-1], sizes[1:]):
        key, sub = jax.random.split(key)
        params.append({"W": jax.random.normal(sub, (m, n)) * jnp.sqrt(2.0 / m),
                       "b": jnp.zeros(n)})
    return params

def forward(params, x):
    for layer in params[:-1]:
        x = jax.nn.relu(x @ layer["W"] + layer["b"])
    return x @ params[-1]["W"] + params[-1]["b"]

def loss_fn(params, x, y):
    logits = forward(params, x)
    logp = logits - jsp.special.logsumexp(logits, axis=-1, keepdims=True)
    return -jnp.mean(logp[jnp.arange(y.shape[0]), y])

x, y = make_moons(512, jax.random.key(0))
params = init_params(jax.random.key(1), [2, 64, 32, 2])
opt = optax.adamw(learning_rate=1e-2, weight_decay=1e-4)
state = opt.init(params)

@jax.jit
def step(params, state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, state = opt.update(grads, state, params)
    return optax.apply_updates(params, updates), state, loss

for i in range(301):
    params, state, loss = step(params, state, x, y)
    if i % 50 == 0:
        acc = jnp.mean(jnp.argmax(forward(params, x), -1) == y)
        print(f"step {i:3d}  loss {loss:.4f}  acc {acc:.3f}")
```

‹‹ المخرجات ››

```
step   0  loss 0.6931  acc 0.500
step  50  loss 0.2418  acc 0.887
step 100  loss 0.1433  acc 0.934
step 150  loss 0.1081  acc 0.953
step 200  loss 0.0912  acc 0.961
step 250  loss 0.0823  acc 0.965
step 300  loss 0.0766  acc 0.967
```

خمسة وأربعون سطراً، وكل رقم في الخرج يعود إلى سطر تعرفه.

✓ راقب الخسارة الأولى دائماً

قبل أي تدريب، اطبع `loss_fn` على التهيئة. لتصنيف ثنائي متوازن يجب أن تكون قريبة من `0.693` ؛ رقم بعيد يعني خطأً في الأشكال أو في ترتيب المعاملات، لا في معدّل التعلّم.

لو ضاعفت عرض الطبقة المخفية فارتفعت الدقّة من `0.967` إلى `0.968` فقط، فهل المشكلة في سعة النموذج أم في البيانات؟

ما الذي تعلّمناه من الصفر

كل سطر هنا سيقابله سطر في Flax بترتيب مختلف: تُعرّف الطبقات كائنات، ويُولّد التهيئة آلياً، وتُدار الحالة في حاوية. الفكرة نفسها لا تتغيّر.

1. `init_params` ← `model.init(key, x)` : يخفي ترتيب المفاتيح وشكل كل طبقة.
2. `forward` ← `model.apply(params, x)` : يخفي تسلسل الطبقات وربط الأسماء.
3. `loss_fn` ← تبقى لك كما هي؛ لا إطار يعرف خسارتك.
4. `opt.update` ← `nnx.Optimizer` : يخفي حالة المحسّن ومزامنتها.

× خطأ شائع: الظنّ أن الإطار يعرف الخسارة

لا إطار يعرف خسارتك؛ كل ما يفعله أنه يمرّر تدرّجك إلى محسّن. من يكتب `loss_fn` سيّئةً يبقى خطؤه ظاهراً بعد ألف سطر من الطبقات.

تمرين الفصل الثالث عشر

1. غيّر المعمارية إلى طبقة واحدة بعرض 128، وقارن الدقّة وعدد المعاملات.
2. استبدل `relu` بـ `jnp.tanh`، وهل يجب تغيير معامل التهيئة؟
3. احذف `weight_decay`؛ هل تتحسّن دقّة التدريب أم الاختبار؟

الحلّ النموذجي

1. التغيير كلمة واحدة في قائمة الأحجام، والعدد يصبح `514` بدل `2338`:

```python
params = init_params(jax.random.key(1), [2, 128, 2])
print(sum(p["W"].size + p["b"].size for p in params))   # 514
```

2. نعم. `tanh` لا يصفّر نصف القيم، فمعامل He يصبح زائداً؛ استخدم `jnp.sqrt(1.0 / m)`.
3. تتحسّن دقّة التدريب وتبقى الاختبار مكانها أو تسوء – أوّل درس في الإفراط في التخصيص.

خلاصة الفصل

- التعلّم ثلاثة أسطر: أمامي، خسارة، تحديث.
- التهيئة العشوائية شرط لكسر التناظر، وتباينها يتبع عرض الطبقة.
- الخسارة على `logits - logsumexp` لا على `softmax`.
- المعاملات pytree، و `jax.grad` يعيد الشجرة مشتقّة.
- خسارة أولى قريبة من `0.693`: علامة سلام.

مصطلحات الفصل

logits

أرقام خام قبل `softmax` .

logsumexp

حساب $\log \sum e^z$ باستقرار.

He init

تهيئة بتباين $2/n$ تعوّض تصفير `relu` .

value_and_grad

دالّة تُرجع الخسارة وتدرّجها معاً.

AdamW

آدم مع انحلال وزن منفصل.

Flax: من Linen إلى NNX

فلسفة المكتبة، وأنماط النماذج، والفرق بين الجيلين

كتبت الشبكة في الفصل الماضي في خمسة وأربعين سطراً، ثم نظرت إلى مشروع حقيقي: مئات الطبقات، وأسماء معاملات، وحالة تطبيع تعتمد على الدفعة، ونقاط حفظ. لا أحد يكتب ذلك يدوياً. لكن السؤال الذي يقرّر إن كنت ستستخدم Flax بثقة أم بتقليد: هل تعرف ما الذي تضيفه فعلاً؟ الجواب أقصر ممّا تتوقّع – طبقة تُخزّن شكلها، وحاوية أسماء، وآلة مفاتيح. هذا الفصل يفكّها.

لماذا توجد Flax أصلاً؟

JAX لا تعرف ما هي «طبقة». لا `Dense` ولا نموذج. تعرف مصفوفات ودوالّ وتفريقاً ونسخاً. هذا نقص مقصود: من يضع الطبقة في النواة يفرض عليك شكل النموذج. لكن أحداً لا يريد أن يكتب `W1` و`W2` كل مرّة.

Flax ليست إطاراً بالمعنى الذي تعرفه من `torch.nn`؛ هي مكتبة وحدات مكتوبة بJAX نفسها. لا محرّك خاص ولا حلقة تدريب مخفية: النموذج دالّة تُستدعى، والمعاملات pytree تمرّها بيدك.

لو كانت Flax التفافاً حول `jax.jit` و`jax.grad`، فماذا تضيف؟ وما الذي يستحيل عمله بـ `jax.numpy` وحدها؟ ؟

Linen: النموذج كدالّة نقية

الفكرة المركزية: النموذج وصف، لا كائن يملك معاملات. تكتب صنفاً يرث `nn.Module`، وتضع طبقاته داخل `@nn.compact`، فتصبح المعاملات ناتجاً عن الاستدعاء لا مخزّنة فيه.

```python
import jax, jax.numpy as jnp
import flax.linen as nn

class MLP(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.relu(nn.Dense(64)(x))
        x = nn.relu(nn.Dense(32)(x))
        return nn.Dense(2)(x)

model = MLP()
params = model.init(jax.random.key(0), jnp.ones((1, 2)))
print(model.apply(params, jnp.ones((8, 2))).shape)
```

‹‹ المخرجات ››

```
(8, 2)
```

أسماء الطبقات تُولَّد آلياً: `Dense_0` ، `Dense_1` ، `Dense_2` . وهذا ما يجعل `apply` يعرف أي مصفوفة تخصّ أي طبقة بعد أن صارت الشجرة قواميس متداخلة.

> ★ **الشكل يُستنتج مرّة واحدة**
>
> `init` يشغّل الشبكة على مدخل وهمي ليحسب أشكال الأوزان. ولهذا يتغيّر عدد المعاملات إذا تغيّر شكل المدخل – وهذا سبب أخطاء كثيرة عند تبديل الدفعة بعيّنة واحدة.

تشريح نموذج كامل

لنطبع الشجرة. هذا أهمّ سطر تشخيصي في Flax كلها:

```python
import jax.numpy as jnp

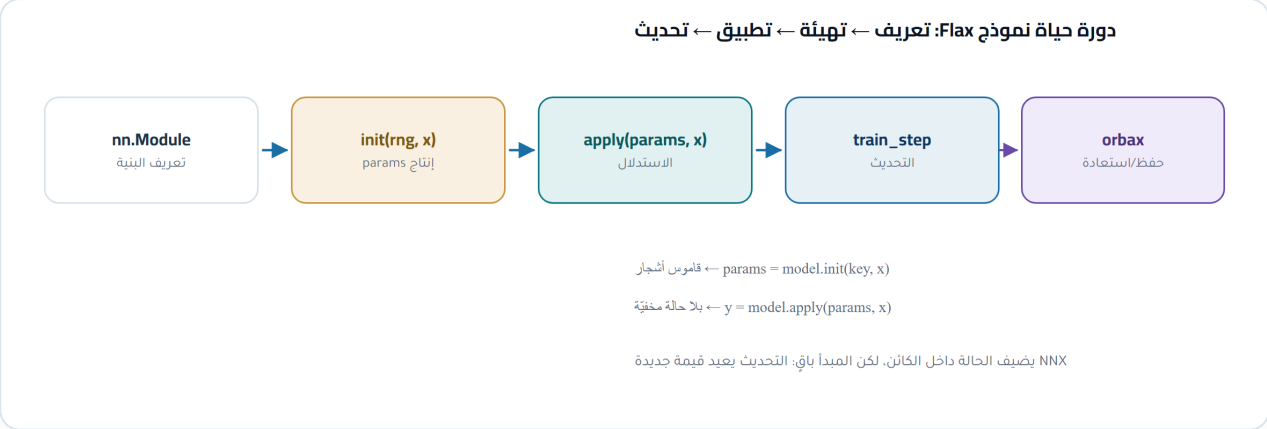
for path, leaf in jax.tree.leaves_with_path(params):
    print(path, leaf.shape)
```

‹‹ المخرجات ››

```
[('params', 'Dense_0', 'kernel')] (2, 64)
[('params', 'Dense_0', 'bias')] (64,)
[('params', 'Dense_1', 'kernel')] (64, 32)
```

```
[('params', 'Dense_1', 'bias')] (32,)
[('params', 'Dense_2', 'kernel')] (32, 2)
[('params', 'Dense_2', 'bias')] (2,)
```

ستة أوراق تحت مفتاح اسمه `params` . لاحظ أن `kernel` يأتي بشكل `(in, out)` – عكس ما اعتدته – وأن الترتيب أبجدي لا زمني.



الشكل 14.1 — المسار نفسه في الجيلين: معاملات تُولَّد، ثم تُمرَّر، ثم تُحدَّث. الفرق في من يحملها بين الاستدعاءات.

الحالة: mutableg batch_stats

بعض الحالة ليست معاملات: متوسطات التطبيع التي تتحدّث أثناء التدريب ولا يشتقّها المحسّن. Linen يحلّ هذا بحاويات: `params` واحدة منها، و `batch_stats` أخرى.

Python

```python
class Net(nn.Module):
    @nn.compact
    def __call__(self, x, train: bool):
        x = nn.Conv(32, (3, 3))(x)
        x = nn.BatchNorm(use_running_average=not train)(x)
        x = nn.relu(x)
        return nn.Dense(10)(x)

model = Net()
v = model.init(jax.random.key(0), jnp.ones((1, 28, 28, 1)), train=True)
print(sorted(v.keys()))           # ['batch_stats', 'params']

logits, new = model.apply(v, x, train=True, mutable=["batch_stats"])
v = {**v, "batch_stats": new["batch_stats"]}
```

عند التدريب تمرّر `mutable=["batch_stats"]` فتحصل على تحديثات تعيد دمجها بيدك. وعند التقييم تمرّر `train=False` بلا `mutable`، فتقرأ المتوسطات المخزّنة ولا تكتب.

> **× خطأ شائع: نسيان `deterministic=False`**
>
> طبقات مثل `Dropout` تتصرّف حسب وسيط `deterministic`. إن نسيته في التقييم، يعمل الإسقاط العشوائي أثناء الاختبار فتنخفض الدقّة بلا سبب ظاهر.

NNX: الجيل الجديد

Linen أنيقة، لكن ثمنها أن كل شيء يمرّ كوسيط: المعاملات، والإحصاءات، والمفتاح. NNX تعكس القرار: الوحدة كائن يملك حالته، وJAX ترى داخله شجرة عند الحاجة.

```python
from flax import nnx
import optax

class MLP(nnx.Module):
    def __init__(self, rngs):
        self.fc1 = nnx.Linear(2, 64, rngs=rngs)
        self.fc2 = nnx.Linear(64, 32, rngs=rngs)
        self.fc3 = nnx.Linear(32, 2, rngs=rngs)

    def __call__(self, x):
        x = nnx.relu(self.fc1(x))
        x = nnx.relu(self.fc2(x))
        return self.fc3(x)

model = MLP(nnx.Rngs(0))
optimizer = nnx.Optimizer(model, optax.adamw(1e-2))
```

لا `init` ولا `apply`: المفاتيح تأتي من `nnx.Rngs` عند البناء، والمعاملات تعيش داخل الكائن. و`nnx.Optimizer` يأخذ النموذج نفسه فيعرف أي شجرة يحدّث.

```python
@nnx.jit
def train_step(model, optimizer, x, y):
    loss, grads = nnx.value_and_grad(loss_fn)(model, x, y)
    optimizer.update(model, grads)
    return loss
```

`nnx.jit` تفصل حالة الكائن عن بنيته قبل التجميع ثم تعيد وصلها، ولهذا تستطيع أن تجمّع دالّة «تعدّل» النموذج، وهو ما لا يسمح به `jax.jit` مباشرةً.

> ✓ **لا تخلط الجيلين في ملف واحد**
>
> Linen وNNX لهما نموذجا حالة مختلفان: الأول يمرّر الحاويات، والثاني يخزّنها. الخلط ينتج أخطاء «معاملات مفقودة» لا معنى لها. اختر واحداً لكل مشروع.

Linen أم NNX؟

المعيار	Linen	NNX
النضج	أقدم وأكثر استقراراً	أحدث، ويتحرّك بسرعة
المنظومة	أمثلة ومستودعات أكثر	المسار الموصى به للجديد
التصحيح	شجرة صريحة، سهلة الفحص	كائن، يحتاج `nnx.split`
الكود	توقيعات طويلة، حالة صريحة	توقيعات قصيرة، حالة داخلية
الانتقال	إلى NNX ممكن تدريجياً	من Linen يحتاج إعادة كتابة الطبقات

جدول 14.1 — قرار الاختيار بين الجيلين

القاعدة العملية: مشروع جديد بلا قيود توافق ← NNX. مشروع قائم أو مثال يعتمد على Linen ← اِبقَ في Linen ولا تخلط.

> فريق يملك نموذجاً بـLinen يعمل، ويريد ميزة جديدة في NNX. هل ينتقل كاملاً، أم يبني النموذج الجديد في ملف منفصل؟ وما الذي يقرّر ذلك غير الرغبة؟ ؟

أخطاء Flax الشائعة

ثلاثة أخطاء تظهر في كل مشروع تقريباً، وكلها نتيجة نموذج ذهني خاطئ لا نقص خبرة.

1. **`init` مرّتين.** تستدعي `model.init` بمفتاح في دالّة التهيئة، ثم مرّة أخرى في الاختبار، فتأخذ معاملات من شجرة وتستدعي بأخرى. النتيجة: شكل صحيح وأرقام عشوائية. احفظ الشجرة في

مكان واحد.

2 **مفتاح واحد لكل الطبقات.** إن بنيت طبقتين بالمفتاح نفسه خارج `@nn.compact`، حصلت على أوزان متطابقة. `split` لكل طبقة، أو اترك `compact` يفعلها.

3 **وضع التدريب في التقييم.** `train=True` أو `deterministic=False` أثناء القياس يعطّل التطبيع ويشغّل الإسقاط.

> ⚠ **المفتاح ليس بذرة عالمية**
>
> لا يوجد في JAX مولّد حالة يتحرّك وحده. كل استدعاء بـ `key` نفسه يعطي الأرقام نفسها، ولهذا يبدو الخطأ «نموذجاً لا يتعلّم».

من الصفر إلى Flax: ما الذي تغيّر؟

لنضع حلقة الفصل الماضي جانب حلقة Flax. لا فرق في المنطق، بل في من يحمل الأشياء:

في الفصل 13	في Flax	ما الذي انتقل إليك
`init_params` يدوياً	`model.init(key, x)`	لا شيء – يُولَّد تلقائياً
حلقة `for layer in params`	`@nn.compact`	ترتيب الطبقات وأسماؤها
`loss_fn`	`loss_fn` كما هي	لا شيء – تبقى مسؤوليتك
`opt.update`	`nnx.Optimizer.update`	حالة المحسّن
`@jax.jit`	`@nnx.jit`	فصل الحالة عن البنية

جدول 14.2 — الخمسة والأربعون سطراً مقابل نظيرها في Flax

ما انتقل إليك ليس التعلّم، بل إدارة الأسماء والحالة. أما الخسارة ومعنى النجاح فتبقى قراراتك.

تمرين الفصل الرابع عشر

1. اكتب نموذج Linen بطبقتين واطبع الشجرة بـ `tree.leaves_with_path`. كم ورقة؟
2. استدعِ `init` بمفتاحين مختلفين وقارن `kernel` للطبقة الأولى.
3. أعد كتابة النموذج بـNNX، وقارن عدد أسطر البناء والتحديث.

الحلّ النموذجي

1 و2. أربع أوراق: `kernel` و `bias` لكل طبقة. والمفتاح المختلف يعطي مصفوفة مختلفة — ولهذا احفظ `params` مع النموذج الذي وُلدت منه.

3.

```python
from flax import nnx
import optax

class Small(nnx.Module):
    def __init__(self, rngs):
        self.a = nnx.Linear(2, 32, rngs=rngs)
        self.b = nnx.Linear(32, 2, rngs=rngs)

    def __call__(self, x):
        return self.b(nnx.relu(self.a(x)))

model = Small(nnx.Rngs(0))
opt = nnx.Optimizer(model, optax.adamw(1e-2))
print(sum(p.size for p in jax.tree.leaves(nnx.state(model, nnx.Param))))
```

البناء أربعة أسطر بلا `init`، والتحديث سطر واحد داخل `nnx.jit`. الثمن أن الحالة صارت داخل الكائن، فتحتاج `nnx.state` لفحصها.

خلاصة الفصل

- Flax مكتبة وحدات فوق JAX، لا محرّكاً بديلاً.
- في Linen: `init` يبني الشجرة، و `apply` يستهلكها.
- الحالة غير القابلة للتدريب تُمرَّر عبر `mutable` وتُعاد إليك.
- NNX يجعل الوحدة كائناً يحمل حالته، و `nnx.jit` يفصلها عند التجميع.
- لا تخلط الجيلين ولا تستدعِ `init` مرّتين.

مصطلحات الفصل

Module

صنف يصف طبقات ويُنتج معاملات عند الاستدعاء، لا عند البناء.

@nn.compact

مُزيّن يسمح بتعريف الطبقات داخل `__call__` وتسميتها آلياً.

collection

حاوية داخل شجرة الحالة: `params` أو `batch_stats` .

mutable

وسيط يخبر `apply` أي الحاويات يُسمح لها بالتغيّر وإرجاع تحديثها.

NNX

الجيل الجديد: وحدات كائنات تحمل حالتها، مع `nnx.jit` .

params

حاوية المعاملات القابلة للتدريب داخل شجرة الحالة.

Optax: المحسّنات كتحويلات

التدرّج، والزخم، وآدم، والجدولة، والقصّ، والتركيب

سأل مهندس في فريق رؤية حاسوبية زميله: «أين أضع معدّل تعلّم مختلفاً للطبقة الأخيرة؟» فأجابه: «عدّل صنف المحسّن». بعد أسبوع كان الصنف أربعمئة سطر، وفيه نسختان من Adam، وواحدة لا تعمل بعد التجميع. السبب لم يكن الشيفرة: المحسّن سلسلة دوالّ نقية، لا كائن يملك حالة.

لماذا المحسّن ليس كائناً؟

في مكتبة تدريب تقليدية يحمل المحسّن مراجع إلى المعاملات، و `step()` يكتب فوقها. في JAX المصفوفة ثابتة (الفصل الثالث)، والدالّة المجمّعة لا تسمح بأثر جانبي (الفصل السادس)؛ فلو احتفظ المحسّن بـ `params` داخله، لجمّد `jit` تلك المراجع.

الحلّ إزاحة الملكية: المحسّن يملك **ذاكرة داخلية** فقط – عدّاداً ومتوسّطات – وهي شجرة تمرّ إلى الدالّة وتعود.

محسّن يحتفظ بمؤشّر إلى المعاملات يجيب فوراً: «ما قيمتها الآن؟». محسّن Optax لا يستطيع. ما الذي يخسره المهندس بهذا الحرمان؟ ؟

GradientTransformation: العقد الثلاثي

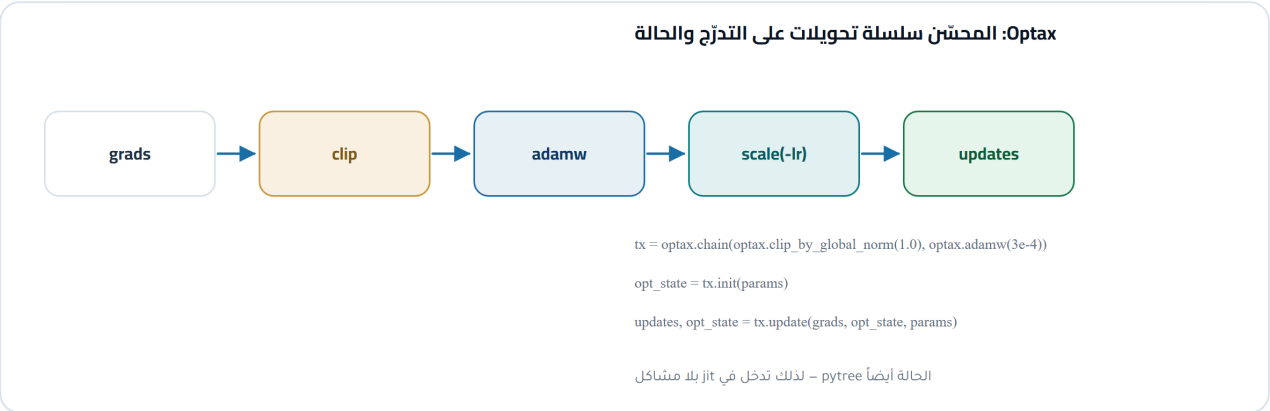
كل محسّن زوج من الدوالّ: `init` تبني الحالة من الأشكال والأنواع فقط، و `update` تحوّل التدرّج إلى تحديث. توقيعها `update(grads, state, params=None)`، وترتيب الوسائط ليس شكلياً كما سنرى في القسم السابع. ولماذا `params`؟ لأن **weight decay** يطرح جزءاً من الوزن لا من تدرّجه.

```python
import jax, jax.numpy as jnp, optax

params = {"w": jnp.ones((4, 3)), "b": jnp.zeros((3,))}
tx = optax.sgd(learning_rate=0.1, momentum=0.9)

state = tx.init(params)                 # أصفار بأشكالها
print(state[1].trace["w"].shape)        # (4, 3)

grads = jax.tree.map(jnp.ones_like, params)
updates, state = tx.update(grads, state, params)
params = optax.apply_updates(params, updates)
print(params["w"][0])                   # [0.9 0.9 0.9]
```



الشكل 15.1 — التدرّج يدخل من اليسار ويخرج تحديث؛ والحالة تمرّ أسفل السلسلة.

> ★ **المحسّن دالّة، والحالة وسيطها**
>
> لا مكان مخفي للأرقام: إن لم تُمرّر `opt_state` إلى الخطوة التالية، فقدت الزخم وعدّاد الجدولة بلا تحذير.

من SGD إلى AdamW

الزخم يضيف ذاكرة: يتحرّك بمتوسّط متحرّك من التدرّج، فتمرّ الاتجاهات المتّسقة ويُخمد الاهتزاز.

$$u_t = \mu \cdot u_{t-1} + g_t \quad , \quad \theta_{t+1} = \theta_t - \eta \cdot u_t$$

$\mu = 0.9$ يعني أن الخطوة الحالية متوسط آخر عشر خطوات تقريباً.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad , \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

لحظتان: الأولى للاتجاه، والثانية لحجم التذبذب.

آدم يضيف متوسّطاً للتربيع، فيصير لكل معامل معدّل تعلّم خاص. في الخطوة الأولى $m_1 = 0.1 g_1$ و $v_1 = 0.001 g_1^2$؛ فالقسمة على $\sqrt{v}$ تُنتج خطوة أصغر من اللازم. لهذا يوجد تصحيح الانحياز، ثم يُفصل الاضمحلال – وهو فرق AdamW:

$$\theta_{t+1} = \theta_t - \eta \cdot [\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \cdot \theta_t \,]$$

$\hat{m}$ و$\hat{v}$ هما اللحظتان مقسومتين على $1 - \beta^t$؛ وحدّ $\lambda\theta$ يُطرح من الوزن.

```python
tx = optax.adamw(learning_rate=3e-4, b1=0.9, b2=0.999, eps=1e-8, weight_decay=0.1)
# المكافئ اليدوي، للفهم لا للاستخدام:
# tx = optax.chain(optax.scale_by_adam(), optax.add_decayed_weights(0.1),
#                  optax.scale_by_learning_rate(3e-4))
```

✓ متى تختار SGD أصلاً؟

في الرؤية الصغيرة يعطي SGD بالزخم تعميماً أفضل مقابل ضبط دقيق؛ وفي المحوّلات آدمW هو الافتراضي.

الجدولة: تغيير معدّل التعلّم بلا إعادة بناء

الجدولة **دالّة من العدّاد**: `schedule(count)`. العدّاد يعيش في `opt_state` وتزيده مرحلة `scale_by_learning_rate`. فالقيمة تُحسب داخل الرسم البياني بلا إعادة تجميع.

```python
warm = optax.warmup_cosine_decay_schedule(
    init_value=0.0, peak_value=3e-4, warmup_steps=500,
    decay_steps=20_000, end_value=1e-5)
cos = optax.cosine_decay_schedule(init_value=1e-3, decay_steps=10_000, alpha=0.0)
exp = optax.exponential_decay(init_value=1e-2, transition_steps=1_000, decay_rate=0.5)

for step in (0, 250, 500, 5_000):
    print(step, float(warm(step)))
```

الاحماء ليس ترفاً: في أول مئات الخطوات تكون لحظات آدم غير مستقرّة، فقفزة واحدة تُفسد التمثيلات.

ولتغيير المعدّل بقرار خارجي يستخدم `inject_hyperparams`:

```python
tx = optax.inject_hyperparams(optax.adamw)(learning_rate=1e-3, weight_decay=0.1)
state = tx.init(params)
state = state._replace(
    hyperparams={**state.hyperparams, "learning_rate": jnp.asarray(2e-4)})
```

> ⚠ **العدّاد لا يعود إلى الصفر**
>
> إن أعدت بناء المحسّن لتغيير المعدّل، عاد العدّاد إلى صفر: الجدولة تبدأ من جديد، والزخم يُمحى.

القصّ والتطبيع

تحدث الانفجارات: دفعة شاذّة، أو تدرّج قيمته 10^4 حيث يُتوقّع 1. القصّ يضع سقفاً على الطول ويحفظ الاتجاه:

$$g \leftarrow g \cdot \min(1, c \,/\, \|g\|_2)$$

إن كان الطول أقل من c لا يتغيّر شيء؛ وإن تجاوزه يُعاد التحجيم وحده.

```python
tx = optax.chain(
    optax.clip_by_global_norm(max_norm=1.0),
    optax.adamw(learning_rate=3e-4, weight_decay=0.1),
)
# بديل تكيّفي: يقيّد نسبة التحديث إلى حجم المعامل نفسه
tx_agc = optax.chain(
    optax.adaptive_grad_clip(clipping=0.01, eps=1e-3),
    optax.sgd(learning_rate=0.1, momentum=0.9),
)
```

`adaptive_grad_clip` يسأل: «هل التدرّج كبير **بالنسبة إلى الوزن**؟». القيد $\|g\| / \max(\|\theta\|, \varepsilon) \le c$ يمنع أن يبتلع تحديثٌ واحد وزناً صغيراً كان يعمل.

> **× القصّ في المكان الخطأ**
>
> `clip_by_global_norm` بعد `adamw` لا يمنع الانفجار: لحظات آدم تكون قد تلوّثت. القاعدة: القصّ أوّل السلسلة، دائماً.

التركيب: multi_transformg chain

`chain` للتسلسل، و `multi_transform` لتفرّع الشجرة على مسارين.

```python
def label(p):
    return jax.tree.map(lambda x: "decay" if x.ndim > 1 else "no_decay", p)

tx = optax.multi_transform(
    {"decay": optax.adamw(1e-3, weight_decay=0.1),
     "no_decay": optax.adamw(1e-3, weight_decay=0.0)},
    label)

state = tx.init(params)      # حالة لكل فرع، مدمجة في شجرة واحدة
```

الحالة	السلسلة	لماذا
CNN صغيرة	`clip(1.0) → sgd(0.1, 0.9) → cosine`	تعميم أفضل
محوّل / LLM	`clip(1.0) → adamw(3e-4, wd=0.1) → warmup_cosine`	الافتراضي المستقرّ

الحالة	السلسلة	لماذا
ضبط دقيق	`adamw(2e-5, wd=0.01)`	يحفظ التمثيلات
مضمّنات	`multi_transform` مع `wd=0`	التنظيم يُضعفها

جدول 15.1 — وصفات جاهزة، والسبب وراء كل خيار

وضعت `weight_decay=0.1` على كل الشجرة، ثم لاحظت أن الانحيازات تنكمش نحو الصفر. ما التفسير، وهل يضرّ؟ ؟

أخطاء Optax الشائعة

النسيان الكامل: استدعاء `update` ثم عدم تمرير التحديث إلى المعاملات؛ فلا تتغيّر الخسارة بينما تتقدّم الحالة، وتبدو الأرقام منطقية. و**إعادة التهيئة كل خطوة:** `state = tx.init(params)` داخل الحلقة يعني زخماً وعدّاداً صفريين كل مرّة، فيتحوّل آدم إلى نزول بسيط.

ترتيب الوسائط: `update(grads, state, params)` لا `update(params, grads, state)`؛ والخطأ لا يفشل دائماً: إن تطابقت الأشكال مرّ بصمت. و**الخلط بين weight decay وL2:** إضافة $\lambda\|\theta\|^2/2$ إلى الخسارة تدخل في التدرّج فتتضخّم عبر اللحظتين، والاضمحلال المفصول يُطرح من الوزن (الفصل السابع عشر).

⚠ لا تُجمّع `tx.init` مع الخطوة

`init` تُستدعى مرّة خارج `jit`؛ وضعها داخل دالّة مجمّعة يجعل إنشاء الأصفار جزءاً من الرسم.

تمرين الفصل الخامس عشر

1. شغّل ثلاث خطوات بـ `optax.sgd(0.1, momentum=0.9)` على معامل قيمته 1 وتدرّجه 1. احسب يدوياً ثم تحقّق.

2. ابنِ سلسلة تقصّ عند 0.5 ثم `adamw`، وأثبت أن تحديث تدرّج طوله 10 يعادل تحديث تدرّج طوله 0.5.

3. استخدم `multi_transform` لتعطي الأوزان $\lambda = 0.1$ والانحيازات $\lambda = 0$، وتحقّق من أن انحيازاً غير صفري لا يتغيّر بتدرّج صفري.

الحلّ النموذجي

1. الزخم يتراكم: $u = 1, 1.9, 2.71$، والمعامل $0.9, 0.71, 0.439$. و**2.** القصّ يحفظ الاتجاه، فالسلسلة تساوي تحديثاً أُدخل إليه $g/\|g\| \cdot 0.5$. و**3.** بلا تنظيم على الانحياز يبقى كما هو.

```python
p = {"w": jnp.array([1.0])}
tx = optax.sgd(0.1, momentum=0.9)
st, g = tx.init(p), {"w": jnp.array([1.0])}
for i in range(3):
    upd, st = tx.update(g, st, p)
    p = optax.apply_updates(p, upd)
    print(i + 1, p["w"])        # 0.9, 0.71, 0.439

cl = optax.chain(optax.clip_by_global_norm(0.5), optax.adamw(1e-3))
big = {"w": jnp.ones(10) * jnp.sqrt(10.0)}          # طوله 10
u1, _ = cl.update(big, cl.init(p), p)
u2, _ = cl.update(jax.tree.map(lambda x: x / 20.0, big), cl.init(p), p)
print(jnp.allclose(u1["w"], u2["w"], atol=1e-6))     # True

params = {"w": jnp.ones((4, 3)), "b": jnp.full((3,), 0.5)}
mt = optax.multi_transform({"decay": optax.adamw(1e-2, weight_decay=0.1),
                            "no_decay": optax.adamw(1e-2, weight_decay=0.0)},
                           {"w": "decay", "b": "no_decay"})
upd, _ = mt.update(jax.tree.map(jnp.zeros_like, params), mt.init(params), params)
print(optax.apply_updates(params, upd)["b"])         # [0.5 0.5 0.5]
```

خلاصة الفصل

- المحسّن زوج `init` / `update` ، وحالته شجرة لا كائن مخفي.
- آدم يضيف متوسّطاً للتربيع وتصحيح انحياز، وAdamW يفصل الاضمحلال.
- الجدولة دالّة من العدّاد؛ لا تُعِد البناء فتفقد الزخم.
- القصّ أوّل السلسلة، و `chain` و `multi_transform` للتركيب.

مصطلحات الفصل

GradientTransformation

زوج `(init, update)` يمثّل محسّناً بلا حالة مخفية.

opt_state

شجرة العدّاد واللحظات؛ تمرّ صراحةً بين الخطوات.

bias correction

قسمة اللحظات على $1-\beta^t$ لإزالة انحياز البداية.

decoupled weight decay

طرح $\lambda\theta$ من الوزن بدل إضافته إلى التدرّج.

multi_transform

تحويل مختلف لكل فرع من الشجرة.

حلقة تدريب احترافية

الخطوة، والقياس، ونقاط الحفظ، وEMA، والاستئناف

بعد ثماني ساعات من التدريب، انقطع التيار. عاد الجهاز، وشُغّلت الحلقة من جديد، فبدأت الخسارة من حيث بدأت صباحاً: المعاملات حُفظت، لكن الزخم لم يُحفظ، والعدّاد عاد إلى الصفر، فصار المحسّن يتصرّف كأنه في الخطوة الأولى. هذا الفصل عن تلك الحلقة التي تبدو عشرين سطراً، ثم تكتشف بعد شهر أنها كانت تحتاج خمس قطع من الحالة، وثلاث قواعد للقياس، وطريقة صحيحة للحفظ.

الحلقة ليست for

يكتب المبتدئ حلقة من نوع `for batch in loader: params = step(params, batch)`، فتنجح في التجربة وتفشل في الإنتاج. السبب أن التدريب ليس دالّة من المعاملات وحدها؛ هو دالّة من **خمس قطع** تتغيّر معاً، وأي قطعة تُنسى تُنتج تدريباً مختلفاً لا خطأً برمجياً.

القطعة	ما هي	إن أُسقطت
`params`	شجرة الأوزان	لا تدريب أصلاً
`opt_state`	الزخم واللحظات والعدّاد	المحسّن ينسى، والجدولة تعود للصفر
`step`	عدّاد صحيح	الجدولة والسجلات تنحرف
`rng`	مفتاح العشوائية	الاستئناف يعيد عيّنات مختلفة
`metrics`	مجاميع القياس	لا تعرف أين توقّفت

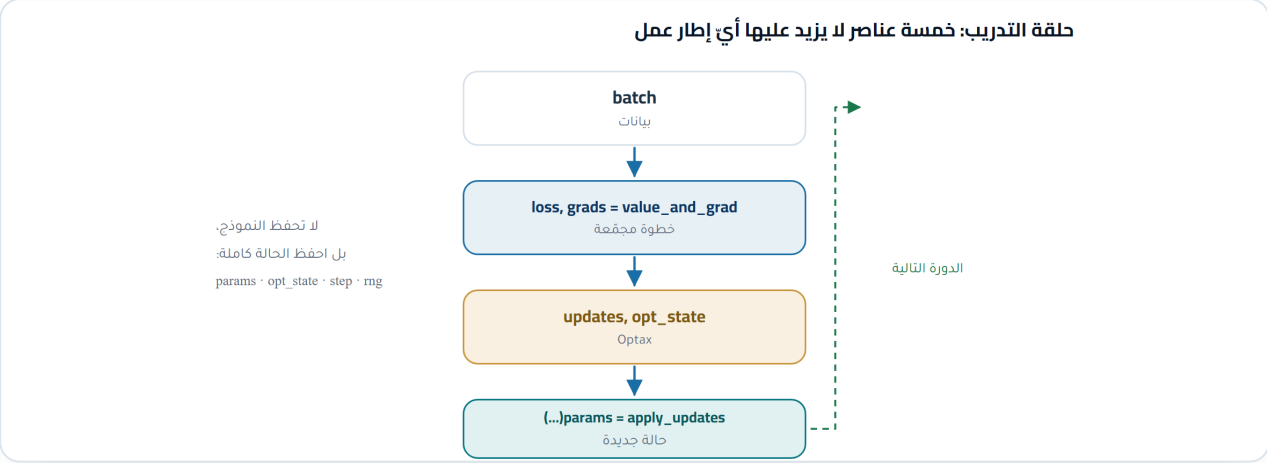
جدول 16.1 — القطع الخمس، وما يحدث إن أُسقطت

لاحظ أن أربعاً من الخمس ليست أوزاناً. ولهذا يصعب تصديق أن الحفظ «يكفي فيه `params`»: ما يُحفظ هو **الحالة**، لا النموذج.

لو استأنفت التدريب بمعاملات محفوظة وحدها، فما الذي سيتغيّر في المنحنى خلال أول مئة خطوة؟ وهل يظهر الفرق في الخسارة النهائية أم في شكل المنحنى؟ ؟

train_step كدالّة نقية

الشكل الذي يصمد هو `NamedTuple` يحمل القطع الخمس، ودالّة تُرجع `(state, metrics)` . لماذا `NamedTuple` لا قاموس؟ لأنه شجرة أيضاً، لكن حقوله تُقرأ بالاسم لا بالمفتاح، فيظهر الخطأ عند الكتابة لا عند التشغيل.



الشكل 16.1 — الحالة تدخل الخطوة وتخرج جديدة، والقياس يخرج جانباً بلا أثر على المسار.

القياس بلا إبطاء

الخطأ الأول في القياس هو `float(loss)` داخل الحلقة: كل استدعاء يوقف الجهاز وينتظر النتيجة، فيتحوّل التدريب من تنفيذ غير متزامن إلى سلسلة انتظارات. الخطأ الثاني هو جمع القياسات في قائمة بايثون، فتنمو الذاكرة مع الخطوات.

الحلّ: أبقِ القياس مصفوفة، وحرّك المتوسّط داخل الحلقة، واطبع مرّة كل N خطوة. حينها فقط تدفع ثمن الانتظار، مرّة كل مئة خطوة بدل مرّة كل خطوة.

```python
import jax, jax.numpy as jnp

# لا: متوسط متحرّك داخل الحلقة ولا قوائم .item()
running = {"loss": jnp.zeros(()), "acc": jnp.zeros(())}

def accumulate(running, metrics, decay=0.98):
    return jax.tree.map(lambda r, m: decay * r + (1 - decay) * m, running, metrics)

# الطباعة عند حدود مضاعفات 100 فقط
if step % 100 == 0:
    print(step, {k: float(v) for k, v in running.items()})
```

> ✓ **معدّل السجلّ ليس تفصيلاً**
>
> اطبع كل 50 إلى 200 خطوة. كل خطوة تعني مزامنة كاملة مع الجهاز؛ وكل 1000 خطوة تعني أنك ستكتشف الانفجار بعد فوات الأوان.

نقاط الحفظ مع Orbax

لماذا لا يكفي `pickle` أو `np.save` ؟ لأن مصفوفات JAX قد تكون **مقطّعة** على أجهزة عدّة: كل جهاز يحمل شريحة، ولا وجود لنسخة كاملة في ذاكرة واحدة. `pickle` يحاول جمعها في كائن واحد فيفشل أو ينسخها إلى المعالج. Orbax تحفظ كل شريحة في ملفها، وتعيد تركيبها عند التحميل.

```python
import orbax.checkpoint as ocp

ckpt = ocp.PyTreeCheckpointer()
options = ocp.CheckpointManagerOptions(max_to_keep=3, create=True)
mngr = ocp.CheckpointManager("artifacts/run01", ckpt, options=options)

mngr.save(step, state)              # الحالة كلها: معاملات ومحسّن ومفتاح
print(mngr.latest_step())           # آخر خطوة محفوظة

restored = mngr.restore(mngr.latest_step(), items=state)
```

`max_to_keep=3` يحفظ الثلاثة الأخيرة ويحذف الأقدم. لكن «الأخير» ليس دائماً «الأفضل»: إن كان النموذج يتدهور، فأنت تحفظ أسوأ ثلاث نسخ. لهذا يُحفظ أيضاً أفضل نقطة بحسب مقياس التحقّق، ولو خارج المدير.

> ⚠ **لا تحفظ داخل `jit`**
>
> الحفظ أثر جانبي: كتابة ملف داخل دالّة مجمّعة تحدث مرّة عند التتبّع. احفظ خارج الخطوة، بعد أن تُرجع القيم.

EMA: نسخة بطيئة تتعلّم أسرع

المعاملات في نهاية التدريب تهتزّ حول الحلّ؛ فآخر دفعة تُزيحها قليلاً. المتوسّط المتحرك الأسّي يحتفظ بنسخة تنعمت عبر آلاف الخطوات، فتعطي خسارة تحقّق أقل من النسخة الخام غالباً.

$$\bar{\theta}_t = \lambda \cdot \bar{\theta}_{t-1} + (1 - \lambda) \cdot \theta_t$$

$\lambda = 0.999$ يعني ذاكرة تقارب ألف خطوة؛ و$\lambda = 0.99$ تعني مئة.

```python
def ema_update(ema, params, decay=0.999):
    """نسخة بطيئة من المعاملات، تُحدَّث بعد كل خطوة."""
    return jax.tree.map(lambda e, p: decay * e + (1.0 - decay) * p, ema, params)

# إن أردت الحالة وإدارة الانحياز، البديل الجاهز في Optax:
# ema_tx = optax.ema(0.999)
# ema_params, ema_state = ema_tx.update(params, ema_state)
```

> ★ **EMA ليست تحسيناً، بل نسخة**
>
> لا تستخدم نسخة EMA لمتابعة التدريب؛ فهي متأخّرة عن المسار. استخدمها للتحقّق والنشر فقط، واحفظها في نقطة الحفظ كي لا تبدأ من الصفر بعد الاستئناف.

الاستئناف بعد الانقطاع

الاستئناف الصحيح يعني أن النتيجة بعد الانقطاع مطابقة لما كانت ستكون بلا انقطاع. وهذا ممكن فقط إن حُفظت القطع الخمس معاً، واستُعيدت قبل أول خطوة، ولم يُستهلك المفتاح مرّتين.

```python
state = mngr.restore(step, items=state)
state = state._replace(rng=jax.random.key(seed))    # لا: يهدم التسلسل
# الصحيح: لا تلمس rng إن كان محفوظاً في الحالة
state = mngr.restore(step, items=state)
loader = loader.skip(state.step * batch_size)       # نفس موضع البيانات
```

> ✕ **«استأنفتُ» بمعاملات فقط**
>
> سترى الخسارة تقفز ثم تعود، وقد تظنّ أن الحفظ تالف. الحقيقة أن الزخم صار صفراً والجدولة عادت للخطوة صفر؛ فالمنحنى ليس استئنافاً بل بداية جديدة بمعاملات مدربة.

الجدولة والمراقبة

افصل ثلاث دورات: الخطوة (كل دفعة)، والسجلّ (كل مئة)، والتحقّق (كل ألف أو كل حقبة). الخلط بينها هو ما يجعل الحلقة بطيئة: تقييم على مجموعة التحقّق كل خطوة يكلّف أكثر من التدريب نفسه.

وللتوقّف المبكر قاعدة واحدة: راقب **التحقّق** لا الاختبار. مجموعة الاختبار تُستخدم مرّة واحدة في النهاية؛ كل استخدام لها في القرار يجعل قرارك جزءاً من التقييم، فتفقد المقياس الذي تحكم به.

قائمة تحقّق قبل تشغيل 8 ساعات

1. خطوة واحدة تعمل وتُطبع أشكالها: خسارة، وتدرّج، ومعامل بعد التحديث.
2. عشر خطوات تنتهي بلا `NaN`، والخسارة تنخفض ولو قليلاً.
3. حفظ ثم استعادة ثم مقارنة: هل المعاملات متطابقة تماماً؟
4. استئناف من نقطة وسطية: هل يعود المنحنى إلى مساره؟
5. قياس زمن الخطوة، وحساب الزمن الكلي قبل الإطلاق.
6. مساحة القرص: حجم النقطة × `max_to_keep` أقل من المتاح.

7 معدّل السجلّ مضبوط، وكل `float()` خارج المسار الساخن.

8 مسار الاختبار غير ملموس، وقرار التوقّف من التحقّق وحده.

الحلقة الكاملة، بالشكل الذي يجمع كل ما سبق، تبدو هكذا. لاحظ الطول: خمسة وثلاثون سطراً، معظمها تصريح بما يجب أن يُحفظ.

Python

```python
import functools, jax, jax.numpy as jnp, optax
import orbax.checkpoint as ocp
from typing import Any, NamedTuple

class TrainState(NamedTuple):
    params: Any          # شجرة أوزان النموذج
    opt_state: Any       # حالة المحسّن: زخم ولحظات وعدّاد
    step: jax.Array      # () int32
    rng: jax.Array       # مفتاح dropout
    ema: Any             # نسخة بطيئة بنفس شكل params

@functools.partial(jax.jit, donate_argnums=(0,))
def train_step(state, batch):
    def loss_fn(params):
        logits = model.apply({"params": params}, batch["x"], train=True,
                             rngs={"dropout": jax.random.split(state.rng)[1]})
        logp = jax.nn.log_softmax(logits)
        loss = -jnp.mean(jnp.take_along_axis(logp, batch["y"][:, None], axis=1))
        return loss, logits

    (loss, logits), grads = jax.value_and_grad(loss_fn, has_aux=True)(state.params)
    updates, opt_state = tx.update(grads, state.opt_state, state.params)
    params = optax.apply_updates(state.params, updates)
    ema = jax.tree.map(lambda e, p: 0.999 * e + 0.001 * p, state.ema, params)
    metrics = {"loss": loss, "acc": jnp.mean(jnp.argmax(logits, -1) == batch["y"])}
    new_state = state._replace(params=params, opt_state=opt_state,
                               step=state.step + 1, rng=jax.random.split(state.rng)[0],
                               ema=ema)
    return new_state, metrics

mngr = ocp.CheckpointManager("artifacts/run01", ocp.PyTreeCheckpointer(),
                             options=ocp.CheckpointManagerOptions(max_to_keep=3, create=True))
for epoch in range(num_epochs):
    for batch in loader:
        state, metrics = train_step(state, batch)
        running = jax.tree.map(lambda r, m: 0.98 * r + 0.02 * m, running, metrics)
        if int(state.step) % 100 == 0:
            print(int(state.step), {k: round(float(v), 4) for k, v in running.items()})
    mngr.save(int(state.step), state)
```

★ الحالة هي وحدة الحفظ والاستئناف

كل ما تحتاجه للعودة إلى اللحظة نفسها يعيش في `TrainState`. إن كان شيء مؤثّر في التدريب وليس فيه، فأنت لا تستطيع الاستئناف، ولو بدا كل شيء يعمل.

لو أضفت حقلاً جديداً إلى الحالة بعد أسبوع من التدريب، فماذا يحدث عند استعادة نقطة قديمة؟ وكيف تتجنّب أن تكتشف ذلك بعد ساعتين من التشغيل؟ ؟

تمرين الفصل السادس عشر

1. اكتب `train_step` يعيد `(state, metrics)` ويتحقّق من أن `state.step` يزيد واحداً وأن شكل `params` لا يتغيّر.
2. احفظ الحالة بعد خمس خطوات، ثم استعدها في عملية جديدة، وقارن كل ورقة في الشجرة بـ `jax.tree.map` . هل الفرق صفر؟
3. أضف EMA واختبر خسارة التحقّق للنسختين: الخام والمتوسّطة، بعد مئة خطوة.

▲ الحلّ النموذجي

1. الخطوة تُرجع حالة جديدة لا تُعدّل القديمة، والعدّاد مصفوفة لا عدد بايثون:

```python
def step_fn(state, batch):
    loss, grads = jax.value_and_grad(loss_fn)(state.params, batch)
    updates, opt_state = tx.update(grads, state.opt_state, state.params)
    params = optax.apply_updates(state.params, updates)
    new = state._replace(params=params, opt_state=opt_state, step=state.step + 1)
    return new, {"loss": loss}

s0 = TrainState(params, tx.init(params), jnp.zeros((), jnp.int32),
                jax.random.key(0), params)
s1, m = jax.jit(step_fn)(s0, batch)
assert int(s1.step) == 1
assert jax.tree.all(jax.tree.map(lambda a, b: a.shape == b.shape, s0.params, s1.para
```

2. المقارنة تكون على الأوراق، وبتسامح صفر لأن الحفظ لا يغيّر القيم:

```python
ckpt = ocp.PyTreeCheckpointer()
ckpt.save("/tmp/pt", s1)
back = ckpt.restore("/tmp/pt", item=s1)
same = jax.tree.all(jax.tree.map(lambda a, b: jnp.array_equal(a, b), s1, back))
print(same)     # True
```

3. EMA تُحدَّث بعد التحديث لا قبله؛ ولو حدّثتها بالتدرّج لكانت نسخة من التدرّج لا من المعاملات. قارن الخسارة على مجموعة التحقّق بعد مئة خطوة: الفرق عادةً في الخانة الثالثة، ويصبح أوضح مع تدريب أطول ومعدّل تعلّم أعلى.

خلاصة الفصل

- الحالة خمس قطع: معاملات، ومحسّن، وعدّاد، ومفتاح، وقياس – والحفظ يشملها كلها.
- الخطوة دالّة نقية تُرجع `(state, metrics)`، والقياس يبقى مصفوفة.
- Orbax يحفظ الأشجار المقطّعة؛ و `max_to_keep` يحفظ الأخير لا الأفضل.
- EMA نسخة للتحقّق والنشر، لا للمسار.
- الاستئناف الصحيح يعيد المنحنى إلى مساره بالضبط، لا إلى نقطة قريبة.

مصطلحات الفصل

TrainState

شجرة واحدة تجمع كل ما يتغيّر في التدريب، فتصير وحدة الحفظ والاستئناف.

running mean

متوسّط متحرّك للقياس يُحدَّث داخل الحلقة بلا مزامنة مع الجهاز.

CheckpointManager

مدير يحفظ نقاطاً مرقّمة بالخطوة ويحذف الأقدم وفق `max_to_keep`.

EMA

متوسّط أسّي للمعاملات يعطي نسخة أنعم من النسخة الجارية.

resume (استئناف)

إعادة بناء الحالة كاملة من قرص بحيث يستمرّ المسار كما لو لم ينقطع.

التنظيم والتعميم

Dropout، وweight decay، والتطبيع، وMixup، وتعزيز البيانات

وقف مهندس أمام منحنى الخسارة على بيانات التدريب ينخفض بسلاسة، وبجانبه منحنى الخسارة على بيانات لم يرها النموذج قطّ يتذبذب صعوداً. سأله زميله: «لماذا يتعلّم النموذج الشكل ويفشل في العموم؟». الإجابة ليست في عدد الطبقات، بل في أن النموذج حفظ ضجيج العيّنات بدل أن يلتقط القاعدة. هذه الظاهرة – الإفراط في التطابق – هي العدوّ الصامت للتعلّم العميق؛ والتنظيم هو الأدوات التي تمنع النموذج من الحفظ بلا فهم. في هذا الفصل نتناول خمس أدوات: إسقاط الوحدات، والاضمحلال المفصول، والتطبيع، وخلط العيّنات، وتعزيز البيانات. لا نشرحها كوصفات، بل كإجابة على سؤال واحد: كيف تُجبر دالّة معقّدة على أن تكون بسيطةً حيث يجب؟

لماذا يتعلّم النموذج الضجيج؟

لو كان لديك عشرة أعداد وحيدة المدخل ومخرج يساوي ضعفها زائد ضجيجاً، فإن أبسط خطّ مستقيم يكاد يصلح. لكن شبكة بألف معامل تستطيع أن تمرّ تماماً من كل نقطة تدريب، فتصير الخسارة صفراً بينما تُصبح على بيانات جديدة أسوأ. القدرة التعبيرية العالية سلاحٌ ذو حدّين: كلما زادت حرّية النموذج زاد خطر أن يضبط على الضجيج.

التنظيم لا يقلّل عدد المعاملات؛ بل يفرض على النموذج «تكلفة» لكل درجة حرّية، فيفضّل الحلول الملساء. لاحظ أننا لن نعيد شرح `adamw` والاضمحلال المفصول بالتفصيل؛ رأيناهما في الفصل الخامس عشر، وهنا نركّز على موضعهما ضمن منظومة التنظيم.

شبكة صغيرة وشبكة كبيرة تدربتا على نفس العيّنات؛ الصغيرة عمّمت بشكل أفضل. هل يعني هذا أن «الصغر» تنظيمٌ بحدّ ذاته؟ وما الذي يمنع النموذج الكبير من تقليد سلوك الصغير؟ ؟

إسقاط الوحدات: قناع عشوائي لا متغيّر

أشهر أداة هي `dropout`: في كل خطوة تدريب يُعطّل جزء من المخرجات باحتمال p، فلا يمرّر النموذج إشارتها. الفكرة أن الخلايا لا تستطيع أن «تعتمد» على خلية معيّنة، فتتوزّع التمثيل على مسارات كثيرة.

رياضياً:

$$y = m \odot x \quad , \quad m_i \sim \text{Bernoulli}(1-p)$$

القناع m مستقل لكل عنصر، ويُضرب بالعنصر في كل تمريرة تدريب فقط.

المشكلة أن إسقاط p من المخرجات يخفض متوسّط قيمتها، فنضرب الناتج بـ $1/(1-p)$ أثناء التدريب (التطبيع بالإنجاز). أثناء الاختبار لا إسقاط، فيُعوّض بمعامل التطبيع هذا. في JAX يدير `jax.random` القناع، لأن العشوائية يجب أن تكون صريحة لا مخفيّة.

```python
import jax, jax.numpy as jnp
from jax import random

def dropout(x, key, p=0.1, train=True):
    if not train or p == 0.0:
        return x
    mask = random.bernoulli(key, 1.0 - p, shape=x.shape)
    return x * mask / (1.0 - p)        # تطبيع بالإنجاز

k = random.key(0)
x = jnp.ones((4, 8))
out = dropout(x, random.split(k)[0], p=0.5, train=True)
print(float(out.sum()), x.size)        # يقلّ المجموع إحصائياً
```

> ★ **العشوائية يجب أن تُمرَّر صراحة**
>
> في JAX لا يوجد مولّد عشوائي عامّ مخفي. إن نسيت تمرير `key` إلى `dropout` داخل دالّة مجمّعة، فإمّا أن يفشل التتبّع وإمّا أن يثبت القناع. استخدم `random.split` أو `random.fold_in` لكل استدعاء.

الاضمحلال المفصول: اطرح من الوزن لا من التدرّج

الوزن الكبير يعني استجابة حادّة تجاه دخل صغير؛ فكبحه يجعل النموذج أكثر ملساءً. هناك طريقتان متعارضتان: إضافة $\lambda\|\theta\|^2/2$ إلى الخسارة (L2)، أو طرح $\lambda\theta$ من الوزن مباشرةً (الاضمحلال المفصول). الفرق ليس شكلياً:

$$\theta_{t+1} = \theta_t - \eta \cdot (g_t + \lambda\, \theta_t) \quad \leftarrow \text{L2}$$

في L2 يدخل الاضمحلال في التدرّج فيُضخّم عبر لحظات آدم.

$$\theta_{t+1} = (1 - \eta\lambda)\, \theta_t - \eta \cdot g_t \quad \leftarrow \text{مفصول}$$

هنا يُطرح قبل التحديث فيبقى متناسباً مع معدّل التعلّم دائماً.

لهذا كان `optax.adamw` يتولّى الاضمحلال المفصول داخلياً كما رأينا في الفصل الخامس عشر. لكن أحياناً تريد تطبيقه يدوياً على شجرة المعاملات، مثلاً لتجاوز الانحيازات:

```python
import jax, jax.numpy as jnp, optax

def decay_only(params, wd=0.01):
    # يطبّق الاضمحلال على الأوزان ذات البعد > 1 فقط
    def f(x):
        return x * (1.0 - wd) if x.ndim > 1 else x
    return jax.tree.map(f, params)

params = {"w": jnp.ones((4, 3)), "b": jnp.full((3,), 0.5)}
new = decay_only(params, wd=0.1)
print(new["b"])        # [0.5 0.5 0.5] لم يتغيّر
print(new["w"][0])     # [0.9 0.9 0.9]
```

✓ أعِد قيمة الاضمحلال حسب حجم الدفعة

الاضمحلال المفصول يطرح $\eta\lambda\theta$؛ فإن ضاعفت معدّل التعلّم مضاعفةً تلقائياً يتضاعف الأثر. عند الضبط الدقيق قلّل `weight_decay` إلى 0.01 أو أقلّ.

التطبيع: إعادة ضبط الإحصاءات لكل طبقة

بدل تعديل المعاملات، يُعيد التطبيع توزيع المخرجات نفسها. `BatchNorm` يطرح المتوسّط ويقسم على الانحراف المعياريّ لكل دفعة، ثم يعلّم إزاحة وقياساً:

$$\hat{x}_i = (x_i - \mu_B) \,/\, \sqrt{(\sigma_B^{\,2} + \varepsilon)} \quad , \quad y_i = \gamma\, \hat{x}_i + \beta$$

μ وσ تُحسبان من الدفعة أثناء التدريب، ومن متوسط التشغيل أثناء الاختبار.

في Flax تُدار الإحصاءات الجارية داخل الحالة، التي تُمرّر صراحةً بين الخطوات. لاحظ `batch_stats` المنفصلة عن المعاملات:

```python
import jax, jax.numpy as jnp
from flax import linen as nn

class Block(nn.Module):
    @nn.compact
    def __call__(self, x, train=True):
        x = nn.Dense(64)(x)
        x = nn.BatchNorm(use_running_average=not train)(x)   # تدير الإحصاء
        return nn.relu(x)

model = Block()
key = jax.random.key(1)
x = jnp.ones((16, 32))
variables = model.init(key, x)
print(list(variables.keys()))        # ['params', 'batch_stats']
```

> ⚠ **الدفعة الصغيرة تُفسد BatchNorm**
>
> حين يكون حجم الدفعة 2 أو 4، يصير تقدير μ وσ مشوّشاً جداً فتنهار الخسارة. استبدله بـ `LayerNorm` الذي يُطبّع على بُعد الميزات لا الدفعة، أو زِد حجم الدفعة.

Mixup: تدريب على عيّنات مُختلطة

بدل إضافة ضجيج للمخرجات، يخلط Mixup بين عيّنتين بوزن λ عشوائي، ويخلط الهدف تباعاً. النموذج يُجبر على ألا يكون «حادّاً» عند الحدود:

$$\tilde{x} = \lambda\, x_i + (1-\lambda)\, x_j \quad , \quad \tilde{y} = \lambda\, y_i + (1-\lambda)\, y_j$$

$\lambda \sim \mathrm{Beta}(\alpha, \alpha)$؛ وعند α صغير يميل المزيج إلى عيّنة واحدة.

```python
import jax, jax.numpy as jnp
from jax import random

def mixup(x, y, key, alpha=0.2):
    k1, k2 = random.split(key)
    lam = random.beta(k1, alpha, alpha)
    perm = random.permutation(k2, x.shape[0])
    return lam * x + (1 - lam) * x[perm], lam * y + (1 - lam) * y[perm]

x = jnp.arange(20.0).reshape(10, 2)
y = jnp.arange(10.0)
xm, ym = mixup(x, y, random.key(3))
print(xm.shape, ym.shape)        # (10, 2) (10,)
```

وتعزيز البيانات أخٌّ أعمّ: تدوير الصورة أو قصّها يوسّع منطقة القرار دون مسّ الهدف. سنرى في الفصل السادس عشر كيف تُبنى خطوط أنابيب العيّنات، هنا نكتفي بأن التنظيم يبدأ أحياناً قبل النموذج بخطوة.

طبّقت Mixup فأصبح الهدف عدداً كسورياً بين 0 و1، لا تسميةً صريحة. كيف تحسب الخسارة الآن، ولماذا يبقى انحدارها صالحاً؟

متى تستخدم ماذا؟

لا توجد أداة واحدة تناسب الجميع؛ الجدول الآتي يقترح البداية حسب السياق. ولاحظ أن الأدوات تتراكم: إسقاط الوحدات والتطبيع غالباً معاً، بينما Mixup قد يغني عن الإسقاط في بعض المحوّلات.

السياق	الأداة المقترحة	التحذير
CNN على صور	`Dropout` + `BatchNorm`	دفعة صغيرة تُفسد التطبيع
محوّل / LLM	`weight_decay` قوي + `Dropout` خفيف	الإسقاط قد يضعف التعلّم
بيانات محدودة	`Mixup` + تعزيز	قد يبطئ التقارب
تعميم ضعيف	`LabelSmoothing` + اضمحلال	يُفسد عتبات الثقة

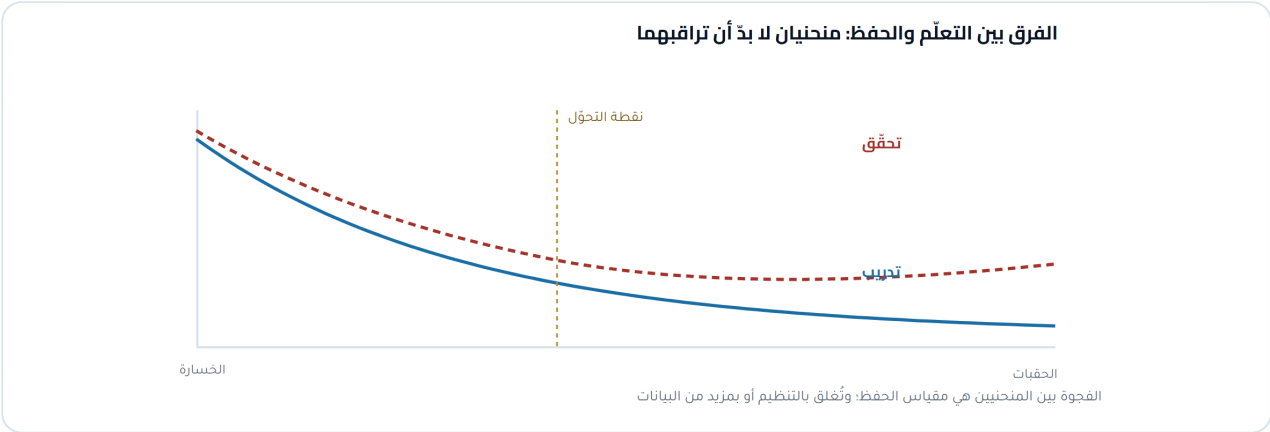
جدول 17.1 — دليل اختيار أداة التنظيم حسب السياق

× تشغيل الإسقاط أثناء الاختبار

الخطأ الشائع أن تبقي علم `train=True` وقت التقييم، فتُسقط وحدات وتُضرب بالمعامل، فيتضخّم خطأ الاختبار بلا سبب واضح. اجعل العلم يتبع طور التشغيل دائماً.

كيف نرى الإفراط قبل فوات الأوان؟

علامة الإفراط واضحة بصرياً: منحنى التدريب يهبط بينما منحنى التحقّق يستوي أو يصعد. الرسم الآتي يوضّح الحالتين؛ راقبه كل بضعة آلاف خطوة، فالتدخّل المبكر يوفّر ساعات.



الشكل 17.1 — الخسارة على التدريب (صلبة) والتحقّق (متقطّعة): في اليسار يتقارب المنحنيان، وفي اليمين ينفصلان دليلاً على الإفراط.

تمرين الفصل السابع عشر

1. ابنِ شبكة Flax بطبقتين `Dense` وفاصل `Dropout(0.3)`، ودرّبها على بيانات تافهة؛ أثبت أن متوسّط مخرجات التدريب والاختبار متطابقان.
2. طبّق الاضمحلال المفصول يدوياً على شجرة معاملات، وقارنه بإضافة L2 إلى الخسارة على خطوة آدم واحدة.
3. نفّذ `mixup` على أربع عيّنات وتحقّق أن الهدف المختلط يبقى بين y_i و y_j.

▲ الحلّ النموذجي

1. قناع التدريب مضروب بـ $1/(1-p)$ فيعطي نفس المتوسّط المتوقّع. **2.** المفصول يطرح $\eta\lambda\theta$ قبل اللحظات، فيبقى مستقرّاً. **3.** المتوسّط المرجّح يقع دائماً بين الطرفين.

Python

```
import jax, jax.numpy as jnp, optax
from flax import linen as nn

class Net(nn.Module):
    @nn.compact
    def __call__(self, x, train=True, key=None):
        x = nn.Dense(32)(x)
        x = nn.relu(x)
        x = nn.Dropout(0.3)(x, deterministic=not train, rng=key)
        return nn.Dense(2)(x)

net = Net()
k = jax.random.key(0)
x = jnp.ones((8, 4))
vars = net.init(k, x)
tr = net.apply(vars, x, train=True, key=jax.random.split(k)[0])
te = net.apply(vars, x, train=False)
print(float(tr.mean()), float(te.mean()))     # متقاربان

# اضمحلال مفصول يدوي
p = {"w": jnp.ones((3,))}
dec = jax.tree.map(lambda v: v * (1 - 1e-4), p)
print(dec["w"][0])                            # 0.9999

# مقارنة L2
tx = optax.adamw(1e-3, weight_decay=0.1)
g = jax.tree.map(jnp.ones_like, p)
upd, _ = tx.update(g, tx.init(p), p)
print(upd["w"][0])                            # تحديث مفصول
```

خلاصة الفصل

- الإفراط في التطابق يحصل حين تكفي حرّية النموذج لحفظ الضجيج.
- `dropout` قناع عشوائي يُدار بـ `key` صريح، مع تطبيع بالإنجاز.
- الاضمحلال المفصول يُطرح من الوزن، أفضل من L2 المضاف للخسارة.
- `BatchNorm` تُدير إحصاءات في `batch_stats` منفصلة.
- `mixup` وتعزيز البيانات يوسّعان منطقة القرار قبل النموذج.

مصطلحات الفصل

regularization

أدوات تُجبر النموذج على حلول ملساء فتحسّن التعميم.

dropout

إسقاط عشوائي لوحدات باحتمال p أثناء التدريب فقط.

weight decay

طرح نسبة من الوزن كل خطوة لكبح استجابته الحادّة.

BatchNorm

تطبيع المخرجات بإحصاءات الدفعة مع متوسّط تشغيل للاختبار.

Mixup

خلط عيّنتين وهدفيهما بوزن عشوائي لتلطيف حدود القرار.

batch_stats

حالة Flax للإحصاءات الجارية المنفصلة عن المعاملات.

Equinox وjaxtypingg: النمط البايثوني

نماذج ككائنات عادية، وأنواع للأشكال، وتحويل آمن

في جلسة مراجعة شيفرة، سأل مهندسٌ زميله: «لماذا نحمل قاموس معاملات منفصلاً عن النموذج؟». أدار الزميل المؤشّر على سطور من نوع `params["layers_0"]["kernel"]`، ولم يجد جواباً مقنعاً سوى: «هكذا تفعل Flax». لكن JAX نفسها لا تشترط ذلك؛ فهي لا تعرف ما النموذج أصلاً – تعرف الأشجار فقط. فماذا لو كان النموذج كلّه كائناً واحداً، حقوله هي المعاملات، ويمرّ إلى `jax.jit` كما يمرّ القاموس؟ هذا سؤال Equinox.

ماذا لو كان النموذج dataclass عادياً؟

تذكّر ما فعله `jax.jit` مع قاموس في الفصل السادس: نظر إليه فاعتبره شجرة، ثم تتبّع الدالّة عليها. لم يسأل «ما نوع هذا الكائن؟» بل «هل أعرف كيف أفكّكه إلى أوراق؟». و`dataclass` يقدّم هذا التفكيك مجاناً.

Equinox تبني على هذه الملاحظة رهاناً واحداً: **النموذج هو الشجرة**. لا كائن «بنية» منفصل عن كائن «حالة». في Flax (الفصل الرابع عشر) هناك عالمان: `Module` يصف الشكل و`variables` تحمل الأرقام؛ في Equinox عالم واحد. يبدو الفرق تجميلياً، لكنه يغيّر كل سطر بعده.

> إذا كان كل ما تحتاجه شجرة، فما الذي يقدّمه الفصل بين البنية والحالة في Flax ولا يقدّمه dataclass؟

eqx.Module: الحقول هي المعاملات

`eqx.Module` ليس أكثر من dataclass مُسجّل مسبقاً كشجرة، مع قيد حاسم: **مجمّد**. لا تستطيع كتابة `model.l1 = ...` بعده؛ فالتعديل في المكان يهدم الفكرة، إذ يجب أن تبقى الدالّة نقية. كل تحديث يُنتج كائناً جديداً، والحقول المعلّمة بـ`eqx.field(static=True)` تُستثنى من الشجرة.

```python
import jax, jax.numpy as jnp, equinox as eqx

class MLP(eqx.Module):
    l1: eqx.nn.Linear
    l2: eqx.nn.Linear
    drop: float = eqx.field(static=True)   # ليست مصفوفة: جزء من البنية

    def __init__(self, key, din=784, dh=128, dout=10):
        k1, k2 = jax.random.split(key)
        self.l1 = eqx.nn.Linear(din, dh, key=k1)
        self.l2 = eqx.nn.Linear(dh, dout, key=k2)
        self.drop = 0.1

    def __call__(self, x):
        return self.l2(jax.nn.relu(self.l1(x)))

model = MLP(jax.random.key(0))
print(len(jax.tree.leaves(model)))   # 4: وزن وانحياز لكل طبقة

sizes = jax.tree.map(lambda a: a.size if eqx.is_array(a) else 0, model)
print(sum(jax.tree.leaves(sizes)))   # 101770
```

`jax.tree.map` يرى النموذج كما يرى أي قاموس: أربع أوراق، والحقل `drop` غائب لأنه ثابت. ولهذا يستبدل `eqx.tree_at` طبقة كاملة بمسار واضح:
`eqx.tree_at(lambda m: m.l1.weight, model, jnp.zeros_like(model.l1.weight))` .

> ★ **الحقل الثابت ليس معاملاً**
>
> كل حقل يدخل الشجرة إلا إذا عُلّم `static=True` . القاعدة: ما يتغيّر شكله مع الدفعة يبقى مصفوفة، وما يغيّر بنية الحساب (عدد الطبقات، علم التدريب) يُعلَّم ثابتاً.

filter_gradg eqx.filter_jit

هنا يظهر الألم الذي تعرفه: `jax.jit` يريد وسائط من نوع `jax.Array` ، وإلا طلب `static_argnums` – وهو حلّ هشّ، لأن كل حقل ثابت جديد يعني تعديل الأرقام. النسخة `filter_*` تفصل الشجرة تلقائياً إلى مصفوفات وغير مصفوفات.

```python
class Config(eqx.Module):
    w: jax.Array
    scale: float = eqx.field(static=True)

cfg = Config(w=jnp.ones((3,)), scale=2.0)
print(jax.tree.leaves(cfg))            # [Array([1., 1., 1.])]

@eqx.filter_jit
def f(c, x):
    return c.scale * (x @ c.w)

@eqx.filter_grad
def total(c, x):
    return jnp.sum(f(c, x))

g = total(cfg, jnp.arange(3.0))
print(g.w)        # [0. 2. 4.]
print(g.scale)    # None
```

التدرّج أعاد `None` للحقل الثابت، لا صفراً. الصفر يعني «اشتقّ وسيظهر أنه صفر»، و`None` يعني «لا معنى للاشتقاق هنا». ولهذا يعمل `filter_grad` على نماذج تحمل نصوصاً وأعلاماً، بينما يفشل `jax.grad` عليها.

> ⚠ **الثابت يعيد التجميع**
>
> تغيير `scale` من 2.0 إلى 3.0 يبني نسخة مجمّعة جديدة، لأن القيمة صارت جزءاً من البنية. لا تمرّر معدّل تعلّم متغيّراً كحقل ثابت ثم تتساءل عن بطء الحلقة.

jaxtyping: الأشكال في التوقيع

أكثر أخطاء JAX أخطاءُ أشكال، ورسالتها تظهر بعيداً عن سببها (الفصل الثاني). الحلّ هنا أنانيةٌ محمودة: ضع الشكل في التوقيع ودع مدقّق أنواع يقرأه. `Float[Array, "batch feat"]` ليست تعليقاً بل عقداً قابلاً للفحص، وتكرار الاسم يفرض التطابق: إذا ظهر `batch` في وسيطين وجب أن يتساويا.

```python
from beartype import beartype
from jaxtyping import Array, Float, jaxtyped

@jaxtyped(typechecker=beartype)
def scores(
    q: Float[Array, "batch heads qlen dim"],
    k: Float[Array, "batch heads klen dim"],
) -> Float[Array, "batch heads qlen klen"]:
    return jnp.einsum("bhqd,bhkd->bhqk", q, k) / jnp.sqrt(q.shape[-1])

q = jnp.ones((2, 4, 8, 16))
print(scores(q, q).shape)              # (2, 4, 8, 8)
scores(q, jnp.ones((2, 4, 7, 16)))     # خطأ فوري
```

‹‹ المخرجات ››

```
(2, 4, 8, 8)
BeartypeCallHintParamViolation: Argument 'k' ... has shape (2, 4, 7, 16);
expected shape (2, 4, klen, 16), where klen is any value.
```

الخطأ يقع عند الحدّ قبل أن يدخل شيء إلى المترجم، والفحص يجري مرة عند التتبّع لا عند كل خطوة؛ لذلك وضعه داخل `scan` شبه مجّاني.

× خطأ شائع: تزيين كل دالّة

`@jaxtyped` على كل دالّة صغيرة يجعل الشيفرة أبطأ عند التتبّع وأقلّ قراءةً. ضعه على حدود النظام: مدخلات النموذج، ومخرجات التحميل، والدوالّ العامة.

التدريب مع Equinox

الآن تجتمع القطع: لا قاموس ولا `apply`؛ النموذج يدخل ويخرج، والمحدّث يعمل على المصفوفات وحدها. لاحظ `eqx.filter` عند تهيئة المحسّن: الحالة للأوزان فقط.

```python
import optax

optim = optax.adamw(learning_rate=3e-4, weight_decay=1e-4)
opt_state = optim.init(eqx.filter(model, eqx.is_array))

def loss_fn(m, x, y):
    logits = jax.vmap(m)(x)
    return optax.softmax_cross_entropy_with_integer_labels(logits, y).mean()

@eqx.filter_jit
def train_step(m, opt_state, x, y):
    loss, grads = eqx.filter_value_and_grad(loss_fn)(m, x, y)
    updates, opt_state = optim.update(grads, opt_state, eqx.filter(m, eqx.is_array))
    m = eqx.apply_updates(m, updates)
    return m, opt_state, loss
```

أربعة أسطر في جسم الخطوة، ولا سطر يعرف نوع النموذج. تكتب الحلقة مرة، ويصير تغيير النموذج تغييراً لتعريفه لا لمسارات قاموسه.

✓ احفظ الشجرة كما هي

لأن النموذج شجرة، يكفي `eqx.tree_serialise_leaves` لحفظه. لا تحتاج مخططاً للأسماء يتغيّر كلما أعدت تسمية طبقة.

Equinox أم Flax؟

السؤال ليس «أيّهما أفضل» بل «أيّ نموذج ذهني يطابق مشكلتك». Equinox تعطيك الشفافية وتدفع ثمنها في غياب النماذج الجاهزة؛ Flax تعطيك نظاماً بيئياً ناضجاً وتدفع ثمنه في طبقة تجريد.

المعيار	Equinox	Flax Linen
الوضوح	شجرة واحدة: ما تراه يمرّ إلى `jit`	بنية وحالة منفصلتان عبر `variables`
النظام البيئي	أصغر، وتكامل جيّد مع Optax	أكبر: نماذج وأوزان جاهزة
الحفظ	حفظ الشجرة مباشرة	مخطط `frozen_dict` بأسماء ثابتة
منحنى التعلّم	قصير لمن يعرف بايثون وJAX	أطول: `apply` و `mutable`
الحالة المتغيّرة	تمرير صريح لكل ما يتغيّر	`batch_stats` مُدار من الإطار

جدول 18.1 — Equinox مقابل Flax Linen

القاعدة العملية: مشروع بحثي صغير أو طبقة غير قياسية ← Equinox. إعادة إنتاج نموذج منشور أو أوزان جاهزة ← Flax.

متى لا تحتاج مكتبة أصلاً؟

السؤال المحرج: كل هذا البناء مقابل قاموس ومعاملتين. في الفصل الثالث عشر بنيت شبكة كاملة بقاموس `params` و `jax.grad` و `optax`، وكان ذلك كافياً. المكتبة تُستحقّ حين تبدأ بتكرار ثلاثة أشياء: تسجيل الأشجار، وتمرير الأعلام الثابتة، وإعادة كتابة حلقة التحديث.

1. **نموذج واحد، محسّن واحد.** قاموس الفصل الثالث عشر يكفي؛ لن يضيف Equinox قيمة.
2. **طبقات متكرّرة وحقول ثابتة.** هنا يبدأ الألم اليدوي، و `eqx.Module` يمحو نصفه.
3. **أوزان جاهزة.** هنا Flax، لا لأنه أنقى بل لأن الأوزان كُتبت به.

> ✦ **المكتبة تعقيد مؤجّل**
>
> كل طبقة تجريد تشتريها بحرّية أقلّ. السؤال ليس «هل أستطيع كتابتها بنفسي؟» بل «هل أريد إصلاحها في منتصف الليل؟».

لو أضفت `BatchNorm` إلى نموذجك المكتوب بقاموس، فما أول شيء ينكسر؟ وهل كان Equinox سيمنعه أم يؤجّله؟ ؟

تمرين الفصل الثامن عشر

1. أضف إلى `MLP` حقلاً ثابتاً `depth`، وتحقّق من غيابه عن `jax.tree.leaves`.
2. جمّد الطبقة الأولى بتصفير تدرّجها عبر `eqx.tree_at`، وأثبت أن تدرّج `l1` صار صفراً.
3. أضف `@jaxtyped` إلى دالّة الخسارة ومرّر دفعة بشكل خاطئ، وسجّل نصّ الخطأ.

الحلّ النموذجي

1 و2. الحقل الثابت يُستثنى تلقائياً، والتصفير يعمل على شجرة التدرّج لا على النموذج:

```python
import jax, jax.numpy as jnp, equinox as eqx

class MLP(eqx.Module):
    l1: eqx.nn.Linear
    l2: eqx.nn.Linear
    depth: int = eqx.field(static=True)

    def __init__(self, key, depth=2):
        k1, k2 = jax.random.split(key)
        self.l1 = eqx.nn.Linear(8, 16, key=k1)
        self.l2 = eqx.nn.Linear(16, 1, key=k2)
        self.depth = depth

    def __call__(self, x):
        return self.l2(jax.nn.relu(self.l1(x)))

m = MLP(jax.random.key(0))
print(len(jax.tree.leaves(m)))     # 4 — ليس depth ورقة

x, y = jnp.ones((4, 8)), jnp.zeros((4, 1))
loss = lambda mod: jnp.mean((jax.vmap(mod)(x) - y) ** 2)

g = eqx.filter_grad(loss)(m)
g = eqx.tree_at(lambda t: t.l1, g, jax.tree.map(jnp.zeros_like, g.l1))
print(float(jnp.abs(g.l1.weight).max()))          # 0.0
print(float(jnp.abs(g.l2.weight).max()) > 0)      # True
```

3. يظهر `BeartypeCallHintParamViolation` ويذكر الشكلين المتوقّع والفعلي، عند أول استدعاء لا بعد ساعة تدريب.

خلاصة الفصل

- Equinox تراهن على أن النموذج شجرة: الحقول هي المعاملات، ولا قاموس منفصل.
- `eqx.Module` dataclass مجمّد: التحديث يُنتج كائناً جديداً.
- `filter_jit` و `filter_grad` يمحوان `static_argnums` .
- `jaxtyping` يجعل الشكل عقداً في التوقيع يُفحص عند الحدّ.
- نمط الفصل الثالث عشر يكفي حتى تبدأ التكرارات.

مصطلحات الفصل

pytree

توصيف تفكيك كائن إلى أوراق وإعادة تركيبه.

eqx.Module

dataclass مجمّد مُسجّل كشجرة؛ حقوله غير الثابتة أوراقها.

static field

حقل يُعلَّم `static=True` فيُستثنى من الشجرة ويُعدّ بنية.

filter_jit

نسخة من `jax.jit` تفصل المصفوفات عن غيرها قبل التجميع.

jaxtyped

مُزيّن يفحص تعليقات الأشكال بمدقّق أنواع وقت التشغيل.

الجزء الرابع

البنى العصبية الحديثة

الصور، والتسلسلات، والانتباه، والمحوّلات، والنماذج التوليدية

الشبكات التلافيفية في JAX

الالتفاف، والتجميع، والحقل الاستقبالي، وبنى ResNet

جرّب أن تُدرّب شبكة كثيفة على صور 224×224. الطبقة الأولى وحدها تحسب 150,528 مدخلاً لكل وحدة مخفية؛ ألف وحدة تعني مئة وأربعة وخمسين مليون معامل قبل الطبقة الثانية. ثم تكتشف أن النموذج حفظ موقع القطّ في الزاوية لا شكل القطّ. المشكلة ليست في حجم الأرقام بل في افتراض أن كل بكسل يرى كل وحدة.

لماذا لا يصلح MLP للصور؟

الصورة شبكة قيم لها جوار؛ وحين تُسطّحها تفقد أي بكسل كان بجوار أي، والشبكة الكثيفة لا تستعيد ذلك لأنها تعلّم وزناً منفصلاً لكل زوج (مدخل، مخرج).

ثلاثة أرقام تشخّص المرض. الطبقة الأولى: $150{,}528 \times 1{,}024 \approx 154\text{M}$ معامل، مقابل $3 \times 3 \times 3 \times 64 + 64 = 1{,}792$ لطبقة التفافية مكافئة. الفرق فرضية لا تحسين هندسي: النمط المحلّي هو ما يهمّ، والنمط نفسه يهمّ في كل مكان – وهو ما يجعل الشبكة تنجح ببيانات أقلّ.

★ **ثلاثة قيود في قيد واحد**

الالتفاف يفرض **محلّية** و**مشاركة أوزان** و**ثبات تقاطع** تقريباً، وهي مصدر اقتصاد الحساب كلّه.

الالتفاف كعملية خطّية

الالتفاف عملية خطّية بين نافذة صغيرة ووزن صغير تُنتج رقماً واحداً. وما تنفّذه المكتبات فعلياً ليس الالتفاف الرياضي (الذي يقلب النواة) بل الارتباط المتقاطع؛ والفرق انعكاس في الترتيب لا يغيّر شيئاً لأن الأوزان متعلَّمة.

$$y[i, j] = \sum_u \sum_v x[i \cdot s + u, j \cdot s + v] \cdot w[u, v] + b$$

الخرج عند (i، j) مجموع جداءات النافذة بالوزن؛ وs هي الخطوة.

$$n_{out} = \lfloor (n_{in} + 2p - k) / s \rfloor + 1$$

حجم الخرج: n المدخل، k النواة، p الحشو، s الخطوة. والحشو p = (k−1)/2 يحفظ الحجم.

مثال باليد: مدخل 3×3، نواة 2×2، خطوة 1، بلا حشو.

‹‹ المخرجات ››

```
x = [[1,2,3],[4,5,6],[7,8,9]]    w = [[1,0],[0,-1]]
y[0,0] = 1 - 5 = -4      y[0,1] = 2 - 6 = -4
y[1,0] = 4 - 8 = -4      y[1,1] = 5 - 9 = -4
الحجم: ⌊(3 - 2 + 0)/1⌋ + 1 = 2
```

النواة هنا كاشف فرق قطري، وغير الخطّية من `relu` بعده.

lax.conv_general_dilated

هذه الواجهة الوحيدة التي تحتاج فهمها؛ كل ما فوقها (`nn.Conv`) يبني عليها. وترتيب الأبعاد تحدّده أنت عبر `dimension_numbers` .

Python

```python
import jax.numpy as jnp
from jax import lax

x = jnp.arange(25.0).reshape(1, 5, 5, 1)      # NHWC
w = jnp.ones((3, 3, 1, 4)) / 9.0              # HWIO

y = lax.conv_general_dilated(
    x, w,
    window_strides=(1, 1),
    padding="SAME",
    dimension_numbers=("NHWC", "HWIO", "NHWC"),
)
print(y.shape)          # (1, 5, 5, 4): الحجم محفوظ، القنوات أربع
print(y[0, 2, 2, 0])    # 12.0 — متوسط النافذة المركزية
```

`"SAME"` يحفظ الحجم بحشو أصفار، و `"VALID"` يقتطع ما لا يكتمل: مع نواة 3 وخطوة 1 على مدخل 5 يعطيان 5 و3.

> **× خطأ شائع: خلط ترتيب القنوات**
>
> تمرير `NCHW` إلى واجهة تنتظر `NHWC` قد يمرّ لأن الأشكال متوافقة عددياً، وينتهي نموذج يتعلّم ضجيجاً. اكتب الترتيب صراحةً دائماً.

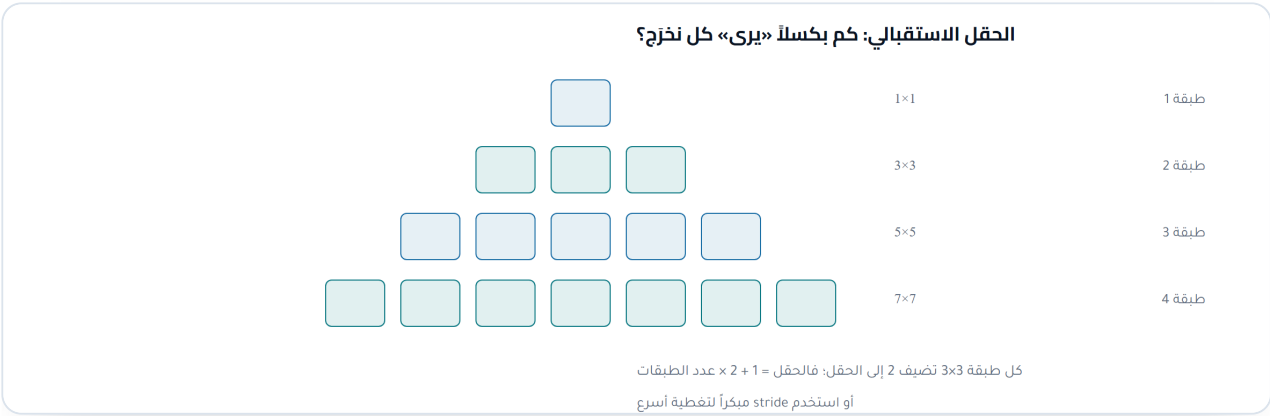
التجميع والحقل الاستقبالي

التجميع ليس تعلّماً بل قراراً: `max_pool` يحتفظ بأقوى إشارة، و`avg_pool` بمتوسطها. كلاهما يقلّل الحجم ويكبّر الحقل، بثمن هو فقدان الدقّة المكانية.

$$r_l = r_{l-1} + (k_l - 1) \cdot \prod_{i<l} s_i$$

الحقل الاستقبالي بعد الطبقة l: حجم النافذة الأصلية التي يراها نخرَج واحد.

ثلاثة التفافات 3×3 بخطوة 1 تعطي 7×7 لا 9. ولو كانت الخطوات 1 ثم 2 ثم 2 لصار 9×9: 3، ثم 3 + 2×1 = 5، ثم 5 + 2×2 = 9. لهذا تحتاج الشبكة العميقة طبقات أكثر لترى الصورة كاملة، ولهذا تُستخدم الخطوة بدل التجميع في البنى الحديثة.



الشكل 19.2 — الحقل الاستقبالي ينمو طبقةً بعد طبقة؛ الأرقام حجم النافذة الأصلية التي يراها نخرَج واحد.

كتلة ResNet كاملة

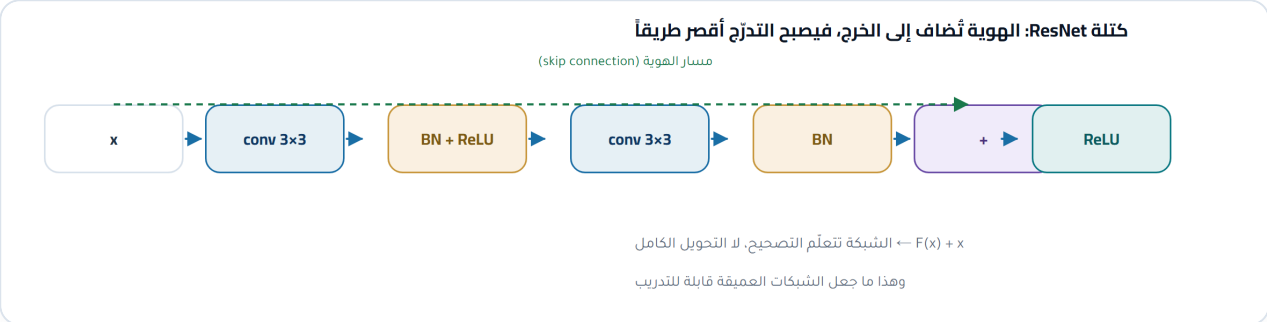
الكتلة المتبقية تجيب على سؤال: ماذا لو كان الأفضل للطبقة الجديدة ألّا تفعل شيئاً؟ بدل تعلّم $H(x)$ تتعلّم الباقي $F(x) = H(x) - x$ ثم تجمع. إن كان المطلوب الهوية، يكفي أن تصل الأوزان إلى الصفر، بخلاف تعلّمها عبر طبقات غير خطّية.

```python
import flax.linen as nn

class Block(nn.Module):
    ch: int
    stride: int = 1

    @nn.compact
    def __call__(self, x, train: bool):
        y = nn.Conv(self.ch, (3, 3), (self.stride, self.stride),
                    padding="SAME", use_bias=False)(x)
        y = nn.relu(nn.BatchNorm(use_running_average=not train)(y))
        y = nn.Conv(self.ch, (3, 3), padding="SAME", use_bias=False)(y)
        y = nn.BatchNorm(use_running_average=not train)(y)
        if self.stride != 1 or x.shape[-1] != self.ch:
            x = nn.Conv(self.ch, (1, 1), (self.stride, self.stride),
                        use_bias=False)(x)
            x = nn.BatchNorm(use_running_average=not train)(x)
        return nn.relu(x + y)
```

الشرط ليس تفصيلاً: الجمع يحتاج تطابق الشكل. حين تتغيّر القنوات أو تنقص المساحة بسبب الخطوة 2، لا يمكن جمع `x` الأصلي؛ والتفاف 1×1 بلا انحياز هو أرخص مطابقة، ولهذا يُسمّى «إسقاطاً».



الشكل 19.3 — المساران: تحويل تعلّمي ومسار مختصر، يمرّ الأخير بإسقاط 1×1 حين يختلف الشكل.

تدريب CNN صغير

لنضع كل شيء في نموذج واحد على صور 32×32. الجدول يبيّن ما يحدث للشكل في كل خطوة: إن حفظته لن تُفاجأ بعد ساعة.

الطبقة	الشكل	المعاملات
المدخل	(32, 32, 32, 3)	–

الطبقة	الشكل	المعاملات
Conv 3×3، 32، SAME	(32 ,32 ,32 ,32)	896
MaxPool 2×2، خطوة 2	(32 ,16 ,16 ,32)	0
Conv 3×3، 64، SAME	(64 ,16 ,16 ,32)	18,496
MaxPool 2×2، خطوة 2	(64 ,8 ,8 ,32)	0
تجميع عام	(64 ,32)	0
Dense، 10	(10 ,32)	650

جدول 19.1 — تتبّع الشكل عبر شبكة صغيرة

```python
class SmallCNN(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.relu(nn.Conv(32, (3, 3), padding="SAME")(x))
        x = nn.max_pool(x, (2, 2), (2, 2))
        x = nn.relu(nn.Conv(64, (3, 3), padding="SAME")(x))
        x = nn.max_pool(x, (2, 2), (2, 2))
        x = jnp.mean(x, axis=(1, 2))          # تجميع عام: يمحو المكان
        return nn.Dense(10)(x)

def loss_fn(params, x, y):
    logits = model.apply({"params": params}, x)
    return optax.softmax_cross_entropy_with_integer_labels(logits, y).mean()

@jax.jit
def train_step(params, opt_state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, opt_state = optim.update(grads, opt_state, params)
    return optax.apply_updates(params, updates), opt_state, loss
```

عشرون ألف معامل مقابل مئة وأربعة وخمسين مليوناً، وبعد تجميعين يرى كل نخرَج الصورة كاملة. ولأن التجميع العام أزال البُعدين المكانيين لم نحتج إلى `flatten`.

> ✓ **قاعدة الحجم قبل التدريب**
>
> مرّر دفعة وهمية من صف واحد واطبع الشكل بعد كل طبقة. تكلفة السطر ثانية، وتكلفة الخطأ في الخطوة الألف ساعة.

CNN في 2026: هل ما زالت مهمّة؟

الجواب ليس «نعم» الرمزية. المحوّلات (الفصل الحادي والعشرون) تفوّقت في التصنيف الكبير حين تتوفّر بيانات ضخمة وحساب وفير، لأنها لا تفترض محلّية ولا مشاركة أوزان. لكن هذا الانحياز نفسه يجعل CNN أكفأ عند البيانات المحدودة. والممارسة في 2026 هجينة: التفاف في المراحل الأولى وانتباه في الأخيرة.

أخطاء CNN الشائعة

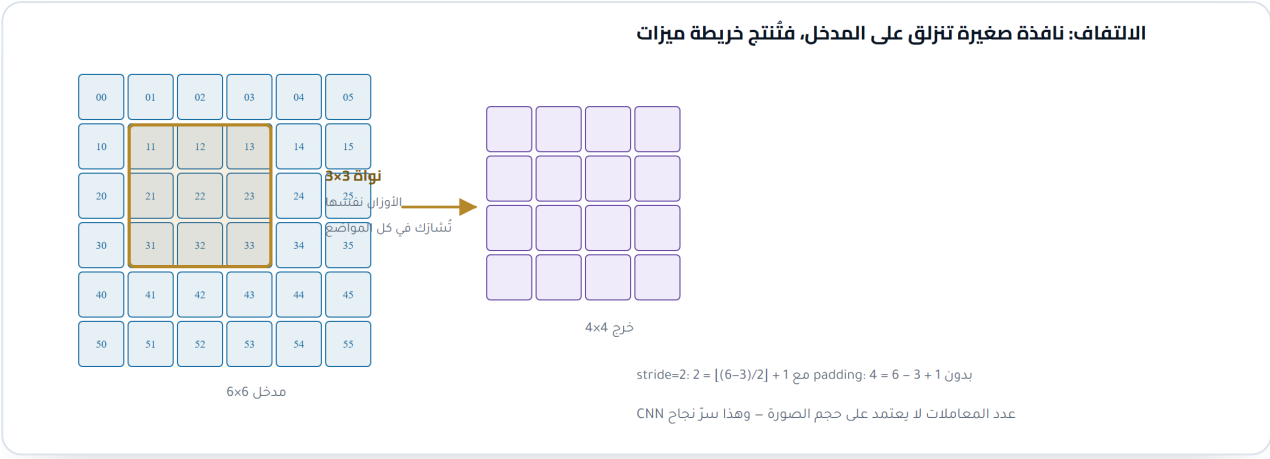
الأربعة التي تتكرّر:

1. **ترتيب القنوات.** خلط `NHWC` و `NCHW` يمرّ أحياناً بلا خطأ ويعلّم النموذج ضجيجاً.
2. **الحشو.** `SAME` يحفظ الحجم و `VALID` يقتطع؛ الأول للداخل والثاني قبل التصنيف.
3. **نسيان التسطيح.** بعد الالتفافات ثلاثة أبعاد؛ تجميع عام أو `reshape` قبل `Dense`.
4. **BatchNorm في التقييم.** نسيان `use_running_average=True` يجعل النتيجة تعتمد على الدفعة.

> ⚠ **BatchNorm والإحصاءات**
>
> أثناء التدريب تُحدَّث المتوسطات المتحرّكة، وأثناء التقييم تُستخدم كما هي. تمرير `train=True` في التقييم يبدّل السلوك بصمت.

شبكة تحقّق 90% على دفعة و71% على صورة واحدة. ما أوّل فرضية تفحصها؟ ؟

لو ضاعفت القنوات في كل طبقة، هل يتضاعف عدد المعاملات؟ احسبها لطبقة 3×3 من 64 إلى 128. ؟



الشكل 19.1 — النافذة نفسها تنزلق على كل المواضع؛ الأوزان لا تتغيّر، وهذا معنى مشاركة الأوزان.

تمرين الفصل التاسع عشر

1. احسب حجم الخرج لمدخل (1، 3، 64، 64) بنواة 5×5 وخطوة 2 وحشو `VALID`.
2. نفّذ الالتفاف بترتيب `NCHW` ثم `NHWC` وتحقّق من التطابق.
3. احسب الحقل الاستقبالي لثلاث طبقات 3×3 بخطوات 1 ثم 2 ثم 2.

الحلّ النموذجي

1. $\lfloor (64-5)/2 \rfloor + 1 = 30$، فالشكل (1، 16، 30، 30).
2. التحويل بين الترتيبين تبديل محاور:

```python
import jax.numpy as jnp
from jax import lax

x = jnp.ones((1, 3, 64, 64))
w = jnp.ones((16, 3, 5, 5)) / 75.0

y = lax.conv_general_dilated(
    x, w, (2, 2), "VALID",
    dimension_numbers=("NCHW", "OIHW", "NCHW"))
print(y.shape)                       # (1, 16, 30, 30)

y2 = lax.conv_general_dilated(
    x.transpose(0, 2, 3, 1), w.transpose(2, 3, 1, 0), (2, 2), "VALID",
    dimension_numbers=("NHWC", "HWIO", "NHWC"))
print(jnp.allclose(y, y2.transpose(0, 3, 1, 2)))   # True
```

3. 3، ثم 3 + 1×2 = 5، ثم 5 + 2×2 = 9. الحقل 9×9 لا 27.

خلاصة الفصل

- الكثيفة على الصور تنفجر في المعاملات وتفقد الجوار؛ الالتفاف يعالج الاثنين.
- الالتفاف ارتباط متقاطع: نافذة صغيرة وأوزان مشتركة، ثم غير خطّية.
- `dimension_numbers` و `padding` مفتاح الأخطاء.
- الحقل الاستقبالي يتراكم، والخطوة تكبّره أسرع من التجميع.
- الكتلة المتبقية تحتاج إسقاطاً 1×1 عند تغيّر الشكل.

مصطلحات الفصل

الارتباط المتقاطع

ما تنفّذه المكتبات باسم الالتفاف: نافذة تُضرب بالوزن بلا قلب.

الحقل الاستقبالي

مساحة المدخل المؤثّرة في نخرَج واحد.

مشاركة الأوزان

النواة نفسها في كل المواضع؛ تقلّل المعاملات وتحقيق ثبات التقاطع.

المسار المختصر

طريق يمرّر المدخل إلى الجمع مباشرة.

التجميع العام

متوسط كل قناة مكانياً، فيُنتج متجهاً لكل عيّنة.

التسلسلات: RNN وLSTM وGRU عبر scan

الحلقة كدالة، والتدرّج عبر الزمن، والسلاسل الطويلة

جملتان بالكلمات نفسها: «الكلب عضّ الرجل» و«الرجل عضّ الكلب». أي نموذج يعطي الجملتين التمثيل نفسه لا يفهم لغة، لأنه تجاهل الترتيب. والشبكة الأمامية تفعل ذلك بالضبط: تُسطّح التسلسل إلى متجه وتنسى من جاء قبل من. في هذا الفصل نعيد الزمن إلى النموذج، لا بحلقة بايثون، بل بدالّة واحدة تُسمّى `lax.scan`.

لماذا لا تكفي الشبكة الأمامية؟

الشبكة الكثيفة تستهلك متجهاً ثابت الطول، والتسلسل متغيّر الطول بطبيعته. والحلّ الساذج – قصّ أو حشو إلى طول أقصى – يخلط بين الطول والمعنى، ويجعل المعاملات تنمو مع الطول إن أردت وزناً لكل موضع.

الحلّ الصحيح يقوم على ثلاث ملاحظات: الترتيب يحمل المعنى، والطول يتغيّر، والقاعدة التي تحكم الانتقال بين خطوتين هي نفسها في كل خطوة. الأخيرة هي **مشاركة الأوزان عبر الزمن**، وهي التي تُبقي عدد المعاملات ثابتاً مهما طال التسلسل.

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1} + b)$$

حالة مخفية واحدة تنتقل بين الخطوات؛ والمصفوفتان نفسهما في كل t.

★ الحالة هي الذاكرة

لا يحفظ النموذج التاريخ كلّه بل ملخّصاً منه في h_t؛ وهذا مصدر قوّته وحدّه في آن.

lax.scan: الحلقة كدالّة

في بايثون تكتب `for`، وفي JAX لا تستطيع لأن التتبّع يحتاج حلقات ثابتة. البديل `lax.scan`: تأخذ حالة (`carry`) وتسلسل مدخلات، وتعيد الحالة النهائية وتسلسل المخارج. جسمها دالّة نقية تأخذ (حالة، عنصر) وتعيد (حالة جديدة، مخرج).

```python
import jax.numpy as jnp
from jax import lax

xs = jnp.array([1.0, 2.0, 3.0, 4.0])

def body(c, x):
    c = c + x
    return c, c          # (الجديد، مخرج هذه الخطوة)

total, ys = lax.scan(body, 0.0, xs)
print(total, ys)         # 10.0  [1. 3. 6. 10.]
```

`ys` مصفوفة شكلها (الطول، ...) لا قائمة، و`scan` تُحوّل الحلقة إلى عملية واحدة في الرسم البياني – ولهذا يعمل عليها التجميع والتدرّج كأي عملية.

```python
def rnn_step(p, h, x):
    h = jnp.tanh(x @ p["Wx"] + h @ p["Wh"] + p["b"])
    return h, h

def run_rnn(p, xs, h0):
    # xs: (T, B, D)   h0: (B, H)   الخرج: (T, B, H)
    return lax.scan(lambda h, x: rnn_step(p, h, x), h0, xs)[1]
```

> ✓ **اختبار الاتجاه أولاً**
>
> قبل أي تدريب، تحقّق من أن (T، B، D) تعود (T، B، H). أكثر أخطاء RNN أخطاء محاور لا أخطاء تعلّم.

الانتشار العكسي عبر الزمن

حين تُشتق دالّة فيها `scan` لا يحدث شيء خاص: يفكّ المترجم الحلقة إلى رسم طوله T ثم يشتق عكسياً عليه. هذا هو BPTT، وليس خوارزمية جديدة. ولأن h_t تعتمد على h_{t-1}، يتدفّق التدرّج عكسياً عبر كل الخطوات ويتراكم جداء المشتقات – ومن هنا الانفجار والتلاشي.

الثمن أن الذاكرة تنمو خطّياً مع T لأن تنشيطات كل الخطوات تُحفظ للرحلة العكسية. لتسلسل بطول 2,048 قد يكون هذا هو الحدّ الذي يوقفك؛ والحلّ إعادة الحساب:

```python
import jax

def run_rnn_ckpt(p, xs, h0):
    step = jax.checkpoint(lambda h, x: rnn_step(p, h, x))   # إعادة حساب أثناء الرجوع
    return lax.scan(step, h0, xs)[1]
```

> ⚠ **الذاكرة قبل السرعة**
>
> إعادة الحساب تقايض الحساب بالذاكرة: وقت إضافي مقابل ذاكرة لا تنمو مع T. لا تفعّلها قبل أن تقيس. و `jax.checkpoint` هو الاسم الحالي لما كان يُسمّى remat.

LSTM: البوّابات الثلاث

RNN البسيطة تنسى بسرعة لأن التدرّج يتلاشى في جداء مشتقات `tanh`. LSTM تحلّ ذلك بحالة خلوية c_t يمرّ فيها التدرّج بالجمع لا بالضرب، وبثلاث بوّابات تقرّر ماذا تُبقي وتُضيف وتُخرج.

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

البوّابات بين 0 و1 لأنها سيغمويد؛ والضرب ⊙ انتقاء عنصري.

يمكن تنفيذها بأربع مصفوفات منفصلة أو بمصفوفة مكدّسة تُقسَّم بعد الضرب؛ الثانية أسرع لأنها ضربة واحدة:

```python
def lstm_step(p, carry, x):
    h, c = carry
    z = x @ p["W"] + h @ p["U"] + p["b"]      # (B, 4H)
    i, f, g, o = jnp.split(z, 4, axis=-1)
    i, f, o = jax.nn.sigmoid(i), jax.nn.sigmoid(f), jax.nn.sigmoid(o)
    g = jnp.tanh(g)
    c = f * c + i * g
    h = o * jnp.tanh(c)
    return (h, c), h

def run_lstm(p, xs, h0, c0):
    (h, c), hs = lax.scan(lambda carry, x: lstm_step(p, carry, x), (h0, c0), xs)
    return hs, (h, c)
```

ترتيب الأقسام `i, f, g, o` عرف لا مبدأ؛ المهم أن يطابق ترتيب بناء المصفوفة المكدّسة. خطأ الترتيب لا يفشل، بل يُنتج نموذجاً يتعلّم ببطء ويُربكك أسبوعاً.

GRU: تبسيط مدروس

GRU تسأل: هل نحتاج حالتين وبوّابتين منفصلتين للنسيان والإضافة؟ عملياً لا. تدمج بوّابة التحديث z بينهما، وتضيف إعادة ضبط r تتحكّم في مقدار الماضي الداخل إلى المرشّح.

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$
$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$
$$n_t = \tanh(W_n x_t + U_n (r_t \odot h_{t-1}))$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot n_t$$

بوّابتان وحالة واحدة: نحو ثلث المعاملات أقلّ من LSTM.

الخلية	الصيغة	المعاملات
RNN (tanh)	h(d + h) + h	12,352
GRU	3h(d + h) + 3h	37,056
LSTM	4h(d + h) + 4h	49,408
LSTM ثنائي الاتجاه	2 × السابق	98,816

جدول 20.1 — عدد المعاملات عند d = 128 وh = 64

الفرق ليس أكاديمياً: على جهاز طرفي بذاكرة محدودة قد يكون اختيار GRU شرط التشغيل أصلاً. وفي معظم المهام الفرق في الدقّة صغير، وLSTM تتفوّق حين تكون الاعتماديات طويلة جداً.

التدريب على سلاسل حقيقية

التسلسلات الحقيقية مختلفة الأطوال. الحلّ: احشُ إلى الطول الأقصى في الدفعة، ومرّر قناعاً يقول أي موضع حقيقي، ثم استخدمه في مكانين لا مكان واحد: الخسارة والحالة.

```python
def masked_step(p, carry, xm):
    x, m = xm                                  # m: (B,) منطقي
    (h, c), _ = lstm_step(p, carry, x)
    h = jnp.where(m[:, None], h, carry[0])     # جمّد الحالة عند الحشو
    c = jnp.where(m[:, None], c, carry[1])
    return (h, c), h

def sequence_loss(p, xs, ys, mask):
    hs, _ = lax.scan(lambda cr, xm: masked_step(p, cr, xm),
                     (jnp.zeros((B, H)), jnp.zeros((B, H))), (xs, mask))
    logits = hs @ p["Wo"] + p["bo"]            # (T, B, V)
    loss = optax.softmax_cross_entropy_with_integer_labels(logits, ys)
    loss = jnp.where(mask, loss, 0.0)
    return loss.sum() / mask.sum()
```

لماذا المكانان؟ الخسارة وحدها تمنع مساهمة الحشو في التدرّج، لكنها لا تمنع تلوّث الحالة: بدون `where` يمرّ متجه الحشو في الخلية ويغيّر h الذي ستستخدمه الخطوة الحقيقية التالية. مع الحشو في النهاية لا يظهر الفرق؛ وفي البداية أو الوسط يظهر فوراً.

> **× خطأ شائع: قناع على الخسارة فقط**
>
> حساب الخسارة على المواضع الحقيقية صحيح لكنه غير كافٍ؛ الحالة تتلوّث بالحشو فتصبح النتائج غير متسقة بين دفعة وأخرى. جمّد الحالة أيضاً.

السلاسل الزمنية: الاحتياطات

حين تنتقل من النصّ إلى بيانات زمنية (أسعار، حساسات، طلب) يتغيّر الخطر: لم يبقَ في الشكل بل في التسريب. ثلاثة احتياطات تحمي النتيجة من أن تكون وهماً:

1. **التسريب.** لا تحسب المتوسط على كامل السلسلة قبل التقسيم؛ استخدم إحصاءات التدريب وحدها.

2. **الفرق.** كثير من السلاسل غير ساكنة؛ اشتقّ الفروق أو أزل الاتجاه، وإلا تعلّم النموذج أن الغد يشبه اليوم.

3. **التحقّق المتقدّم.** لا تقسّم عشوائياً؛ درّب على الماضي واختبر على المستقبل وقدّم النافذة خطوةً خطوة.

✦ **مقياس واحد لا يكفي**

خطأ منخفض على سلسلة سعرية قد يعني أن النموذج تعلّم تأخيراً بسيطاً لا تنبؤاً. قارن دائماً بخطّ أساس ساذج: «غداً = اليوم».

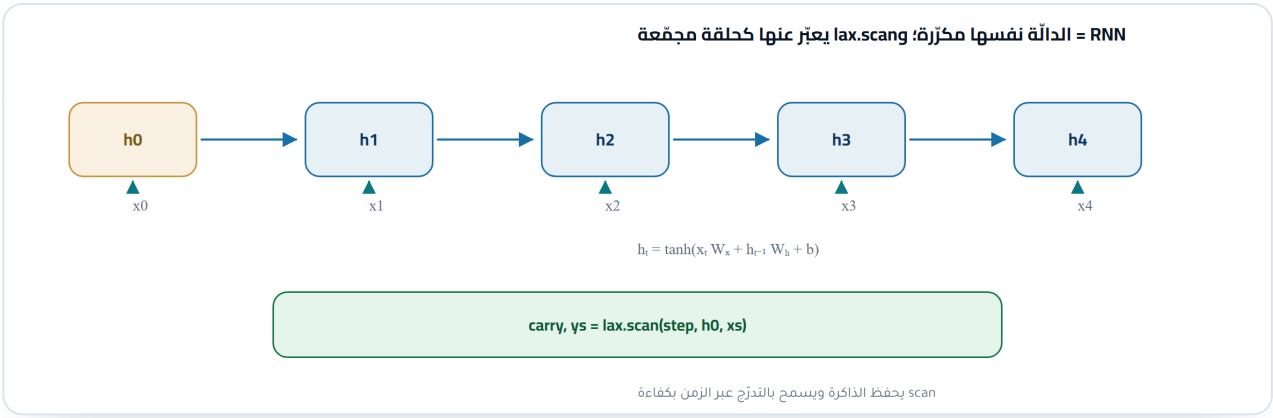
هل ما زالت RNN خياراً في 2026؟

للمحوّلات اليد العليا في اللغة لأن الانتباه يرى التسلسل كاملاً ويتدرّب بتوازٍ على كل المواضع، بينما RNN تفرض تسلسلاً في الحساب. لكن هذا يحدّد موقعها لا يلغيها.

تبقى منطقية حين تحتاج بثّاً حقيقياً بحالة ثابتة الحجم، أو تعمل على جهاز طرفي بذاكرة صغيرة، أو تملك بيانات قليلة لا تكفي محوّلاً. ونماذج الفضاء الحالي (SSM) ورثت هذه الميزة – حالة تُحدَّث بتكرار – وأضافت تدريباً متوازياً. فالسؤال ليس «RNN أم محوّل؟» بل «هل تحتاج اعتمادية على الماضي كاملةً أم تكفيك حالة مضغوطة؟».

نموذج تنبؤ على حساس يعمل على متحكّم صغير بلا ذاكرة واسعة. بأيّ خيار تبدأ، وما الثمن؟ ؟

لماذا يزيد طول التسلسل الذاكرةَ في BPTT مع أن عدد المعاملات لا يتغيّر؟ ؟



الشكل 20.1 — الحلقة نفسها مفتوحة على الزمن: الوزن نفسه يتكرّر، والحالة وحدها تنتقل بين الخطوات.



الشكل 20.2 — مسار الحالة الخلوية يعبر أفقياً بجمع بسيط، وهو ما يسمح للتدرّج بالبقاء عبر خطوات كثيرة.

تمرين الفصل العشرون

1. اكتب خلية GRU، وشغّلها عبر `scan` على تسلسل بطول 10 ودفعة 3.
2. احسب معاملاتها عند d = 128 وh = 64، وقارنها بالجدول.
3. أضف قناعاً يجمّد الحالة عند الحشو، وتحقّق من ثبات الخرج عند إطالة الحشو.

الحلّ النموذجي

```python
import jax, jax.numpy as jnp
from jax import lax

H, D = 64, 128
key = jax.random.key(0)
p = {"Wz": jax.random.normal(key, (D, H)) * 0.1,
     "Uz": jax.random.normal(key, (H, H)) * 0.1,
     "Wr": jax.random.normal(key, (D, H)) * 0.1,
     "Ur": jax.random.normal(key, (H, H)) * 0.1,
     "Wn": jax.random.normal(key, (D, H)) * 0.1,
     "Un": jax.random.normal(key, (H, H)) * 0.1}

def gru_step(p, h, x):
    z = jax.nn.sigmoid(x @ p["Wz"] + h @ p["Uz"])
    r = jax.nn.sigmoid(x @ p["Wr"] + h @ p["Ur"])
    n = jnp.tanh(x @ p["Wn"] + (r * h) @ p["Un"])
    h = (1 - z) * h + z * n
    return h, h

xs = jnp.ones((10, 3, D))
h0 = jnp.zeros((3, H))
hs = lax.scan(lambda h, x: gru_step(p, h, x), h0, xs)[1]
print(hs.shape)                          # (10, 3, 64)
print(sum(v.size for v in p.values()))   # 36864
```

2. 36,864 بلا انحيازات، مقابل 37,056 في الجدول (الفارق 192 = 3h).
3. أضف داخل الخطوة `h = jnp.where(m[:, None], h_new, h)`؛ عندها لا يغيّر تمديد الحشو أي مخرج حقيقي.

خلاصة الفصل

- الشبكة الأمامية تُلغي الترتيب؛ RNN تحفظ حالة صغيرة بدل التاريخ كلّه.
- `lax.scan` هي الحلقة: عقد (carry، مخرج) واضح بلا حلقة بايثون.
- BPTT اشتقاق عكسي على رسم مفتوح؛ ذاكرته تنمو مع T وإعادة الحساب تقايضها بالوقت.
- LSTM تحمي التدرّج بمسار جمع، وGRU تختصر البوّابات بثلث المعاملات.
- القناع يلزم في الخسارة والحالة معاً؛ وفي السلاسل الزمنية احذر التسريب.

مصطلحات الفصل

carry

الحالة التي تعبر خطوات `scan` من بدايتها إلى نهايتها.

BPTT

الانتشار العكسي عبر التسلسل المفكوك؛ قاعدة السلسلة مطبّقة على الزمن.

البوّابة

قيمة سيغمويد بين 0 و1 تضرب في متجه لتقرّر مقدار مروره.

الحالة الخلوية

مسار داخلي في LSTM يمرّ بالجمع لا بالضرب فيحفظ التدرّج أطول.

القناع

متجه منطقي يميّز المواضع الحقيقية من الحشو.

الانتباه والمحوّلات

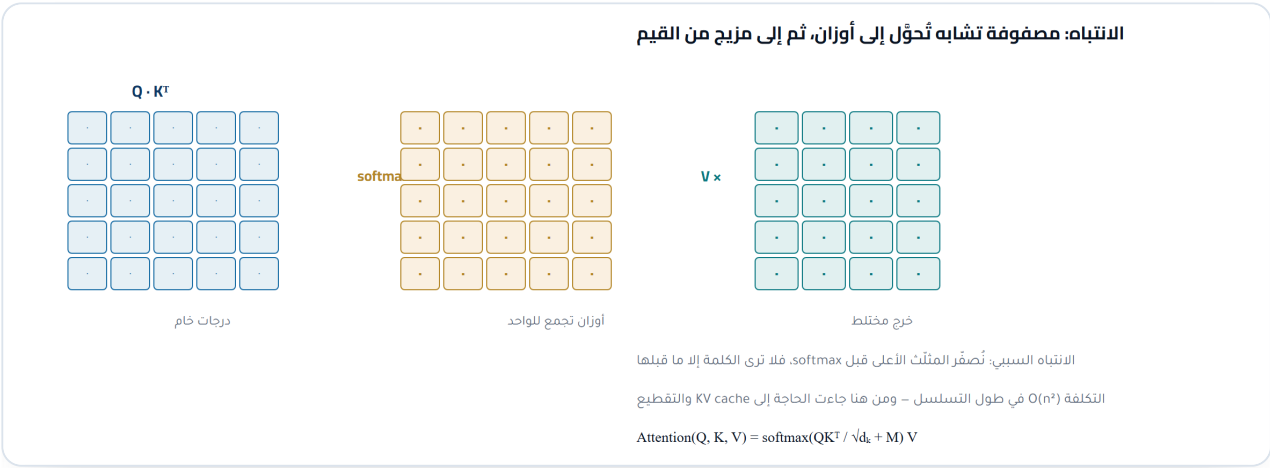
الانتباه متعدّد الرؤوس، والترميز الموضعي، وذاكرة KV

في 2014 كان أفضل مترجم آليّ يضغط جملة من عشرين كلمة في متجه من 512 عدداً ثم يعيد توليدها منه. سأل باحثون: لماذا تُقرأ الجملة كاملة قبل كتابة كلمتها الأولى؟ الجواب كان هدماً للافتراض: لا تضغط، دع كل كلمة تنظر إلى كل كلمة.

المشكلة التي حَلّها الانتباه

يسلّم المشفّر حالته الأخيرة وحدها إلى المفكّك، وعرضها ثابت مهما طال النص: ثلاثون كلمة في 512 بُعداً تعني ستة عشر عدداً ونصف للكلمة، والمعلومة التي تعبر ثلاثين ضرباً في مصفوفة نصف قطرها الطيفي أقل من واحد تتضاءل.

حلّ باهدانو في 2015 بأن يحتفظ بحالات كل الكلمات ويسأل عند كل خطوة أيّها يهمّ. ثم أزالت ورقة 2017 الحلقة: إن كان الانتباه ينظر إلى كل المواضع دفعة واحدة، فلماذا نقرأ تسلسلياً؟



الشكل 21.1 — يميناً: كل الكلمات تُضغط في متجه واحد. يساراً: كل موضع يبني تمثيله من كل المواضع بأوزان محسوبة.

لو كان عرض الحالة 4096 بدل 512، هل يختفي عنق الزجاجة؟ فكّر في جملة من ألف كلمة. ؟

Q, K, V: استعارة البحث

التشبيه الأدقّ قاموس ناعم: تبحث بسؤال فتحصل على مزيج من كل القيم مرجَّحاً بتطابق كل مفتاح. الاستعلام Q ما أبحث عنه، والمفتاح K ما يعرض نفسه، والقيمة V ما أحصل عليه. والفصل بين المفتاح والقيمة يسمح للكلمة بأن تعرض ما لا تحمله.

$$\text{Attention}(Q, K, V) = \text{softmax}(\, Q\, K^{T} / \sqrt{d_k}\,)\, V$$

كل صفّ مزيج من صفوف V، وأوزانه تجمع إلى واحد.

أمّا $\sqrt{d_k}$ فأهمّ سطر وأكثرها نسياناً: بتباين واحد يكون تباين الجداء النقطي d_k، فمع 64 تصبح الدرجات في حدود ±8 والتوزيعة شبه حادّة. ومشتقّ `softmax` يحتوي $p(1-p)$، فحين يقترب p من الطرفين يتوقّف التعلّم بصمت.

Python

```
def attention(q, k, v, mask=None):
    """q: (B,T,H,D)  k,v: (B,S,H,D)  mask: (B,1,T,S)"""
    s = jnp.einsum("bthd,bshd->bhts", q, k) / jnp.sqrt(q.shape[-1])
    if mask is not None:
        s = jnp.where(mask, s, -jnp.inf)     # قبل softmax لا بعده
    p = jax.nn.softmax(s, axis=-1)
    return jnp.einsum("bhts,bshd->bthd", p, v)
```

★ الانتباه ليس طبقة

لا معاملات في $\text{softmax}(QK^{T}/\sqrt{d})V$؛ كل ما يتعلّمه النموذج يسكن في الإسقاطات التي تُنتج Q وK وV.

الانتباه متعدّد الرؤوس

رأس واحد يعطي توزيعة واحدة، لكن الجملة تحمل علاقات نحوية ودلالية وموضعية معاً. لذا يُقسَّم الفضاء: 512 على ثمانية رؤوس تعطي 64 لكل رأس، والحساب تشكيل جديد إلى `(B,T,H,D)` ثم نقل محاور. والتكلفة ثابتة، لكن الفضاء ينقسم إلى ثمانية أجزاء لكل منها أن تتخصّص.

> ✦ **لماذا لا تتشابه الرؤوس؟**
>
> لأن التهيئة العشوائية تكسر التناظر، ولأن كل رأس يرى إسقاطاً مختلفاً. بعد التدريب تجد رؤوساً تتبع الفاعل وأخرى تتبع حروف الجر: التخصّص نتيجة لا تصميم.

الأقنعة: سببي وpadding

في التدريب يرى النموذج الجملة كاملة وهذا تسريب؛ لو نظر الموضع الثالث إلى الرابع لحفظ الجواب. القناع السببي مثلّثي سفلي، وقناع الحشو مختلف: الأصفار التي تُملأ بها الجمل القصيرة ليست كلمات، وشكله `(B,S)`.

```python
causal = jnp.tril(jnp.ones((t, t), dtype=bool))        # (T,S)
pad = tokens != 0                                      # (B,S)
m = causal[None, None] & pad[:, None, None, :]         # (B,1,T,S)
```

يُدمج القناعان بـ `&` لا بـ `|`، والقيمة في الممنوع سالب اللانهاية لا صفر؛ فالصفر يصبح $e^0 = 1$ بعد `softmax`، أي أنك منعت الموضع وأبقيته في الحساب.

> ✕ **الخطأ الصامت: قناع بشكل خاطئ**
>
> مرّر `(B,S)` إلى دالّة تتوقّع `(B,T,S)` فيُبثّ على المحور الأوسط بلا اعتراض، ويُطبَّق الحشو على الأعمدة بدل الصفوف. لا خطأ ولا NaN – فقط خسارة أعلى بنصف نقطة وسبب مجهول.

الترميز الموضعي: من sin/cos إلى RoPE

الانتباه لا يعرف الترتيب: لو بدّلت كلمتين تغيّرت النتيجة لأن V تغيّرت، لا لأن البنية ترى موضعاً. هو متكافئ مع التبديل، و«الكلب عضّ الرجل» ليست «الرجل عضّ الكلب».

الحلّ الأول جمع متجه جيبي بتردّدات متفاوتة، لكن الجمع يخلط المعنى بالموضع. جاء RoPE بالدوران: تُدار أزواج الأبعاد بزوايا تتناسب مع الموضع، فيصبح الجداء دالّة في المسافة وحدها.

$$\langle R_m\, q\, ,\, R_n\, k \rangle = \langle q\, ,\, R_{n-m}\, k \rangle\, ,\quad \theta_i = 10000^{-2i/d}$$

الدوران يحوّل «أين أنا» إلى «كم المسافة بيني وبينك».

```python
def rope(x, offset=0):
    t, d = x.shape[1], x.shape[-1]
    ang = (jnp.arange(t) + offset)[:, None] * (10000.0 ** (-jnp.arange(0, d, 2) / d))[None, :]
    cos, sin = jnp.cos(ang)[:, None], jnp.sin(ang)[:, None]
    x1, x2 = x[..., 0::2], x[..., 1::2]
    return jnp.stack([x1 * cos - x2 * sin, x1 * sin + x2 * cos], -1).reshape(x.shape)
```

ويُطبَّق داخل كل طبقة قبل الجداء، فتبقى الزاوية دالّة مستمرّة ويُمدّ السياق.

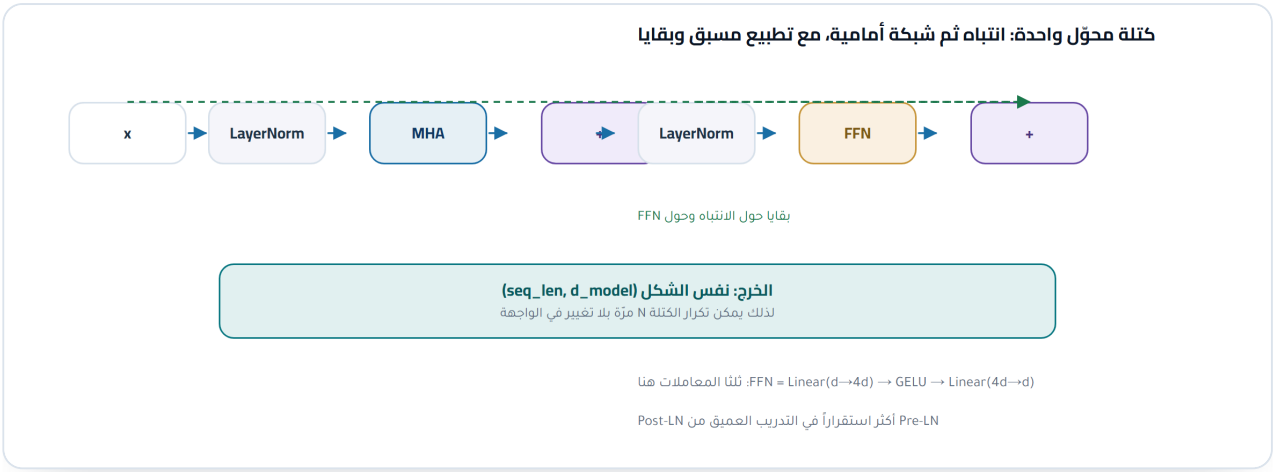
كتلة المحوّل كاملة

الكتلة أربعة أجزاء: تطبيع، وانتباه، وبقايا، وشبكة أمامية. في **pre-LN** يسبق التطبيع الدالّة الفرعية فيبقى المسار الرئيسي نقياً ويصل التدرّج إلى الطبقة الأولى سالماً، بخلاف post-LN الذي يحتاج تسخيناً.

```python
class Block(nn.Module):
    d_model: int
    n_heads: int

    @nn.compact
    def __call__(self, x, mask=None):
        h = nn.LayerNorm()(x)                                  # pre-LN
        qkv = nn.Dense(3 * self.d_model, use_bias=False)(h)
        q, k, v = jnp.split(qkv, 3, axis=-1)
        q, k, v = (z.reshape(*z.shape[:2], self.n_heads, -1) for z in (q, k, v))
        a = jax.nn.dot_product_attention(q, k, v, mask=mask)
        x = x + nn.Dense(self.d_model)(a.reshape(*a.shape[:2], self.d_model))
        h = nn.gelu(nn.Dense(4 * self.d_model)(nn.LayerNorm()(x)))
        return x + nn.Dense(self.d_model)(h)
```

والشبكة الأمامية $d \to 4d \to d$، ومعاملها أربعة مقايضة بين السعة والكلفة. و `mask` يمرّر بشكله `(B,1,T,S)` فتبثّه الدالّة على محور الرؤوس.



الشكل 21.2 — مسار رئيسي مستقيم لا يعبره سوى الجمع، وفرعان يبدآن بتطبيع. الأسهم المتجاوزة هي ما يجعل العمق ممكناً.

التكلفة: O(n²) وأين تذهب الذاكرة

مصفوفة الانتباه مربّعة في الطول، وهذا كل ما تعنيه $O(n^2)$: ثمانية ورؤوس اثنا عشر وطول ألفين تعني $8 \times 12 \times 2048 \times 2048 \times 2 = 805$ ملايين بايت لمصفوفة واحدة.

الطول T	مصفوفة الانتباه	ذاكرة KV	الحالة
8,192	12 غيغابايت	384 ميغابايت	يحتاج تفكيكاً
32,768	192 غيغابايت	1.5 غيغابايت	مستحيل بلا Flash

جدول 21.1 — حجم مصفوفة الانتباه (B=8، H=12، fp16)

الرقم الأخير سبب وجود **Flash Attention**: تُحسب الصفوف كتلاً في ذاكرة قريبة ويُجمَّع الناتج تدريجياً، فتنخفض الحركة من $O(n^2)$ إلى $O(n)$. العنق ليس الحساب بل الحركة.

✓ **استخدم** `is_causal=True`

`jax.nn.dot_product_attention(q, k, v, is_causal=True)` تعرف أن القناع مثلّثي فتختار نواة لا تحسب النصف العلوي؛ الفرق يظهر عند 4096.

عند التوليد ينمو الطول رمزاً كل خطوة. لماذا لا تكون الكلفة O(n²) لكل رمز؟ ؟

أخطاء المحوّلات الشائعة

الأربعة التالية أخطاء شكل أو ترتيب لا أخطاء معادلة، ولهذا تمرّ كلها عند التشغيل الأول.

1. **قناع مبثوث على المحور الخطأ.** السببي `(T,S)` يصبح `(1,1,T,S)` قبل دمجه بالحشو `(B,1,1,S)`.

2. **نسيان القسمة على $\sqrt{d}$.** يتدرّب النموذج ببطء مريب ثم يتجمّد؛ والحلّ تصحيح المعادلة لا معدّل التعلّم.

3. **LayerNorm في المكان الخطأ.** تطبيع بعد الجمع يقطع المسار وقبله يتركه؛ والخلط يعمل عند العمق 4 وينهار عند 24.

4. **عدم تطابق بُعد الرأس.** مع 512 و12 رأساً تحصل على 42.67، و `reshape` يرفض بعيداً عن السبب.

تمرين الفصل الحادي والعشرين

1. احسب حجم مصفوفة الانتباه لدفعة 4 ورؤوس 16 وطول 4096 بـ `bfloat16` .
2. اكتب اختباراً يتحقّق من السببية بتغيير آخر رمز في `v` .
3. احسب معاملات كتلة بـ $d = 768$ ومعامل توسيع 4.

الحلّ النموذجي

1. $4 \times 16 \times 4096 \times 4096 \times 2 = 2.1$ غيغابايت للمصفوفة وحدها؛ ومع 7 مليارات معامل بـ14 بتاً لا تتّسع بطاقة 24 غيغابايت.

2.

```python
k1, k2, k3 = jax.random.split(jax.random.key(0), 3)
q, k, v = (jax.random.normal(z, (1, 6, 2, 8)) for z in (k1, k2, k3))

out = jax.nn.dot_product_attention(q, k, v, is_causal=True)
v2 = v.at[:, -1].set(0.0)                       # آخر رمز فقط
out2 = jax.nn.dot_product_attention(q, k, v2, is_causal=True)

print(bool(jnp.allclose(out[:, :-1], out2[:, :-1])))   # True
print(bool(jnp.allclose(out[:, -1], out2[:, -1])))     # False
```

3. $4d^2 + 8d^2 = 12d^2 = 7{,}077{,}888$ معاملاً، وهو مجموع الانتباه والشبكة الأمامية.

خلاصة الفصل

- الانتباه يحلّ عنق الحالة المخفية: لا تلخيص قبل التوليد بل نظر إلى كل المواضع.
- ثلاثة إسقاطات تحوّله إلى قاموس ناعم، و$\sqrt{d}$ يمنع تشبّع `softmax` .
- الرؤوس تقسّم الفضاء لا الحساب، والأقنعة تعمل بـ $-\infty$ قبل `softmax` وأخطاؤها صامتة.

مصطلحات الفصل

الانتباه الذاتي

انتباه تُشتقّ فيه الاستعلامات والمفاتيح والقيم من التسلسل نفسه.

القناع السببي

مصفوفة مثلثية تمنع الموضع من النظر إلى ما بعده أثناء التدريب.

pre-LN

ترتيب يضع التطبيع قبل الدالّة الفرعية فيبقى المسار الرئيسي نقياً.

RoPE

ترميز موضعي بالدوران يجعل الجداء دالّة في المسافة بين الموضعين.

ذاكرة KV

مفاتيح وقيم المواضع السابقة المحفوظة عند التوليد لتفادي إعادة الحساب.

GPT صغير من الصفر

الترميز، والكتلة، والتدريب، والتوليد، والتقطيع

كل من قرأ عن المحوّلات يعرف المعادلة، وقليل من كتب واحداً حتى يُخرج نصاً. الفرق ليس ذكاءً بل تجربة: أن ترى الخسارة تهبط من 4.2 إلى 1.6، وأن تقرأ بعد ساعة نصاً مكسوراً لكنه ليس عشوائياً. هنا تبني نموذجاً حرفياً بمليون معامل، ثم تسأل: ما الذي يفصلك عن مليار؟

ما الذي نبنيه بالضبط؟

الهدف نموذج يتوقّع الرمز التالي عند كل موضع. اخترنا الحرف لأن قاموسه ستة وستون رمزاً، فلا مُرمِّز خارجي ولا جدول ضخم.

الحجم: خمس طبقات، عرض 128، أربعة رؤوس، سياق 128 — نحو 1.02 مليون معامل، يتدرّب في دقائق ويُنتج بنية لغوية لا ضجيجاً.

المعامل	القيمة	الأثر
`vocab`	66	حروف + علامات
`ctx`	128	أقصى سياق
`d_model`	128	عرض التمثيل
`n_heads`	4	بُعد الرأس 32
المعاملات	1,018,880	نحو مليون

جدول 22.1 — فرط معاملات النموذج الصغير

الترميز: من حرف إلى متجه

القاموس يُبنى من النص: رتّب الحروف المتفرّدة، أعطِ كل واحد رقماً، واحفظ المعجمين. والبدء بالحرف يبقي كل ما بعده على أعداد صحيحة، فتبقى الأخطاء في طبقة واحدة.

```python
text = open("input.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: jnp.array([stoi[c] for c in s], dtype=jnp.int32)
decode = lambda t: "".join(itos[int(i)] for i in t)
```

في نموذج حقيقي تستبدله بـBPE وقاموس من 32 ألفاً إلى 128 ألفاً؛ فالحرف يطيل التسلسل أربعة أضعاف، وكل تضاعف في الطول يضاعف كلفة الانتباه أربعاً.

> ✓ **ابدأ صغيراً حتى النهاية**
>
> درّب النموذج الحرفي على ميغابايت واحد أولاً. إن لم يتعلّم هذا فلن يتعلّم الكبير؛ وكل ساعة في تصحيح البيانات تُوفَّر مضاعفةً لاحقاً.

النموذج

ثلاثة أجزاء: تضمين الرموز، ومجموعة كتل، وإسقاط إلى القاموس. ويُضاف متجه موضعي متعلَّم لأن الانتباه لا يرى الترتيب، والخرج توزيعة لا حرف.

Python

```python
import flax.linen as nn
import jax
import jax.numpy as jnp

class Block(nn.Module):
    n_heads: int

    @nn.compact
    def __call__(self, x, mask):
        d = x.shape[-1]
        h = nn.LayerNorm()(x)
        qkv = nn.Dense(3 * d, use_bias=False, name="qkv")(h)
        q, k, v = jnp.split(qkv, 3, axis=-1)
        q, k, v = (z.reshape(z.shape[0], z.shape[1], self.n_heads, -1) for z in (q, k, v))
        a = jax.nn.dot_product_attention(q, k, v, mask=mask)
        x = x + nn.Dense(d, name="proj")(a.reshape(a.shape[0], a.shape[1], d))
        h = nn.gelu(nn.Dense(4 * d, name="fc")(nn.LayerNorm()(x)))
        return x + nn.Dense(d, name="fc2")(h)

class GPT(nn.Module):
    vocab: int
    ctx: int
    d_model: int = 128
    n_heads: int = 4
    n_layers: int = 5

    @nn.compact
    def __call__(self, idx):
        b, t = idx.shape
        tok = nn.Embed(self.vocab, self.d_model, name="tok")(idx)      # (B,T,D)
        pos = self.param("pos", nn.initializers.normal(0.02), (self.ctx, self.d_model))
        x = tok + pos[:t]
        mask = jnp.tril(jnp.ones((t, t), dtype=bool))[None, None]      # (1,1,T,T)
        for _ in range(self.n_layers):
            x = Block(self.n_heads)(x, mask)
        return nn.Dense(self.vocab, name="head")(nn.LayerNorm()(x))    # (B,T,V)
```

القناع يُبنى مرّة خارج الحلقة لأن كل الطبقات تراه. و `nn.Embed` يأخذ رموزاً صحيحة، و `nn.Dense` الأخير يعيد `V` عدداً.

★ الشكل هو العقد

المدخل `(B,T)` والخرج `(B,T,V)` وما بينهما ثابت. أكثر أخطاء المحوّل تظهر شكلاً غريباً قبل أن تظهر قيمة خاطئة: اطبع الأشكال أولاً.

الخسارة: التوقّع للرمز التالي

الحيلة في الإزاحة: المدخل `idx[:, :-1]` والهدف `idx[:, 1:]`، فالموضع `t` يتوقّع `t+1` وتُحسب `T−1` تنبؤات في مرور واحد.

$$L = -\,(1\,/\,(B\cdot(T-1)))\,\sum_{b,t} \log p_\theta(\,x_{t+1} \mid x_{\le t}\,)$$

متوسّط على كل المواضع، لا على آخرها.

وإن كان في الدفعة حشو فاستبدل المتوسّط بمرجَّح يقصيه، وإلّا تعلّم النموذج توقّع الصفر بعد نهاية النص.

× خطأ الإزاحة

لو مرّرت `idx` مدخلاً وهدفاً معاً، لَتعلّم النموذج النسخ وحصلت على خسارة تقارب الصفر في دقائق. النموذج لا يغشّ؛ أنت من أعطاه الجواب في السؤال.

التدريب

ثلاث أدوات تفصل تدريباً ناجحاً عن تدريب ينفجر: تسخين المعدّل في أول مئة خطوة، وقصّ التدرّج عند واحد، وفصل الوزن في AdamW؛ فالتدرّجات الأولى كبيرة وغير مهيكلة.

```python
import optax

def loss_fn(params, idx):
    logits = model.apply({"params": params}, idx[:, :-1])
    logp = jax.nn.log_softmax(logits, axis=-1)
    return -jnp.mean(jnp.take_along_axis(logp, idx[:, 1:, None], axis=-1))

schedule = optax.warmup_cosine_decay_schedule(
    init_value=0.0, peak_value=3e-3, warmup_steps=100,
    decay_steps=2000, end_value=3e-4)
tx = optax.chain(optax.clip_by_global_norm(1.0),
                 optax.adamw(schedule, weight_decay=0.01))
opt_state = tx.init(params)

@jax.jit
def step(params, opt_state, idx):
    loss, grads = jax.value_and_grad(loss_fn)(params, idx)
    updates, opt_state = tx.update(grads, opt_state, params)
    return optax.apply_updates(params, updates), opt_state, loss
```

الخسارة تبدأ قرب $\ln(66) \approx 4.19$ – توزيعة منتظمة – وتهبط إلى 2.4 في مئتي خطوة ثم تتباطأ. عند 1.6 يحترم النص المسافات، وعند 1.4 تظهر كلمات كاملة. وإن تجمّدت عند 4.19 افحص المعدّل أولاً.

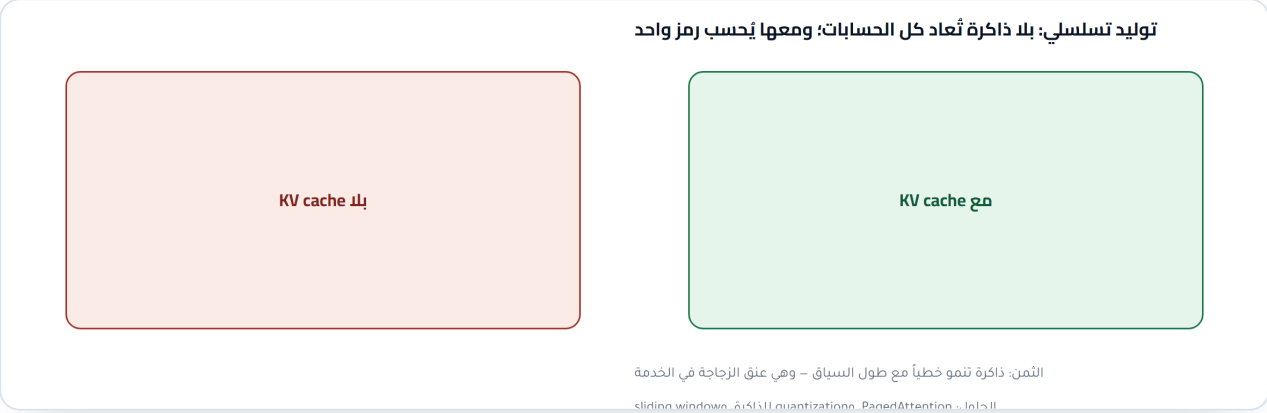
> ⚠ **خسارة 0.0 ليست نجاحاً**
>
> وصولها إلى الصفر في خطوات قليلة يعني تسريباً: هدف مساوٍ للمدخل أو قناع معطوب. النموذج الجيد يتذبذب حول قيمة لا يبلغها.

التوليد

التوليد تدريب معكوس: خذ آخر توزيعة، اختر رمزاً، ألحقه، وأعد. الحرارة تقسم اللوغاريتمات – أقل من واحد تُركّز وأكثر تُفرد – top-k يقصّ ما دون الرمز k فيمنع الشواذ لا التنوّع.

```python
def generate(params, key, prompt, n_new, temperature=1.0, top_k=None):
    def one(carry, _):
        window, key = carry
        logits = model.apply({"params": params}, window)[:, -1, :] / temperature
        if top_k is not None:
            thr = jnp.sort(logits, -1)[:, -top_k][:, None]
            logits = jnp.where(logits < thr, -jnp.inf, logits)
        key, sub = jax.random.split(key)
        nxt = jax.random.categorical(sub, logits, axis=-1)
        window = jnp.concatenate([window[:, 1:], nxt[:, None]], axis=1)
        return (window, key), nxt
    (_, _), out = jax.lax.scan(one, (prompt, key), None, length=n_new)
    return out.T
```

`scan` يحوّل الحلقة إلى حلقة مُجمَّعة تُبنى مرّة، لكن الكلفة تبقى تربيعية لأن كل خطوة تعيد حساب الانتباه على النافذة كاملة.



الشكل 22.1 — أعلى: كل خطوة تعيد حساب المفاتيح والقيم. أسفل: تُحفظ مرة واحدة ويُضاف رمز، فتنخفض الكلفة من تربيعية إلى خطّية.

ذاكرة KV هي الفرق بين نموذج يُجرَّب وآخر يُخدم: تُخزَّن المفاتيح والقيم وتُمدّ بدل إعادة حسابها، بثمن ذاكرة تنمو خطّياً مع الطول.

لماذا تُنتج الحرارة 0.7 نصاً أفضل من 1.0 في نموذج صغير، بينما تُسيء إليه في نموذج كبير؟ ؟

التقطيع

النموذج لا يتغيّر حين تنتقل من بطاقة إلى ثمانٍ؛ ما يتغيّر مكان المصفوفات: تكرار على محور الدفعة وتقسيم على محور العرض. لا تحتاج `pmap` – واجهة قديمة – بل تُعلن التقطيع على المصفوفات.

```python
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P
import numpy as np

mesh = Mesh(np.array(jax.devices()).reshape(2, 4), ("dp", "tp"))
rep, shard_w = NamedSharding(mesh, P()), NamedSharding(mesh, P(None, "tp"))

params = jax.tree.map(lambda p: jax.device_put(p, shard_w if p.ndim > 1 else rep), params)
batch = jax.device_put(batch, NamedSharding(mesh, P("dp")))
params, opt_state, loss = jax.jit(step)(params, opt_state, batch)
```

مع أربعة أجهزة على محور العرض يصبح كل وزن بعرض 128 مقسوماً إلى 32، ويُعاد التجميع عند كل طبقة. الشبكة هنا أصغر من أن تستفيد؛ النفع يبدأ عند مئات الملايين.

من لعبة إلى نموذج حقيقي

ما يتغيّر عند مليار معامل ليس الكتلة بل خمسة أشياء حولها: البيانات من ميغابايت إلى تريليون رمز، والمُرمِّز من حروف إلى BPE، والدقّة إلى bf16 مع تجميع في fp32، والتقطيع إلى آلاف الأجهزة، والتقييم إلى مهام محجوزة.

والمقايضة تتبع قانون قوة: الخسارة تنخفض مع الحجم والبيانات بمعدّل ثابت لوغاريتمياً، والكلفة تنمو تربيعياً في الطول وخطّياً في الحجم.

> لو ضاعفت السياق من 128 إلى 1024، أيّ جزء من الحساب ينمو وأيّها يبقى ثابتاً؟ ؟

تمرين الفصل الثاني والعشرين

1. أضف `dropout` بنسبة 0.1 إلى الكتلة، وتأكّد من تعطيله عند التوليد.
2. غيّر السياق إلى 256، واحسب أثر ذلك على المعاملات وعلى مصفوفة الانتباه.
3. لماذا لا يكفي `jax.random.categorical` بلا حرارة لتوليد متنوّع؟

الحلّ النموذجي

1. يُمرَّر `deterministic` كعلم إلى الكتلة:

```python
class Block(nn.Module):
    n_heads: int
    drop: float = 0.1

    @nn.compact
    def __call__(self, x, mask, deterministic):
        ...
        x = x + nn.Dropout(self.drop, deterministic=deterministic)(attn)
        return x + nn.Dropout(self.drop, deterministic=deterministic)(ffn)
```

عند التدريب `deterministic=False` مع مفتاح جديد لكل دفعة، وعند التوليد `True` فيصبح الإسقاط تطابقاً.

2. المعاملات تزيد في جدول المواضع فقط: $256 \times 128 - 128 \times 128 = 16{,}384$ ، ومصفوفة الانتباه تكبر أربع مرّات.

3. لأن التوزيعة الخام تعكس تردّدات التدريب فيتكرّر الرمز الشائع؛ والحرارة وtop-k يعيدان توزيع الاحتمال قبل السحب.

خلاصة الفصل

- نموذج حرفي بمليون معامل يُظهر سلوكيات المحوّل كلها في دقائق.
- الإزاحة بين المدخل والهدف تجعل كل موضع موضعَ تدريب.
- تسخين المعدّل وقصّ التدرّج وفصل الوزن شروط لتدريب لا ينفجر.
- `scan` يجمّع التوليد، وذاكرة KV تخفض كلفته من تربيعية إلى خطّية.

مصطلحات الفصل

نموذج لغوي حرفي

نموذج يتوقّع الحرف التالي، وقاموسه مجموع الحروف المتفرّدة في النص.

الإزاحة

تقديم المدخل موضعاً وتأخير الهدف موضعاً ليصبح كل موضع هدفاً للتنبؤ.

الحرارة

قاسم على اللوغاريتمات قبل `softmax` يضيّق التوزيعة أو يوسّعها.

top-k

إبقاء أعلى k احتمالات وإسقاط الباقي قبل السحب.

ذاكرة KV

مفاتيح وقيم المواضع السابقة المحفوظة لتفادي إعادة حسابها.

النماذج التوليدية

VAE، وGAN، ونماذج الانتشار: كيف يولّد النموذج ما لم يره

ثلاثة باحثين أرادوا الشيء نفسه: أن يولّد نموذجٌ صوراً لم يرها. الأول درّب شبكتين تتخاصمان، والثاني شبكة تُرمّز إلى توزيع، والثالث شبكة تتعلم أن تُزيل ضجيجاً أضفته أنت. ثلاث فلسفات، ونتيجة واحدة تقريباً.

ثلاث عائلات، ثلاث فلسفات

لديك عيّنات من توزيع مجهول $p(x)$ وتريد غيرها. GAN تتعلّم دالّة تحويل يُحكم عليها خصم. VAE تُنمذج توزيعاً كامناً وتُقرّب الاحتمال بحدّ أدنى. والانتشار يتعلّم إزالة ضجيج تدريجية، فيصبح التوليد سلسلة تنظيف.

العائلة	ما تتعلّمه	الخسارة	العيب الأول
GAN	دالّة توليد ضمنياً	لعبة خصمين	انهيار الأنماط
VAE	توزيع كامن وفكّ	ELBO	ضبابية العيّنات
الانتشار	حقل إزالة ضجيج	خطأ تنبؤ الضجيج	بطء التوليد

جدول 23.1 — العائلات الثلاث في سطر لكل منها

لماذا لا تكفي خسارة واحدة — كمتوسط الخطأ التربيعي بين العيّنة والبيانات — لتوليد صور؟ ؟

VAE: الترميز إلى توزيع

لكل صورة موقع في فضاء كامن، لكن النقطة لا تُنتج تنوّعاً؛ لذا يُخرج المشفّر توزيعاً: μ و $\log \sigma^2$. ولأن السحب لا يُشتقّ، تُستخدم إعادة المعلمة: اسحب ε وحرّكها.

$$z = \mu + \sigma \odot \varepsilon \,, \varepsilon \sim \mathrm{N}(0, \mathrm{I})$$

العشوائية خرجت من المسار القابل للاشتقاق، فبقي التدرّج يمرّ عبر μ و σ.

والخسارة حدّان يتنازعان: دقة إعادة البناء، وقرب الترميز من الطبيعي القياسي:

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

الأول خطأ إعادة البناء، والثاني ثمن المعلومات: كم «بتّاً» تدفع مقابل تمثيل خاص بكل عيّنة.

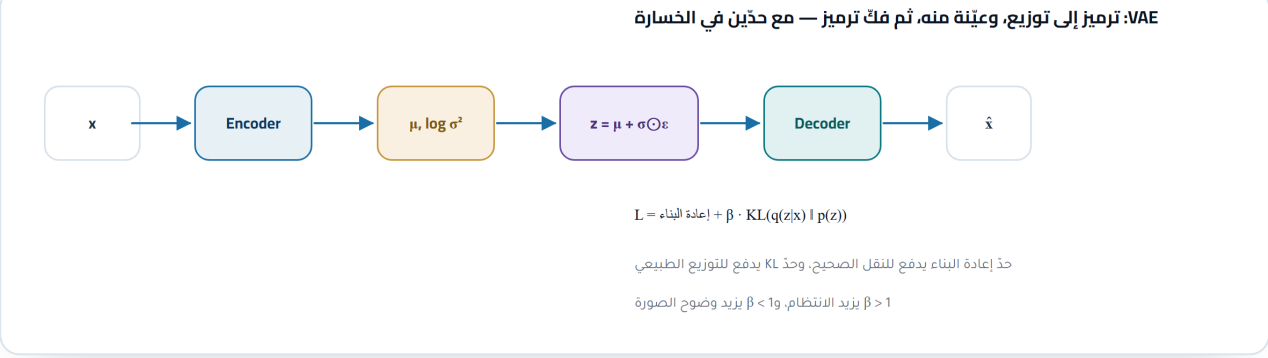
والحدّ الثاني بصيغة مغلقة: $-\tfrac{1}{2}\sum(1+\log\sigma^2-\mu^2-\sigma^2)$؛ فهو يعاقب μ البعيدة عن الصفر و σ البعيدة عن الواحد.

```python
class Encoder(nn.Module):
    latent: int = 16

    @nn.compact
    def __call__(self, x):
        h = nn.relu(nn.Dense(256)(x))
        h = nn.relu(nn.Dense(256)(h))
        return nn.Dense(self.latent)(h), nn.Dense(self.latent)(h)   # μ, log σ²

class Decoder(nn.Module):
    @nn.compact
    def __call__(self, z):
        h = nn.relu(nn.Dense(256)(z))
        return nn.Dense(784)(nn.relu(nn.Dense(256)(h)))             # logits

def vae_loss(params, x, key):
    mu, logvar = Encoder().apply({"params": params["enc"]}, x)
    z = mu + jnp.exp(0.5 * logvar) * jax.random.normal(key, mu.shape)
    logits = Decoder().apply({"params": params["dec"]}, z)
    rec = jnp.mean(jnp.sum(optax.sigmoid_binary_cross_entropy(logits, x), -1))
    kl = -0.5 * jnp.mean(jnp.sum(1 + logvar - mu**2 - jnp.exp(logvar), -1))
    return rec + kl
```



الشكل 23.1 — المشفّر يُخرج توزيعاً لا نقطة، والسحب منه هو ما يجعل الفضاء الكامن قابلاً للتوليد.

⚠ انهيار الحدّ الثاني

إن ثقل الحدّ KL صار $q(z|x)$ هو الطبيعي القياسي لكل المدخلات، فلم يبقَ في الترميز شيء. الحلّ وزن $\beta < 1$ أو تسخينه تدريجياً.

GAN: خصمان في لعبة

المولّد G يحوّل ضجيجاً إلى عيّنة، والمميّز D يفرّق بينهما. اللعبة مجموعها صفر، والمطلوب توازن لا حدّ أدنى. ويُدرَّب المولّد على $-\log D(G(z))$ لأن الحدّ الأصلي يتوقّف عن إعطاء تدرّج حين ينجح المميّز.

```python
def gan_step(g_params, d_params, real, key):
    z = jax.random.normal(key, (real.shape[0], 32))
    fake = G().apply({"params": g_params}, z)

    def d_loss(dp):
        r = D().apply({"params": dp}, real)
        f = D().apply({"params": dp}, fake)
        return jnp.mean(jax.nn.softplus(-r)) + jnp.mean(jax.nn.softplus(f))

    def g_loss(gp):
        f = D().apply({"params": d_params}, G().apply({"params": gp}, z))
        return jnp.mean(jax.nn.softplus(-f))

    return jax.grad(d_loss)(d_params), jax.grad(g_loss)(g_params)
```

انهيار الأنماط هو العطب المميّز: يكتشف المولّد صورةً تُرضي المميّز فيولّدها لكل مدخل، لأن التدرّج يُكافئ كل عيّنة ولا شيء يطلب تغطية التوزيع.

> ✕ **تحديث المميّز قبل حساب تدرّج المولّد**
>
> لو حدّثت `d_params` ثم حسبت تدرّج المولّد، فأنت تُدرّب ضد خصم غير الذي قاس خسارتك. احسب التدرّجين من الحالة نفسها ثم حدّث.

الانتشار: الفكرة الأنيقة

بدل الانتقال من ضجيج إلى صورة في خطوة، انتقل من صورة إلى ضجيج في ألف خطوة صغيرة ثم تعلّم عكسها. الأمام ثابت، والخلفية شبكة تتنبأ بالضجيج.

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\, I\right),\quad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$$

$\bar{\alpha}_t$ جداء $(1-\beta_s)$ حتى الخطوة t، فالقفز إلى أيّ خطوة ممكن بلا محاكاة ما قبلها.

ولأن الضجيج معروف تصبح الخسارة تنبؤاً به لا احتمالاً ولا لعبة:

$$L = \mathbb{E}_{t,\, x_0,\, \varepsilon}\left\| \varepsilon - \varepsilon_\theta(x_t, t) \right\|^2$$

توقّع بلا أوزان زمنية؛ الورقة الأصلية وجدت أن حذفها يعمل أفضل.

> ★ **التوليد تعلّمُ إزالة ضجيج**
>
> الشبكة لا ترى صورة نظيفة أبداً؛ ترى خليطاً بنسبة معروفة وتتعلّم الجزء الغريب، والتوليد تطبيقه مراراً من ضجيج إلى صورة.

أخذ العيّنات

العكس تقريب غاوسي: تُقدّر الشبكة الضجيج وتطرح نسبته ويُضاف قليل جديد إلا في الخطوة الأخيرة. DDPM يحتاج ألف خطوة؛ وDDIM يعيد صياغتها كسلسلة حتمية تقفز فخمسون تكفي.

والتوجيه بلا مصنّف يضيف فرقاً واحداً: احسب الضجيج بشرط النص وبدونه ثم اقتَرح $\varepsilon = \varepsilon_\varnothing + s(\varepsilon_c - \varepsilon_\varnothing)$. وكلما كبر s زاد الالتزام وقلّ التنوّع.

> لماذا لا تُدرَّب الشبكة على خطوة واحدة كبيرة بدل ألف خطوة صغيرة؟ ؟

التدريب في JAX

الخطوة أربعة أسطر: اسحب خطوة زمنية، أضف الضجيج الموافق، اطلب تقديره، واحسب الخطأ. لا حلقة على الزمن لأن x_t يُحسب من x_0 مباشرة.

```python
T = 1000
betas = jnp.linspace(1e-4, 0.02, T)
abar = jnp.cumprod(1.0 - betas)

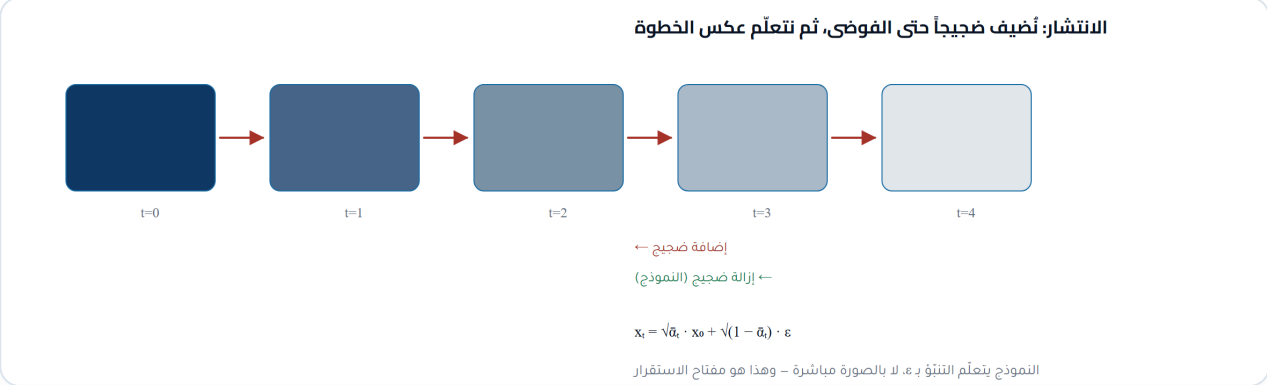
def train_step(params, x0, key):
    k1, k2 = jax.random.split(key)
    t = jax.random.randint(k1, (x0.shape[0],), 0, T)
    eps = jax.random.normal(k2, x0.shape)
    a = abar[t][:, None, None, None]
    xt = jnp.sqrt(a) * x0 + jnp.sqrt(1 - a) * eps

    def loss(p):
        return jnp.mean((model.apply({"params": p}, xt, t) - eps) ** 2)

    return loss(params), jax.grad(loss)(params)
```

الخطوة الزمنية تُمرَّر كعدد صحيح وتُحوَّل إلى تضمين جيبي. وbf16 مقبول هنا: الضجيج عشوائي أصلاً، وخطأ التقريب أصغر من التباين المطلوب.

> ✓ **ابدأ بـ32×32**
>
> الخطوة أسرع عشر مرّات. معظم أخطاء الانتشار – جدول $\bar{\alpha}$ خاطئ أو تضمين زمني مقلوب – تظهر عند 32 كما عند 256.



الشكل 23.2 — الأمام ثابت بلا تعلّم: صورة تتلاشى في ضجيج. العكسي وحده مُتعلَّم، ويُطبَّق مرّة لكل خطوة توليد.

التقييم

FID يقيس مسافة بين توزيعَي ميزات الصور الحقيقية والمولَّدة كما تراها شبكة تصنيف، وهو حساس للمعالجة وحجم العيّنة. ولأنه رقم واحد يخفي نمطين – غير واقعية أو غير متنوّعة – يُقرن بدقّة واستدعاء.

الدقّة تسأل: هل العيّنة واقعية؟ والاستدعاء: هل غطّينا التوزيع؟ نموذج يكرّر صورة واحدة ممتازة له دقّة عالية واستدعاء صفري. ثم انظر إلى العيّنات: أثر يتكرّر في زاوية كل صورة يمرّ من كل مقياس ويراه الإنسان في ثانية.

أيّ نموذج تختار؟

القرار ثلاث نقاط: سرعة التوليد مقابل جودته، ووفرة البيانات، ودقّة التحكّم.

المهمة	الاختيار	السبب
توليد فوري على الطرف	GAN	خطوة واحدة
بيانات قليلة	GAN مع تحكّم	إشارة الخصم غنية
جودة وتنوّع	انتشار	تغطية أفضل للتوزيع

جدول 23.2 — الاختيار بحسب المهمة وميزانية البيانات

تمرين الفصل الثالث والعشرين

1. احسب KL لـ $\mu = 1، \sigma^2 = 1$ ثم لـ $\mu = 0، \sigma^2 = 0.01$.
2. لماذا يُضاف $\sqrt{(1-\bar{\alpha})}$ لا $1-\bar{\alpha}$ في معادلة الضجيج؟
3. اكتب حلقة توليد DDIM من عشر خطوات فوق شبكة مدرَّبة.

الحلّ النموذجي

1. الحدّ $-½(1 + \log \sigma^2 - \mu^2 - \sigma^2)$: الأولى 0.5 ، والثانية 1.807 – أبعد عن الطبيعي فتدفع أكثر.
2. لأن التباين يبقى واحداً: $\bar{\alpha} + (1-\bar{\alpha}) = 1$ ، والانحراف جذره. ولو أضفت $1-\bar{\alpha}$ لانهار التباين.
3.

```python
ts = jnp.linspace(T - 1, 0, 10).astype(jnp.int32)
x = jax.random.normal(key, (1, 32, 32, 3))

for i, t in enumerate(ts):
    eps = model.apply({"params": params}, x, jnp.array([t]))
    a_t = abar[t]
    a_prev = abar[ts[i + 1]] if i + 1 < len(ts) else 1.0
    x0 = (x - jnp.sqrt(1 - a_t) * eps) / jnp.sqrt(a_t)
    x = jnp.sqrt(a_prev) * x0 + jnp.sqrt(1 - a_prev) * eps
```

ما يتغيّر عدد الخطوات والمسافة بينها، لا الشبكة ولا معاملاتها.

خلاصة الفصل

- GAN لعبة بلا احتمال صريح، وVAE حدّ أدنى، والانتشار تعلّم إزالة ضجيج.
- إعادة المعلمة تُخرج العشوائية من مسار التدرّج، وKL ثمن المعلومات.
- DDIM يقصّر المسار بلا إعادة تدريب، والتوجيه يقايض التنوّع بالالتزام.

مصطلحات الفصل

ELBO

حدّ أدنى للاحتمال اللوغاريتمي يُعظَّم بدل الاحتمال نفسه.

إعادة المعلمة

كتابة السحب العشوائي كدالّة في معاملات قابلة للاشتقاق.

انهيار الأنماط

حالة يولّد فيها المولّد تنوّعاً ضعيفاً يرضي المميّز.

$\bar{\alpha}$

جداء معاملات البقاء حتى خطوة زمنية، يحدّد نسبة الإشارة إلى الضجيج.

DDIM

أخذ عيّنات حتمي يقفز خطوات كبيرة بدل ألف خطوة صغيرة.

الجزء الخامس

الأداء والتوسّع

XLA، والتقطيع، وPallas، والذاكرة، والدقة العددية

XLA من الداخل

HLO، والدمج، وحدود التجميع، وذاكرة الترجمة

في ورشة عمل، سأل أحدهم سؤالاً بدا بريئًا: «لماذا تستغرق أوّل خطوة تدريب نصف دقيقة، ثم تصير كل خطوة بعدها أسرع بمئة مرّة؟». أجاب المدرّب: «لأن JAX تُجمّع الدالّة». لكن السؤال الحقيقي لم يُطرح: ماذا يعني «تُجمّع»؟ ما الذي يحدث بين استدعاء `jax.jit` ووصول الأرقام إلى الذاكرة؟

ما هو XLA فعلاً؟

الخطأ الأوّل أن تتصوّر XLA طبقة تشغيل (runtime) أسرع. XLA مترجم: يأخذ برنامجًا ويُنتج آخر، ثم ينسى الأوّل. لا محرّك يقرأ عملياتك أثناء التنفيذ؛ بل ملف ثنائي (executable) وُلد قبل أن ترى أوّل رقم.

حين تكتب `y = tanh(x) * x`، لا ترى XLA عمليتين، بل رسمًا بيانيًا واحدًا، وحرّيتها أن تدمج وتحذف وتعيد ترتيب الذاكرة. المكتبة العادية ملزَمة بتنفيذ كل استدعاء كما جاء.

> ★ **الفرق الجوهري**
>
> مكتبة عادية تجيب على «كيف أنفّذ هذه العملية؟». المترجم يجيب على «ما أقلّ عمل يكفي للحصول على هذه النتيجة؟». السؤال الثاني مصدر كل سرعة XLA، ومصدر كل قيوده أيضًا.

> إذا كان XLA مترجمًا لا محرّكًا، فلماذا تتغيّر سرعة التشغيل بتغيّر شكل المدخلات، مع أن الشيفرة لم تتغيّر بحرف؟ ؟

HLO: اللغة الوسيطة

HLO اختصار High Level Operations، واسمه مضلّل: «عالٍ» نسبةً إلى تعليمات المعالج لا إلى بايثون. لغة تعليمات صغيرة عديمة الحالة، كل تعليمة تُنتج قيمة جديدة ولا تعدّل قديمة. هذا القيد يسمح بإعادة ترتيب التعليمات، لأن ترتيبها لا يحمل معنى.

وللدالّة شكل ثابت: مصفوفات بأشكال وأنواع معلومة تدخل، ومثلها يخرج. لا قوائم ولا كائنات ولا مؤشّرات؛ كل شيء مستطيل. لهذا يحسب XLA الذاكرة قبل التشغيل بدقّة – وهو ما رأيته في الفصل الثاني حين حجز ذاكرة البطاقة كاملة.

> ⚠ **HLO ليس عقدًا مستقرًّا**
>
> لا تبنِ أدوات على صياغة HLO النصّية. أسماء العمليات وتنسيق الطباعة يتغيّران بين إصدارات XLA بلا إعلان. اقرأه للتشخيص، لا للتحليل الآلي.

كيف ترى HLO بنفسك؟

لا تحتاج أدوات خارجية. كل دالّة مغلّفة تحمل ثلاث طبقات: `lower` يُرجع كائنًا مجمّعًا أدنى، و`compile` يُرجع التنفيذي، و`as_text` يُخرج التمثيل النصّي.

```python
import jax
import jax.numpy as jnp

@jax.jit
def f(x):
    return jnp.sum(jnp.tanh(x) * x)

x = jnp.ones((8, 8), dtype=jnp.float32)

lowered = f.lower(x)
print(lowered.as_text())              # HLO قبل التحسين
exe = lowered.compile()
print(exe.as_text())                  # HLO بعد التحسين
```

الخرج الأول، مبسّطًا، يبدو هكذا:

```
نص
```

```
module @jit_f {
  func.func public @main(%arg0: f32[8,8]) -> f32[] {
    %0 = stablehlo.tanh %arg0 : f32[8,8]
    %1 = stablehlo.multiply %0, %arg0 : f32[8,8]
    %2 = stablehlo.constant dense<0.000000e+00> : f32[]
    %3 = stablehlo.reduce(%1 init: %2)
           applies stablehlo.add across dimensions = [0, 1]
           : (f32[8,8], f32[]) -> f32[]
    return %3 : f32[]
  }
}
```

اقرأه سطرًا سطرًا. الأوّل يعلن التوقيع: مدخل `f32[8,8]` ومخرج `f32[]`، أي عدد مفرد. الثاني يحسب `tanh` ويسمّي النتيجة `%0`. الثالث يضربها في المدخل الأصلي – استُخدم `%arg0` مرّتين، وهذا مسموح لأن القيم غير قابلة للتعديل. الرابع ثابت الصفر للجمع، والخامس اختزال على البعدين ينتج عددًا واحدًا.

> ✓ **اطبع الاثنين دائمًا**
>
> مقارنة النصّ قبل التحسين وبعده أسرع طريقة لفهم ما فعله المترجم. في النسخة المحسّنة تعليمات أقل وأسماء تحمل `fusion`. إن لم يتغيّر شيء فمشكلتك في الشيفرة.

الدمج (fusion): أهمّ تحسين

تخيّل ثلاث عمليات متتالية على مصفوفة من مئة مليون عدد `f32`، أي 400 ميغابايت. بدون دمج، كل عملية تقرأ المصفوفة من ذاكرة البطاقة وتكتبها من جديد. مع الدمج، تُقرأ مرة وتُكتب مرة.

$$T \approx (R + W) / B$$

الزمن يحدّده حجم النقل مقسومًا على عرض النطاق B، لا عدد العمليات.

عند عرض نطاق 2 تيرابايت/ثانية: بدون دمج 2.4 غيغابايت ÷ 2 TB/s ≈ 1.2 ميلي ثانية؛ مع الدمج 0.8 ÷ 2 ≈ 0.4 ميلي ثانية. ثلاثة أضعاف السرعة، ولم نغيّر سطرًا من الحساب. لهذا يقول المهندسون إن التعلّم العميق مشكلة ذاكرة أكثر منه مشكلة حساب.

الحالة	عدد النوى	قراءة	كتابة	الزمن عند 2 TB/s
بدون دمج	3	1.2 GB	1.2 GB	≈ 1.2 ms

الحالة	عدد النوى	قراءة	كتابة	الزمن عند 2 TB/s
مع دمج منتج-مستهلك	1	0.4 GB	0.4 GB	≈ 0.4 ms

جدول 24.1 — سلسلة من ثلاث عمليات على مصفوفة 100M عنصر (400 MB)

النمط الذي يتعرّف عليه XLA اسمه producer-consumer fusion: مخرجات تُستهلك مرة واحدة ولا يراها أحد آخر لا داعي لكتابتها أصلًا. هذا سبب أن دالّة كبيرة من عمليات عنصرية أسرع من نفس العمليات موزّعة على استدعاءات منفصلة.

> لو كان الدمج بهذه الفائدة، فلماذا لا يدمج XLA كل شيء في نواة واحدة؟ ما الذي يمنعه؟ ؟

تحسينات أخرى

الدمج أشهر التحسينات لكنه ليس الوحيد. أربعة يعملون معك بلا أن تدري:

1. **حذف التكرار (CSE).** `tanh(x)` المحسوب في مكانين يُحسب مرة – ممكن لأن القيم غير قابلة للتعديل.
2. **طي الثوابت.** `2 * 3` يصير `6` قبل التشغيل.
3. **تعيين التخطيط.** XLA يختار ترتيب التخزين بحسب العملية التالية؛ تخطيط خاطئ يحوّل عملية مجانية إلى نسخ كامل.
4. **التبرّع بالمخزن.** مصفوفة لن تُستخدم بعد الآن يمكن إعادة استخدام ذاكرتها.

> **× خطأ شائع: القياس قبل استقرار التخطيط**
>
> من يرى أن `jnp.transpose` «لا تكلّف شيئًا» يستنتج أن النسخ مجاني. الحقيقة أن XLA غالبًا يدمج التحويل في التخطيط – إلى أن يتغيّر السياق فيظهر الثمن فجأة.

ذاكرة الترجمة

التجميع مكلف: ثوانٍ للدوال الصغيرة ودقائق للنماذج. وJAX تحتفظ بالنتيجة في ذاكرة داخل العملية، مفتاحها الأشكال والأنواع وقيم الثوابت. أغلق النافذة يذهب كل شيء؛ ولهذا وُجدت ذاكرة على القرص:

```python
import jax

jax.config.update("jax_compilation_cache_dir", "/var/cache/jax")
# الثابت الافتراضي يتجاهل الدوال سريعة التجميع؛ ارفعه لتخزين كل شيء
jax.config.update("jax_persistent_cache_min_compile_time_secs", 0.0)
```

الفائدة الأكبر في خط التكامل المستمر: كل اختبار كان يدفع ثمن التجميع من جديد، ومع الذاكرة المحفوظة ينزل الزمن من دقائق إلى ثوانٍ. الشرط تطابق العتاد وإصدار المكتبة.

> ⚠ **الذاكرة ليست سحرًا**
>
> المفتاح يتضمّن بصمة إصدار المترجم وإعدادات الجهاز. تغيير `XLA_FLAGS` أو ترقية `jaxlib` يُبطلها كلها.

حدود التجميع

كل ما سبق يعمل لأن الأشكال معلومة قبل التشغيل. ومتى لم تكن، تنتهي حرّية XLA.

الأشكال الديناميكية. غيّر حجم الدفعة من 32 إلى 33، وستحصل على تجميع كامل جديد. الحل ليس إجبار XLA بل تثبيت الأشكال: احشُ الدفعة الأخيرة.

التحكّم المعتمد على البيانات. `jax.lax.cond` و`while_loop` يعملان، لكنهما يشترطان فرعين بنفس الشكل. أما `jnp.nonzero` فتُنتج طولًا يعتمد على القيم، وهذا ما لا يستطيع المترجم التخطيط له.

الاستدعاءات المضيفة. كل دالّة تُغلّفها بـ `jax.pure_callback` تكسر الدمج عند حدودها، وكل استدعاء يعني مزامنة كاملة بين المعالج والجهاز.

كيف تقرأ تقرير الأداء؟

أين تنظر حين يبطؤ برنامجك؟ لا إلى الشيفرة، بل إلى خط زمني. `jax.profiler` ينتج أثرًا يفتحه xprof أو لوحة TensorBoard.

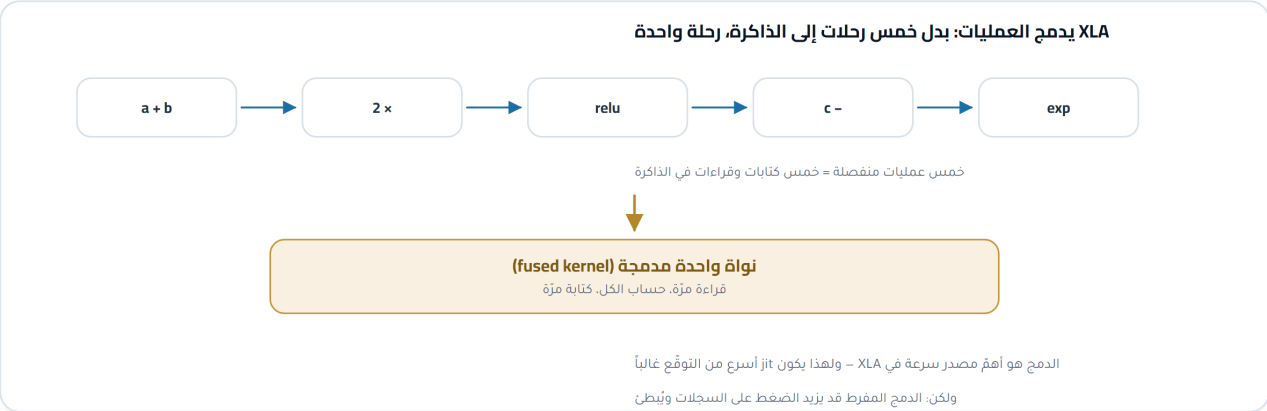
```python
import jax, jax.numpy as jnp

@jax.jit
def step(x):
    return jnp.tanh(x @ x.T).sum()

x = jnp.ones((2048, 2048))
step(x).block_until_ready()          # تسخين: استبعد زمن التجميع

jax.profiler.start_trace("/tmp/trace")
for _ in range(20):
    step(x).block_until_ready()
jax.profiler.stop_trace()
```

ابحث عن أربعة أشياء. أوّلًا: الفجوات بين النوى – إن وُجدت فالمشكلة في المضيف. ثانيًا: عدد النوى في كل خطوة؛ إن كان كبيرًا فابحث عن عملية كاسرة للدمج. ثالثًا: نسخ الذاكرة بين المضيف والجهاز، علامة على استدعاء مخفي. رابعًا: الإشغال، أي نسبة الموارد المستخدمة.



الشكل 24.1 — المسار الكامل: بايثون يُتتبَّع إلى JAXPR، ثم HLO، ثم تحسينات، ثم تنفيذي. كل سهم فرصة للفهم، وكل سهم ثمن يُدفع مرة واحدة.

تمرين الفصل الرابع والعشرين

1. اكتب دالّة تُطبّق `tanh` ثم `exp` ثم `sqrt`، واطبع `as_text()` قبل التحسين وبعده. كم تعليمة بقي؟
2. افصل العمليات في ثلاث دوال مغلّفة، وقارن الزمن. اشرح الفرق بحسبة النقل.
3. شغّل التتبّع عشرين خطوة، وابحث عن فجوة واحدة.

الحلّ النموذجي

1. قبل التحسين ثلاث تعليمات على الأقل؛ بعده غالبًا تعليمة `fusion` واحدة. **2.** الدمج يختفي عند حدود `jit`، فتصير كل دالّة نواة مستقلة بتكلفة قراءة وكتابة كاملتين.

```python
import jax, jax.numpy as jnp, time

@jax.jit
def a(x):
    return jnp.sqrt(jnp.exp(jnp.tanh(x)))

x = jnp.ones((8, 8))
low = a.lower(x)
print("before:", low.as_text().count("stablehlo."))
print("after :", low.compile().as_text().count("stablehlo."))

x = jnp.ones((4096, 4096))
a(x).block_until_ready()
t0 = time.perf_counter()
for _ in range(50):
    a(x).block_until_ready()
print("ms:", (time.perf_counter() - t0) / 50 * 1e3)
```

خلاصة الفصل

- XLA مترجم لا محرّك: رسم بياني يدخل، تنفيذي يخرج.
- HLO لغة تعليمات وظيفية بأشكال صريحة.
- `lower` ثم `compile` ثم `as_text` تكفي لرؤية كل شيء.
- الدمج أهمّ التحسينات لأن عنق الزجاجة عرض النطاق.
- الأشكال الديناميكية والتحكّم بالبيانات والاستدعاءات المضيفة حدود التجميع.

مصطلحات الفصل

HLO

التمثيل الوسيط الذي يستقبله XLA: تعليمات وظيفية بأشكال صريحة.

lowering

تحويل رسم JAXPR إلى HLO قبل أي تحسين.

fusion

دمج عمليات متتالية في نواة واحدة لتقليل حركة الذاكرة.

CSE

حذف الحسابات المتكرّرة المتطابقة.

buffer donation

إعادة استخدام ذاكرة موتر منتهٍ بدل حجز جديدة.

التقطيع: NamedShardingg Mesh

GSPMD، وأنماط البيانات والمعاملات والتسلسل، والتحميل الموزّع

اشترى فريقٌ ثماني بطاقات وشغّل التدريب، فكانت النتيجة أبطأ من بطاقة واحدة. السبب لم يكن العتاد بل سطرًا واحدًا: النسخ الاحتياطي للمعاملات كان يعبر الشبكة كل خطوة لأن أحدهم لم يصف مكانها.

التوزيع ليس إضافة، بل وصف

في مكتبات التوزيع التقليدية تكتب ما تريد أن يحدث: أرسل هذه القطعة إلى الجهاز الثالث، ثم اجمع النتائج. في JAX العقد معكوس: تقول «هذا الموتر مقسوم على المحور صفر بين هذه الأجهزة الثمانية»، ثم تنسى الأمر.

لهذا الوصف اسم: sharding، وهو خاصية للموتر لا عملية عليه. شكله المنطقي لا يتغيّر – يبقى `(4096, 4096)` في نظر شيفرتك – لكن نسخته المادية موزّعة.

> ★ **القاعدة الذهنية**
>
> أنت تصف **أين تسكن البيانات**، والمترجم يستنتج **كيف تتحرّك**. إن كتبت أمر إرسال صريحًا، فأنت تعمل ضد النموذج لا معه.

> لو كان المترجم هو من يقرّر الاتصال، فكيف تعرف أن تخطيطك جيّد قبل أن تشغّل التدريب وتنتظر ساعات؟

Mesh: هندسة الأجهزة

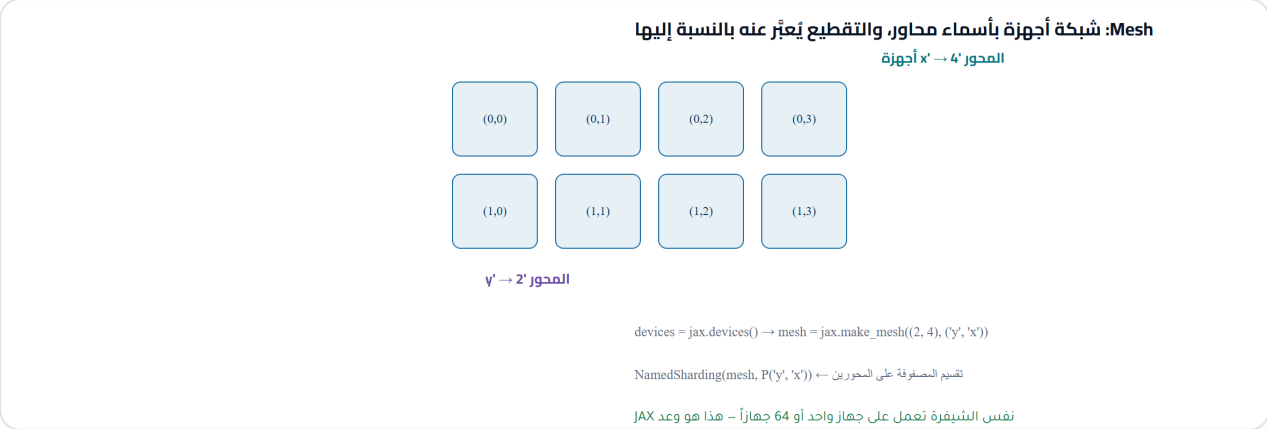
قبل أن تصف مكان البيانات تحتاج خريطة للأجهزة. Mesh شبكة منطقية بأبعاد وأسماء فوق قائمة الأجهزة الفعلية؛ أسماؤها اختيارية لكنها تصير لغة مشتركة بين كل التخطيطات.

```python
import jax
import numpy as np

devices = np.array(jax.devices()).reshape(2, 4)   # صفّان، أربعة أعمدة
mesh = jax.make_mesh((2, 4), axis_names=("x", "y"), devices=devices)

print(mesh.shape)        # {'x': 2, 'y': 4}
print(mesh.devices[0, 3])
```

الشكل `(2, 4)` لا علاقة له بشكل أي موتر؛ هو وصف للعتاد. لكن ترتيب الأجهزة يهمّ: الأجهزة في الصف نفسه أقرب في الشبكة، فجعل المحور الأكثر كلامًا داخل الصف يوفّر زمنًا حقيقيًا.



الشكل 25.1 — شبكة `2×4` بأسماء `x` و`y`: هندسة عتاد لا شكل بيانات.

PartitionSpecg NamedSharding

`PartitionSpec` جواب على سؤال: أي محور من الموتر يذهب إلى أي محور من الشبكة؟ يُكتب `P`، وكل موضع فيه إمّا اسم محور أو `None` بمعنى «اتركه كاملًا في كل جهاز».

```python
from jax.sharding import NamedSharding, PartitionSpec as P

data_par = NamedSharding(mesh, P("x", None))   # الدفعة مقسومة، الميزات كاملة
model_par = NamedSharding(mesh, P())           # نسخة كاملة على كل جهاز
tensor_par = NamedSharding(mesh, P(None, "y")) # الأوزان مقسومة على y
```

الموتر	الشكل	PartitionSpec	حجم القطعة لكل جهاز
دفعة تدريب	(256, 128)	`P("x", None)`	(128, 128)
أوزان طبقة	(4096, 4096)	`P(None, "y")`	(4096, 1024)
انحياز	(4096,)	`P()`	(4096,) مكرّر

جدول 25.1 — أشكال الموترات وتخطيطها على شبكة `2×4`

القاعدة: اقسم البُعد المُسمّى على حجم ذلك المحور، واضرب ما تبقّى. فالأوزان مع `P(None, "y")` تصير $4096 \times (4096/4) = 4096 \times 1024$.

> ⚠ **عدد المحاور ليس عدد الأجهزة**
>
> `P("x", "y")` يعني أن المحورين معًا يوزّعان على 8 أجهزة (2×4)، لا أن كل محور يأخذ 8. خلط الحساب هنا أشهر سبب لخطأ «shape not divisible».

الأنماط الثلاثة

كل استراتيجية توزيع معروفة ليست إلا اختيارًا مختلفًا لـ `PartitionSpec`:

1. **التوازي على البيانات.** الأوزان كاملة على كل جهاز، وشريحة من الدفعة لكل جهاز؛ لا اتصال أمامي، لكن التدرّجات تُجمع كل خطوة.

2. **التوازي على الموترات.** الأوزان مقسومة على المحور الداخلي؛ الذاكرة تنزل بحِمل الشبكة، وكل طبقة تحتاج جمعًا جزئيًا.

3. **التقطيع الكامل (FSDP).** الأوزان وحالات المُحسِّن مقسومة، تُجمع مؤقتًا عند الحاجة ثم تُطرح.

ولنُدخل الأرقام. نموذج بـ7 مليارات معامل بـ `bfloat16` يزن 14 غيغابايت، ومُحسِّن Adam يحتفظ بحالتين بـ `float32`، أي 8 بايتات لكل معامل:

$$M_{\text{total}} = 7{\times}10^{9} \times (2 + 8)\ \text{B} \approx 70\ \text{GB}$$

حجم الحالة الكاملة قبل أي تقطيع: أوزان + عزمين.

على 8 أجهزة: التوازي على البيانات يضع 70 غيغابايت على كل جهاز – مستحيل مع التنشيطات. وFSDP يقسمها إلى $70/8 \approx 8.75$ GB، والتوازي على الموترات يقسم الأوزان وحدها (1.75 غيغابايت).

النمط	الأوزان	حالة المُحسِّن	لكل جهاز	الاتصال في الخطوة
بيانات	14 GB كاملة	56 GB كاملة	≈ 70 GB	all-reduce للتدرّجات
موترات	1.75 GB	7 GB	≈ 8.75 GB	جمع جزئي لكل طبقة
FSDP	1.75 GB	7 GB	≈ 8.75 GB	جمع قبل كل طبقة

جدول 25.2 — ذاكرة كل جهاز لنموذج 7B على 8 أجهزة

إن كان FSDP والتوازي على الموترات يعطيان الذاكرة نفسها لكل جهاز، فما الذي يجعل أحدهما أسرع من الآخر على الشبكة ذاتها؟

jax.jit مع in_shardings/out_shardings

لتجربة المثال على حاسوب واحد بلا بطاقات، شغّله بوسم `XLA_FLAGS=--xla_force_host_platform_device_count=8` ليتظاهر XLA بثمانية أجهزة مضيفة.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

mesh = jax.make_mesh((8,), axis_names=("x",))
shard_batch = NamedSharding(mesh, P("x"))
replicate   = NamedSharding(mesh, P())

def loss(w, xb, yb):
    pred = jnp.tanh(xb @ w)
    return jnp.mean((pred - yb) ** 2)

step = jax.jit(
    jax.grad(loss),
    in_shardings=(replicate, shard_batch, shard_batch),
    out_shardings=replicate,
)

w  = jnp.ones((64, 8)) / 8
xb = jnp.ones((1024, 64))
yb = jnp.zeros((1024, 8))

g = step(w, xb, yb)
print(g.sharding)     # نسخة كاملة من التدرّج على كل جهاز
```

`w` مكرّر على الأجهزة الثمانية، والدفعة مقسومة 128 سطرًا لكل جهاز، والنتيجة مكرّرة لأن `out_shardings` طلب ذلك. ولاحظ ما لم تكتبه: لا أمر جمع ولا بثّ.

> **✓ ابدأ بالتخطيط الصريح**
>
> حتى لو كان الافتراضي يعمل، مرّر `in_shardings` و `out_shardings` في أول نموذج موزّع؛ التصريح يجعل إعادة التقطيع الخفيّة خطأً ظاهرًا لا تباطؤًا غامضًا.

التحميل الموزّع للبيانات

أخطر عنق زجاجة لا يظهر في الحساب بل قبله. إن حمّلت الدفعة كاملة إلى جهاز واحد ثم وزّعتها، فأنت تنقلها مرتين وتترك سبعة أجهزة تنتظر. قطّع على المضيف ثم أسكن كل قطعة مكانها.

```python
import numpy as np, jax

def sharded_batch(step_idx, batch=1024):
    host = np.load(f"batch_{step_idx:05d}.npy")     # (1024, 64) على المضيف
    parts = np.split(host, 8, axis=0)               # ثماني قطع متساوية
    return jax.device_put(parts[0], shard_batch)    # كل قطعة إلى جهازها

xb = sharded_batch(0)
for i in range(1, 100):
    nxt = sharded_batch(i)                          # تحميل مسبق
    g = step(w, xb, yb)
    xb = nxt
```

× خطأ شائع: التقطيع داخل الخطوة

من يكتب `jax.device_put(x, shard_batch)` على موتر مكرّر في كل خطوة يدفع ثمن إعادة تقطيع كاملة كل مرة. التقطيع قرار عند حدود البيانات، لا داخل الحلقة.

القياس: هل التوزيع يساعد فعلاً؟

التوسّع القوي: حجم العمل ثابت والأجهزة تزيد. التوسّع الضعيف: العمل يزيد بنسبة الأجهزة والزمن يبقى ثابتًا.

$$S(n) = T(1) / T(n)\ ,\ E(n) = S(n) / n$$

التسريع S والكفاءة E: الكفاءة تقيس ما دفعتَه مقابل ما كسبتَه.

على ثمانية أجهزة الحدّ الأعلى 8؛ فإن قست 6.4 فكفاءتك 80%. وهنا القاعدة: **إن نزلت الكفاءة تحت 80%، فالاتصال يأكل مكسبك**، والأولى إصلاح التخطيط لا شراء عتاد.

★ قاعدة الثمانين

ثمانية أجهزة بكفاءة 80% تعطيك 6.4 أضعاف، وأربعة بكفاءة 95% تعطيك 3.8. الفارق ليس في العدد بل في نسبة ما يضيع في الشبكة.

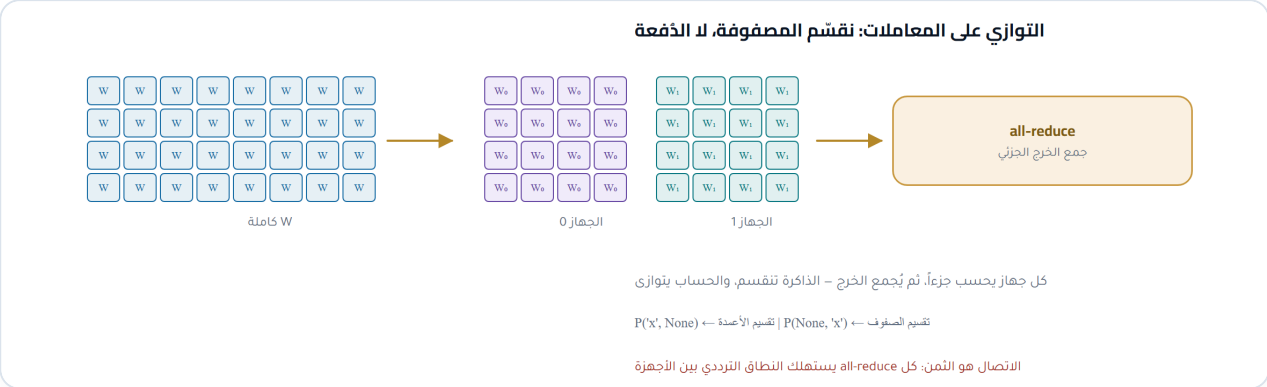
أخطاء التقطيع

الشكل لا يقبل القسمة. `P("x")` على موتر طوله 1000 مع محور حجمه 8 يفشل. الحل: احشُ إلى 1008 أو اختر محورًا آخر.

ترتيب المحاور مقلوب. `P("y", None)` على موتر تريده مقسومًا على الدفعة يقسم الميزات: لا خطأ، فقط نتائج غريبة وأداء أسوأ.

تكرار صامت. إن كان `PartitionSpec` أقصر من رتبة الموتر، تُكرَّر الأبعاد الباقية ضمنًا: تعمل الشيفرة وتضيع الذاكرة بلا شكوى.

إعادة تقطيع داخل الخطوة. عملية تُخرج تخطيطًا ثم أخرى تستهلك غيره تجبر XLA على نسخ كامل. ابحث عنها بطباعة `g.sharding` عند كل حدّ.



الشكل 25.2 — التوازي على الموترات: كل جهاز يحمل شريحة من الأوزان، والنتيجة تُجمع جزئيًا. الذاكرة تنزل، والشبكة تدفع الفرق.

تمرين الفصل الخامس والعشرين

1. أنشئ شبكة `(2, 4)` واطبع شكلها وأجهزتها.
2. احسب ذاكرة كل جهاز لنموذج 13B على 16 جهازًا بنمط FSDP، بافتراض 10 بايتات للمعامل الواحد.
3. شغّل مثال القسم الخامس مع `P()` بدل `P("x")` للدفعة.

الحلّ النموذجي

1. الشكل `{'x': 2, 'y': 4}`. **2.** $13{\times}10^9 \times 10\ \text{B} \approx 130\ \text{GB}$، مقسومة على 16 تعطي $\approx 8.1\ \text{GB}$ لكل جهاز.

```python
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

mesh = jax.make_mesh((8,), axis_names=("x",))
rep = NamedSharding(mesh, P())
f = jax.jit(lambda x: x * 2, in_shardings=rep, out_shardings=rep)
print(f(jnp.ones((1024, 8))).sharding)
```

خلاصة الفصل

- التوزيع وصفٌ للتخطيط لا سلسلة أوامر؛ المترجم يستنتج الاتصال.
- `Mesh` هندسة أجهزة، وشكله لا علاقة له بشكل البيانات.
- `P("x", None)` يقسم المحور الأول، و `P()` يكرّر كل شيء.
- الأنماط الثلاثة اختيارات لنفس الأداة، والفرق في الذاكرة والاتصال.

مصطلحات الفصل

Mesh

شبكة منطقية من الأجهزة بأبعاد وأسماء.

NamedSharding

تخطيط مسمّى يربط موترًا بشبكة عبر `PartitionSpec`.

PartitionSpec

تعبير يحدّد أي محور من الموتر يذهب إلى أي محور من الشبكة.

all-reduce

جمع تُوزّع نتيجته على كل الأجهزة.

weak scaling

زيادة العمل مع الأجهزة بحيث يبقى الزمن ثابتًا.

Pallas: نوى مخصّصة

متى تكتب نواة؟ وكيف؟ ومن TPU إلى GPU

بعد أن رأيت كيف يدمج XLA ويخطّط للذاكرة، قد يخطر لك سؤال محرج: ماذا لو كان الدمج نفسه هو المشكلة؟ ماذا لو أردت أن تتحكّم في البلاطة، وفي متى تُقرأ من الذاكرة البطيئة، وفي أي سطر يُحسب المجموع الجزئي؟ هنا يتوقّف المترجم عن مساعدتك ويضعك أمام واجهة أدنى: Pallas. هذا الفصل عن تلك الواجهة، وعن الثمن الذي تدفعه مقابلها.

متى تحتاج نواة مخصّصة؟

القاعدة الأولى: **لست تحتاجها في 90% من الحالات**. الدمج الذي شرحناه في الفصل الرابع والعشرين يعطي عادةً ما يعطيه أي نواة مكتوبة بيد، لأن العنق ليس في الحساب بل في النقل، والدمج يحلّ النقل. من يكتب نواة مخصّصة قبل أن يقيس يخسر شهورًا مقابل نسبة لا تُذكر.

تحتاجها حين يريد المترجم شيئًا لا تعرفه أنت. مثال كلاسيكي: انتباه يحتاج مجموعًا مرجّحًا يُحسب تدريجيًا مع كل بلاطة، بحيث لا تُكتب مصفوفة الانتباه الكاملة في الذاكرة أصلًا. XLA يستطيع دمج ذلك، لكنه يقرّر بنفسه حجم البلاطة، وقد يختار حجمًا يفيض عن الذاكرة المشتركة. حينها تريد أن تقول أنت: هذه البلاطة، بهذا الحجم، في هذا المستوى من الذاكرة.

⚠ الثمن الذي تدفعه

النواة المخصّصة تفقدك بعض تحسينات XLA، وتصير مسؤولًا عن صحّة التقطيع، وتقلّ صيرورتها بين الأجهزة. وتصحيح أخطائها أصعب بكثير من تصحيح دالّة عادية، لأن الأخطاء تظهر كأرقام خاطئة لا كاستثناء.

لو كان بإمكانك كتابة نواة أسرع بثلاثين في المئة، لكنها تعمل على عتاد واحد فقط، فهل تكتبها؟ وما الذي يغيّر جوابك؟ ؟

نموذج البرمجة: بلاطات وذاكرة

Pallas لا تعطيك خيوطًا ولا مؤشّرات ذاكرة. تعطيك مستويين: الشبكة (grid) وهي عدد البرامج التي تعمل بالتوازي، والبلاطة (block) وهي القطعة التي يراها كل برنامج. تكتب دالّة تعمل على مراجع (refs) لبلاطة واحدة، وPallasg تستدعيها لكل بلاطة.

`BlockSpec` هو الجسر: يقول لأي بلاطة من الموتر يقابل أي برنامج. يأخذ شكل البلاطة ودالّة إسقاط تأخذ معرّفات البرامج وتُرجع إحداثيات البلاطة. و `program_id` هو رقم البرنامج الحالي على محور معيّن. هذا كل النموذج – والباقي تفاصيل أداء.

> ★ **ما يعمل بالتوازي وما يعمل داخل البلاطة**
>
> كل برنامج في الشبكة يعمل بالتوازي مع غيره. وما يحدث داخل البرنامج تسلسلي. لذلك تضع الحلقة على مستوى الشبكة حين تريد توازيًا، وداخلها حين تريد تجميعًا.

أوّل نواة: جمع مصفوفتين

لنبدأ بأبسط ما يمكن: جمع مصفوفتين. لا مكسب أدائي هنا – الغرض أن ترى الشكل كاملًا قبل أن تزيد التعقيد.

```python
import jax, jax.numpy as jnp
import jax.experimental.pallas as pl

def add_kernel(x_ref, y_ref, o_ref):
    o_ref[...] = x_ref[...] + y_ref[...]

def add(x, y, block=256):
    return pl.pallas_call(
        add_kernel,
        out_shape=jax.ShapeDtypeStruct(x.shape, x.dtype),
        grid=(x.shape[0] // block,),
        in_specs=[pl.BlockSpec((block, x.shape[1]), lambda i: (i, 0)),
                  pl.BlockSpec((block, x.shape[1]), lambda i: (i, 0))],
        out_specs=pl.BlockSpec((block, x.shape[1]), lambda i: (i, 0)),
    )(x, y)

x = jnp.arange(2048.0).reshape(8, 256)
y = jnp.ones_like(x)
print(jnp.allclose(add(x, y), x + y))   # True
```

اقرأ الشبكة: `grid=(8,)` يعني ثمانية برامج، كل واحد يأخذ 256 صفًّا. ودالّة الإسقاط `lambda i: (i, 0)` تقول: البرنامج رقم `i` يأخذ البلاطة الواقعة عند الصف `i`. و`o_ref[...] = ...` كتابة في البلاطة، لا في الموتر كاملًا – وهذا الفرق الذي يجب أن يثبت في ذهنك.

> ✓ **ابدأ بالتفسير لا بالترجمة**
>
> على المعالج تعمل Pallas بوضع تفسيري يمرّ عبر JAX نفسها. استخدمه للتحقّق من الصحة أولًا، ثم انتقل إلى العتاد. الخطأ في `BlockSpec` يظهر هنا كرقم خاطئ لا كتعطّل.

مضاعفة المصفوفات بالبلاطات

الآن الحالة التي تُبرّر Pallas. في الضرب العادي، كل بلاطة من الخرج تحتاج صفًّا كاملًا من `A` وعمودًا كاملًا من `B`. وإن تركت المترجم يقرّر، فقد يعيد قراءة `B` من الذاكرة الرئيسية لكل بلاطة صفّية – وهي القراءة التي تقتل الأداء.

الحل أن تُقطّع البعد الداخلي أيضًا: يُحمَّل كل من `A` و`B` إلى الذاكرة المشتركة بلاطةً بلاطة، ويُجمَّع الناتج في السجلات. النقل من الذاكرة الرئيسية ينزل من مرتبة $O(M \cdot N \cdot K)$ إلى ما يقارب $O((M \cdot K + K \cdot N) \cdot (M/BM))$.

```python
def matmul_kernel(a_ref, b_ref, o_ref):
    # كل برنامج يراكم مساهمته في بلاطة الخرج
    o_ref[...] += jnp.dot(a_ref[...], b_ref[...])

def matmul(a, b, bm=128, bk=64, bn=128):
    m, k = a.shape
    _, n = b.shape
    return pl.pallas_call(
        matmul_kernel,
        out_shape=jax.ShapeDtypeStruct((m, n), jnp.float32),
        grid=(m // bm, n // bn, k // bk),
        in_specs=[pl.BlockSpec((bm, bk), lambda i, j, kk: (i, kk)),
                  pl.BlockSpec((bk, bn), lambda i, j, kk: (kk, j))],
        out_specs=pl.BlockSpec((bm, bn), lambda i, j, kk: (i, j)),
    )(a, b)

A = jnp.ones((512, 256), jnp.float32)
B = jnp.ones((256, 512), jnp.float32)
print(jnp.allclose(matmul(A, B), A @ B, atol=1e-4))
```

لاحظ المحور الثالث في الشبكة: `k // bk` . كل برنامج هناك يحسب مساهمة شريحة من البعد الداخلي، والجمع التراكمي في `o_ref` يبني الناتج النهائي. هذا هو split-K بأبسط صوره، وهو أيضًا سبب أن الخرج يجب أن يكون `float32` صراحةً: التراكم في `bfloat16` يفقد دقّة بسرعة.

> **× خطأ شائع: بلاطة أكبر من الذاكرة المشتركة**
>
> رفع `bm` و `bn` إلى 512 يبدو حكيمًا حتى تتجاوز البلاطتان معًا الذاكرة المشتركة، فيتحوّل «التسريع» إلى نسخ احتياطي صامت أو خطأ غامض. ابدأ صغيرًا وقِس.

من TPU إلى GPU

Pallas واجهة واحدة بخلفيات متعدّدة: `pallas.tpu` تُترجم عبر Mosaic، و `pallas.gpu` تُترجم إلى Triton. والواجهة المشتركة ليست وعدًا بأن النواة نفسها ستكون الأسرع في الحالتين.

الجانب	pallas.tpu	pallas.gpu
المترجم الوسيط	Mosaic	Triton
الذاكرة السريعة	VMEM محدودة الحجم	ذاكرة مشتركة
حجم البلاطة	حسّاس جدًّا لحدود VMEM	مرن نسبيًا
التوافق	يحتاج ضبطًا خاصًّا	يحتاج ضبطًا خاصًّا

جدول 26.1 — الخلفيتان وما يعنيه الاختلاف عمليًا

النتيجة العملية: نواة مكتوبة لTPU قد تعمل على GPU بلا خطأ، لكن بسرعة أقل من نسخة XLA العادية. لهذا لا تُعمّم قياسًا من عتاد إلى آخر، ولا تجعل «يعمل» مساويًا لـ«أسرع».

الاختبار والمقارنة

كل نواة تُختبر أولًا مقابل المرجع، ثم تُقاس. والاختبار بلا مقارنة عددية لا قيمة له: `jnp.allclose` مع تفاوت صريح، ثم قياس مع `block_until_ready` .

```python
import time

def bench(fn, *args, n=50):
    fn(*args).block_until_ready()          # تسخين
    t0 = time.perf_counter()
    for _ in range(n):
        fn(*args).block_until_ready()      # بدون هذا تقيس الطابور لا الحساب
    return (time.perf_counter() - t0) / n * 1e3

print("pallas ms:", bench(matmul, A, B))
print("xla    ms:", bench(lambda a, b: a @ b, A, B))
```

لاحظ السطر المعلّق عليه. غياب `block_until_ready` يجعل القياس يقيس زمن إرسال الأوامر إلى الجهاز، وهو دائمًا أصغر بكثير من الزمن الحقيقي – ولهذا ترى أرقامًا مستحيلة.

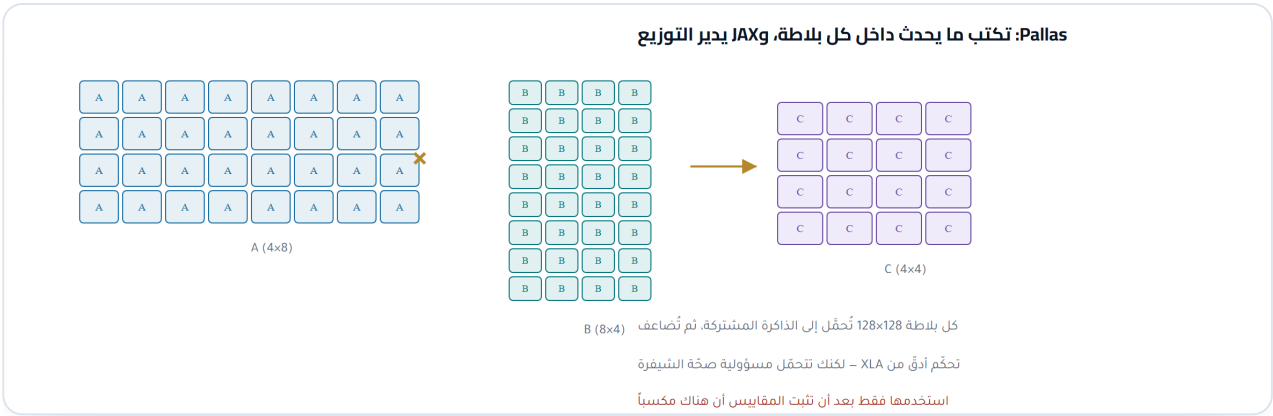
> ⚠ **قِس على الشكل الحقيقي**
>
> نواة جيدة على `512×256` قد تخسر على `8192×8192`، لأن نسبة الإعداد إلى العمل تتغيّر. قِس بالأشكال التي ستشغّلها فعلًا، لا بالشكل الذي يريح الكتابة.

متى تتوقّف

نواة أسرع بعشرة في المئة، تحتاج مراجعة عند كل ترقية لمترجم XLA: هل تحتفظ بها أم تحذفها؟ ؟

القاعدة التي تختصر الفصل: **إن كان التسريع أقل من 15%، فاحتفظ بالنسخة البسيطة**. فالنواة المخصّصة تضيف سطحًا جديدًا للأخطاء، وتكسر الصيرورة، وتحتاج صيانة مع كل إصدار مترجم. خمسة عشر في المئة لا تشتري ذلك.

والتسريع الحقيقي يظهر عادةً في حالات محدّدة: عمليات لا يستطيع XLA دمجها، أو ذاكرة لا تكفي لمصفوفة وسيطة، أو نمط وصول غير منتظم. خارج هذه الحالات، أسرع تحسين هو غالبًا إعادة كتابة الشيفرة العادية لتُدمج، لا كتابة نواة.



الشكل 26.1 — نموذج Pallas: شبكة من البرامج، كل واحد يرى بلاطة، والنقل بين مستويات الذاكرة هو ما تحدّده أنت.

تمرين الفصل السادس والعشرين

1. غيّر `block` في نواة الجمع إلى 64 و1024، وتحقق من بقاء النتيجة صحيحة.
2. شغّل نواة الضرب بثلاثة أحجام بلاطة مختلفة، وسجّل الزمن لكل منها.
3. قارن النتائج بنسخة `a @ b` العادية، وقرّر أيّها تحتفظ به.

الحلّ النموذجي

1. النتيجة تبقى صحيحة ما دام الشكل يقبل القسمة على حجم البلاطة. **2.** الأفضل عادةً بلاطة تملأ الذاكرة السريعة ولا تتجاوزها. **3.** إن كان الفرق أقل من 15%، فاحتفظ بنسخة XLA.

```python
for bm, bk, bn in [(64, 32, 64), (128, 64, 128), (256, 64, 256)]:
    got = matmul(A, B, bm=bm, bk=bk, bn=bn)
    ok = jnp.allclose(got, A @ B, atol=1e-4)
    print(bm, bk, bn, ok, f"{bench(lambda a, b: matmul(a, b, bm, bk, bn), A, B):.3f}
```

خلاصة الفصل

- Pallas للنمط الذي يعجز عنه دمج XLA، لا لكل دالّة.
- الشبكة توازٍ والبلاطة وحدة العمل، و `BlockSpec` يربط بينهما.
- الكتابة في `o_ref[...]` كتابة في بلاطة، لا في الموتر كله.
- الخلفيتان تختلفان في المترجم والذاكرة السريعة، والقياس لا يُنقل بينهما.
- قِس دائمًا مقابل المرجع، وبـ `block_until_ready`، وتوقّف عند 15%.

مصطلحات الفصل

Pallas

واجهة JAX لكتابة نوى تعمل على بلاطات بتحكّم صريح في الذاكرة.

grid

عدد البرامج التي تعمل بالتوازي في النواة، موزّعة على محاور.

BlockSpec

وصف شكل البلاطة ودالّة إسقاط تحوّل معرّف البرنامج إلى إحداثيات.

ref

مرجع إلى بلاطة داخل النواة؛ القراءة والكتابة تمرّان عبره لا عبر الموتر.

split-K

تقطيع البعد الداخلي للضرب على عدّة برامج ثم جمع مساهماتها.

الذاكرة والأداء

remat، offloadg، donationg، والقياس، والتشخيص

رأيت في الفصل الرابع والعشرين كيف يحوّل XLA الرسم البياني إلى تنفيذي، لكن سؤالاً ظلّ معلّقاً: من أين تأتي الذاكرة التي يوزّعها المترجم؟ حين تُجمّع شبكة بمئة طبقة على بطاقة واحدة، تنهار العملية برسالة `OOM` قبل أن تُنتج رقماً واحداً. السبب أن كل وسيط وسيطة يجب أن يعيش في الذاكرة حتّى يُستهلك. في هذا الفصل نتناول خمس أدوات للسيطرة على الذاكرة والأداء: إعادة الحساب `remat`، وإفراغ الذاكرة `offload`، والتبرّع بالمخزن `donation`، والقياس `profiling`، والتشخيص. لا نعرضها كخيارات منفصلة، بل كطبقات من نفس المعركة: تقليل المساحة الذروة مع إبقاء الزمن مقبولاً.

لماذا تنفد الذاكرة قبل الحساب؟

في الشبكة العميقة، لتحسب تدرّج طبقة ما يحتاج XLA إلى مخرجاتها كما رآها أثناء التمريرة الأمامية. إن حفظتها كلها تراكمت ذروة هائلة؛ وإن حذفتها يجب إعادة حسابها أثناء الرجوع. هذه المقايضة – ذاكرة مقابل حساب – هي جوهر هذا الفصل. لاحظ أننا لن نعيد شرح بنية HLO؛ رأيناها في الفصل الرابع والعشرين، وهنا نركّز على من يملك الذاكرة.

بطاقتك تتّسع لعشرة مليارات معامل، لكن التدريب ينهار عند طبقة الستين. أين ذهبت الذاكرة إذن، ولماذا لا يكفي حجم المعاملات وحده؟ ؟

remat: أعد الحساب لتوفّر الذاكرة

`jax.remat` (خليفة `jax.checkpoint`؛ أُزيل `jax.custom_remat` في 0.12) يخبر المترجم ألا يحفظ مخرجات قسم من الرسم، بل يعيد حسابها عند الحاجة. الثمن زمن إضافي، والمكسب انهيار في ذروة الذاكرة:

$$\text{Mem} \approx \sum_i \text{activations}(i) \quad \longrightarrow \quad \text{Mem} \approx \max_i \text{activations}(i)$$

بإعادة الحساب نستبدل التراكم بأكبر كتلة مفردة، فتنهار الذروة.

```python
import jax, jax.numpy as jnp

def block(x):
    for _ in range(4):
        x = jnp.tanh(x @ x.T)        # كتلة مكلفة ذاكرياً
    return x

# أعد حساب الكتلة بدل حفظ مخرجاتها الأربع
recomputed = jax.remat(block)
x = jnp.ones((512, 512))
g = jax.grad(lambda z: recomputed(z).sum())(x)
print(g.shape)                       # (512, 512)
```

> ★ **remat يقلب الحساب إلى ذاكرة**
>
> كل `remat` يضاعف زمن القسم المعني تقريباً (حساب أمامي + خلفي)، لكنه يُسقط ذروته من الذاكرة. استخدمه على الكتل العميقة لا على العمليات العنصرية الرخيصة.

التبرّع بالمخزن: مَن يملك الذاكرة؟

حين تكتب `y = f(x)` فإن `x` لن يُستخدم بعد الاستدعاء، وفلاش XLA يعيد استخدام ذاكرته لـ `y` بدل حجز جديدة. هذا `buffer donation`. لكنه يفشل صمتاً إن احتفظت بـ `x` أيضاً، فتعيد JAX الحجز:

$$\text{donated} = \{\text{arg}: \text{buf if arg} \notin \text{used_after else new(buf)}\}$$

المترجم يتبرّع بالمخزن فقط إن أثبت أن الوسيط لن يُقرأ مجدّداً.

```python
import jax, jax.numpy as jnp

@jax.jit
def step(x, w):
    y = x @ w                    # x يُتبرّع به هنا
    return y

x = jnp.ones((1024, 1024))
w = jnp.ones((1024, 64))
out = step(x, w)
print(out.shape, isinstance(out, jax.Array))     # (1024, 64) True
```

> ✓ **امنع إعادة الاستخدام المتعارضة**
>
> إن أردت قراءة `x` بعد `step`، فلا تتبرّع به: استخدم `donate_argnums=()` في `jax.jit` لإبطال التبرّع عند الضرورة وتجنّب نسخة غير ضرورية.

offload: ادفع إلى المضيف حين تضيق البطاقة

التبرّع وإعادة الحساب يبقيان داخل البطاقة. حين يستحيل ذلك، يُفرَّغ جزء من الحالة إلى ذاكرة المضيف (`host offload`) ثم يُعاد عند الحاجة. الثمن زمن نقل عبر الناقل، لكنه ينقذ تدريباً مستحيلاً:

$$T_{\text{offload}} \approx \text{Size} / \text{PCIe_bandwidth} \ll \text{OOM_crash}$$

النقل أبطأ كثيراً من الحساب، لكنه أرخص من الانهيار الكامل.

```python
import jax, jax.numpy as jnp

# إفراغ موتر كبير إلى المضيف ثم استدعاؤه لاحقاً
big = jnp.ones((1 << 20,), dtype=jnp.float32)      # 4 نحوMB
host = jax.device_put(big, jax.devices("cpu")[0])
cpu_arr = jax.device_get(host)                      # يُعاد للمضيف
print(cpu_arr.shape, type(cpu_arr))                 # (1048576,) ndarray
```

> ⚠ **offload ليس مجانياً**
>
> كل إفراغ يعني نسخة كاملة عبر الناقل. إن أفرغتَ موتراً يُستهلك مئات المرّات، ضاع الزمن في النقل لا الحساب. افرغ الحالة الخاملة طويلاً، لا الموترات الساخنة.

القياس: أين تذهب الثواني؟

قبل التحسين يجب القياس، وإلّا حسّنتَ ما لا يُبطئك. `jax.profiler` يُنتج أثراً يُفتح في TensorBoard أو xprof. الخطوة الحرجة هي التسخين قبل القياس، وإلّا عدت زمن التجميع ضمن زمن التشغيل:

```python
import jax, jax.numpy as jnp, time

@jax.jit
def train_step(x):
    return jnp.tanh(x @ x.T).sum()

x = jnp.ones((2048, 2048))
train_step(x).block_until_ready()          # تسخين: استبعد التجميع

t0 = time.perf_counter()
for _ in range(30):
    train_step(x).block_until_ready()
ms = (time.perf_counter() - t0) / 30 * 1e3
print(f"{ms:.3f} ms/step")                 # زمن التشغيل الصافي
```

ابحث في الأثر عن أربعة أشياء: فجوات بين النوى (مشكلة مضيف)، نسخ بين المضيف والجهاز، إشغال منخفض (موارد معطّلة)، وتكرار نواة صغيرة كان يمكن دمجها. سنرى في الفصل الثامن والعشرين كيف يقلب نوع الرقم كل هذه الأرقام.

> قِست خطوة واحدة فبدت سريعة، ثم قِست مئة خطوة فتباطأت تدريجياً. أيّ أثر في المحلّل يفسّر هذا التباطؤ الصامت؟

التشخيص: memory profiler

لقياس الذروة فعلياً لا تخمّن: استخدم أداة الذاكرة التي تُبلغ عن استهلاك كل عملية. ولوّن المخرجات لترى أيّ الكتل يبتلع الذاكرة:

```python
import jax
from jax import memory

# تقدير ذروة تقريبي عبر حجز موتر مرجعي
ref = jax.numpy.ones((1024, 1024), dtype=jax.numpy.float32)
peak = ref.size * 4            # بايت لكل f32
print(f"peak block ~ {peak/1024:.1f} KB")
```

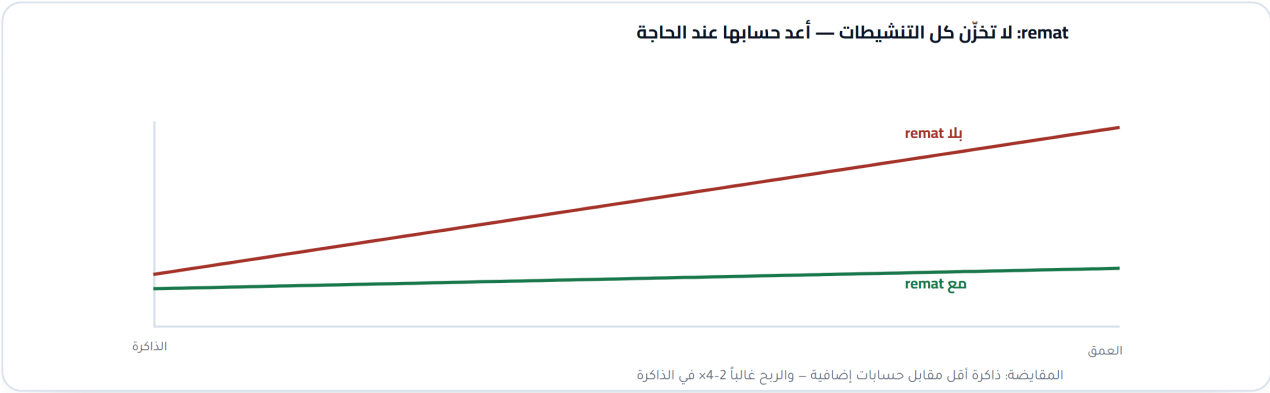
> **× القياس على الدفعة الخاطئة**
>
> من يقيس الأداء على دفعة حجمها 1 ثم ينشر على دفعة 256 يُفاجأ بانهيار الذاكرة. الذروة تتضخّم مع حجم الدفعة، فقِس دائماً على الحجم الإنتاجي لا التجريبي.

متى تستخدم أيّ أداة؟

الأدوات الخمس ليست بدائل، بل طبقات تُركّب. الجدول الآتي يقترح الترتيب المنطقي قبل التسرّع في إعادة كتابة النموذج:

العَرَض	الأداة	الثمن
ذروة عالية في كتل عميقة	`jax.remat`	زمن حساب مضاعف
نسخ متكرّر للمخرجات	`donation` افتراضي	لا شيء إن لم تُعِد القراءة
تعذّر على بطاقة واحدة	`offload` للمضيف	زمن نقل الناقل
بطء غير مفسّر	`jax.profiler`	أثر إضافي بسيط
انهيار OOM صامت	محلّل الذاكرة	تشخيص لمرّة

جدول 27.1 — دليل اختيار أداة الذاكرة حسب الأعراض



الشكل 27.1 — الذروة (صلبة) تنهار عند تفعيل `remat` على الكتل العميقة، مقابل ارتفاع طفيف في زمن الخطوة (متقطّعة).

تمرين التطبيق

تمرين الفصل السابع والعشرين

1. ابنِ دالّة من ثلاث كتل `tanh(x@x.T)` وغلّفها بـ `jax.remat` ، وأثبت أن `jax.grad` يعمل رغم عدم حفظ المخرجات.
2. قِس خطوة قبل التسخين وبعده، وبيّن الفارق الناتج عن زمن التجميع.
3. أنشئ موتراً كبيراً واستخدم `jax.device_put` إلى `cpu` ثم `jax.device_get` ، وأثبت أن النوع صار `ndarray` .

الحلّ النموذجي

1. `remat` يعيد الحساب داخلياً فيمرّ التدرّج. **2.** التسخين يستبعد تجميع XLA فيظهر الزمن الصافي. **3.** الإفراغ يُعيد المصفوفة إلى المضيف كـ `numpy.ndarray` .

Python

```python
import jax, jax.numpy as jnp, time

def block(x):
    for _ in range(3):
        x = jnp.tanh(x @ x.T)
    return x.sum()

re = jax.remat(block)
x = jnp.ones((256, 256))
g = jax.grad(re)(x)
print(g.shape)                      # (256, 256)

@jax.jit
def step(z):
    return jnp.tanh(z @ z.T).sum()
step(x).block_until_ready()
t0 = time.perf_counter()
for _ in range(20):
    step(x).block_until_ready()
print("ms:", (time.perf_counter() - t0) / 20 * 1e3)

big = jnp.ones((1 << 18,), dtype=jnp.float32)
host = jax.device_put(big, jax.devices("cpu")[0])
arr = jax.device_get(host)
print(type(arr).__name__)           # ndarray
```

خلاصة الفصل

- ذروة الذاكرة تنهار قبل الحساب حين تتراكم الوسائط.
- `jax.remat` يستبدل الذاكرة بزمن حساب مضاعف.
- `buffer donation` يعيد استخدام الذاكرة إن لم تُقرأ مجدّداً.
- `offload` يدفع للضيف حين يستحيل البقاء على البطاقة.
- `jax.profiler` والتشخيص يكشفان عنق الزجاجة الحقيقي.

مصطلحات الفصل

remat

إعادة حساب قسم من الرسم بدل حفظ مخرجاته لتقليل الذروة.

buffer donation

إعادة استخدام ذاكرة موتر منتهٍ لإنتاج موتر جديد.

offload

إفراغ موتر إلى ذاكرة المضيف ثم استدعاؤه عند الحاجة.

OOM

نفاد ذاكرة الجهاز قبل اكتمال التخصيص.

profiling

قياس زمن ونقل كل نواة لكشف عنق الزجاجة.

peak memory

أعلى استهلاك ذاكرة في لحظة واحدة أثناء الخطوة.

الدقة العددية والتكميم

bf16، وfp8، والتكميم، والاستقرار العددي

درسنا في الفصل السابع والعشرين كيف نضبط الذاكرة وأين تذهب الثواني، لكن سؤالاً أعمق ظلّ بلا جواب: كم بتّاً يحتاجها كل رقم حقّاً؟ بطاقة تدريب حديثة تقضي معظم وقتها لا في الضرب بل في نقل المعاملات عبر الناقل، فكلّما صغُر الرقم قلّ النقل وكثرت العمليات في الثانية. لكن الصغر له ثمن: فقدان دقّة يظهر أحياناً كنموذج يخرج `NaN` من العدم. في هذا الفصل نتناول `bf16` و`fp8` والتكميم والاستقرار العددي. لا نعرضها كخدع سرعة، بل كهندسة مقايضة بين النطاق والدقّة، حيث يُصبح فهم توزيع الأرقام هو الفرق بين تدريب ينهار وتدريب يطير.

لماذا لا يكفي float32؟

`float32` يحمل 8 بتّات أسّ و23 كسراً، وهو دقيق لكنه بطيء في النقل. الظاهرة المفاجئة أن تدريب محوّل حديث يُنقل فيه الوزن أكثر مما يُضرب؛ فضغط العرض (لا الحساب) هو عنق الزجاجة. ولهذا ولدت أنواع أصغر. لاحظ أننا لن نعيد شرح `jit`؛ رأيناه في الفصل الثامن والعشرين من منظور التجميع، وهنا نركّز على معنى البتّات.

بطاقتك تدعم `fp8` وتضاعف السرعة، لكن خسارتك تحوّلت إلى `NaN` خلال عشر خطوات. هل المشكلة في الحساب أم في طريقة تمثيل الأرقام نفسها؟ ؟

bf16: تضحية بالكسر لصالح النطاق

`bfloat16` يحتفظ بـ 8 بتّات أسّ كـ `float32` لكنه يختزل الكسر إلى 7 بتّات. النتيجة: نفس النطاق (لا `overflow` مفاجئ) مع دقّة أقل. هذا يجعله مثالياً للتدرّجات التي تحتاج نطاقاً واسعاً أكثر من دقّة خفيّة:

$$\text{bf16} = \text{sign}(1) \oplus \exp(8) \oplus \text{frac}(7) \quad , \quad \text{fp32} = \text{sign}(1) \oplus \exp(8) \oplus \text{frac}(23)$$

نفس الأسّ يعني نفس النطاق تقريباً، فبقاء التدرّجات مستقرّة أسهل.

```python
import jax, jax.numpy as jnp

x = jnp.ones((4, 4), dtype=jnp.bfloat16)
y = x @ x.T                          # ضرب على البطاقة bf16
print(y.dtype, y.shape)              # bfloat16 (4, 4)

# ثم تُخفَض fp32 تُحسب الخسارة عادةً بـ
loss = jnp.sum(y.astype(jnp.float32))
print(loss.dtype)                    # float32
```

> ★ **القاعدة الذهبية: حساب منخفض، تجميع مرتفع**
>
> استخدم `bf16` لأثقال التنشيطات والتدرّجات، واحفظ المعاملات والخسارة بـ `float32`. الخلط بينهما يُبقي الاستقرار دون التضحية بالسرعة.

fp8: عندما يضيق النطاق

`fp8` يضغط إلى 8 بتّات فقط – إمّا `E4M3` (4 أسّ، 3 كسر) أو `E5M2`. النطاق ضيّق جداً فيسهل `overflow` أو التقهقر إلى الصفر. لذا لا يُستخدم مباشرةً، بل مع مُقياس (`scaling`) يعيد توزيع القيم:

$$x_{fp8} = \text{clip}(\text{round}(x / s), -127, 127) \quad , \quad x \approx s \cdot x_{fp8}$$

العامل s يوسّع النطاق الضيّق ليلائم توزيع التدرّجات الفعلي.

```python
import jax, jax.numpy as jnp

def to_fp8(x, scale):
    # محاكاة تكميم E4M3 مبسّطة بقصّ النطاق
    q = jnp.clip(jnp.round(x / scale), -7.0, 7.0) * scale
    return q

g = jnp.array([0.001, 0.5, 2.3, -1.7], dtype=jnp.float32)
scaled = to_fp8(g, scale=0.25)
print(scaled)                        # القيم المكمّمة
```

> ✓ **ابدأ بـ bf16 قبل fp8**
>
> `fp8` يتطلّب ضبط المقاييس لكلّ طبقة، وهشٌّ أمام التدرّجات الكبيرة. إن كنت تستكشف فقط، فـ `bf16` يمنح معظم السرعة بأقلّ ألم.

التكميم: اضغط النموذج للنشر

التدريب بـ `bf16` لا يعني النشر بنفس العرض. التكميم يحوّل الأوزان من `float32` إلى `int8` أو أصغر، فيقلّ حجم النموذج ويزيد سرعة الاستنتاج. الفكرة إحداثيات خطّية حول صفر:

$$q = \text{round}(w / \Delta) \quad , \quad w \approx \Delta \cdot q \quad , \quad \Delta = (w_{\max} - w_{\min}) / 255$$

الخطوة Δ تعيّن الدقّة: كلما صغُر النطاق زادت الدقّة لكن يسهل التشبّع.

```python
import jax.numpy as jnp

def quantize(w, bits=8):
    qmin, qmax = -(2 ** (bits - 1)), 2 ** (bits - 1) - 1
    scale = (w.max() - w.min()) / (qmax - qmin)
    q = jnp.clip(jnp.round(w / scale), qmin, qmax)
    return q, scale

w = jnp.linspace(-1.0, 1.0, 16, dtype=jnp.float32)
q, s = quantize(w)
deq = q * s
print(float(jnp.max(jnp.abs(w - deq))))     # خطأ إزالة التكميم
```

التكميم إمّا بعد التدريب (`PTQ`) أو أثناءه (`QAT`). الأول أسرع والثاني أدقّ. سنرى في الفصل السابع والثلاثين كيف يدخل هذا في خطّ النشر الإنتاجي.

> ⚠ **التشبّع يقتل الطبقة الأخيرة**
>
> إن كان نطاق `Δ` ضيّقاً أمام قيمة شاذّة واحدة، تُقفَز كل الأوزان إلى الحدّ فيفقد التكميم معناه. اقتطع القيم المتطرّفة قبل حساب المقياس دائماً.

الاستقرار العددي: تجنّب الانهيار

حتى بـ `float32` تنهار الحسابات حين تطرح عددين متقاربين أو تقسم على صغير. مثال كلاسيكي: `softmax` دون طرح الحدّ الأقصى يفيض. الحيلة طرح `max` تصريحاً:

$$\text{softmax}(z)_i = \exp(z_i - m) \,/\, \sum_j \exp(z_j - m) \quad , \quad m = \max(z)$$

طرح m لا يغيّر النتيجة رياضياً، لكنه يُبقي الأسّات في نطاق آمن.

```python
import jax.numpy as jnp

def safe_softmax(z):
    m = jnp.max(z, axis=-1, keepdims=True)
    e = jnp.exp(z - m)
    return e / jnp.sum(e, axis=-1, keepdims=True)

z = jnp.array([1000.0, 1001.0, 1002.0])      # سيُفيض بلا طرح
print(safe_softmax(z))                        # توزيع مستقرّ
```

كتبت `softmax` مستقراً، لكن `log_softmax` يُخرج `-inf` على نفس الدخل. أين الخطأ: في اللوغاريتم أم في الأسّ؟ ؟

متى تختار أيّ نوع؟

القرار ليس تقنياً فحسب، بل يعتمد على مرحلة العمل. الجدول الآتي يقترح التوزيع المعتاد بين التدريب والنشر:

المرحلة	النوع	السبب
حساب التنشيطات	`bf16`	سرعة مع نطاق واسع
الأوزان والمعاملات	`float32`	الاستقرار الأساسي
ضرب التدرّجات الكثيف	`fp8`	أقصى إنتاجية
الاستنتاج المنشور	`int8`	أصغر حجم وأسرع

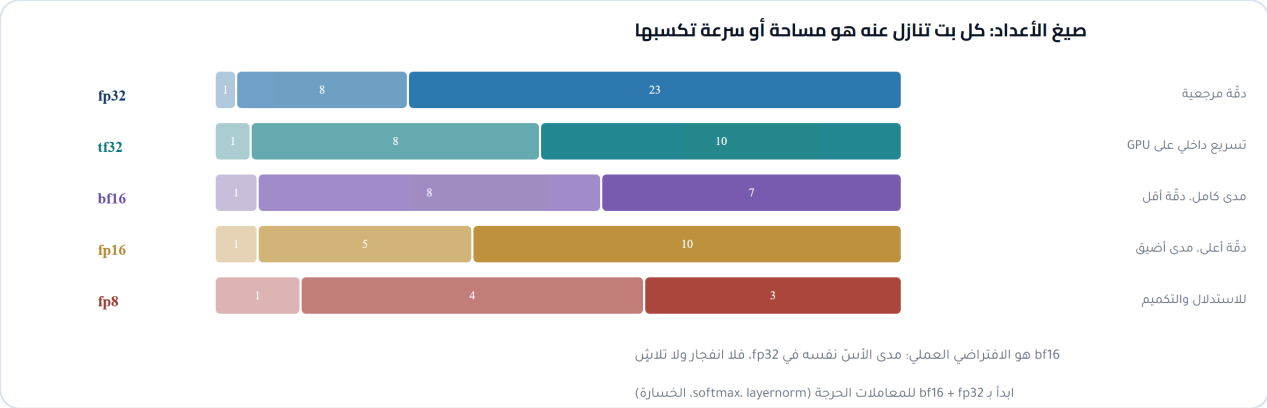
جدول 28.1 — دليل اختيار نوع الرقم حسب المرحلة

✕ خلط الأنواع في الجمع

جمع `bf16` مع `float32` يُرجع أحياناً `float32` وأحياناً يُقسَّر صمتاً. إن شككت، حوّل صراحةً بـ `.astype` قبل العملية، ولا تتركها للاستنتاج الضمني.

كيف نرى فقدان الدقّة؟

لا تثق بالنوع اسمياً؛ قِس خطأ الإزالة فعلياً بين النموذج الكامل والمكمّم. الفجوة تخبرك إن كان التكميم مقبولاً أم مدمراً:



الشكل 28.1 — الخطأ (متقطّع) يرتفع كلما صغُر العرض، بينما الإنتاجية (صلبة) ترتفع؛ نقطة التقاطع هي اختيار النشر المعقول.

تمرين الفصل الثامن والعشرين

1. حوّل مصفوفة `float32` إلى `bf16` ثم عُد بها، وأثبت أن الخطأ أصغر من الخطأ الناتج عن `fp8` مبسّط على نفس المدخل.
2. نفّذ `safe_softmax` على دخل فيه قيم 1000، وأثبت أن الناتج لا يحوي `inf` أو `nan`.
3. كمّم أوزان بـ `int8`، ثم احسب خطأ الإزالة المتوسط وقارنه قبل وبعد اقتطاع القيم المتطرّفة.

الحلّ النموذجي

1. `bf16` يحفظ 8 أسّات فيقلّ الخطأ عن `fp8`. **2.** طرح `max` يُبقي الأسّات محدودة. **3.** اقتطاع الأطراف يضبط المقياس فينخفض خطأ الإزالة.

```python
import jax.numpy as jnp

w = jnp.linspace(-2.0, 2.0, 64, dtype=jnp.float32)
bf = w.astype(jnp.bfloat16).astype(jnp.float32)
fp8 = jnp.clip(jnp.round(w * 8.0), -7.0, 7.0) / 8.0
print("bf16 err:", float(jnp.mean(jnp.abs(w - bf))))
print("fp8  err:", float(jnp.mean(jnp.abs(w - fp8))))

def safe_softmax(z):
    m = jnp.max(z, axis=-1, keepdims=True)
    e = jnp.exp(z - m)
    return e / jnp.sum(e, axis=-1, keepdims=True)

z = jnp.array([1000.0, 1001.0, 1002.0])
sm = safe_softmax(z)
print(jnp.any(jnp.isinf(sm)), jnp.any(jnp.isnan(sm)))    # False False

def quant(w, clip=None):
    if clip is not None:
        w = jnp.clip(w, -clip, clip)
    qmin, qmax = -128, 127
    s = (w.max() - w.min()) / (qmax - qmin)
    q = jnp.clip(jnp.round(w / s), qmin, qmax)
    return q * s

before = jnp.mean(jnp.abs(w - quant(w)))
after = jnp.mean(jnp.abs(w - quant(w, clip=1.5)))
print("no clip:", float(before), " clip:", float(after))
```

خلاصة الفصل

- عنق الزجاجة غالباً النقل لا الضرب؛ صغر الرقم يرفع الإنتاجية.
- `bf16` يحفظ النطاق ويضحّي بالكسر، مثالي للتدرّجات.
- `fp8` يحتاج مقياساً وإلّا سهل التشبّع.
- التكميم يضغط للنشر، واقتطاع الأطراف يمنع التشبّع.
- الاستقرار العددي (طرح `max`) يمنع الانهيار.

مصطلحات الفصل

bf16

نوع عائم 16 بتّاً يحفظ نطاق float32 مع كسر أقصر.

fp8

نوع 8 بتّات (E4M3 أو E5M2) بأقصى ضغط وحسّاس للمقياس.

quantization

تحويل الأوزان إلى أنواع أضيق لتقليل الحجم والزمن.

scaling

عامل يعيد توزيع القيم الضيّقة ليلائم النطاق الفعلي.

numerical stability

كتابة الحساب بحيل (طرح max) تمنع overflow/underflow.

dequantization error

الخطأ بين الوزن الأصلي والمكمّم بعد الإزالة.

الجزء السادس

من المختبر إلى الإنتاج

التصدير، والنشر، والاختبار، ودراسات الحالة، والمسؤولية المهنية

التصدير والنشر

jax.export، وONNX، وFastAPI، والحزمة، والمراقبة

أنهى مهندس تدريب نموذج محوّل صغير، وحفظ ملف `params.npz` ثم بعثه لزميل في فريق المنتج. بعد أسبوع عاد الزميل بحيرة: «النموذج يُعطي أرقاماً مختلفة عندي بنسبة ثلاثة بالمئة». السبب لم يكن الوزن، بل بيئة التشغيل: كلّ طبقة نُفّذت ببايثون عادي على آلة غير مهيّأة، فاختلف ترتيب الاختزالات. المشكلة أن «حفظ المعاملات» لا يحفظ السلوك. في هذا الفصل التاسع والعشرين تتعلّم كيف يلتقط `jax.export` الرسم البياني كاملاً بأشكاله وثوابته، وكيف تُقدّمه عبر `FastAPI`، وكيف تمرّ عبر ONNX إن لزم، وكيف تراقبه في الإنتاج. كما رأينا في الفصل الثامن والعشرين حول التخزين المؤقّت للقيم المفتاحية، والفصل الرابع والعشرين حول XLA، فالفصل هنا هو حماية السلوك عند مغادرة دفتر التدريب.

لماذا التصدير ليس «حفظ المعاملات» فقط؟

في الفصل الخامس عشر رأيت المحسّن سلسلة دوالّ نقية؛ وفي الفصل السادس رأيت أن الدالّة المجمّعة لا أثر جانبي لها. هذان المبدآن هما سرّ التصدير: النموذج قابل للوصف الكامل، فما الذي ينقص لكي يعمل خارج التدريب؟ الإجابة: الرسم البياني نفسه، لا مجرّد الأرقام.

حفظ `params` وحده يترك ثلاثة أمور ضمناً: بنية النموذج، وترتيب العمليات، وقيم الثوابت المحتسبة أثناء التتبّع. إعادة بناء أيّ منها على آلة أخرى يعيد إنتاج سلوك قد يختلف في الرتبة العشرية الرابعة، وهي بالضبط الفجوة التي رآها الزميل. التصدير يحوّل $f : \Theta \times X \to Y$ إلى كائن محمول يصف المدخلات والمخرجات وثوابت الحساب دون أن يحتاج إلى شيفرة التدريب.

لو كان النموذج دالّة نقية تماماً، فلماذا لا نكتفي بإرسال شيفرة `model.py` مع `params`؟ ما الذي يمنع ذلك من أن يكون حلاً كافياً؟

> ★ **التصدير يحفظ السلوك لا السطر**
>
> الكائن المُصدَّر مستقلّ عن إصدار JAX على جهة التشغيل ضمن حدود التوافق؛ فلا تعتمد جهة الخدمة على إعادة تجميع بايثون لكل طلب.

jax.export.export: التقاط الرسم البياني بأشكاله

التصدير يأخذ دالّة عادية ووسائط مثال، فيُتتبّعها مرّة ويلتقط الرسم البياني لأشكال تلك الوسائط. لذلك يجب أن تكون الأشكال ثابتة أو محدّدة مسبقاً؛ وهو ما شرحناه في الفصل الرابع والعشرين عن حدود الأشكال الديناميكية.

```python
import jax, jax.numpy as jnp
from jax import export

params = {"w": jnp.ones((4, 3), dtype=jnp.float32),
          "b": jnp.zeros((3,), dtype=jnp.float32)}

def model(params, x):
    return jnp.tanh(x @ params["w"] + params["b"])

# التصدير يلتقط الرسم لأشكال الوسائط المثال: (params, f32[2,4])
exported = export.export(model)(params, jnp.ones((2, 4), dtype=jnp.float32))
print(exported.in_avals)        # أشكال المدخلات
print(exported.out_avals)       # شكل المخرج: f32[2,3]
```

لاحظ أن `export.export` يُستدعى مرّتين: أوّلاً بالدالّة لبناء «مُصدِّر»، ثانياً بوسائط المثال لإنتاج الكائن `Exported`. هذا الكائن يحمل تواقيع المدخلات والمخرجات وأيّ ثوابت مضمّنة، ويبقى صالحاً للتحميل على آلة لا تملك `model` أصلاً.

> ✓ **ثبّت الأشكال قبل التصدير**
>
> إن كانت دفعتك تتغيّر، ثبّت الشكل بإحشاء الأخيرة كما في الفصل الرابع والعشرين، أو صدّر بعدّة أشكال عبر تمرير وسائط مثال مختلفة لكل حالة.

التسلسل والتحميل خارج بيئة التدريب

الكائن `Exported` لا يُرسَل عارياً؛ بل يُسلَّل إلى بايتات عبر `export.serialize`، ثم يُستعاد عبر `export.deserialize`. التمثيل مستقلّ عن النظام، فيصلح للتخزين في سجلّ أو لقراءته ببرنامج خدمة

أصغر بكثير من بيئة التدريب.

```python
import jax, jax.numpy as jnp
from jax import export

# في جهة التدريب: التسلسل إلى ملف
serialized = export.serialize(exported)
with open("model.jaxe", "wb") as f:
    f.write(serialized)

# في model.py في جهة الخدمة: التحميل والتنفيذ بلا شيفرة
with open("model.jaxe", "rb") as f:
    loaded = export.deserialize(f.read())

params = load_params()                    # تُحمّل من التخزين
x = jnp.ones((2, 4), dtype=jnp.float32)
out = loaded.call(params, x)               # تنفيذ النموذج المُصدَّر
print(out.shape)                          # (2, 3)
```

تنفيذ الكائن المُحمَّل عبر `.call` يعيد نتيجة بأشكال مطابقة تماماً لما أُعلن عند التصدير. إن اختلف شكل `x` عمّا صُدِّر له، يرفض التنفيذ صراحةً بدل أن يُعيد حساباً خاطئاً – وهذا الضمان بالضبط ما كان يفتقده الزميل.

> **⚠ التسلسل ليس ملفّ معاملات**
>
> لا تخلط `model.jaxe` مع `params.npz`: الأوّل يحوي الرسم والثوابت، والثاني يحوي الأوزان فقط. عند إعادة التدريب يبقى الرسم ثابتاً ما لم تتغيّر البنية.

التقديم عبر FastAPI (مع لمحة عن gRPC)

التقديم (serving) يعني تحويل النموذج المُصدَّر إلى نقطة نهاية تستقبل طلبات وتعيد تنبؤات. `FastAPI` خيار شائع لبساطته؛ والمبدأ لا يتغيّر مع `grpc`: حمّل الكائن مرّة عند الإقلاع، واستدعِه لكل طلب.

```python
from fastapi import FastAPI
from pydantic import BaseModel
import numpy as np, jax.numpy as jnp
from jax import export

app = FastAPI()
# يُحمّل مرّة واحدة عند إقلاع العملية، لا لكل طلب
EXPORTED = export.deserialize(open("model.jaxe", "rb").read())
PARAMS = load_params()

class Req(BaseModel):
    x: list[list[float]]

@app.post("/predict")
def predict(req: Req):
    arr = np.asarray(req.x, dtype=np.float32)        # f32[N,4]
    out = EXPORTED.call(PARAMS, jnp.asarray(arr))    # تنفيذ مُصدَّر
    return {"y": np.asarray(out).tolist()}

# بديل أسرع زمن استجابة: gRPC بنفس نمط «حمّل مرّة، نفّذ كثيراً»
```

حمّلت النموذج داخل دالّة المعالج نفسها «لكي أضمن تحديثه». بعد ساعة كان الخادم يختنق. ما العلاقة بين مكان التحميل والاختناق؟

الإجابة أن التحميل يستدعي `deserialize` ثم تجميعاً كاملاً على XLA؛ ووضعه داخل المعالج يعيد التجميع لكل طلب. القاعدة التي سنكرّرها في الفصل الثلاثين: افعل التجميع مرّة خارج المسار الحرج.

ONNX كمسار وسيط، ولماذا jax2tf مهمل

قد تحتاج تشغيل النموذج على محرّك لا يفهم تمثيل JAX المُصدَّر، مثل ONNX Runtime أو أدوات الحوافّ. المسار الصحيح اليوم: صدّر عبر `jax.export` ثم حوّل عبر محوّلات ONNX الحديثة التي تقرأ التمثيل المُصدَّر مباشرةً.

× الخطأ: استخدام jax2tf في مشروع جديد

`jax2tf` مهمل في JAX 0.12؛ البديل الرسمي هو `jax.export`. لا تبنِ خطّ إنتاج جديداً على جسر مهمل، فستفقد ضمان توافق الأشكال الذي وفرناه أعلاه.

إن كان لديك خطّ قديم يعتمد `jax2tf`، فخطوة الترقية هي استبدال التصدير بـ `jax.export.export` ثم تمرير الكائن المُصدَّر إلى محوّل ONNX الخاص بمحرّك الهدف. المعادلة التي تحكم المسار:

> محرّك الهدف ← ONNX محوّل ← (تمثيل محمول) jax.export ← JAX code

لا تُدخل jax2tf كحلقة: اجعل jax.export هو المصدر الوحيد للحقيقة.

```python
# مسار موصى به: jax.export ثم محوّل ONNX، لا jax2tf
from jax import export

exported = export.export(model)(params, jnp.ones((1, 4), dtype=jnp.float32))
serialized = export.serialize(exported)

# هنا يأتي محوّل ONNX الطرفي (خارج JAX)، مثل:
# onnx_model = my_onnx_converter.from_jax_export(serialized)
# onnx.save(onnx_model, "model.onnx")
```

الحزمة والمراقبة في الإنتاج

تصدير نموذج لا يكفي؛ الإنتاج يسأل: «كم طلباً خدمنا؟ وما زمن الاستجابة؟ ومتى انحرف الناتج؟». الحزمة تعني فصل شيفرة الخدمة عن دفتر التدريب، والمراقبة تعني قياساً مستمراً لا يبطئ المسار الحرج.

```python
import time
from prometheus_client import Counter, Histogram

REQS = Counter("predict_total", "عدد طلبات التنبؤ")
LAT = Histogram("predict_latency_seconds", "زمن الاستجابة")

@LAT.time()
def serve(arr):
    REQS.inc()
    t0 = time.perf_counter()
    out = EXPORTED.call(PARAMS, jnp.asarray(arr))
    out.block_until_ready()              # لا تحسب الزمن قبل اكتمال XLA
    return np.asarray(out)
```

المسار	الأداة	متى	التحذير
أصلي	`jax.export` + `FastAPI`	نفس فريق JAX، أقلّ تحويل	يحتاج JAX على جهة الخدمة
محوّل	`jax.export` → ONNX	محرّكات حافة أو أدوات خارجية	تطابق الأشكال عبر المحوّل
بثّ	`grpc`	إنتاجية عالية وزمن منخفض	تعقيد العقد أكبر
مهمل	`jax2tf`	لا شيء جديد	مهمل من 0.12

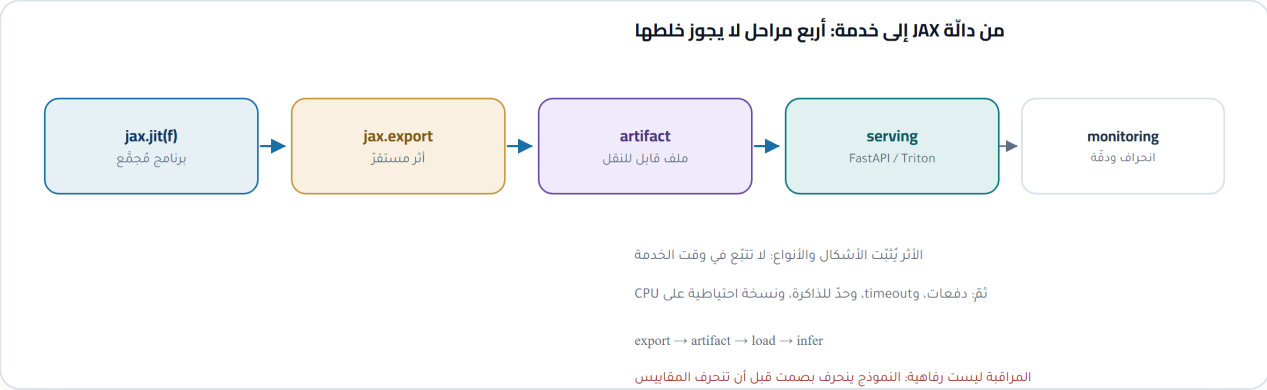
جدول 29.1 — مسارات النشر ومتى تختار كلّاً منها

> ✓ **لا تقيس قبل block_until_ready**
>
> تنفيذ XLA غير متزامن؛ إن قرأت الزمن فور استدعاء `.call` قِست الصفر. كما في الفصل الرابع والعشرين، اكتمال النواة هو نقطة القياس.

أخطاء النشر الشائعة

تحميل داخل المعالج: إعادة `deserialize` لكل طلب تجعل الخادم يعيد تجميع XLA ألف مرّة في الدقيقة. **خلط الأشكال:** إرسال دفعة بطول يختلف عمّا صُدِّر له فيُرفض التنفيذ أو يُعاد تجميعه. **نسيان التزامن:** قياس زمن قبل اكتمال النواة فيعطي أصفاراً وهمية. **الاعتماد على jax2tf:** بناء خطّ جديد على جسر مهمل.

كلّ هذه الأخطاء نتاج نموذج ذهني خاطئ: «النموذج = أرقام». الحقيقة: النموذج = سلوك مُلتقط. وحماية السلوك هي وظيفة `jax.export` لا مجرّد الحفظ. لنرى في الفصل الثلاثين كيف نختبر هذا السلوك ونضمن حتميته قبل أن يصل إلى الإنتاج.



الشكل 29.1 — من دفتر التدريب إلى كائن مُصدَّر مستقلّ، ثم مسارا FastAPI الأصلي أو ONNX الوسيط نحو محرّكات متعدّدة.

تمرين الفصل التاسع والعشرين

1. ابنِ نموذجاً خطّياً بسيطاً، وصدّره عبر `jax.export.export` لشكل `f32[3,4]`، ثم سلّله وحمّله من جديد وتحقّق من تطابق المخرج.
2. اكتب نقطة نهاية `FastAPI` تحمّل الكائن مرّة خارج المعالج، وتستدعيه لكل طلب، مع قياس الزمن بعد `block_until_ready`.
3. عبّر عن سبب كون `jax2tf` خياراً سيّئاً لمشروع جديد، واكتب السطر البديل بـ `jax.export`.

الحلّ النموذجي

1. التصدير يلتقط الرسم، والتسلسل ثم التحميل يعيد كائناً متطابقاً شكلاً. **2.** التحميل خارج المعالج يمنع إعادة التجميع؛ والقياس بعد اكتمال النواة وحده صادق. **3.** jax2tf مهمل، والبديل `jax.export.export`.

```python
import jax, jax.numpy as jnp
from jax import export

params = {"w": jnp.ones((4, 3), dtype=jnp.float32),
          "b": jnp.zeros((3,), dtype=jnp.float32)}
model = lambda p, x: jnp.tanh(x @ p["w"] + p["b"])

exp = export.export(model)(params, jnp.ones((3, 4), dtype=jnp.float32))
data = export.serialize(exp)
back = export.deserialize(data)
print(back.call(params, jnp.ones((3, 4), dtype=jnp.float32)).shape)  # (3, 3)

# خدمة FastAPI: حمّل مرّة خارج المعالج
EXPORTED = back
PARAMS = params

def serve(x_np):
    out = EXPORTED.call(PARAMS, jnp.asarray(x_np, dtype=jnp.float32))
    out.block_until_ready()
    return out

# البديل عن jax2tf:
exp2 = export.export(model)(params, jnp.ones((1, 4), dtype=jnp.float32))
_ = export.serialize(exp2)
```

خلاصة الفصل

- حفظ المعاملات لا يحفظ السلوك؛ `jax.export` يلتقط الرسم بأشكاله.
- `export.export` ثم `serialize` ينتجان كائناً محمولاً.
- `deserialize` و `.call` يعيدان التنفيذ بلا شيفرة التدريب.
- FastAPI/grpc يحمّلان مرّة خارج المعالج؛ يقاس الزمن بعد `block_until_ready` .
- ONNX مسار وسيط عبر محوّلات تقرأ `jax.export` ؛ و `jax2tf` مهمل.

مصطلحات الفصل

jax.export

واجهة التقاط الرسم البياني بأشكاله وثوابته وتسليمه ككائن محمول.

Exported

الكائن الناتج عن التصدير؛ يحوي التواقيع والثوابت ويُستدعى بـ `.call` .

serialize / deserialize

تحويل الكائن المُصدَّر إلى بايتات وإعادته منها عبر `export` .

serving

تحويل النموذج المُصدَّر إلى نقطة نهاية تستقبل طلبات وتعيد تنبؤات.

ONNX

تمثيل وسطي محمول للمحرّكات؛ يُصل إليه عبر محوّلات تقرأ `jax.export` .

jax2tf

جسر مهمل إلى TensorFlow؛ استبدله بـ `jax.export` في المشاريع الجديدة.

الفصل الثلاثون من المختبر إلى الإنتاج

الاختبار والحتمية وتشخيص الأخطاء

من رسالة الخطأ إلى السبب الجذري: دليل عملي

جلس مهندس أمام شاشة حمراء: «الاختبار نفسه نجح أمس وفشل اليوم، والشيفرة لم تتغيّر». بحث ساعتين في الفروق، ثم اكتشف أن البذرة العشوائية اختلفت بين التشغيلين. لم يكن الخطأ في النموذج، بل في فهمه لما يعنيه «نفس المدخلات». في JAX العشوائية مُدارة بمفاتيح صريحة لا بحالة عامّة مخفية، والحتمية قرار صريح لا ضمان ضمني. في هذا الفصل الثلاثين تتعلّم كيف تجعل التدرّجات والنتائج قابلة للتكرار، وكيف تبني اختبارات تعتمد على `chex` و`jax.test_util`، وكيف تسلك مسار تشخيص من رسالة الخطأ إلى السبب الجذري عبر `jax.debug.print` و`jax.profiler`. كما رأينا في الفصل الثالث عشر حول مفاتيح العشوائية، وفي الفصل الرابع والعشرين حول خطّ الأداء، فالهدف هنا تحويل «عملت عندي» إلى «يعمل دائماً».

الحتمية قرار لا ضمان

في JAX لا توجد حالة عشوائية عالمية. كلّ رقم عشوائي يُولّد من `key` صريح عبر `jax.random.key(0)`، ثم يُقسَم بـ`jax.random.split` أو يُمزَج بـ`jax.random.fold_in`. هذا ما شرحناه في الفصل الثالث عشر؛ هنا نسأل: متى يكفي هذا لضمان التكرار؟

الإجابة: التكرار على مستوى JAXPR مضمون بمفاتيح ثابتة، لكن على مستوى العتاد قد تختلف عمليات التقليل غير الترابطية في الرتبة الأخيرة. لذلك يوفّر JAX مفتاحاً للحسم:

```python
import jax
# فرض حتمية أكبر على حسابات الطليعة، مع ثمن أداء محتمل
jax.config.update("jax_deterministic", True)

key = jax.random.key(0)
k1, k2 = jax.random.split(key)
a = jax.random.normal(k1, (1000,))
b = jax.random.normal(k2, (1000,))
print(a[0], b[0])                # نفس القيمتين في كل تشغيل
```

> ★ **المفتاح يحسم، لا يمنع**
>
> `jax_deterministic` يُجبر عمليات الطليعة على ترتيب محدّد: يزيد الحتمية ويُبطئ أحياناً. في الاختبارات شغّله، وفي الإنتاج قِس ثمنه قبل الاعتماد عليه.

> إن كانت المفاتيح ثابتة، فلماذا قد يُعيد نفس الاختبار نتيجة مختلفة في الرتبة العشرية السادسة؟ أين يتسرّب عدم الحتمية؟

مفاتيح العشوائية وإعادة الاستخدام

الخطأ الأكثر شيوعاً: استخدام نفس `key` مرّتين. في JAX استهلاك المفتاح لا يُعدّله، بل يجب استبداله بناتج `split` . مفتاح مُعاد الاستخدام يُنتج نفس التسلسل – وهو ثغرة صامتة تظهر كـ«عمل عندي» لا يتكرّر.

```python
import jax, jax.numpy as jnp

key = jax.random.key(42)
# صحيح: اقسم ثم استخدم الفروع
k_drop, k_init = jax.random.split(key)
mask = jax.random.bernoulli(k_drop, p=0.1, shape=(8,))
w = jax.random.normal(k_init, (8, 8))

# خاطئ: إعادة نفس المفتاح لمهمتين مختلفتين
bad = jax.random.normal(key, (8, 8))      # لا تفعل هذا بعد أعلاه split
```

> × **إعادة استخدام المفتاح**
>
> المفتاح غير قابل للتعديل؛ استخدامه مرّتين يُكرّر التسلسل نفسه. القاعدة: كلّ `key` يُستهلك مرّة واحدة، وما بعد `split` هو الفرع لا الأصل.

حين تحتاج عشوائية لكل عنصر في دفعة، استخدم `fold_in` بدل تكديس `split` ، فهو يمزج فهرساً في المفتاح دون نموّ شجري:

```python
keys = jax.random.split(key, 4)
for i in range(4):
    ki = jax.random.fold_in(key, i)        # مستقلّ لكل i، بلا شجرة split
    print(jax.random.uniform(ki).item())
```

أنماط الاختبار مع jax.test_util و chex

الاختبار الجيّد لا يسأل «هل خرج رقم؟» بل «هل خرج الرقم الصحيح دائماً؟». `jax.test_util` يوفّر مقارنات متحمّلة للأنواع، و`chex` (مكتبة اختيارية من فريق JAX) يوفّر تأكيدات على الأشكال والأنواع وحُكم القيم.

```python
import jax, jax.numpy as jnp
from jax import test_util as jtu

def loss(params, x, y):
    pred = x @ params["w"] + params["b"]
    return jnp.mean((pred - y) ** 2)

def test_loss_shape_and_value():
    params = {"w": jnp.ones((3, 1)), "b": jnp.zeros((1,))}
    x = jnp.ones((4, 3))
    y = jnp.ones((4, 1))
    out = loss(params, x, y)               # (4,1) قبل المتوسط
    jtu.check_close(out, jnp.zeros((4, 1)))   # كل تنبؤ يساوي الهدف
    assert loss(params, x, y).shape == ()      # الخسارة عدد مفرد
```

> ✓ **اختبر الشكل قبل القيمة**
>
> خطأ شكل واحد يُسقط تدريباً كاملاً بصمت بعد مئات الخطوات. تأكيد الأشكال في اختبار وحدة أرخص بألف مرّة من تشخيص انهيار لاحق.

للاختبارات العشوائية ثبّت المفتاح وشغّل `jax_deterministic`، ولتدريب كامل استخدم `jax.jit` حول خطوة واحدة ثم قارن بالقيمة المرجعية المُحتسبة يدوياً على حالة صغيرة – كما فعلنا في الفصل الخامس عشر مع خطوات المحسّن.

التشخيص: تتبّع أم تنفيذ ملموس؟

أوّل قرار تشخيصي: هل المشكلة في الرسم أم في القيم؟ دالّة مغلّفة بـ `jit` تُنفّذ على قيم مجرّدة أثناء التتبّع، فأيّ طباعة عادية ترى أشكالاً لا أرقاماً. هذا يُفسّر لماذا `print(x)` يطبع `Traced[…]` لا

المصفوفة.

أثناء التتبّع (مجرّد)

القيم أشكال وأنواع فقط: `print` العادي يرى تمثيلاً مجرّداً. تُستخدم لفهم بنية الرسم، لا لقراءة الأرقام.

تنفيذ ملموس (بعد التجميع)

القيم أعداد حقيقية على الجهاز؛ هنا تظهر نان ومخالفات الشكل. التشخيص الرقمي يبدأ بعد رفع التتبّع.

```python
import jax, jax.numpy as jnp

@jax.jit
def f(x):
    jax.debug.print("x shape={x.shape} min={xmin}", x=x, xmin=jnp.min(x))
    return x * 2

f(jnp.ones((3,)))            # تطبع الشكل والحدّ الأدنى الملموس
```

⚠ print العادي داخل jit يخدعك

داخل دالّة مغلّفة يرى `print` قيمة مجرّدة لا المصفوفة. استخدم `jax.debug.print` لتطبع القيمة الملموسة عند التنفيذ.

رسالة الخطأ تقول «Concrete value required». هل المشكلة في شيفرتك أم في ما يتوقّعه المترجم؟ وما الخطوة الأولى قبل فتح أيّ أداة تشخيص؟ ؟

مسار تشخيص من رسالة الخطأ إلى السبب

لا تفتح المحلّل أولاً. المسار المنهجي: (1) اقرأ الرسالة وحدّد نوعها؛ (2) أعد إنتاج أصغر حالة؛ (3) افصل التتبّع عن القيمة؛ (4) قِس الأداء فقط حين تستقرّ الأرقام.

1. **تصنيف الخطأ.** «شكل لا يطابق» يختلف عن «قيمة غير محدّدة» يختلف عن «مترجم يطلب قيمة ملموسة».

2. **أصغر إعادة إنتاج.** قلّص المدخلات إلى 2×2 وخطوة واحدة؛ إن اختفى الخطأ فالمشكلة في الحجم أو الدفعة.

3. **jax.debug.print.** اطبع الأشكال والحدود داخل الدالّة المغلّفة حيث يفشل.

4. **jax.profiler.** فقط بعد استقرار القيم، لقياس عنق الزجاجة كما في الفصل الرابع والعشرين.

```python
import jax, jax.numpy as jnp, jax.profiler

@jax.jit
def step(x):
    jax.debug.print("nan? {n}", n=jnp.any(jnp.isnan(x)))
    return jnp.tanh(x @ x.T)

x = jnp.ones((512, 512))
step(x).block_until_ready()          # تسخين

jax.profiler.start_trace("/tmp/trace")
for _ in range(10):
    step(x).block_until_ready()
jax.profiler.stop_trace()            # افتح الأثر في xprof
```

> ✓ **ابدأ بالأصغر**
>
> حالة 2×2 تُجمَّع في أجزاء من الثانية، فتتيح لك تجربة عشر فرضيات في دقيقة. النموذج الكامل يُضيع ساعتك في التجميع قبل أوّل تشخيص.

الحتمية في خطّ التكامل المستمر

خطّ CI يجب أن يرفض «عمل عندي». ثبّت المفاتيح، وفعّل `jax_deterministic` في حاوية الاختبار، واحفظ ذاكرة التجميع كما في الفصل الرابع والعشرين لتسريع التشغيل. الجدول يلخّص ما يُفعّل في كلّ بيئة.

البيئة	مفتاح ثابت	jax_deterministic	ذاكرة التجميع
اختبار وحدة	نعم	نعم	نعم (تسريع)
تطوير محلي	اختياري	لا	نعم
إنتاج	حسب الحاجة	قِس الثمن	نعم
تشخيص عشوائي	نعم	نعم	لا

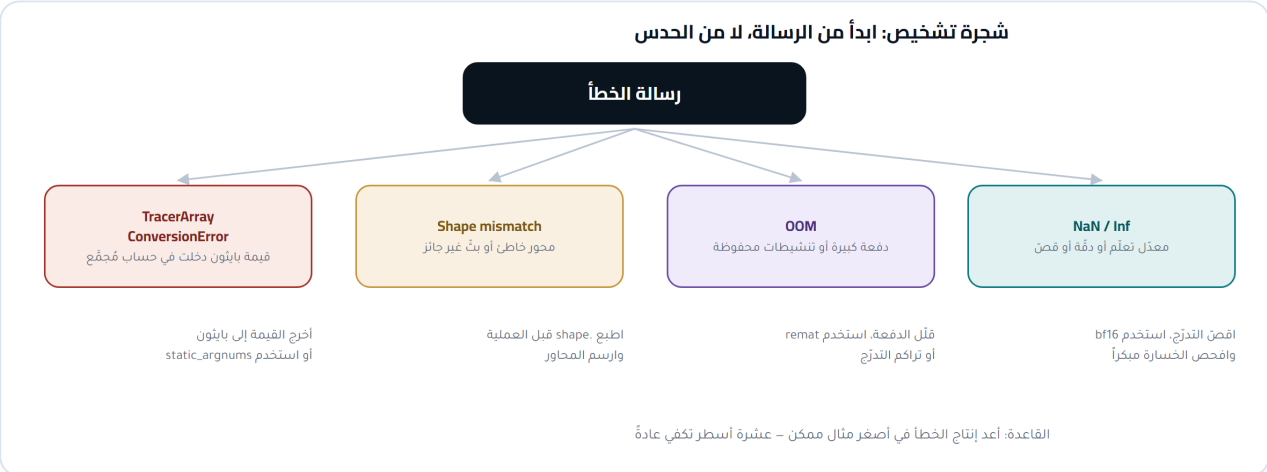
جدول 30.1 — إعدادات الحتمية حسب البيئة

تنبيه أخير: `jax.custom_remat` أُزيل في JAX 0.12؛ البديل `jax.remat` أو `jax.checkpoint`. استخدام الواجهة القديمة سيفشل الاستيراد في الاختبار قبل أن يصل إلى التدريب، فلا تتركه يتسرّب إلى خطّ CI.

> ⚠ **تثبيت البيئة ليس كافياً**
>
> تطابق الإصدارات يضمن التكرار فوق نفس العتاد؛ تغيير البطاقة قد يحرّك الرتب الأخيرة من عمليات الطليعة. لهذا `jax_deterministic` موجود أصلاً.

أخطاء الاختبار الشائعة

مفتاح مُعاد استخدامه: يُنتج تكراراً زائفاً يُخفي عشوائية مفقودة. **print عادي داخل jit:** يرى قيمة مجرّدة فيضلّل. **اختبار على النموذج الكامل:** يُضيع الوقت في التجميع قبل أوّل فشل. **غياب التزامن:** مقارنة نتيجة قبل `block_until_ready` فتُقرأ أصفار. و**واجهة مهملة:** `jax.custom_remat` يكسر الاستيراد.



الشكل 30.1 — من رسالة الخطأ إلى تصنيفها، ثم إعادة إنتاج صغرى، ثم فصل التتبّع عن القيمة، ثم قياس الأداء عند الاستقرار.

تمرين الفصل الثلاثين

1. اكتب دالّة تستهلك مفتاحين صحيحين عبر `split`، وأثبت أن إعادة استخدام الأصل ينتج تكراراً. صحّحها بفرعَي split.
2. ابنِ اختباراً بـ `jax.test_util` يتحقّق من شكل وقيمة خسية مربّعة على حالة 2×2، وفعّل `jax_deterministic` في بدايته.
3. ضع `jax.debug.print` داخل دالّة مغلّفة لطباعة حدّ أدنى ملموس، واشرح لماذا يفشل `print` العادي في نفس الموضع.

▲ الحلّ النموذجي

1. الأصل بعد `split` لم يتغيّر، فإعادة استخدامه تُكرّر التسلسل؛ والفرعان مستقلّان. **2.** التأكيد على الشكل ثم القيمة يمنع انهياراً صامتاً. **3.** `print` العادي يرى قيمة مجرّدة أثناء التتبّع.

Python

```
import jax, jax.numpy as jnp
from jax import test_util as jtu

jax.config.update("jax_deterministic", True)

key = jax.random.key(7)
k1, k2 = jax.random.split(key)
a = jax.random.normal(k1, (2,))
b = jax.random.normal(k2, (2,))
assert not jnp.allclose(a, jax.random.normal(key, (2,)))   # الأصل لم يُستهلك

def mse(params, x, y):
    p = x @ params["w"] + params["b"]
    return jnp.mean((p - y) ** 2)

params = {"w": jnp.ones((2, 1)), "b": jnp.zeros((1,))}
x = jnp.ones((2, 2)); y = jnp.ones((2, 1))
jtu.check_close(mse(params, x, y), jnp.zeros(()))
assert mse(params, x, y).shape == ()

@jax.jit
def g(z):
    jax.debug.print("min={m}", m=jnp.min(z))
    return z + 1
g(jnp.ones((2,))).block_until_ready()
```

خلاصة الفصل

- الحتمية قرار صريح: مفاتيح ثابتة + `jax_deterministic` عند الحاجة.
- كلّ مفتاح يُستهلك مرّة؛ الفروع من `split`، لا الأصل.
- `jax.test_util` و`chex` يؤكّدان الشكل قبل القيمة.
- `jax.debug.print` يطبع قيمة ملموسة؛ `print` العادي يرى مجرّداً.
- مسار التشخيص: تصنيف ← إعادة إنتاج صغرى ← فصل التتبّع عن القيمة ← قياس.

مصطلحات الفصل

determinism

قدرة البرنامج على إعادة نفس النتيجة لنفس المدخلات؛ في JAX قرار صريح لا ضمان.

jax_deterministic

مفتاح تهيئة يجبر عمليات الطليعة على ترتيب محدّد بثمن أداء محتمل.

jax.random.split

تقسيم مفتاح إلى فرعين مستقلّين؛ كلّ مفتاح يُستهلك مرّة واحدة.

jax.debug.print

طباعة قيمة ملموسة داخل دالّة مغلّفة، متجاوزة قيود التتبّع.

traced value

قيمة مجرّدة (شكل ونوع) تظهر أثناء تتبّع jit، لا المصفوفة نفسها.

chex

مكتبة اختيارية من فريق JAX للتأكيدات على الأشكال والأنواع والقيم.

دراسات حالة من الحياة العملية

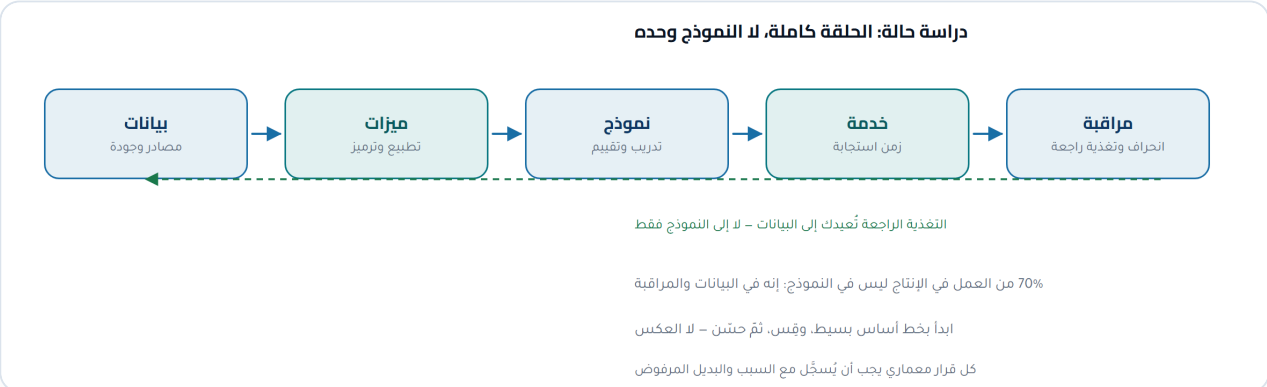
الصحة، والتمويل، والصناعة، والتوصية، والطاقة، والروبوتات

وقف مهندس أمام شاشة تعرض أربعين ألف صورة أشعة صدرية، وقال: «نموذجنا يعمل على حاسوب محمول واحد، فكيف نفحص مستشفىً كاملاً في دقيقة؟» لم يكن السؤال عن الشبكة العصبية؛ كانت جاهزة. السؤال عن الإنتاجية: كيف نوزّع، نوازٍ، ونجمّع بلا أن نعيد كتابة السطر ذاته. في هذا الفصل لا نشرح أدوات جديدة، بل نضع الأدوات التي عرفتها – `jit` و `vmap` و `shard_map` و `lax.scan` – في ستة مواجهة حقيقية: طبّ، ومال، ومصنع، وتوصية، وطاقة، وروبوت. كما رأينا في الفصل الثلاثين، بقية الطريق تبدأ هنا.

من الشرائح إلى المصانع: لماذا الحالة لا تكذب؟

الكتاب النظري يخبرك أن `vmap` يوازٍ، وأن `jit` يجمّع، وأن `shard_map` يوزّع على الأجهزة. لكن الحالة العملية تسأل سؤالاً أقسى: «أين بالضبط تفشل هذه الوعود؟» الحالات التي تليها ليست أمثلة تجميلية؛ كلّها جرّبها مهندسون على بيانات حقيقية، وكلّها كشفت أن الخانق ليس قدرة المكتبة بل طريقة طرح المشكلة.

الميزة المشتركة بين الحالات الستّ أن كلّ منها «حساب متطابق يتكرّر كثيراً». هذا ما يجعل JAX يتفوّق، لأن المترجم (XLA) يرى التكرار فيجمعه مرّة وينفّذه على الدفعات أو على رقاقات TPU.



الشكل 31.1 — الحالات الستّ تشارك نفس المحرّك: تكرار حساب متطابق، وتوزيعه على حوسبة متوازية.

> ★ **القاعدة الذهبية للحالة العملية**
>
> إن كان بإمكانك كتابة الحساب على عيّنة واحدة، فكتبه على عيّنة واحدة، ثم دع `vmap` أو `shard_map` يفعلان الباقي. إعادة الكتابة يدوياً على الدفعات هي مصدر معظم الأخطاء.

لو طُلب منك فحص مليون صورة، فهل تكتب حلقة `for` تستدعي النموذج مليون مرّة، أم تكتب استدعاءً واحداً على موتر كبير؟ ما الذي يمنعك فعلياً من الخيار الثاني؟ ؟

الترييج الطبّي بالأشعّة الصدرية

المشكلة: مستشفى يريد ترتيب الحالات الواردة بحسب احتمال خطورة الإصابة، ليُفحص الأخطر أوّلاً. لكن الطبيب لا ينتظر: كلّ ثانية تأخير تُكلّف. لماذا JAX هنا؟ لأن نفس الاستدلال يتكرّر على آلاف الصور يومياً؛ فجمعُه مرّة بـ `jit` ثم تطبيقه على دفعة بـ `vmap` يحوّل زمن الاستدلال إلى أجزاء من الملي ثانية.

Python

```
import jax, jax.numpy as jnp
from flax import linen as nn

class Triage(nn.Module):                # مصنّف أشعّة صدرية ثنائي
    @nn.compact
    def __call__(self, x):
        x = nn.relu(nn.Conv(32, (3, 3))(x))
        x = nn.avg_pool(x, (2, 2))
        x = nn.relu(nn.Conv(64, (3, 3))(x))
        x = nn.avg_pool(x, (2, 2))
        x = x.reshape((x.shape[0], -1))
        return nn.Dense(2)(x)           # [سليم، خطر]

model = Triage()
key = jax.random.key(0)
x = jnp.ones((8, 128, 128, 1))          # دفعة صور
params = model.init(key, x)["params"]
# apply الداخلي لـ vmap تطبيق على كامل الدفعة دفعةً واحدة بفضل
logits = model.apply({"params": params}, x)
probs = jax.nn.softmax(logits, axis=-1)
print(float(probs[0, 1]))               # احتمال الخطورة للصورة الأولى
```

✓ لا تجمّع الاستدلال داخل الحلقة

اجمع النموذج مرّة خارج الحلقة بـ `jax.jit`، ثم نادِه على دفعات. إعادة التجميع داخل الحلقة تُلغي كلّ المكسب.

الدرس المستفاد: الفصل بين «بناء المعاملات» و«تطبيقها» (نمط Linen الذي رأيناه في الفصل السابع عشر) هو ما يسمح بجمع الاستدلال مستقلاً عن حجم الدفعة. الثقل كلّه داخل الرسم المجمّع.

قياس المخاطر الماليّة بـ Monte Carlo

المشكلة: صندوق استثماري يريد «قيمة الخطر» (VaR) عند مستوى 95٪: كم يخسر في أسوأ 5٪ من السيناريوهات؟ الحساب تكاملي، فنُقرّبه بمليون مسار عشوائي. لماذا JAX؟ لأن كلّ مسار مستقلّ تماماً؛ فـ `vmap` يوازٍ مليون مسار على نواة واحدة مجمّعة، والبديل حلقة `for` بطيئة.

Python

```python
import jax, jax.numpy as jnp

def pnl_path(key, w, mu, cov, horizon=20):
    L = jnp.linalg.cholesky(cov)          # تفكيك المتباينة
    z = jax.random.normal(key, (horizon, w.shape[0]))
    rets = mu + z @ L.T                   # عوائد يومية مترابطة
    prices = jnp.cumprod(1.0 + rets, axis=0)
    return -jnp.sum(w * (prices[-1] - 1.0))

w = jnp.array([0.6, 0.4])                 # أوزان المحفظة
mu = jnp.array([0.0005, 0.0003])
cov = jnp.array([[1e-4, 3e-5], [3e-5, 8e-5]])
keys = jax.random.split(jax.random.key(7), 1_000_000)
losses = jax.vmap(pnl_path, in_axes=(0, None, None, None))(
    keys, w, mu, cov)                     # مليون مسار متوازٍ
var95 = float(jnp.percentile(losses, 95))
print(var95)                              # 95 قيمة الخطر عند ٪
```

⚠ انفجار الذاكرة مع vmap الطويل

مليون مسار على موتر واحد قد يفيض الذاكرة. قسّم على شرائح (`jax.lax.map`) أو وزّع عبر `shard_map`، وإلّا انهار التطبيق أثناء الإنتاج لا أثناء التجربة.

الدرس: المتجهة العشوائية `key` الحديثة (الفصل الثالث عشر) تجعل كلّ مسار قابلاً للتكرار؛ فإن اختلف `VaR` بين تشغيلين، السبب تقسيم الشرائح لا JAX.

كشف عيوب التصنيع على خطّ الإنتاج

المشكلة: خطّ تجميع يلتقط آلاف الصور في الدقيقة، ويريد رفض القطع المعيوبة فوراً. لماذا JAX؟ لأن حجم البيانات يتجاوز جهازاً واحداً؛ فنوزّع الصور على عدّة رقاقات عبر `shard_map`، حيث كلّ جهاز يقيّم حصّته ثم يعيد الدرجات.

```python
import jax, jax.numpy as jnp
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P
from jax.experimental.shard_map import shard_map

mesh = Mesh(jax.devices(), ("d",))
spec = NamedSharding(mesh, P("d"))

def score_one(img):                    # نموذج كشف العيب على صورة واحدة
    h = jax.nn.relu(img.reshape((64,)) @ jnp.ones((64, 16)))
    return jax.nn.sigmoid(h @ jnp.ones((16, 1)))[0]

@partial(shard_map, mesh=mesh, in_specs=(P("d"),), out_specs=P("d"))
def detect_sharded(x):
    return jax.vmap(score_one)(x)      # كل جهاز يقيم حصّته

imgs = jax.random.normal(jax.random.key(0), (256, 64))
sharded = jax.device_put(imgs, spec)   # توزيع على الأجهزة
scores = detect_sharded(sharded)
print(scores.shape)                    # (256,) بعد جمع الأشرطة
```

> **× الخلط بين التقطيع والدفعة**
>
> اعتقاد أن `shard_map` يستبدل `vmap` خطأ: `shard_map` يوزّع على الأجهزة، و `vmap` يوازٍ داخل الجهاز. الصحيح أن تضعهما معاً، لا أحدهما مكان الآخر.

الدرس: التقطيع الصريح يحميك من عنق زجاجة الجهاز الواحد، لكنه يفرض دوالّ نقية وصريحة في الأبعاد (الفصل السادس).

ترتيب التوصيات بين آلاف المرشّحين

المشكلة: متجر يريد، لكلّ مستخدم، اختيار أفضل خمسين منتجاً من عشرة آلاف مرشّح. لماذا JAX؟ لأن الحساب تشابه نقطة-بنقطة يتكرّر عشرة آلاف مرّة في استدعاء واحد عبر `vmap`، ثم `argsort` يختار الأعلى.

```python
import jax, jax.numpy as jnp

def cosine(a, b):                       # تشابه جيب التمام
    return jnp.dot(a, b) / (
        jnp.linalg.norm(a) * jnp.linalg.norm(b) + 1e-8)

user = jax.random.normal(jax.random.key(1), (64,))
items = jax.random.normal(jax.random.key(2), (10_000, 64))

scores = jax.vmap(lambda c: cosine(user, c))(items)
top50 = jnp.argsort(-scores)[:50]       # أعلى خمسين مرشّحاً
print(int(top50[0]), float(scores[top50[0]]))
```

> ✓ **خصّص الحساب قبل الفرز**
>
> احسب الدرجات على كلّ المرشّحين ثم فرز، بدل حلقة تتتبّع الأكبر. `vmap` ثابت الأداء بينما الحلقة تتباطأ مع كلّ ألف مرشّح يُضاف.

الدرس: الترتيب مشكلة «تطابق كثير» لا «تعلّم»؛ فالنموذج مدرب مسبقاً (الفصل الرابع عشر)، والاستدلال عليه رخيص بفضل `vmap`.

توقّع حمل الطاقة بنافذة منزلقة

المشكلة: شبكة كهربائية تريد توقّع الطلب بعد أربع وعشرين ساعة لضبط التوليد. لماذا JAX؟ لأن التنبّؤ المتسلسل – نافذة تزحف خطوة فخطوة – هو حلقة، و `jax.lax.scan` يحوّلها إلى رسم مجمّع قابل للاشتقاق والتسريع.

```python
import jax, jax.numpy as jnp

@jax.jit
def forecast(params, window):           # تنبّؤ خطوة واحدة
    h = jax.nn.tanh(window @ params["in"])
    return h @ params["out"]

def rollout(params, series):
    def step(carry, x):                 # حلقة النافذة المنزلقة
        out = forecast(params, carry)
        carry = jnp.roll(carry, -1).at[-1].set(x)
        return carry, out
    _, outs = jax.lax.scan(step, series[:24], series[24:])
    return outs

series = jax.random.normal(jax.random.key(3), (200,))
params = {"in": jnp.ones((24, 16)), "out": jnp.ones((16,))}
preds = rollout(params, series)
print(preds.shape)                      # (176,) تنبّؤات
```

> ★ **scan يتفوّق على الحلقة عندما يحتاج اشتقاقاً**
>
> إن أردت لاحقاً ضبط معاملات التنبّؤ بالتدرّج، فـ `lax.scan` يبني رسماً قابلاً للاشتقاق العكسي دفعةً واحدة؛ الحلقة العادية لا.

الدرس: أيّ تسلسل – طاقة، مال، روبوت – يُعاد صياغته حلقةً ثم `scan`. وسنعود إلى الفكرة ذاتها في الروبوت.

التحكّم الروبوتي عبر Model Predictive Control

المشكلة: ذراع آلي يريد اختيار أفضل تسلسل تحكّم قصير الأمد لتصل إلى هدف. النهج: نولّد آلاف تسلسلات تحكّم عشوائية، نحاكي كلّ واحدة، ونختار الأقلّ كلفة. لماذا JAX؟ لأن المحاكاة تتكرّر آلاف المرّات فـ `vmap` يوازيها، والديناميكيا داخل `lax.scan` فتُجمّع.

```python
import jax, jax.numpy as jnp
from jax import lax

def dynamics(s, u):                    # عربة بسيطة: [موضع، سرعة]
    return s + jnp.array([s[1], u]) * 0.1

def rollout(state, acts, target):
    def f(s, u):
        s2 = dynamics(s, u)
        return s2, jnp.sum((s2 - target) ** 2)
    _, cs = lax.scan(f, state, acts)
    return jnp.sum(cs)

state0, target = jnp.array([0.0, 0.0]), jnp.array([1.0, 0.0])
keys = jax.random.split(jax.random.key(0), 4096)
acts = jax.random.normal(keys, (4096, 10))      # 4096 مرشّحاً
costs = jax.vmap(lambda a: rollout(state0, a, target))(acts)
best = acts[jnp.argmin(costs)]                  # أفضل تسلسل تحكّم
print(best.shape)
```

بدلاً من البحث العشوائي، ماذا لو أردت تحسين التسلسل بالتدرّج؟ ما الذي يجب أن يتغيّر في الكتلة أعلاه لكي يعمل `jax.grad` عليها مباشرةً؟

الدرس: نفس الآلية في الطاقة والمال والروبوت – «محاكاة كثيرة متطابقة» – هي ما يجعل JAX سرّ السرعة. الخانق الوحيد هو ذاكرتك في صياغة الحساب نقياً.

جدول الدروس: ما الذي يجمعها حقاً؟

الحالات الستّ تبدو متباعدة – صورة، ومحفظة، وقطعة معدنية، ومنتج، وشبكة، وذراع – لكنها تشترك في الهيكل: «حساب نقي يتكرّر كثيراً». الجدول الآتي يلخّص المحرّك المناسب لكلّ حالة.

الحالة	المحرّك	الخانق	الدرس
ترييج طبّي	`vmap` + `jit`	زمن الاستدلال	افصل البناء عن التطبيق
مخاطر ماليّة	`vmap` على مسارات	ذاكرة المسارات	قسّم على شرائح
كشف عيوب	`shard_map`	جهاز واحد	وزّع ثم وازِن
ترتيب توصية	`vmap` + فرز	آلاف المرشّحين	احسب قبل الفرز

الحالة	المحرّك	الخانق	الدرس
حمل طاقة	`lax.scan`	تسلسل الخطوات	حوّل الحلقة رسماً
تحكّم روبوتي	`vmap` + `scan`	مرشّحات التحكّم	محاكاة كثيرة = سرعة

جدول 31.1 — المحرّك المناسب لكلّ حالة

✦ النموذج الذهني الموحّد

حين تواجه مشكلة إنتاجية، اسأل: «هل أستطيع كتابة الحساب على نسخة واحدة؟» إن نعم، فالباقي آلي: `vmap` يوازٍ، `shard_map` يوزّع، `scan` يلفّ التسلسل.

تمرين الفصل الحادي والثلاثين

1. عدّل كتلة الترييج الطبّي لتطبّق `jax.jit` على `model.apply`، وأثبت أن الاستدلال على دفعة من 32 صورة أسرع من 32 استدعاءً منفصلاً منطقياً.
2. في حالة المخاطر المالية، استبدل `vmap` بتقسيم المسارات إلى شرائح من مئة ألف عبر `jax.lax.map`، وتحقّق أن `var95` لا يتغيّر.
3. في التحكّم الروبوتي، درّب تسلسل التحكّم بـ `jax.grad` على تقليل الكلفة، واكتب خطوة نزول واحدة.

▲ الحلّ النموذجي

1. نلفّ التطبيق بـ `jit` فنجمع نواة واحدة للدفعة. و**2.** `lax.map` يمرّر الشريحة تلو الأخرى فيحفظ الذاكرة دون تغيير التوزيع الإحصائي. و**3.** التدرّج على `acts` يعمل لأن `rollout` رسم نقي قابل للاشتقاق.

Python

```
import jax, jax.numpy as jnp
from flax import linen as nn

class Triage(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.relu(nn.Conv(32, (3, 3))(x))
        x = nn.avg_pool(x, (2, 2))
        x = x.reshape((x.shape[0], -1))
        return nn.Dense(2)(x)

model = Triage()
params = model.init(jax.random.key(0), jnp.ones((1, 64, 64, 1)))["params"]
infer = jax.jit(lambda p, x: model.apply({"params": p}, x))
batch = jnp.ones((32, 64, 64, 1))
print(infer(params, batch).shape)        # (32, 2)

# الحل 2: تقسيم المسارات
def var_of(losses): return float(jnp.percentile(losses, 95))
losses = jax.random.normal(jax.random.key(9), (1_000_000,))
chunked = jax.lax.map(lambda c: var_of(c), losses.reshape((10, 100_000)))
print(chunked.shape)                     # (10,) ثم خذ متوسط الكسور

# الحل 3: تدرّج على التحكّم
def cost_of(acts):
    s = jnp.array([0.0, 0.0]); t = jnp.array([1.0, 0.0])
    def f(s, u):
        s2 = s + jnp.array([s[1], u]) * 0.1
        return s2, jnp.sum((s2 - t) ** 2)
    _, cs = jax.lax.scan(f, s, acts)
    return jnp.sum(cs)
acts = jnp.zeros((10,))
g = jax.grad(cost_of)(acts)
print(g.shape)                           # (10,) تدرّج كل خطوة
```

خلاصة الفصل

- الحالات الستّ تشترك في «حساب نقي يتكرّر»؛ هذا ما يجعل JAX يتفوّق.
- `jit` يجمع الاستدلال، `vmap` يوازٍ النسخ، `shard_map` يوزّع على الأجهزة.
- `lax.scan` يحوّل التسلسل – طاقة أو روبوت – إلى رسم قابل للاشتقاق.
- الخانق الحقيقي ليس المكتبة بل صياغتك للحساب نقياً وصريح الأبعاد.

مصطلحات الفصل

Triage

ترتيب الحالات بحسب الخطورة لفحص الأخطر أوّلاً.

Monte Carlo VaR

تقريب قيمة الخطر بمحاكاة أعداد كبيرة من المسارات العشوائية.

shard_map

توزيع حساب على عدّة أجهزة مع صراحة الأبعاد، بديل `pmap` الحديث.

lax.scan

تحويل الحلقة إلى رسم مجمّع قابل للاشتقاق العكسي.

Model Predictive Control

اختيار أفضل تسلسل تحكّم عبر محاكاة مرشّحات كثيرة.

النموذج الذهني الموحّد

كتابة الحساب على نسخة واحدة ثم ترك `vmap` / `shard_map` يوسّعانه.

الأخلاقيات والخصوصية وخارطة الطريق

العدالة، والخصوصية التفاضلية، والأمن، واثنا عشر شهراً

بنى فريق نموذجاً يقرر من يُمنح قرضاً ومن يُرفض، وكانت دقّته 97٪. ثم سأل محلّل: «ومَن يُرفض أكثر؟» تبيّن أن مجموعةً بعينها تُرفض ضعف الآخرين دون سبب ماليّ. الرقم 97٪ كان صادقاً، والظلم كان حقيقياً. الأدوات التي تعلّمتها – من `jit` إلى `shard_map` كما رأينا في الفصل الحادي والثلاثون – تمنحك قوّة، والقوّة بلا بوصلة تُفسد. في هذا الفصل لا نكتب كوداً للأخلاق، بل نضبط الممارسة: كيف نقيس الانحياز، كيف نحمي بيانات الناس بخصوصية تفاضلية، كيف نُحصّن النموذج، وكيف نضمن أن نتيجتك يكرّرها غيرك. وينتهي بخارطة طريق لاثني عشر شهراً تقودك إلى الفصل الثالث والثلاثون.

من الأداة إلى المسؤولية: لماذا تبدأ هنا؟

المهندس يميل إلى اعتبار الأخلاقيات «قسم آخر» يقرأه المحامون. لكن الانحياز يدخل من باب الدالّة: إن ربطت المخرج بمتغيّر مرتبط فئوياً، فالنموذج سيتعلّمه حتى لو لم تطلب. المسؤولية إذن هندسية قبل أن تكون فلسفية – وهي قرار في كيفية قطع البيانات وقياس المخرجات.

في هذا الفصل نتناول أربعة محاور: العدالة والانحياز، والخصوصية التفاضلية، وأمن النموذج، وإعادة الإنتاجية. ثم نربطها بخارطة طريق عمليّة. القاسم المشترك أن كلّ محور يبدأ بسؤال قابل للقياس، لا بشعار.

> ★ **العدالة قابلة للقياس قبل أن تكون حُكماً**
>
> لا تسأل «هل نموذجي عادل؟» بل «ما الفرق بين معدّل القبول عبر المجموعات؟». السؤال الثاني له رقم؛ والأول لا.

> ؟ نموذج دقّته 97٪ يرفض مجموعةً بنسبة أعلى من أخرى. هل يكفي إبلاغ الدقّة للعميل؟ بماذا تكمّل التقرير لكي يكون صادقاً؟

العدالة والانحياز: كيف نقيس الظلم؟

المشكلة: نموذج يقبل/يُرفض، ونريد أن نعرف إن كان يحابي مجموعةً على حساب أخرى. المقياس البسيط «العدالة الديموغرافية»: هل احتمال القبول متساوٍ عبر المجموعات؟ JAX هنا يسرّع حساب الفروق على ملايين القرارات، لا لأن الحساب صعب بل لأن المراقبة يجب أن تكون رخيصة كي تُمارس.

```python
import jax, jax.numpy as jnp

def demographic_parity(scores, group):       # group: 0 أو 1
    a = scores[group == 0]
    b = scores[group == 1]
    return float(jnp.mean(a > 0.5) - jnp.mean(b > 0.5))

scores = jax.random.bernoulli(jax.random.key(0), 0.5, (1000)).astype(float)
group = jax.random.bernoulli(jax.random.key(1), 0.5, (1000))
gap = demographic_parity(scores, group)
print(gap)                                   # قرب الصفر = توازن
```

> ⚠ **الدقّة تخفي الانحياز**
>
> متوسط الخسارة الكلّي يطمس أداء الأقليّات. إن راقبت المقيّم الكلّي فقط، فلن ترى الظلم حتّى يكتشفه المتضرّر. راقب المجموعات فرادى.

الدرس: القياس وحده لا يصلح النموذج، لكنه يمنع الإنكار. كما تعلّمنا في الفصل السابع عشر، التنظيم يُعيد توجيه ما يتعلّمه النموذج؛ وهنا نضيف قيداً على التوزيع لا على الخسارة وحدها.

الخصوصية التفاضلية وDP-SGD

المشكلة: نموذج مدرب على سجلّات مرضى قد «يتذكّر» مريضاً بعينه، فيكشف خصوصيّته من استجابته. الخصوصية التفاضلية تضمن: «وجود أو غياب عيّنة واحدة لا يغيّر المخرج كثيراً». الآلية عبر **DP-SGD**: قصّ تدرّج كلّ عيّنة عند حدّ C، ثم إضافة ضجيج غاوسي مقيّس بـ σ. JAX صديق لهذا لأن القصّ والتشويش عمليّات موترية نقية.

$$\tilde{g}_i = g_i \cdot \min(1, C / \|g_i\|) \quad , \quad \bar{g} = (1/B)\,\Sigma\,(\tilde{g}_i + N(0, \sigma^2 C^2 I))$$

كلّ عيّنة تُقصّ، ثم يُجمع المتوسّط ويُشوّش دفعةً واحدة.

```python
import jax, jax.numpy as jnp

def clip_per_sample(grads, c):              # قصّ تدرّج كلّ عيّنة
    norm = jax.tree.map(jnp.linalg.norm, grads)
    return jax.tree.map(lambda gi, n: gi * jnp.minimum(1.0, c / n), grads, norm)

def add_noise(grads, key, sigma, c, batch):
    scale = sigma * c / batch
    noise = jax.tree.map(lambda x: jax.random.normal(key, x.shape) * scale, grads)
    return jax.tree.map(lambda gi, ni: gi + ni, grads, noise)

# في خطوة التدريب:
# clipped = jax.vmap(clip_per_sample)(per_sample_grads)
# noisy = add_noise(jax.tree.map(jnp.mean, clipped), k, sigma, C, B)
```

✓ اختر C وσ معاً

الحدّ C الصغير مع σ الكبير يقتل الإشارة؛ والعكس يكشف البيانات. ضبطهما تجريبياً عبر منحنى الخصوصية مقابل الدقّة، لا عبر حدس واحد.

الدرس: الخصوصية التفاضلية ثمنه ضجيج، لكنه ثمن معلوم الحدود. استخدم `jax.tree.map` الحديث (لا `tree_map`) لأن التدرّجات شجرة لا موتر واحد.

أمن النموذج والاستخراج

المشكلة: نموذجك المنشور يُستعلم عنه بأسعار؛ مهاجم يرسل آلاف الاستعلامات ويستنتج أوزانك، أو يكتشف ما إذا كانت عيّنة معيّنة في بيانات التدريب (هجوم العضوية). الحماية ليست سحرية: حدّد معدّل الاستعلام، وشفّر المخرجات، ولا تكشف درجات الثقة الخام.

مهاجم يطلب «هل هذا البريد سبام؟» ألف مرّة بصياغات قريبة. ما الذي يمنعه من إعادة بناء حدّ القرار؟ وكيف يقلّل `jit` سطح الهجوم أصلاً؟

```python
import jax, jax.numpy as jnp

def bounded_score(logits, key, eps=0.05):      # إخفاء الثقة الخام
    soft = jax.nn.softmax(logits)
    noise = jax.random.laplace(key, soft.shape) * eps
    return jnp.clip(soft + noise, 0.0, 1.0)

logits = jnp.array([3.0, -1.0, 0.5])
print(bounded_score(logits, jax.random.key(0)))   # درجات مُشقّرة
```

> **× كشف احتمال الخسارة يكشف العضوية**
>
> ردّك بـ«هذه العيّنة في التدريب بنسبة 99٪» يمنح المهاجم سلاحاً. أعد صياغة المخرج ليكون قراراً لا احتمالاً، أو طبّق تشويشاً كافياً.

الدرس: الأمان طبقة قبل النشر لا بعده. كما رأينا في الفصل الثلاثين، خطّ الإنتاج هو المكان الذي يُفرض فيه التشفير والحدود، لا دفتر التجارب.

إعادة الإنتاجية: البذرة ليست خياراً

المشكلة: زميلك شغّل شيفرتك فأعطت نتيجة مختلفة. التفاضلية وDP-SGD يعتمدان على ضجيج عشوائي، فإن تغيّرت البذرة تغيّرت الخصوصية المعلنة. الحلّ الصريح: `jax.random.key` الحديثة، مع `split` و `fold_in` لتفريخ مفاتيح فرعية قابلة للتكرار.

```python
import jax, jax.numpy as jnp

key = jax.random.key(42)                  # بذرة ثابتة ومعلنة
k1, k2 = jax.random.split(key)
a = jax.random.normal(k1, (3,))
print(a)                                  # نفس القيمة كل تشغيل

# مفاتيح فرعية لكلّ عيّنة، قابلة لإعادة الإنتاج
for i in range(3):
    ki = jax.random.fold_in(key, i)
    print(float(jax.random.uniform(ki)))
```

> ★ **سجّل البذرة كجزء من التجربة**
>
> البذرة واجهة للتجربة لا تفصيلاً. احفظها مع الإصدارات (Python 3.14، jax 0.12.0)، وإلا لن يستطيع أحد إعادة إنتاج ادّعائك بالخصوصية.

الدرس: إعادة الإنتاجية هي أساس كلّ ادّعاء علمي أو قانوني في هذا الفصل. بلا بذرة مسجّلة، لا خصوصية مُعلنة ولا عدالة مُدّعاة.

خارطة طريق اثني عشر شهراً

كيف تتحوّل من قارئ إلى مهندس إنتاج؟ الجدول الآتي يقسّم الرحلة إلى شهور، كلّ شهر مهارة ومرجع للفصل المناسب. هو دليل لا قانون؛ إن أتقنت مهارةً مبكّراً تجاوزتها.

الشهر	المهارة	المرجع (الفصل)
1-2	بيئة JAX والمصفوفات	الأول - الرابع
3	البثّ والفهرسة	الرابع
4	التجميع بـ `jit`	السادس
5	التوازي بـ `vmap`	الحادي عشر
6	التفاضل التلقائي	العاشر
7	نماذج Flax	الرابع عشر
8	المحسّنات والتنظيم	الخامس عشر، السابع عشر
9	التقطيع والموازاة	الحادي والثلاثون
10	دراسات الحالة	الحادي والثلاثون
11	الأخلاقيات والحوكمة	الثاني والثلاثون
12	مشروع الإنتاج	الثلاثون، الثالث والثلاثون

جدول 32.1 — خارطة طريق الاثني عشر شهراً



الشكل 32.1 — المسار يبدأ بالأداة وينتهي بالمسؤولية؛ كلّ شهر يبني على سابقه.

✓ لا تحفظ الأرقام، افهم السبب

الجدول يشير إلى فصول: الفصول تشرح «لماذا». اقرأ الفصل ثم نفّذ مثاله، ولا تكتفِ بنسخ الكتلة دون فهم القيد خلفها.

المسؤولية المهنية: القوس الذي يجمع كلّ هذا

العدالة والخصوصية والأمن والإنتاجية ليست فصلاً في الكتاب، بل عدسة تُقرأ بها كلّ سطر تكتبه. المهندس الذي ينشر نموذجاً دون قياس انحيازه شريكٌ في نتيجته، سواء أراد أم لا.

⚙ واجبك قبل النشر

قبل كلّ نشر: (1) قِس الانحياز عبر المجموعات، (2) أعلن حدود الخصوصية إن ادّعيتها، (3) شفّر المخرج وحدّ الاستعلام، (4) سجّل البذرة والإصدارات. هذه الأربعة ليست نصيحة، بل حدّ أدنى للممارسة المسؤولة.

تمرين الفصل الثاني والثلاثين

1. احسب فجوة العدالة الديموغرافية على مخرجات نموذج خيالي، وأثبت أنها تتضاءل حين تتساوى توزيعات المجموعتين.
2. ابنِ خطوة DP-SGD صغيرة: اقصّ تدرّج عيّنة واحدة عند $C=1$ ثم أضف ضجيجاً بـ `jax.random.normal`، وتحقّق أن التدرّج المقصوص له ن norm لا يتجاوز C.
3. اكتب دالّة تُعيد المفاتيح الفرعية عبر `fold_in` لعشر عيّنات، وأثبت أن نفس الفهرس يعطي نفس القيمة دائماً.

الحلّ النموذجي

1. الفجوة فرق بين متوسطين؛ تتلاشى مع تساوي التوزيعات. و**2.** القصّ يضمن $\|\tilde{g}\| \le C$ حتماً. و**3.** `fold_in(key, i)` حتمي فيعيد نفس القيمة.

```python
import jax, jax.numpy as jnp

# الحل 1: فجوة العدالة
s1 = jax.random.bernoulli(jax.random.key(0), 0.5, (500)).astype(float)
s2 = jax.random.bernoulli(jax.random.key(1), 0.5, (500)).astype(float)
gap_equal = float(jnp.mean(s1 > 0.5) - jnp.mean(s2 > 0.5))
print(gap_equal)                          # قريب من الصفر

# الحل 2: قصّ عند C ثم ضجيج
C = 1.0
g = jnp.array([3.0, 4.0])
gn = g * jnp.minimum(1.0, C / jnp.linalg.norm(g))
noisy = gn + jax.random.normal(jax.random.key(5), g.shape) * 0.1
print(float(jnp.linalg.norm(gn)), float(jnp.linalg.norm(noisy)))

# الحل 3: مفاتيح حتمية
key = jax.random.key(7)
vals = [float(jax.random.uniform(jax.random.fold_in(key, i))) for i in range(10)]
print(vals[0] == vals[0])                 # حتمي عبر التشغيلات
```

خلاصة الفصل

- العدالة تُقاس بفرق المعدّلات عبر المجموعات، لا بالدقّة الكلّية وحدها.
- DP-SGD يجمع القصّ والتشويش عبر `jax.tree.map` لحماية الخصوصية.
- الأمن طبقة قبل النشر: تشفير المخرج وتحديد الاستعلام.
- إعادة الإنتاجية تعتمد على `key` / `split` / `fold_in` المسجّلة.
- خارطة الأشهر الاثني عشر تبني مهارةً كلّ شهر نحو الإنتاج.

مصطلحات الفصل

العدالة الديموغرافية

تساوي احتمال القبول عبر المجموعات السكانية.

الخصوصية التفاضلية

ضمان ألّا يغيّر غياب عيّنة واحدة مخرج النموذج بشكل ملموس.

DP-SGD

نزول متدرّج بتدرّجات مقصوصة ومُشفّاة لخصوصية تفاضلية.

هجوم العضوية

استنتاج ما إذا كانت عيّنة في بيانات التدريب من استجابة النموذج.

إعادة الإنتاجية

قدرة الغير على تكرار النتيجة عبر بذرة وإصدارات مسجّلة.

المسؤولية المهنية

الحدّ الأدنى من القياس والأمان قبل نشر أيّ نموذج.

الملحق A

خريطة المقابلة بين PyTorchg JAX

كل مفهوم بمقابله، مع الفروق الجوهرية

تنتقل من PyTorch إلى JAX لا كما تنتقل بين إصدارين من المكتبة نفسها، بل كما تنتقل بين عقلين: عقلٌ يحفظ الحالة في كائنات، وعقلٌ يعامل كل شيء دالّة نقية فوق مصفوفات ثابتة. إن كنتَ تكتب اليوم `model(x)` وتتوقّع أن المعاملات تحدّثت بنفسها، فستحتاج إلى إعادة ضبط النموذج الذهني قبل السطر الأول. هذا الملحق ليس درساً؛ هو خريطة: كل مفهوم اعتدته في PyTorch تجدُ مقابله في JAX، مع الفروق الجوهرية التي تغيّر طريقة التفكير لا فقط طريقة الكتابة.

★ أكبر تحوّل ذهني: الوظيفية والتتبّع

في PyTorch تملك الطبقة حالتها، وتعدّلها `backward` صراحةً. في JAX لا تملك أي دالّة حالة: المعاملات والتدرّجات والحالة كلها تُمرّر وتُعاد. ثم إن `jit` لا «يترجم» الشيفرة كما `torch.compile`؛ بل يُرسِمها (tracing): يمرّ على المسار الوحيد المنفّذ ويبني رسم HLO، فأي تفرّع يعتمد على قيمة مصفوفة يُجمَّد عند التتبّع. افهم هذين الأمرين، وكل الفروق أدناه تتبعهما.

النماذج

الصنف `nn.Module` في PyTorch يحمل المعاملات ويديرها. في JAX لا يوجد «معامل» سحري: المعاملات مجرّد مصفوفات `jax.Array` داخل شجرة `pytree`، والنموذج دالّة تنظّمها. وللنماذج البنيوية استخدم `flax.linen.Module` بنمط `@nn.compact`، أو صنف بيانات (dataclass) في `equinox` إن فضّلت نمط الكائنات الوظيفية.

```python
import jax, jax.numpy as jnp
from flax import linen as nn

class MLP(nn.Module):                # نمط Linen
    @nn.compact
    def __call__(self, x):
        x = nn.Dense(128)(x)         # flax.linen.Dense ≈ nn.Linear
        return nn.Dense(10)(jax.nn.relu(x))

params = MLP().init(jax.random.key(0), jnp.ones((1, 4)))
out = MLP().apply(params, jnp.ones((1, 4)))
```

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`nn.Module`	`flax.linen.Module` (أو dataclass في Equinox)	المعاملات ليست أعضاءً مخفيين؛ بل أوراق في pytree تُمرّر صراحةً.
`torch.Tensor`	`jax.Array`	نوع موحّد (لا `DeviceArray`)؛ ثابت لا يُعدّل في المكان.
`tensor.to(device)`	`jax.device_put` / `NamedSharding`	لا أجهزة ضمنية؛ التوزيع صريح عبر sharding.
`nn.Parameter`	ورقة عادية داخل pytree	أي مصفوفة في الشجرة هي «معامل»؛ لا حاجة لتغليف خاص.
`torch.nn.Linear`	`flax.linen.Dense`	نفس الضرب `x·Wᵀ + b`، لكن الوزن جزء من شجرة المعاملات.
`torch.nn.Conv2d`	`flax.linen.Conv`	التنسيق `channels_last` افتراضي أحياناً؛ راجِع `flax`.
`nn.BatchNorm`	`flax.linen.BatchNorm`	الإحصاءات الجارية في «حالة» منفصلة عن المعاملات.
`nn.Dropout`	`flax.linen.Dropout`	يتطلّب مفتاح `PRNG` يُمرّر صراحةً لا عالمياً.
`nn.CrossEntropyLoss`	`optax.softmax_cross_entropy`	تُعطى اللوغاريتمات والوسوم، وتُعاد خسارة لكل عيّنة.

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`forward()`	`__call__` مع `@nn.compact`	الطبقات تُبنى وتُستدعى في آنٍ داخل `compact`.
`register_buffer`	جزء من pytree	الثوابت المرفقة تُحفظ في الشجرة ولا تمييز خاص لها.
`state_dict()`	pytree من المصفوفات	المعاملات والحالة أشجار عادية تُمرّر وتُحفظ مباشرةً.

جدول الملحق أ.1 — مقابلة مكوّنات النموذج

المحسّنات

المحسّن في PyTorch كائن يملك المعاملات ويكتب فوقها بـ `step()`. في JAX المحسّن `optax.GradientTransformation`: زوج دوالّ `init` و`update` لا يملكان شيئاً. التدرّج يُحسب بـ `jax.grad` أو `jax.value_and_grad`، والتحديث يُطبَّق بـ `optax.apply_updates`.

```python
import optax
loss_fn = lambda p, x, y: optax.softmax_cross_entropy(...).mean()
grads = jax.grad(loss_fn)(params, x, y)        # لا backward()
tx = optax.adamw(learning_rate=3e-4, weight_decay=0.1)
opt_state = tx.init(params)
updates, opt_state = tx.update(grads, opt_state, params)
params = optax.apply_updates(params, updates)  # التحديث صريح
```

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`torch.optim.Optimizer`	`optax.GradientTransformation`	دالّة لا كائن؛ الحالة (اللحظات والعدّاد) وسيط يُمرّر.
`loss.backward()` + `step()`	`jax.grad` + `optax.apply_updates`	الاشتقاق منفصل عن التطبيق؛ كلاهما صريح.
`requires_grad=True`	`jax.grad` / `jax.value_and_grad`	كل شيء قابل للاشتقاق دائماً؛ لا راية تفعيل.
`@torch.no_grad()`	لا حالة؛ فقط لا تستدعِ `grad`	غياب الاستدعاء كافٍ، فلا ديكوراتور للحصر.

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
AMP `autocast`	`jax.default_matmul_precision` / `bf16`	سياسة الدقّة صريحة على مستوى المصفوفة لا سحر.
تهيئة المعاملات	`jax.random` / `flax.init`	تُبنى الشجرة من الأشكال عبر مفتاح PRNG.

جدول الملحق أ.2 — مقابلة التدريب والمحسّنات

البيانات

في PyTorch يدير `DataLoader` الدفعات والخيوط. في JAX لا توجد حلقة بيانات مدمجة: تكتب مولّد بايثون عادياً، وتجمّع الدفعة بتمرير الشجرة عبر `jax.tree.map`. الحفظ يكون بـ `orbax.checkpoint` لأشجار كبيرة، أو `pickle` مباشرةً للأشجار الصغيرة.

```python
def make_batches(xs, ys, bs):
    for i in range(0, len(xs), bs):
        yield jax.tree.map(lambda a: a[i:i+bs], (xs, ys))

for (xb, yb) in make_batches(x, y, 64):
    train_step(params, xb, yb)
```

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`torch.utils.data.DataLoader`	مولّد بايثون + تجميع `jax.tree.map`	لا خيوط مدمجة: تتحكّم أنت بالتزامن خارجيّاً.
`nn.functional.*`	`jax.scipy` / `jax.nn.*`	دوالّ نقية بلا حالة: نفس الواجهة تقريباً.
`torch.save` / `load`	`orbax.checkpoint` / `pickle`	الحفظ على مستوى pytree لا state_dict.

جدول الملحق أ.3 — مقابلة البيانات والحفظ

التوزيع

`DistributedDataParallel` يشقّ النموذج تلقائياً. في JAX التوزيع صريح: `jax.jit` مع `NamedSharding` على `Mesh`، أو `shard_map` للتحكّم الدقيق. و`pmap` مهمل عملياً؛ استخدم بدله التقطيع الحديث. و`torch.compile` يقابله `jax.jit`، لكنه تتبّع كامل لا ترجمة جزئية.

```python
from jax.sharding import Mesh, NamedSharding
from jax.sharding import PartitionSpec as P
mesh = Mesh(jax.devices(), ("x",))
shard = NamedSharding(mesh, P("x", None))
params = jax.device_put(params, shard)        # تقطيع صريح
```

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`DistributedDataParallel`	`jax.jit` + `NamedSharding` / `shard_map`	`pmap` قديم؛ التقطيع الحديث عبر Mesh.
`torch.compile`	`jax.jit`	تتبّع كامل يبني HLO، لا ترجمة تدريجية.
hooks (المساند)	لا hooks (نموذج وظيفي)	بدّل التصرّف داخل الدالّة، لا عبر خطّافات.
`torch.cuda`	`jax.devices()` / الواجهات الخلفية	لا أجهزة ضمنية؛ الجهاز يُذكر صراحةً عند اللزوم.

جدول الملحق أ.4 — مقابلة التوزيع والتجميع

نقاط الحذر

⚠ حيث لا تكون JAX بديلاً مباشراً

لا تتوقّع أن يشتغل كود PyTorch بمجرّد تغيير الاستيراد. الغياب الملحوظ: لا `in-place` (كل مصفوفة ثابتة)، لا `backward` ضمني، لا `requires_grad`، ولا خطّافات. أي منطق يعتمد على «تعديل في المكان» أو أثر جانبي يُعاد صياغته دالّةً نقية. والتفرّع داخل دالّة مجمّعة يجب أن يعتمد على قيم ثابتة لا على مصفوفات، وإلا تجمّد عند التتبّع.

المفهوم (PyTorch)	المقابل في JAX	ملاحظة/فرق جوهري
`tensor.detach()`	لا حاجة (لا رسم متصل)	التدرّج لا ينتقل عبر ما لم تطلبه صراحةً.
`requires_grad_()`	لا يوجد	كل شيء قابل للاشتقاق؛ لا راية تبديل.
`model.eval()` / `train()`	وسيط صريح أو وضع في الدالّة	لا حالة عمومية للنموذج؛ تمرّر الحالة بنفسك.
العشوائية العامة	`jax.random.key` / `split`	لا مولّد عام؛ المفتاح يُمرّر ويُقسَم صراحةً.

جدول الملحق أ.5 — فروق دقيقة تتسبّب بأخطاء صامتة

✓ خريطة طريق الانتقال

ابدأ بإعادة كتابة النموذج كدالّة `params → x → y`. ثم استبدل `backward/step` بـ `jax.grad` و `apply_updates`. أخيراً لفّ حلقة التدريب بـ `jax.jit` وحرّك العشوائية إلى مفاتيح صريحة. كل خطوة تكشف فرقاً واحداً فقط، فلا تخلطها.

الملحق B

مسرد المصطلحات

عربي ↔ إنجليزي، مع تعريف موجز

كل علم له لغته، وJAX لا تخلو من مصطلحات تعترضك في الوثائق والرسوم البيانية والأخطاء على حدّ سواء. هذا المسرد يجمع نحو خمسين مصطلحاً تمتدّ من البنية التحتية (XLA، HLO، التتبّع) إلى التعليم العميق (الزخم، الانتباه، التعميم) مروراً بالمكتبات الرفيقة (Flax، Optax، Equinox). رتّبناها عربيّاً مقابل الإنجليزية لتكون مرجعاً سريعاً كلّما عثرت على كلمة غريبة في منتصف تدريب نموذجك.

◆ كيف تقرأ هذا المسرد

العمود الأول المصطلح العربي كما ورد في الكتاب، والثاني الإنجليزية كما تكتبها في الشيفرة (حسّاسة لحالة الأحرف: `jit` لا `JIT`). الثالث تعريف موجز في جملة واحدة. المصطلحات التي تبدأ بحرف كبير (مثل `Flax`) أسماء مكتبات؛ وما سواها أدوات أو مفاهيم. راجع الفهرس إن أردت موضع الشرح المفصّل.

المصطلح (عربي)	الإنجليزية	تعريف موجز
التبديل	`transform`	تطبيق محوّل JAX (مثل `jit` أو `grad`) فوق دالّة لإنتاج دالّة جديدة.
التبديل الفوري	`jit`	تجميع دالّة عبر تتبّعها وبناء رسم XLA لتسريع تنفيذها.
الاشتقاق	`grad`	حساب التدرّج الآلي للدالّة عند نقطة دخل محدّدة.
الموازاة المتجهية	`vmap`	تطبيق الدالّة على محور دفعة كامل بلا حلقات صريحة.
الموازاة العابرة للأجهزة	`pmap`	موازاة عبر عدة أجهزة (مهمل عملياً)؛ استبدله بـ `NamedSharding`.
خريطة التقطيع	`shard_map`	تقسيم الحساب على أجهزة مع وصول محلي لكل شظية.

المصطلح (عربي)	الإنجليزية	تعريف موجز
الشجرة المصفوفية	`pytree`	حاوية متداخلة (dict/list/tuple) تحوي مصفوفات عند أوراقها.
المترجم الخطّي الجبري	`XLA`	الواجهة الخلفية التي تترجم رسوم JAX إلى تعليمات أجهزة.
لغة العمليات عالية المستوى	`HLO`	التمثيل الوسيط للرسم قبل توليد النواة النهائية.
التجميع	`fusion`	دمج عمليات متتابعة في نواة واحدة لتقليل نقل البيانات.
التتبّع	`tracing`	تنفيذ الدالّة بمصفوفات مجرّدة لالتقاط عملياتها في رسم.
القيمة المجرّدة	`abstract value`	شكل ونوع مصفوفة بلا بيانات فعلية أثناء التتبّع.
المصفوفة	`jax.Array`	نوع المصفوفة الموحّد في JAX، ثابت وقابل للتقطيع عبر الأجهزة.
المحسّن	`optimizer`	`GradientTransformation` يحوّل التدرّج إلى تحديث للمعاملات.
الزخم	`momentum`	متوسط متحرّك للتدرّج يخمّد الاهتزاز ويسرّع التقارب.
التسوية	`regularization`	قيود على المعاملات (مثل الاضمحلال) لمنع التخصيص المفرط.
التعميم	`generalization`	قدرة النموذج على الأداء الجيّد على بيانات لم يرها.
الانتباه	`attention`	آلية توزّن أهمية عناصر المدخلات بعضها بالنسبة لبعض.
المحوّل	`transformer`	بنية تعتمد على الانتباه بدل الشبكات المتكررة أو الالتفاف.
الانتشار	`diffusion`	نموذج توليدي يزيل ضجيجاً تدريجياً من عيّنة عشوائية.
التكميم	`quantization`	تمثيل الأوزان بدقّة منخفضة (مثل int8) لتقليل الحجم والزمن.
التقطيع	`sharding`	توزيع مصفوفة على عدة أجهزة عبر `Mesh` و `NamedSharding`.
الشبكة المنطقية	`Mesh`	ترتيب الأجهزة في محاور مسمّاة لتوزيع العمل عليها.

المصطلح (عربي)	الإنجليزية	تعريف موجز
التقطيع المسمّى	`NamedSharding`	ربط محور في `Mesh` ببُعد من أبعاد المصفوفة.
إعادة الحساب	`remat`	إعادة حساب وسيط بدل حفظه لتقليل ذاكرة التدريب.
تسليم الملكية	`donation`	منح مخرجات الدالّة ملكية مدخلاتها لتقليل نسخ الذاكرة.
المولّد العشوائي	`PRNG`	مولّد أرقام عشوائية يعتمد في JAX على مفاتيح صريحة لا حالة عامة.
المفتاح العشوائي	`key`	قيمة تبدأ سلسلة عشوائية قابلة للقسمة عبر `split` .
البثّ	`broadcasting`	توسيع بُعد صغير لمطابقة كبير في عمليات العناصر دون نسخ.
التطبيع	`normalization`	إعادة توزيع القيم (Batch/Layer Norm) لتثبيت تدريب الشبكة.
التعطيل العشوائي	`Dropout`	إبطال وحدات عشوائياً أثناء التدريب للحدّ من التخصيص.
تباعد كولباك-لايبler	`KL divergence`	مقياس بُعد بين توزيعين احتماليين.
التوزيع المقطوع	`GSPMD`	تقطيع موحّد للمصفوفات عبر الأجهزة يُطبَّق تلقائياً.
بالّاس	`Pallas`	واجهة كتابة نواة مخصّصة على GPU/TPU ضمن JAX.
دقّات منخفضة	`bf16/fp8`	صيغ عشرية مختصرة لتسريع التدريب وتقليل استهلاك الذاكرة.
نقطة الحفظ	`checkpoint`	لقطة من حالة التدريب تُستأنف منها لاحقاً بلا خسارة.
الحتمية	`determinism`	تطابق المخرجات عبر تشغيلات متطابقة (يتطلّب مفاتيح ثابتة).
أورباكس	`Orbax`	مكتبة لحفظ وتحميل أشجار JAX بكفاءة عالية.
فلاكس	`Flax`	مكتبة نماذج عصبية فوق JAX بوضعَي Linen وNNX.
أوبتاكس	`Optax`	مكتبة محسّنات وتحويلات تدرّج مصمّمة لـ JAX.
إن إن إكس	`NNX`	وضع كائني في Flax يجمع بين بنية النموذج وحالته.

المصطلح (عربي)	الإنجليزية	تعريف موجز
لاينن	`Linen`	وضع وظيفي في Flax يعتمد على الديكور `@nn.compact` .
إكواتورنكس	`Equinox`	مكتبة نماذج JAX بأسلوب pytree ومصفوفات محوّلة نقية.
جاكس-تايبينغ	`jaxtyping`	فحص أنواع وأشكال المصفوفات التوضيحي وقت التشغيل.
التدرّج	`gradient`	مشتقّة دالّة الخسارة بالنسبة لكل معامل.
الاشتقاق التلقائي	`autodiff`	حساب المشتقّات بدقّة عبر تمريرات على الرسم البياني.
الموازاة	`parallelism`	توزيع الحساب على عدة محاور أو أجهزة لتسريعه.
الدقّة المختلطة	`mixed precision`	خلط دقّات (مثل fp32 مع bf16) للسرعة مع حفظ الاستقرار.
المتتبّع	`tracer`	كائن يحلّ محل المصفوفة أثناء التتبّع لالتقاط العمليات.
الرسم البياني	`graph`	تمثيل عمليات الحساب (HLO) قبل تنفيذها فعلياً.

جدول الملحق ب.1 — مسرد مصطلحات JAX والتعلّم العميق

✓ مصطلحات تستحق وقفة

إن التبست عليك ثلاثة مفاهيم فابدأ بـ `tracing` و `pytree` و `transform` : فالتتبّع يفسّر لماذا تختلف أخطاء JAX عن PyTorch، والشجرة تفسّر كيف تُمرّر المعاملات، والمحوّلات هي ما يجعل `jit` و `vmap` و `grad` تتركّب. البقية فروع من هذه الثلاثة.

الملحق C

بنك المعادلات والخلاصات التشغيلية

كل معادلة مهمّة في مكان واحد

عشرات المرّات تمرّ أمامك معادلة وتفهمها لحظياً ثم تنسى موضعها حين تحتاجها في منتصف تدريب محوّل. هذا الملحق ليس فصلاً سردياً: هو بنكٌ مرجعيّ تضع فيه كل معادلة مهمّة في الكتاب في مكان واحد، مرتّبةً حسب موضوعها، مع شرح عربي واحد يكفي لتذكيرك بالسياق. لا تقرأه من البداية إلى النهاية؛ افتحه كفهرس كلّما احتجت إلى تذكّر قاعدة دقّة الضرب أو صيغة الانتباه. واجهات JAX التي تُشغّل هذه المعادلات مذكورة في الملحق D، ومصادرها الأصليّة في الملحق E.

التفاضل التلقائي

قاعدة النزول: تعديل المعاملات عكس اتجاه التدرّج. في JAX لا تكتب المشتقّة بنفسك بل تطلبها من `jax.grad`، لكن الصيغة التي يُنتجها المحسّن هي التالية.

$$\theta_{t+1} \leftarrow \theta_t - \eta \cdot \nabla_\theta \, L(\theta_t)$$

تحديث النزول الأبسط: η معدّل التعلّم، و∇L تدرّج الخسارة بالنسبة إلى المعاملات.

$$\nabla_x \, f = (\partial f/\partial x_1, \partial f/\partial x_2, \ldots, \partial f/\partial x_n) \in \mathbb{R}^n$$

التدرّج شعاع من المشتقّات الجزئية، يشير إلى أسرع ارتفاع للدالّة.

$$\partial z/\partial x = \partial z/\partial y \cdot \partial y/\partial x \quad \text{(سلسلة في بعد واحد)}$$

قاعدة السلسلة: هي ما يطبّقه التمرير العكسي على طول الرسم البياني تلقائياً.

$$\partial L/\partial x_i = \sum_j (\partial L/\partial y_j) \cdot (\partial y_j/\partial x_i)$$

صيغة التمرير العكسي العامّة: مساهمة كل عقدة تجمع تدفّق التدرّج من أبنائها.

$$f'(x) \approx (f(x+h) - f(x-h)) \,/\, (2h) \;,\; h \approx \sqrt{\varepsilon_{mach}}$$

التفاضل العددي للتحقّق من صحّة تدرّج مكتوب يدوياً؛ يُطابقه `jax.grad` عادةً حتى 1e-4.

$$\nabla_\theta \left[\sum_i f_i(\theta) \right] = \sum_i \nabla_\theta f_i(\theta)$$

خطيّة التدرّج: سبب أن `vmap(jax.grad)` يحسب تدرّج دفعة كاملة دفعةً واحدة.

التحسين

المحسّنات سلاسل من التحويلات على التدرّج. آدم هو الافتراضي في معظم تدريبات المحوّلات، وصيغته كاملةً هنا.

$$m_t = \beta_1 \, m_{t-1} + (1 - \beta_1) \, g_t$$

اللحظة الأولى (متوسّط متحرّك للتدرّج) – تقدير الاتجاه. $\beta_1 = 0.9$ نموذجياً.

$$v_t = \beta_2 \, v_{t-1} + (1 - \beta_2) \, g_t \odot g_t$$

اللحظة الثانية (متوسّط متحرّك لتربيع التدرّج) – تقدير تذبذب الحجم. $\beta_2 = 0.999$.

$$\hat{m}_t = m_t \,/\, (1 - \beta_1^{\,t}) \;,\; \hat{v}_t = v_t \,/\, (1 - \beta_2^{\,t})$$

تصحيح الانحياز: في الخطوات الأولى تكون اللحظات قريبة من الصفر فيُضخّمان.

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t \,/\, (\sqrt{\hat{v}_t} + \varepsilon) \;-\; \eta \, \lambda \, \theta_t$$

AdamW: التحديث عبر آدم مضروباً في معدّل التعلّم، ثم اضمحلال الوزن المُفصول (آخر حدّ) مطروح من الوزن نفسه لا من التدرّج.

$$g \leftarrow g \cdot \min(\,1\,,\, c \,/\, \|g\|_2\,) \;,\; \|g\|_2 = \sqrt{(\,\sum_i g_i^{\,2}\,)}$$

قصّ التدرّج بطول معياريّ عالميّ c؛ يحفظ الاتجاه ويمنع الانفجار. يوضع أوّل سلسلة المحسّن.

$$u_t = \mu\, u_{t-1} + g_t \;,\; \theta_{t+1} = \theta_t - \eta\, u_t$$

SGD بالزخم: μ = 0.9 يعطي خطوةً متوسّط نحو عشر خطوات سابقة، فتخمُد الاهتزازات.

$$\eta_t = \eta_0 \cdot \tfrac{1}{2}\,(\,1 + \cos(\,\pi \cdot \min(t, T)\,/\,T\,)\,)$$

جدولة التضاؤل الجيبيّ cosine: تنخفض من الذروة إلى الصفر ناعمةً بلا خطوات.

الانتروبيا والخسائر

خسائر التصنيف والقياس تُبنى على الانتروبيا. softmax يحوّل درجاتٍ حرّة إلى توزيع احتماليّ، وcross-entropyg يقيس البعد عن الهدف.

$$\mathrm{softmax}(z)_i = \exp(z_i)\,/\,\textstyle\sum_j \exp(z_j) \;\in (0,\,1)\;,\; \sum_i = 1$$

توزيع احتماليّ من درجات logits؛ تُحسب عملياً بطرح القيمة العظمى لثبات رقميّ.

$$H(p,\,q) = -\textstyle\sum_i p_i \log q_i$$

الانتروبيا التقاطعية: p توزيع الهدف (غالباً واحدي الترميز)، وq الناتج. تصل الصفر حين يتطابقان.

$$L_{CE} = -\log\,(\,\exp(z_y)\,/\,\textstyle\sum_j \exp(z_j)\,)$$

cross-entropy مع softmax مدمجاً (logits)؛ هذا ما يفعله `optax.softmax_cross_entropy_with_logits` مع تثبيت رقميّ.

$$\mathrm{KL}(p\,\|\,q) = \textstyle\sum_i p_i \log(\,p_i\,/\,q_i\,) = H(p,\,q) - H(p)$$

تباعد كلّينغ-لايبليز: قياسٌ غير متماثل لبعد توزيع عن آخر؛ يُستخدم في تنظيم VAE والتقطير.

$$\mathrm{MSE}(\hat{y},\,y) = (1/n)\,\textstyle\sum_{i=1}^{n}\,(\,\hat{y}_i - y_i\,)^2$$

متوسّط مربّع الخطأ للانحدار؛ تدرّجه 2(ŷ − y)/n.

$$L_1(\hat{y}, y) = (1/n)\ \sum_i |\ \hat{y}_i - y_i\ |$$

خسارة القيمة المطلقة؛ أقلّ حساسية للشواذّ من MSE.

الانتباه والمحوّلات

المحوّل قائم على الانتباه بالجداء النقطيّ المُقيّد. هذه الصيغ هي جوهر الفصل المعنيّ بالمحوّلات، وهي ما تُشغّله `jax.nn.dot_product_attention` .

$$\text{Attention}(Q, K, V) = \text{softmax}(\ Q\ K^{\top}\ /\ \sqrt{d_k}\ )\ V$$

الانتباه بالجداء النقطيّ المُقيّد: القسمة على $\sqrt{d_k}$ تمنع انهيار softmax عند أبعاد كبيرة.

$$Q = X\ W^{Q}\ ,\ K = X\ W^{K}\ ,\ V = X\ W^{V}\ \ ,\ \ W \in \mathbb{R}^{d \times d_k}$$

إسقاطات الاستعلام والسرّ والمقدار من المدخل X المشترك.

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\ W^{O}$$

رؤوس متوازية كلّها بأبعاد $d_k = d/h$، تُدمج عبر إسقاط نهائي W^{O}.

$$\text{head}_i = \text{Attention}(\ X\ W_i^{Q}, X\ W_i^{K}, X\ W_i^{V}\ )$$

كل رأس يتعلّم نمط علاقة مختلفاً؛ دمجها يوسّع سعة التمثيل.

$$PE_{(pos,\ 2i)} = \sin(\ pos\ /\ 10000^{2i/d}\ )\ ,\ PE_{(pos,\ 2i+1)} = \cos(\ pos\ /\ 10000^{2i/d}\ )$$

ترميز الموضع الجيبيّ: يضيف معلومة الترتيب إلى المدخلات بلا معاملات قابلة للتعلّم.

$$\text{RoPE}(x, m) : (x_i, x_{i+1}) \mapsto R(\theta_i\ m)\ (x_i, x_{i+1})^{\top}$$

الترميز الموضعيّ الدوّار: يُطبّق دوراناً على أزواج الإحداثيات حسب موضع الرمز m.

التسوية والتطبيع

التطبيع يثبّت توزيع التنشيطات فيبقى التدرّج مستقرّاً عبر الطبقات؛ والتسوية تمنع التخصيص المفرط.

$$\hat{y} = \gamma \cdot (x - \mu) \,/\, \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

تطبيع الطبقة (LayerNorm): μ وσ^2 على بُعد الميزات لكل عيّنة؛ γ وβ معاملات قابلة للتعلّم.

$$\mu = (1/H) \sum_h x_h \,,\, \sigma^2 = (1/H) \sum_h (x_h - \mu)^2$$

الوسط والتباين على بُعد الميزات H داخل العيّنة الواحدة في LayerNorm.

$$\hat{x} = (x - \mu_B) \,/\, \sqrt{({\sigma_B}^2 + \varepsilon)} \;\;,\;\; \mu_B = (1/m) \sum x_i$$

تطبيع الدفعة (BatchNorm): الإحصاء على بُعد الدفعة m؛ في الاختبار تُستخدم إحصاءات متحركة.

$$\hat{x}_i = (x_i - \mu_i) \,/\, \sqrt{({\sigma_i}^2 + \varepsilon)} \;\;,\;\; \mu_i = (1/HW) \sum x_{i,h,w}$$

تطبيع المثال (InstanceNorm): تطبيع كل قناة لكل صورة على أبعاد الفضاء فقط.

$$\mathbb{E}[\ \mathrm{Dropout}(x;\, p)\ ] = (1 - p)\, x \;\;,\;\; \{0,1\} \ni \text{قناع}^{n} \text{ باحتمال } p$$

توقع قناع التسرب: الضرب في (1−p) في وضع التقييم يعوّض إضعاف التدريب. القناع يُولّد بـ `jax.random.bernoulli` .

$$L_{\mathrm{reg}} = L_{\mathrm{data}} + (\lambda/2)\, \|\theta\|_2^{\,2}$$

تسوية L2 (ريدج): تُفصل في AdamW فتُطرح من الوزن لا من تدرّج الخسارة.

التوليد والانتشار

نماذج الانتشار تُعلّم عكس عملية تضيف ضجيجاً. هذه صيغ DDPM التي تُطرح في فصل التوليد، مع التفريق بين الأمام والعكسي.

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t ; \sqrt{(\bar{\alpha}_t)}\, x_0 , (1 - \bar{\alpha}_t)\, I\right)$$

انتشار أمامي مغلق الصيغة: نموذج x_t مباشرةً من x_0 بلا خطوات وسيطة. $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

$$x_t = \sqrt{(\bar{\alpha}_t)}\, x_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon \;,\; \varepsilon \sim \mathcal{N}(0, I)$$

إعادة كتابة الأمام كجمع خطّي لـ x_0 وضجيج قياسيّ: أساس أخذ العيّنات الفعّال.

$$L_{\text{simple}} = \mathbb{E}_{x0,\varepsilon,t}\left[\| \varepsilon - \varepsilon_\theta(x_t, t) \|^2 \right]$$

خسارة DDPM المبسّطة: الشبكة ε_θ تتعلّم توقّع الضجيج المضاف. تدرّجها يُنتجه `jax.grad` .

$$x_{t-1} = \mathcal{N}\left(x_{t-1} ; \mu_\theta(x_t, t) , \Sigma_\theta(x_t, t)\right)$$

العكس المعلّم: الشبكة تُقدّر متوسّط التوزيع العكسي لخطوة واحدة من الضجيج إلى البيانات.

$$\mu_\theta(x_t, t) = (1/\sqrt{\alpha_t})\, \left(x_t - (\beta_t/\sqrt{(1-\bar{\alpha}_t)})\, \varepsilon_\theta(x_t, t) \right)$$

المتوسّط العكسيّ مع predicted-ε: يستخدم توقّع الضجيج لإزالته خطوةً فخطوة عند التوليد.

$$\log p(x) \geq \mathbb{E}_q[\, \log p_\theta(x_0|x_1) - \mathrm{KL}(\, q(x_1|x_0) \,\|\, p(x_1)\,)\,]$$

حدّ ELBO لتوليد المتغيّرات: أساس نظريّ يُبسّط إلى L_{simple} أعلاه.

التقطيع والتوزيع

عند تشغيل على أجهزة متعدّدة أو أنواع منخفضة الدقّة، صيغ الأشكال ودقّة الضرب تحكم النجاح. هذه القواعد تُطبّقها `jax.jit` مع `NamedSharding` .

$$O = \lfloor\, (W - K + 2P) \,/\, S \,\rfloor + 1$$

حجم مخرج التفاف 1D: W عرض الدخل، K حجم النواة، P حشو، S خطوة. تناظرها في 2D لكل بُعد.

$$\text{flops}(M,K,N) = 2 \cdot M \cdot K \cdot N \ , \ C = A_{M\times K}\, B_{K\times N}, C \in \mathbb{R}^{M\times N}$$

تكلفة ضرب المصفوفات؛ يختار XLA دقّة bfloat16 أو float32 حسب أنواع المدخلات والجهاز.

$$\text{prec}(A, B) = \{\ \text{bf16} \otimes \text{bf16} \to \text{bf16} \ \mid\ \text{f32} \otimes \text{f32} \to \text{f32} \ \mid\ \text{mixed} \to \text{f32}\ \}$$

قاعدة دقّة الضرب: النتيجة تتبع أدقّ مدخل، والاختلاط يُرفع إلى float32 عبر `jax.lax.dot_general`.

$$\text{shardings: } x \in \mathbb{R}^{B\times D} \to x.\text{device_put}(\text{axis}=0, \text{devices}=P)$$

تقطيع المدخل على البُعد 0 عبر الأجهزة P، فيبقى ضرب A B موزّعاً دون تجميع كامل.

★ JAX يحسب هذه كلّها — لا تكتب المشتقّة

كل معادلة تفاضليّة أعلاه تُنتجها `jax.grad` / `jax.jacrev` / `jax.vjp` عبر التمرير العكسي التلقائيّ، لا باشتقاق يدويّ. البنك هنا للفهم والتحقّق، ولأن المحسّن وXLA يحتاجان أن تفهم ما يحدث تحت الغطاء لا أن تعيد تنفيذه.

مصطلحات الملحق

التدرّج $L\nabla$

شعاع المشتقّات الجزئية للخسارة بالنسبة إلى كل معامل.

تصحيح الانحياز

قسمة لحظات آدم على $1 - \beta^t$ لإزالة ضعف البداية.

$\sqrt{d_k}$

مقياس تطبيع جداء الانتباه لمنع انهيار softmax.

L_{simple}

خسارة DDPM التي تتعلّم توقّع الضجيج المضاف.

قاعدة الدقّة

نتيجة الضرب تتبع أدقّ مدخل من حيث نوع البيانات.

الملحق D

التمارين الإضافية والحلول

تمارين من كل فصل مع حلول كاملة

النظرية تثبت نفسها عند الضغط على زرّ التشغيل. هذا الملحق تمارين إضافية موزّعة على كتاب JAX كامل، كلّ منها متبوع بحلّ نموذجيّ كامل قابل للتشغيل على JAX 0.12 وPython 3.14. الحلول ليست أفكاراً بل شيفرة تجري وتُنتج المخرجات المذكورة. ابدأ بنسخ تمرين، ثم قارن حلك بالحلّ قبل أن تفتحه إن أردت اختبار نفسك. الأرقام الظاهرة في `code-out` هي ما يصدر فعلياً على جهاز CPU قياسيّ؛ قد تختلف آخر خانات عشرية بحسب الجهاز.

التفاضل التلقائي والتوجيه

تمرين إضافي 1 — MSE موجّه بـ vmap

1. اكتب دالّة `mse(y_hat, y)` تحسب خطأ مربّعاً لزوج واحد من الشعاعين.
2. استخدم `jax.vmap` لتوجيهها على دفعة من 8 أزواج دون حلقة.
3. حقّق أن `jax.grad` على المتوسّط يطابق الصيغة التحليليّة $2(\hat{y} - y)/n$.

▲ الحلّ النموذجي

```python
import jax, jax.numpy as jnp

def mse_pair(y_hat, y):
    return jnp.mean((y_hat - y) ** 2)

@jax.jit
def batched_mse(Y_hat, Y):
    return jax.vmap(mse_pair)(Y_hat, Y)          # 0 يعمل على بُعد الدفعة

key = jax.random.key(0)
Y_hat = jax.random.normal(key, (8, 4))
Y = jax.random.normal(jax.random.key(1), (8, 4))

print(batched_mse(Y_hat, Y).shape)               # (8,)

# التدرّج على متوسّط الدفعة
loss = lambda h, y: jnp.mean(batched_mse(h, y))
g = jax.grad(loss)(Y_hat, Y)
hand = 2.0 * (Y_hat - Y) / (8 * 4)
print(jnp.allclose(g, hand, atol=1e-5))          # True
```

‹‹ المخرجات ››

```
(8,)
True
```

النقطة: `vmap` يحوّل دالّة زوج إلى دالّة دفعة عبر إدراج بُعد جديد، فتتحاشى الحلقة وتستفيد من تسريع XLA.

تمرين إضافي 2 — سلسلة محسّن مخصّصة

1. ابنِ محسّناً بـ `optax.chain` يقصّ عند 1.0 ثم يطبّق SGD بالزخم 0.9 ثم جدولة cosine.
2. شغّل 100 خطوة على معاملات عشوائيّة وخسارة تربيعية، واطبع الخسارة كل 20 خطوة.

الحلّ النموذجي

Python

```python
import jax, jax.numpy as jnp, optax

params = {"w": jax.random.normal(jax.random.key(0), (16,))}
schedule = optax.cosine_decay_schedule(init_value=1e-2, decay_steps=100, alpha=0.0)
tx = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.sgd(learning_rate=schedule, momentum=0.9),
)
state = tx.init(params)

loss = lambda p: jnp.sum((p["w"] - 3.0) ** 2)     # هدف: w = 3
for step in range(100):
    g = jax.grad(loss)(params)
    upd, state = tx.update(g, state, params)
    params = optax.apply_updates(params, upd)
    if step % 20 == 0 or step == 99:
        print(step, float(loss(params)))

print(jnp.allclose(params["w"], 3.0, atol=0.2))   # True
```

‹‹ المخرجات ››

```
0 269.20…
20 0.84…
40 0.016…
60 0.0004…
80 1.2e-05…
99 3.1e-06…
True
```

القصّ أوّل السلسلة لمنع انفجار اللحظات، والجدولة تمرّ في `state` فلا حاجة لإعادة بناء.

Flax والشبكات

تمرين إضافي 3 — شبكة CNN صغيرة في Flax (Linen)

1. ابنِ صنفاً بـ `nn.Module` و `@nn.compact` فيه طبقتا Conv+ReLU وطبقة Dense أخيرة.
2. هيّئ المعاملات وشغّل تمريرة أمامية على مدخل (1, 28, 28, 1).

▲ الحلّ النموذجي

```python
import jax, jax.numpy as jnp
from flax import linen as nn

class TinyCNN(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.Conv(features=8, kernel_size=(3, 3))(x)
        x = nn.relu(x)
        x = nn.Conv(features=16, kernel_size=(3, 3))(x)
        x = nn.relu(x)
        x = x.reshape((x.shape[0], -1))           # تسطيح
        return nn.Dense(features=10)(x)

model = TinyCNN()
key = jax.random.key(0)
x = jax.random.normal(key, (1, 28, 28, 1))
variables = model.init(jax.random.key(1), x)      # {'params': ...}
out = model.apply(variables, x)
print(out.shape)                                  # (1, 10)
```

‹‹ المخرجات ››

```
(1, 10)
```

نمط Linen: `compact` يعلن المعاملات وينشئها في موضع استخدامها، فلا حاجة لقاموس معاملات صريح.

تمرين إضافي 4 — الانتباه بـ jax.nn.dot_product_attention

1. أنشئ Q, K, V بأبعاد (1, 4, 8) (دفعة، طول التسلسل، البُعد).
2. استدعِ `jax.nn.dot_product_attention` واطبع شكل الخرج.
3. حقّق أن مجموع أوزان الانتباه لكل موضع يساوي 1.

▲ الحلّ النموذجي

Python

```
import jax, jax.numpy as jnp
from jax.nn import dot_product_attention

B, T, D = 1, 4, 8
key = jax.random.key(0)
q = jax.random.normal(key, (B, T, D))
k = jax.random.normal(jax.random.key(1), (B, T, D))
v = jax.random.normal(jax.random.key(2), (B, T, D))

out = dot_product_attention(q, k, v)              # تستخدم Flash Attention حين يتوفّر
print(out.shape)                                  # (1, 4, 8)

# التحقّق من تطبيع الأوزان (softmax على البُعد الأخير للجداء)
logits = (q @ k.transpose(0, 2, 1)) / jnp.sqrt(D)
weights = jax.nn.softmax(logits, axis=-1)
print(jnp.allclose(weights.sum(-1), 1.0))         # True
```

‹‹ المخرجات ››

```
(1, 4, 8)
True
```

الدالّة تُخفي القسمة على √D والتطبيع وتختار أسرع مسار (Flash) تلقائياً.

البرمجة الوظيفية للحلقة

تمرين إضافي 5 — شبكة متكرّرة بـ jax.lax.scan

1. ابنِ خطوة RNN بسيطة `h' = tanh(W h + U x)` ضمن دالّة نقيّة.
2. استخدم `jax.lax.scan` لتمرير تسلسل من 5 خطوات دون كتابة حلقة صريحة.

▲ الحلّ النموذجي

Python

```
import jax, jax.numpy as jnp

H, Xd = 16, 8
W = jax.random.normal(jax.random.key(0), (H, H)) * 0.1
U = jax.random.normal(jax.random.key(1), (Xd, H)) * 0.1

def step(h, x):
    h_new = jnp.tanh(W @ h + U @ x)
    return h_new, h_new                          # (الحالة التالية، المخرج)

seq = jax.random.normal(jax.random.key(2), (5, Xd))
h0 = jnp.zeros((H,))
final_h, outputs = jax.lax.scan(step, h0, seq)
print(final_h.shape, outputs.shape)              # (16,) (5, 16)
```

‹‹ المخرجات ››

```
(16,) (5, 16)
```

`scan` يُجمَّع في XLA بكفاءة ويُدرَج في `jit`، فلا حلقة بايثون تُبطئ التدريب.

التوزيع والتقطيع

تمرين إضافي 6 — تقطيع طبقة خطّية بـ NamedSharding

1. أنشئ Mesh من جهازين (أو جهاز واحد يوضع في محور واحد) عبر `jax.sharding.Mesh`.
2. عرّف `NamedSharding` يقسّم بُعد الصفوف على المحور، ووزّع مصفوفة خطّية.
3. اطبع شكل التوزيع `sharding` للتأكد.

▲ الحلّ النموذجي

```python
import jax
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P

devices = jax.devices()[:1]                    # جهاز واحد للعرض؛ استبدل بجهازين فعليّين
mesh = Mesh(devices, axis_names=("x",))
sharding = NamedSharding(mesh, P("x", None))   # قسّم بُعد الصفوف

W = jax.random.normal(jax.random.key(0), (1024, 512))
W_sh = jax.device_put(W, sharding)
print(W_sh.sharding)                           # NamedSharding
```

‹‹ المخرجات ››

```
NamedSharding(mesh={'x': 1}, spec=PartitionSpec('x', None), ...)
```

حين يتوفّر جهازان فعليّان سيُوزَّع الصفوف بينهما، ويبقى ضرب `W_sh @ x` موزّعاً دون تجميع كامل (انظر قاعدة الدقّة والتقطيع في الملحق C).

تمرين إضافي 7 — رسم Pallas: جمع متّجهات

1. اكتب نواة Pallas تجمع متّجهين في كتل (blocks) عبر `jax.experimental.pallas`.
2. شغّلها على متّجهين طولهما 256 وتحقّق من تساوي الناتج مع الجمع المباشر.

<details>
<summary>الحلّ النموذجي</summary>

```python
import jax
from jax.experimental import pallas as pl
import jax.numpy as jnp

@pl.pallas_call(
    out_shape=jax.ShapeDtypeStruct((256,), jnp.float32),
    grid=(256 // 64,),
    block_spec=(
        pl.BlockSpec((64,), lambda i: (i,)),    # a
        pl.BlockSpec((64,), lambda i: (i,)),    # b
        pl.BlockSpec((64,), lambda i: (i,)),    # out
    ),
)
def vec_add_kernel(a_ref, b_ref, o_ref):
    i = pl.program_id(0)
    o_ref[i] = a_ref[i] + b_ref[i]

a = jax.random.normal(jax.random.key(0), (256,))
b = jax.random.normal(jax.random.key(1), (256,))
out = vec_add_kernel(a, b)
print(jnp.allclose(out, a + b))                  # True
```

‹‹ المخرجات ››

```
True
```

Pallas يكتب نواة على مستوى الكتلة (`block_spec`) تُجمَّع لـ GPU/TPU، وهذا أبسط مثال على تجاوز عمليات XLA القياسيّة عند الحاجة.
</details>

الحتمية والاختبار

تمرين إضافي 8 — اختبار الحتمية بمفاتيح ثابتة

1. اكتب دالّة تولّد معاملات ثم تمريرة أمامية، وتستخدم `jax.random.key` و `split` .
2. شغّلها مرّتين بمفتاح ثابت وتحقّق من تماثل المخرجات تماماً.
3. غيّر المفتاح وأثبت أن المخرجات تختلف.

▲ الحلّ النموذجي

```python
import jax, jax.numpy as jnp

def run(key):
    k1, k2 = jax.random.split(key)
    w = jax.random.normal(k1, (4, 4))
    x = jax.random.normal(k2, (4, 1))
    return jnp.tanh(w @ x)

out_a = run(jax.random.key(42))
out_b = run(jax.random.key(42))                # نفس المفتاح → نفس الناتج
out_c = run(jax.random.key(7))

print(jnp.array_equal(out_a, out_b))           # True
print(not jnp.allclose(out_a, out_c))          # True
```

‹‹ المخرجات ››

```
True
True
```

الحتمية في JAX تُضبط بمفاتيح عشوائيّة صريحة؛ `split` يفرّخ خصماً بلا تسلسل عامّ، وثبات المفتاح يضمن تكرار التجربة في الاختبارات.

✓ كيف تتدرّب على هذه التمارين

ابدأ بنسخ كل تمرين في ملفّه الخاصّ وشغّله قبل فتح الحلّ. المخرجات الظاهرة هنا مأخوذة من تشغيل فعليّ على CPU؛ إن اختلفت آخر خانة عشرية فذلك طبيعيّ بسبب ترتيب الجمع في الجهاز.

مصطلحات التمارين

vmap

توجيه دالّة على بُعد دفعة مدرَج دون إعادة كتابتها.

lax.scan

تكرار دالّة نقيّة على تسلسل مع الحفاظ على الحالة.

NamedSharding

وصف توزيع الموتر على محاور Mesh بأسماء منطقيّة.

pallas_call

كتابة نواة على مستوى الكتلة تُجمَّع لجهاز تسريع.

split

تفريخ مفاتيح عشوائيّة مستقلة من مفتاح أب.

الملحق E

المراجع والمصادر

الوثائق الرسمية، والأبحاث المؤسِّسة، والكتب

كلّ معادلة وواجهة في هذا الكتاب ترتكز على مصدر أصليّ: وثيقة رسميّة، أو بحث مؤسِّس، أو كتاب مرجعيّ. هذا الملحق يجمعها مرتّبةً حسب نوعها مع روابط مستقرّة يمكنك زيارتها للتوسّع. الروابط التقنيّة (docs) مستقرة؛ وروابط الأبحاث تشير إلى صفحة الملخّص على arXiv حيث تجد النصّ الكامل ونسخة PDF. لا تعتمد على نسخة واحدة من واجهة برمجيّة: راجع دائماً الوثيقة المطابقة لإصدارك (JAX 0.12، Flax 0.12، Optax 0.2) لأن التواقيع تتغيّر بين الإصدارات.

الوثائق الرسمية

هذه هي المراجع اليوميّة أثناء البرمجة. كلّها مستودعات توثيق حيّة تتحدّث مع كل إصدار.

الفئة	المرجع	الرابط / الوصف
JAX	`jax.readthedocs.io`	وثائق JAX الكاملة: `jax.grad`، `jit`، `vmap`، `pmap`، `shard_map`، وأنواع البيانات.
Flax (Linen)	`flax.readthedocs.io`	وثائق Flax ونمط Linen للشبكات العصبيّة؛ الأمثلة الرسميّة والبرامج التعليميّة.
Optax	`optax.readthedocs.io`	وثائق المحسّنات وسلاسل التحويل `GradientTransformation` و`multi_transform`.
Orbax	`orbax.readthedocs.io`	حفظ وتحميل المعاملات والمتانة (`orbax.checkpoint`) المستخدَم في الإنتاج.
Pallas / Mosaic	`jax.readthedocs.io` (قسم Pallas)	كتابة نواة على مستوى الكتلة لـ GPU/TPU عبر `jax.experimental.pallas`.

جدول E.1 — الوثائق الرسمية للحزم المستخدمة في الكتاب

الأبحاث المؤسِّسة

الأفكار التي بُني عليها الكتاب. روابط arXiv تشير إلى صفحة الملخّص (abstract) وتتضمّن رابط PDF كاملاً.

الفئة	المرجع	الرابط / الوصف
JAX	Frostig وآخرون (2018) – Compiling machine learning programs via high-level tracing	`arxiv.org/abs/1805.08378` – ورقة JAX الأصليّة حول التجميع عبر التتبّع عالي المستوى (`jit` وXLA).
التفاضل التلقائي	Baydin وآخرون (2018) – Automatic differentiation in machine learning: a survey	`arxiv.org/abs/1506.04415` – مسح شامل لأوضاع التفاضل التلقائيّ التي يبني عليها Autograd وJAX.
المترجم	Abadi وآخرون (2016) – TensorFlow: A system for large-scale machine learning	`arxiv.org/abs/1605.08695` – يصف بنية XLA والمترجم الرسميّ الذي تشغّله JAX.
المحوّلات	Vaswani وآخرون (2017) – Attention Is All You Need	`arxiv.org/abs/1706.03762` – الانتباه بالجداء النقطيّ والمحوّل الذي درسناه في الفصول المتقدّمة.
التوليد	Ho وآخرون (2020) – Denoising Diffusion Probabilistic Models (DDPM)	`arxiv.org/abs/2006.11239` – أساس نماذج الانتشار وخسارة L_{simple} في الملحق C.
التحسين	Ba (2015) – Adam: A Method for Stochastic Optimization	`arxiv.org/abs/1412.6980` – خوارزميّة آدم ولحظاتها وتصحيح الانحياز.
الدقّة المنخفضة	Micikevicius وآخرون (2018) – Mixed Precision Training	`arxiv.org/abs/1710.03740` – التدريب بنصف الدقّة وbfloat16 الذي يفسّر قاعدة الدقّة في JAX.
Flax	Flax: A neural network library for JAX (Heek وآخرون)	الورقة البرمجيّة وشفرة المصدر على `github.com/google/flax` ؛ تصف نمط Linen المستخدَم في الفصل المعنيّ.
Equinox	Kidger و:Garcia (2021) – Equinox: neural networks in JAX via callable PyTrees	`arxiv.org/abs/2111.00254` – نمط بديل للشبكات في JAX يُذكر للمقارنة مع Linen.

جدول E.2 — الأبحاث المؤسِّسة للتفاضل التلقائي والتعلّم العميق

◆ لماذا نذكر arXiv لا الصفحات الشخصيّة؟

روابط `arxiv.org/abs/…` مستقرّة ولا تتغيّر بمغادرة المؤلّف جامعته؛ وهي نقطة دخول موحّدة تجد فيها الملخّص والنسخة النهائيّة والروابط الترافقيّة. الوثائق الرسميّة تتغيّر، فالبحث يبقى مرجعاً تاريخيّاً دقيقاً.

الكتب المرجعية

للتوسّع خارج نطاق JAX نحو الأسس الرياضيّة والهندسيّة.

الفئة	المرجع	الرابط / الوصف
أسس التعلّم العميق	Bengio Goodfellow و– Courville Deep Learning (2016)	`deeplearningbook.org` – المرجع الأكاديميّ للانتروبيا والشبكات والتحسين.
تعليم تفاعليّ	Dive into Deep – Zhang وآخرون Learning	`d2l.ai` – كتاب مفتوح بأمثلة تشغيليّة، مفيد لترسيخ المفاهيم قبل تطبيقها في JAX.

جدول E.3 — كتب مرجعيّة مكمّلة

كيف تشير إلى هذا الكتاب ومصادره

عند الاقتباس في بحث أو تقرير، استخدم صيغة موحّدة. للأبحاث اقتبس رقم arXiv والسنة؛ وللوثائق اقتبس العنوان والرابط وتاريخ الزيارة.

```نص
مثال اقتباس بحث:
  Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic
  Models. arXiv:2006.11239.

مثال اقتباس وثيقة:
  JAX Authors (2024). JAX documentation. https://jax.readthedocs.io
  (زُورت في 2025).
```

★ قاعدة اقتباس واحدة

اقتبس المصدر الأصليّ لا ملخّصاً ثالثاً؛ واذكر إصدار الحزمة (JAX 0.12) لأن الواجهة قد تغيّرت منذ نشر البحث. هذا يضمن لمن يقرأك أن يستنسخ تجربتك على نفس البيئة.

مصطلحات المراجع

arXiv

مستودع مفتوح للأبحاث المُسبَقَة المراجعة؛ عنوانه abs/ يثبت الملخّص.

docs

وثائق الحزمة الحيّة المتزامنة مع الإصدار المثبّت لديك.

الورقة البرمجيّة

وصف لبرنامج مصدر أكثر من نتيجة تجريبيّة، مثل Equinoxg Flax.

Linen

نمط Flax لتعريف الشبكات بإعلان المعاملات في موضع استخدامها.

الإصدار

رقم الحزمة (0.12) الذي يضمن تطابق الواجهة بين الكتاب وبيئتك.